PERSONAL TECHNICAL NOTEBOOK

AI Infra Learning Book

完整保存每一次提问与教学回答，包括文字、公式、表格、代码和图示。

---

14 个知识点 · 生成于 2026/08/27 10:51

Contents

目录

01 **在线笔记本使用教程**

如何使用 AI Infra Learning Book，并理解它与知识图谱的分工?

2026/07/27 01:53
重要度 5/5

02 **Overlap 的实现方法和背后的逻辑**

1. overlap 的实现方法和背后的逻辑

2026/07/23 22:07
重要度 5/5

03 **FSDP：完全分片数据并行**

详细讲解大模型中的 FSDP 技术，并解释和其他并行策略有什么区别，最后总结成表格列出异同和适合场景。

2026/07/26 15:09
重要度 5/5

04 **大模型训练中的通信方式**

详细讲解大模型训练中所有通信方式，重点拆开讲解，说明应用位置、时机以及能否和计算 overlap。

2026/07/26 15:26
重要度 5/5

05 **VPP：虚拟流水线并行与 Interleaved 1F1B**

详细讲解 vpp，并且我现在的记事本网址为：https://psgmessi.github.io/ai-infra-learning/ 以后的更新都直接在这个网址上更新，把这个方式记录进skill里，以后每次提问新的知识点都将更新到这里，包括现在这个。随后补充要求：按照 AI Infra Tutor Skill 给出完整讲解并同步更新知识图谱和线上笔记本。

2026/07/27 03:08
重要度 5/5

06 **torch、Megatron、CUDA、Triton：概念、关联与整块知识地图**

torch， Megatron，cuda，triton 这些概念分别是什么？之间有什么关联？有没有其他相关的概念和知识点，把这一块整体的内容都给我列出来，清晰地讲解透彻

2026/07/27 16:15
重要度 5/5

07 **CUDA 编程模型与 GPU 内存层级：从 thread 到 HBM**

继续深入讲解这部分：前置：CUDA 编程模型与 GPU 内存层级 —— 理解 kernel/thread/block、寄存器与 HBM，是理解上层一切性能优化的基础。

2026/07/27 17:18
重要度 5/5

08 **MoE 架构详解：从稀疏路由到 Expert Parallel**

详细讲解 MoE 架构

2026/07/29 20:27
重要度 5/5

09 **大模型并行策略全解：DP、FSDP、TP、PP/VPP、CP、EP 与多维组合**

详细讲解不同的并行策略：PP,EP,DP,CP,TP等，每一种都讲透彻，并且列出各自的异同，适合的场景，具体应用例子，然后全部串起来，举一个简单的数据例子，把各种并行策略都叠加使用，完整的展现每种并行策略的实际应用。

2026/07/29 20:49
重要度 5/5

10 **大模型数值精度全解：FP32、BF16、FP16、FP8、INT8、INT4 与 FP4**

详细讲解不同精度：bf32, 16, 8等等，有什么区别，作用在哪里，如何影响训练的性能和显存等指标

2026/08/02 21:09
重要度 5/5

11 **H2D、D2H 与 CPU – GPU 协同：计算、内存、通信和流水线优化**

详细讲解D2H,H2D，以及GPU和CPU各自的运算和通信特点，还有两者之间的互相通信，什么时候应用，可以得到什么方面的优化提升

2026/08/03 11:42
重要度 5/5

12 **大模型训练优化器全解：作用、主流算法、显存代价与选型**

详细解释训练中优化器的作用以及不同主流优化器的详细讲解，并且归纳总结不同优化器的异同和适用场景

2026/08/07 15:24
重要度 5/5

13 **反向传播全流程：从 Loss 到每一个参数梯度**

从头到尾详细讲解一次反向传播的过程，并且要简单的例子来说明，包含具体数值和shape，使用图片来辅助我完整理解整个流程

2026/08/13 11:14
重要度 5/5

14 **大模型训练 LM Head 全解：前向、反向、FLOPs、显存与优化**

详细讲解大模型训练中lm head这部分，包括前反向的每一步。举一个简单例子说明清楚，给出每一步的shape，计算公式，flops，作用和显存开销等大模型训练关心的数据。配合图片辅助我进行理解。我当前在做一个lm head显存优化的工作，讲完上述内容后再重点关注这部分变化，帮我画出一个动态显存开销图，在网页上清晰地展现出来每一步的计算步骤，计算量和显存开销变化。

2026/08/27 10:35
重要度 5/5

使用教程 · 学习系统 · 知识管理

01

在线笔记本使用教程

> **原始问题：**如何使用 AI Infra Learning Book，并理解它与知识图谱的分工?

提问：2026/07/27 01:53　更新：2026/08/05 11:50　重要度：5/5

这本书的定位：知识图谱负责展示知识点之间的结构与联系；AI Infra Learning Book 负责完整保存每次提问和导师式回答，包括文字、公式、代码、表格和图片。

一、如何阅读

进入网页后先看到封面。点击“翻开学习笔记”进入目录。目录默认按实际提问时间从早到晚排列，第一条使用教程固定置顶；也可以切换为从晚到早或按重要程度排序。

点击目录里的知识点会跳转到对应的完整正文。所有正文同时按目录当前顺序连续排列，因此既可以通过目录跳转，也可以像翻阅一本长书一样持续向下滚动阅读。

二、目录与检索

- **时间顺序：**保留学习轨迹，适合按提问历史复盘。
- **重要程度：**优先展示基础性强、影响范围广、工程实践价值高的知识点。
- **搜索：**可按标题、原始问题和标签过滤目录及正文。
- **阅读进度：**页面顶部红色细线表示整本笔记的阅读进度。

三、每条笔记包含什么

每条笔记都包含原始问题、提问时间、重要程度、完整教学正文，以及回答中生成的关键图、公式、表格、代码与例子。只会省略“开始执行”“生成完成”等工具状态通知，不会删减教学内容。

四、知识图谱与笔记本的分工

载体	保存内容	最适合做什么
三维知识图谱	重点摘要、机制、公式、关键图、关系边、分组	理解知识结构、查看关联、寻找学习路径
在线笔记本	原始问题 + 完整回答正文 + 全部图片/代码/表格	完整阅读、复习、跨设备查看

五、以后如何自动更新

以后每当你提出新的 LLM 或 AI Infra 概念问题，导师会先在 WorkBuddy 正文中完整回答，然后执行双写：

1. 把重点结构化写入三维知识图谱，并建立分组和关系。
2. 把原始问题与完整回答复制到 AI Infra Learning Book。
3. 重新生成静态网页，并在需要时重新发布在线版本。

如果你说“不记录本次”，本次内容不会进入图谱和笔记本；如果你说“不记录刚才”，会撤回最近一次记录。

六、在线版本说明

网页采用纯静态方式发布，手机和其他电脑可以直接通过分享链接阅读。每次新增内容后，本地笔记本会立即更新；在线版本需要重新发布。发布完成后应以最新分享链接为准。

七、完整性原则

> 回答必须完整出现在 WorkBuddy 正文中；笔记本保存的是正文的完整副本，而不是摘要。知识图谱可以提炼重点，在线笔记本不能用摘要替代完整回答。

八、PDF 下载

目录标题右侧的 **Download complete PDF** 可以下载整本 AI Infra Learning Book；每个 Chapter 目录项右侧的 **Download PDF** 可以只下载该知识点。

- **单章 PDF**：从 Chapter 标题、原始问题和元数据开始，完整保留正文直到该章结尾。
- **整本 PDF**：包含封面、目录和当前全部 Chapter，按固定提问时间顺序排列。
- **结构保真**：PDF 与在线正文共用同一份 HTML 和书籍样式，保留表格、列表、代码、公式、SVG 图和配色。
- **自动更新**：每次新增或更新知识点后，单章 PDF 和整本 PDF 都会重新生成并发布。

02

Overlap 的实现方法和背后的逻辑

> **原始问题：**1. overlap 的实现方法和背后的逻辑

提问：2026/07/23 22:07　更新：2026/07/27 01:53　重要度：5/5

一句话结论

Overlap 的本质是：让 GPU 在等待某件慢事情（通信、数据搬运、CPU 下发指令）完成时不要闲着，而是同时去做另一件不依赖该结果的事情，从而用并行把等待时间“藏”起来。

它的实现依赖三个底层能力：异步执行 + 多条独立的执行流(stream/engine) + 依赖关系管理。理想情况下总时间 ≈ max(计算, 通信)，而不是二者相加。

直觉：餐厅上菜模型

把 GPU 想象成厨师，把通信/拷贝想象成“跑堂端菜到隔壁桌”。

- 没有 overlap（串行）：厨师炒完一道菜 → 亲自端出去 → 回来再炒下一道。端菜时厨师完全空闲。
- 有 overlap：厨师炒完一道菜交给跑堂去端（异步），自己立刻开始炒下一道。端菜和炒菜同时进行。

关键前提：炒下一道菜不依赖“上一道菜已经端到桌上”。如果有依赖就无法重叠。类比失效点：现实中跑堂(通信引擎)和厨师(计算SM)可能抢同一资源(如显存带宽、SM)，所以 overlap 不是完全免费的并行。

底层逻辑：为什么 GPU 能 overlap

1) GPU 执行是异步的：CPU 调用 kernel 后不等它执行完，把任务提交到队列就立即返回，GPU 在后台执行。下发与执行解耦，CPU 才有机会提交别的任务。

2) GPU 有多个独立执行引擎：Compute Engine(SM) 执行计算 kernel；Copy Engine(DMA) 专门做显存拷贝(H2D/D2H)，独立于 SM；NCCL 通信通常也是 kernel，占用 SM 或专用路径。Copy Engine 与 Compute Engine 物理独立，所以“拷贝”和“计算”天然可真并行。

3) CUDA Stream：表达“哪些任务可以并行”。同一条 stream 内 FIFO 串行；不同 stream 之间默认无顺序约束，可自由并行。所以 overlap 的实现套路就是：把两件想并行的事放进两条不同 stream，用 event 管理跨流依赖。

关键图：串行 vs 重叠的时间线



公式与量级

```
无 overlap（串行）：  T_total = T_comp + T_comm
理想 overlap:        T_total ≈ max(T_comp, T_comm)

例：T_comp = 100 ms, T_comm = 40 ms
  串行   = 100 + 40 = 140 ms
  重叠   = max(100, 40) = 100 ms   → 加速 1.4×，省约 28.6%

例：T_comm = 120 ms（通信占比更大）
  串行   = 100 + 120 = 220 ms
  重叠   = max(100, 120) = 120 ms  → 加速 1.83×

物理下界：T_total ≥ max(T_comp, T_comm)，永远不可能更低。
```

最小 Sample：拷贝-计算 overlap

```
import torch
N_CHUNKS = 4
copy_stream = torch.cuda.Stream()   # 一条流负责拷贝
comp_stream = torch.cuda.Stream()   # 一条流负责计算

# pinned 内存是 overlap 成立的隐藏前提，否则拷贝退化成同步
host = [torch.randn(1024,1024).pin_memory() for _ in range(N_CHUNKS)]
dev  = [torch.empty(1024,1024, device='cuda') for _ in range(N_CHUNKS)]
out  = [None]*N_CHUNKS

for i in range(N_CHUNKS):
    with torch.cuda.stream(copy_stream):
        dev[i].copy_(host[i], non_blocking=True)   # 异步拷贝第 i 块
    comp_stream.wait_stream(copy_stream)           # 跨流依赖
    with torch.cuda.stream(comp_stream):
        out[i] = dev[i] @ dev[i]                   # 计算第 i 块
torch.cuda.synchronize()

# 效果：算第 i 块时，第 i+1 块的拷贝同时进行
# 要点：overlap = 拆分 + 多流 + 依赖管理
```

常见误区与失败模式

1) 误以为多流一定并行：若两条流都要用 SM 而 SM 已被占满，通信 kernel 抢不到资源，overlap 名存实亡（NCCL 占 SM 是典型）。
2) 忘记 pinned memory：拷贝退化成同步，overlap 失效，而且不报错只是变慢，极难发现。
3) 依赖没管好：跨流没用 event 同步，计算读到没传完的数据，导致数值错误或 NaN。
4) bucket 太小：每次通信太小，启动开销占比过高反而变慢；太大又减少重叠机会。
5) 只看平均不看时间线：必须用 Nsight Systems / torch.profiler 确认计算块和通信块真的在时间上重叠。

FSDP：完全分片数据并行

> **原始问题：**详细讲解大模型中的 FSDP 技术，并解释和其他并行策略有什么区别，最后总结成表格列出异同和适合场景。

提问：2026/07/26 15:09　更新：2026/07/27 01:53　重要度：5/5

一句话结论

FSDP（Fully Sharded Data Parallel，完全分片数据并行）的本质是：在数据并行的语义下，把每张卡上"完整复制一份模型训练状态"改成"每张卡只存 1/N 份参数、梯度和优化器状态"；在真正要用到某一层时，才临时用一次 AllGather 通信把这一层的完整参数拼回来，算完立刻丢掉。

它用通信换显存：把 DDP 里冗余复制的内存开销摊薄到 N 张卡上，从而能训练远超单卡显存的大模型。它是 ZeRO（Zero Redundancy Optimizer）思想在 PyTorch 中的原生实现。

系统位置：它解决/不解决什么

解决的核心问题：单卡装不下完整训练状态。一个 7.5B 模型混合精度+Adam 需 16×7.5e9 = 120GB，单张 80GB H100 都装不下，而 DDP 每张卡都存这 120GB，纯属冗余。

不解决：单层本身太大装不下（需 TP 切算子）；层数太多单卡放不下计算图（需 PP）。它减少的是显存不是计算量——每卡 FLOP 一点没少。

上下游：属于数据并行维度的增强版，可与 TP/PP/EP 正交组合成 3D/4D 并行，替代的是 DDP。

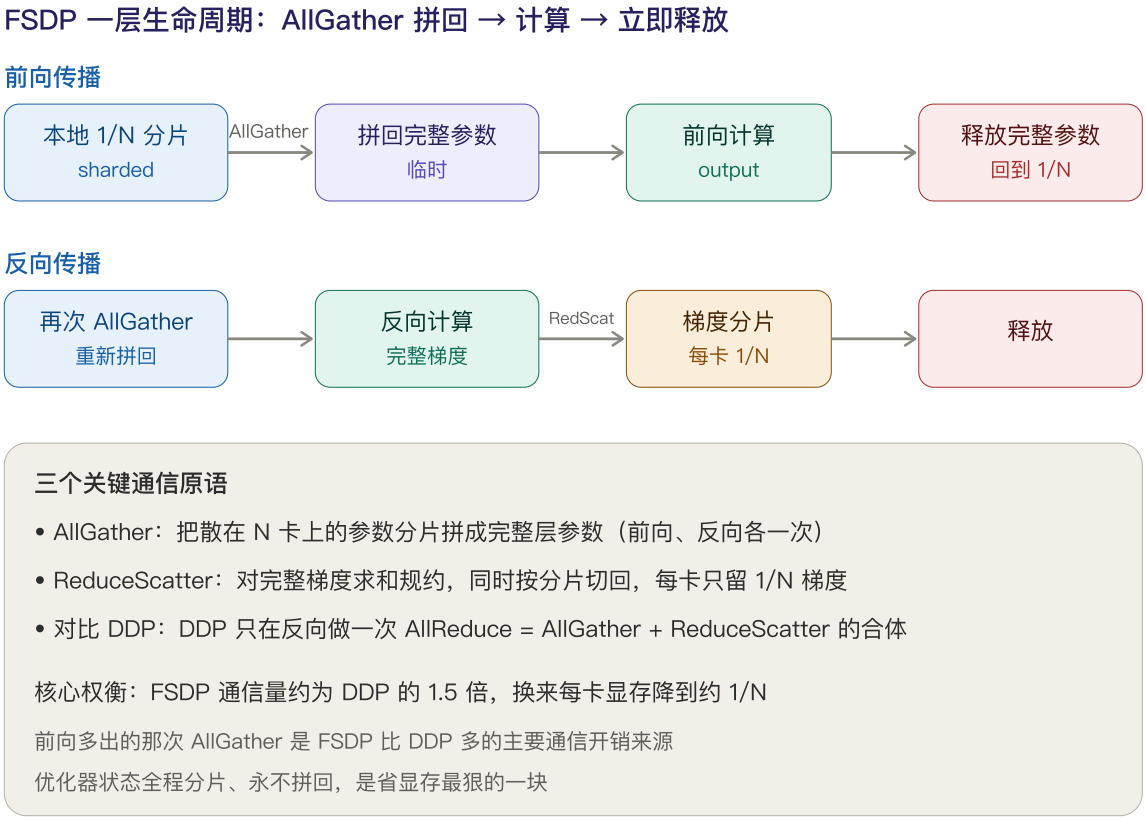
直觉：共享工具间的工厂

DDP：N 个工人每人一整套完整工具（全部参数/梯度/优化器状态），N 套占 N 倍仓库空间，极度浪费。

FSDP：N 个工人共用中央工具库，每人只保管 1/N 工具。轮到用某把扳手（某一层）时，保管它的工人广播复印给所有人（AllGather），大家一起用，用完立刻销毁复印件，只有原保管人留正本。

关键代价：每次用工具都要现场“复印+销毁”（通信+临时显存峰值）。类比失效点：现实中复印(AllGather)和干活(计算)可重叠——用当前扳手时提前复印下一把(prefetch)。

关键图：单层参数生命周期时序



机制：一个训练 step 里做什么

调度单位是 FSDP unit（通常一层）。

前向：1) 本地只存 1/N 参数分片；2) AllGather 拼回完整参数（临时）；3) 前向计算；4) 立即释放完整参数。

反向：5) 再次 AllGather（因为完整参数已被释放）；6) 反向计算得完整梯度；7) ReduceScatter 对梯度求和并按分片切回，每卡只留 1/N；8) 释放完整参数。

优化器更新：每卡用自己的 1/N 梯度+1/N fp32 参数副本+1/N 优化器状态，独立更新 1/N 参数。优化器状态全程分片、永不拼回，这是省显存最狠的一块。

与 DDP 通信对比：DDP 反向做 1 次 AllReduce；FSDP 做 前向AllGather + 反向AllGather + 反向ReduceScatter，通信量约 DDP 的 1.5 倍。

公式与量级

```
每参数训练状态 ≈ 16 Byte
  = fp16参数(2) + fp16梯度(2) + fp32副本(4) + 动量m(4) + 方差v(4)

每卡常驻训练状态（不含激活）：
  DDP        : M = 16 × Ψ             （完整复制，与 N 无关）
  FSDP ZeRO-3: M ≈ 16 × Ψ / N         （全部摊薄到 N 卡）

数值算例（Ψ=7.5B, N=64）：
  DDP  : 16 × 7.5e9        = 120 GB / 卡  → 放不下
  FSDP : (16 × 7.5e9) / 64 ≈ 1.875 GB / 卡 → 轻松放下

通信量：FSDP / DDP ≈ 3P / 2P = 1.5（多了前向 AllGather）

重要：以上只算参数相关状态，未含激活。FSDP 默认不分片激活，大 batch/长序列时激活可能才是 OOM 主因。
```

最小 Sample： PyTorch FSDP

```
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP
from torch.distributed.fsdp import ShardingStrategy, BackwardPrefetch
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy
import functools, torch.nn as nn

# 关键点1：按 transformer block 切分 FSDP unit
policy = functools.partial(transformer_auto_wrap_policy,
            transformer_layer_cls={nn.TransformerEncoderLayer})

model = FSDP(
    model,
    auto_wrap_policy=policy,
    # 关键点2：FULL_SHARD=ZeRO-3（全分片）；SHARD_GRAD_OP=ZeRO-2
    sharding_strategy=ShardingStrategy.FULL_SHARD,
    # 关键点3：反向预取，让 AllGather 与反向计算重叠
    backward_prefetch=BackwardPrefetch.BACKWARD_PRE,
)
# 训练循环与 DDP 几乎一样，分片/拼回/释放由 FSDP 内部完成
for batch in loader:
    loss = model(batch).loss
    loss.backward()     # 内部自动 AllGather + ReduceScatter
    optimizer.step()    # 每卡只更新自己的 1/N 参数
```

应用场景（何时用 FSDP）

- 模型能装进单卡只想加速 → 用 DDP（FSDP 额外通信不划算）
- 训练状态超单卡但单层能装下 → 首选 FSDP
- 单层太大装不下 → FSDP + TP
- 层数极多想提利用率 → FSDP + PP
- MoE 大模型 → FSDP + EP

- 显存极紧 → FSDP + CPU offload

常见误区

1) 误以为 FSDP 减少计算量：错，只减显存，每卡 FLOP 与 DDP 一样。
2) 误以为用了 FSDP 显存就一定够：错，默认不分片激活，大 batch/长序列仍可能 OOM。
3) 把 FSDP 和 TP 混为一谈：FSDP 计算某层时拼回完整参数再算（层计算完整）；TP 是这一层计算本身被切开多卡协同。切分维度不同，可叠加。
4) 以为 ZeRO 和 FSDP 是两个东西：ZeRO 是 DeepSpeed 的算法思想，FSDP 是 PyTorch 原生实现，FULL_SHARD≈ZeRO-3。
5) 切分粒度随意设：太细通信次数暴涨，太粗临时参数撑爆显存峰值，需按 block 粒度并 profiler 调。

大模型训练中的通信方式

> **原始问题：**详细讲解大模型训练中所有通信方式，重点拆开讲解，说明应用位置、时机以及能否和计算 overlap。

提问：2026/07/26 15:26　更新：2026/07/27 01:53　重要度：5/5

一句话结论

大模型训练中的通信只有两大类：集合通信(Collective)和点对点通信(P2P)。集合通信是“一群 GPU 一起参与”(AllReduce/AllGather/ReduceScatter/All-to-All/Broadcast)，由 NCCL 实现；点对点是“一张卡发给另一张卡”(Send/Recv)，主要用于流水线并行传激活。

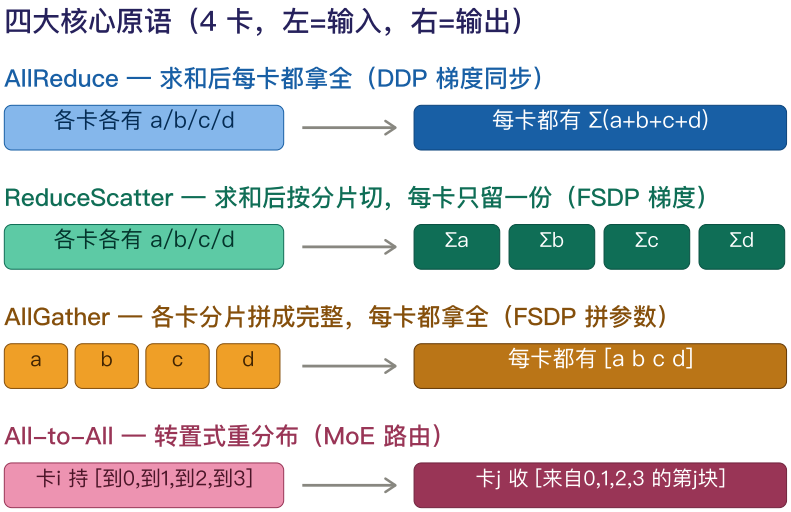
判别口诀：哪种并行策略，决定用哪种通信。DP→AllReduce；FSDP→ReduceScatter+AllGather；TP→AllReduce/AllGather；PP→P2P；MoE→All-to-All。几乎所有通信都能且应与计算 overlap。

地基：通信走哪条物理链路

链路	典型单向带宽	用途
NVLink/NVSwitch(机内)	~450 GB/s (H100)	同机 8 卡，TP 命脉
PCIe(机内)	~64 GB/s (Gen5)	GPU–CPU、offload
IB/RoCE(跨机 RDMA)	~50 GB/s (400Gbps)	跨节点，PP/DP 通信
以太网	更低	一般不用于训练热路径

核心认知：同一次 AllReduce，走 NVLink 和走跨机 IB 慢几个数量级。“哪种并行放机内、哪种跨机”是布局的头等大事。

关键图：四大核心集合通信原语



关键恒等式（务必记住）

AllReduce = ReduceScatter + AllGather（FSDP 拆开 DDP 的数学本质）

Reduce = 求和到一张卡；Gather = 收集到一张卡（原语的“单点”版）

AllReduce —— 数据并行命脉

语义：每张卡有一份等长数据（通常是梯度），逐元素求和，结果每张卡都得到一份完整的和。

用在哪：DDP 梯度同步。N 卡各自在不同 mini-batch 上算梯度，反向后 AllReduce 得全局平均梯度，各自更新出一致参数。

例子：训练 Llama 用 DDP 起 128 卡，每卡吃不同数据，每 step 反向后对全部梯度(7B fp16=14GB)做一次 AllReduce。

通信量：Ring AllReduce 每卡 ≈ 2M，几乎与卡数 N 无关（能 scale 千卡的原因）。

能否 overlap：能，且是头等大事。反向逐层从后往前，某层梯度算完就能立刻 AllReduce，同时继续算前面层。PyTorch DDP 用 gradient bucketing 自动实现。

ReduceScatter + AllGather —— FSDP/ZeRO 两把刀

核心恒等式：AllReduce = ReduceScatter + AllGather，这是理解 FSDP 的钥匙。

ReduceScatter：各卡数据求和后按分片切开，每卡只留 1/N。
AllGather：每卡持一个分片，通信后每卡拿到拼接的完整数据。

用在哪：FSDP/ZeRO-3。前向/反向前用 AllGather 拼回分片参数；反向算完梯度用 ReduceScatter 求和并切片，每卡留 1/N。

为什么不用 AllReduce：AllReduce 后每卡都拿完整梯度仍冗余；ReduceScatter 让每卡只拿 1/N，配合分片优化器状态，显存降到 1/N。FSDP 就是把 DDP 的一次 AllReduce 拆成 ReduceScatter + 平时的 AllGather。

能否 overlap：能，靠 prefetch。算第 i 层时提前 AllGather 第 i+1 层参数；forward_prefetch/backward_prefetch 控制。

All-to-All —— MoE 专属

语义：最"拧巴"的一个。每张卡都持有要发给所有其他卡的不同数据块；通信后每张卡收到来自所有其他卡发给自己的块。本质是跨卡的"矩阵转置式重分布"。

用在哪：MoE 的 Expert Parallel。不同专家在不同卡上，token 经 router 后要发往选中专家所在的卡（dispatch 一次 All-to-All），专家算完再送回（combine 一次 All-to-All）。

例子：训练 Mixtral、DeepSeek-MoE，每个 MoE 层两次 All-to-All。专家跨机时走 IB，极易成瓶颈。

能否 overlap：能但更难。依赖 router 输出（先知道每个 token 去哪），依赖链紧。先进实现(DualPipe/Tutel)会把 dispatch 与其他专家/dense 计算重叠，工程复杂度高于 AllReduce overlap。

P2P Send/Recv —— 流水线并行接力棒

语义：点对点，rank i 把一块数据 Send 给 rank j，rank j Recv。只涉及两张卡。

用在哪：PP 的 stage 间传激活。模型按层切成若干 stage 放不同卡，前向时 stage k 算完把激活 Send 给 stage k+1；反向把梯度 Send 回 stage k。

例子：把 80 层切 8 个 stage，每 stage 10 层，激活沿流水线逐级 Send/Recv，配合 micro-batch 做 1F1B 调度。

通信特点：量相对小(只传激活)，但存在流水线气泡(bubble)——stage 间有依赖，启动收尾阶段部分卡闲置。

能否 overlap：能，这正是流水线调度核心。1F1B 让不同 micro-batch 的前向反向在不同 stage 交错，用一个 micro-batch 的计算掩盖另一个的 P2P 传输，压缩气泡。

overlap 的统一判据

```
能 overlap 的充要条件：
  发起通信 C 后，存在不依赖 C 结果的独立计算 X 可立刻做 → C 与 X 并行
反例（无法 overlap）：
  下一步计算立刻要用这次通信的结果 → 只能干等（暴露在关键路径）

对照：
  DDP AllReduce  → 易（反向逐层有独立计算）
  FSDP AllGather → 易（prefetch 下一层）
  TP AllReduce   → 难（下一步立刻要用，依赖紧 → 必须放 NVLink）
  PP P2P         → 靠 1F1B 调度
  MoE All-to-All → 能但需精心编排

物理下界：单步时间 ≥ max(总计算，总通信)
```

数值算例：感受通信量级

```
DDP 训练 7B 模型，fp16 梯度，64 卡：
  梯度总量 M = 7e9 × 2 Byte = 14 GB
  Ring AllReduce 每卡通信量 ≈ 2M = 28 GB
  跨机 IB 有效带宽 ≈ 40 GB/s：
    T_comm ≈ 28 / 40 ≈ 0.7 s
  若单步计算 T_comp ≈ 1.0 s：
    无 overlap : 1.0 + 0.7 = 1.7 s
    完美 overlap: max(1.0,0.7) = 1.0 s → 加速约 1.7×

结论：通信占计算的比例(70%)决定 overlap 收益上限。
```

五策略通信对照 + 布局原则

DDP : AllReduce | 每step一次(分bucket) | 能overlap(bucketing)

FSDP : AllGather+ReduceScatter| 每层多次 | 能overlap(prefetch)

TP : AllReduce/AllGather | 每层多次(极重) | 难overlap(在关键路径)

PP : P2P Send/Recv | 每micro-batch一次 | 能overlap(1F1B)

MoE(EP) : All-to-All | 每MoE层两次 | 能但复杂

布局原则：越重越频繁的通信放越快的链路。典型 3D 并行：

TP → 机内 8 卡走 NVLink（每层通信最重）

PP → 跨机（P2P 轻，IB 扛得住）

DP/FSDP → 最外层跨机（AllReduce/AllGather 频率较低）

常见误区

1) 误以为 AllReduce 通信量随卡数线性增长：错，Ring 每卡趋近 2M，几乎与 N 无关。
2) 误以为 FSDP 比 DDP 通信更省：反了，FSDP 约 1.5 倍，省的是显存不是通信。
3) 把 TP 放跨机：严重错误，TP 每层通信、依赖紧、难 overlap，必须锁 NVLink 域内(≤8 卡)。
4) 以为开了 overlap 就一定生效：若通信 kernel 与计算抢 SM，或忘用独立 stream/pinned memory，overlap 名存实亡，必须 profiler 验证。
5) All-to-All 当廉价操作：MoE 跨机 All-to-All 常是瓶颈，专家负载不均会拥塞。

VPP · Pipeline Parallel · Interleaved 1F1B · Pipeline Bubble

VPP：虚拟流水线并行与 Interleaved 1F1B

> **原始问题：**详细讲解 vpp，并且我现在的记事本网址为：https://psgmessi.github.io/ai-infra-learning/ 以后的更新都直接在这个网址上更新，把这个方式记录进skill里，以后每次提问新的知识点都将更新到这里，包括现在这个。随后补充要求：按照 AI Infra Tutor Skill 给出完整讲解并同步更新知识图谱和线上笔记本。

提问：2026/07/27 03:08　更新：2026/07/29 19:38　重要度：5/5

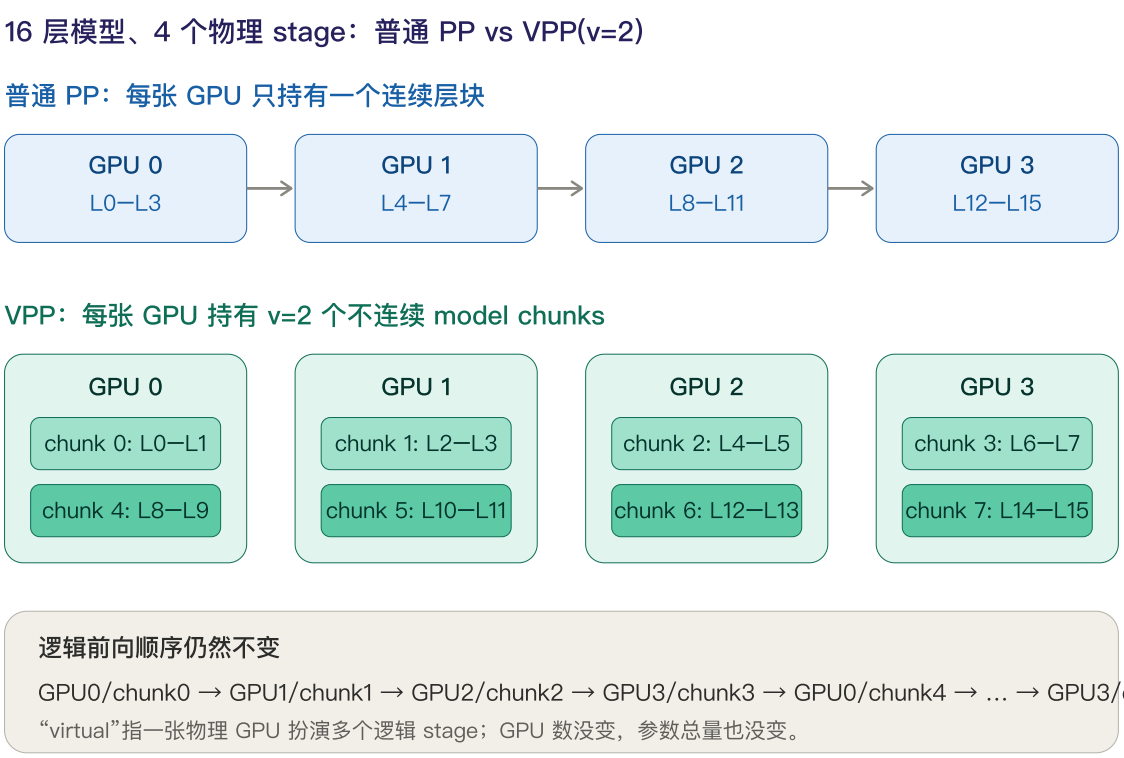
一句话结论

VPP（Virtual Pipeline Parallelism，虚拟流水线并行）不是增加 GPU 数，而是把每个物理 pipeline stage 上的连续层再切成 v 个更小的 model chunks，让同一张 GPU 在调度上扮演多个逻辑 stage，并用 interleaved 1F1B 把这些 chunk 与不同 microbatch 的前后向交错执行。理想情况下，pipeline bubble 相对普通 1F1B 缩小约 v 倍；代价是 P2P 通信次数/总量约增加 v 倍、调度更复杂，并且收益依赖 stage 均衡和通信能被隐藏。

系统位置：VPP 解决什么

普通 Pipeline Parallelism（PP）把模型按层切成 p 个物理 stage，每张 GPU 只负责一个连续层块。即使使用内存友好的 1F1B，流水线启动时仍要 warm-up、结束时仍要 cool-down，因此存在 bubble。VPP 不改变模型数学结果、不改变物理 GPU 数、不替代 TP/DP/FSDP；它是在 PP 内部进一步细化 stage 并优化调度，主要解决流水线粒度太粗导致的 bubble。

关键图：普通 PP 与 VPP 的层映射



模型如何切分

统一例子：L=16 个 Transformer layers，p=4 个物理 pipeline stages，v=2 个 model chunks/GPU。普通 PP 每卡 4 层：GPU0=L0–L3，GPU1=L4–L7，GPU2=L8–L11，GPU3=L12–L15。VPP 把每卡 4 层再分成两个 2 层 chunk：GPU0 持 L0–L1 和 L8–L9；GPU1 持 L2–L3 和 L10–L11；GPU2 持 L4–L5 和 L12–L13；GPU3 持 L6–L7 和 L14–L15。逻辑网络顺序没有改变，只是物理 GPU 0 在逻辑上同时扮演 virtual stage 0 和 4。

深度补充：普通 PP、1F1B 与 VPP Bubble 完整推导

一句话结论

你截图中的公式是正确的，但要真正理解，必须把三个层次分开：

1. **普通 PP 的结构**：模型按层切成 `p` 个连续物理 Stage，一个 Microbatch 的 Forward 依次穿过 Stage 0→1→…→p−1，Backward 再反向返回。

2. **1F1B 的作用**：它把不同 Microbatch 的 Forward/Backward 交错排列，主要减少需要长期保存的 Activation；在经典同步 Flush 语义下，它与 GPipe 的理论 Bubble 大小相同。
3. **VPP 的变化**：每张物理 GPU 不再只持有一个大 Stage，而是持有 `v` 个更小的 Model Chunks。每个 Chunk 的时间约缩短到原来的 `1/v`，所以填充/排空气泡的持续时间约缩小 `v` 倍；但每张 GPU 的总有效计算量并没有减少。

最容易记错的地方是：

```
VPP 把 Bubble 缩小约 v 倍
≠ 把整个 Step 时间缩小 v 倍
```

在统一例子 `p=4、m=8、v=2、t_f=t_b=1` 中：

```
普通 1F1B: Useful=16, Bubble=6, Total=22
VPP v=2:    Useful=16, Bubble=3, Total=19

理想加速 = 22 ÷ 19 ≈ 1.158×
```

Bubble 虽然减半，但总时间只理想下降约 13.6%，吞吐理想提升约 15.8%。

一、统一符号、假设和计数口径

整节使用同一组参数：

```
L = 16            Transformer 层数
p = 4             物理 Pipeline Stage / GPU 数
m = 8             一个训练 Step 中的 Microbatch 数
v = 2             每个物理 GPU 的 Model Chunk 数

t_f = 1 time slot  普通物理 Stage 处理一个 Microbatch Forward 的时间
t_b = 1 time slot  普通物理 Stage 处理一个 Microbatch Backward 的时间
```

核心理想化假设：

- 四个物理 Stage 计算完全均衡；
- 暂时忽略 P2P 通信时间；
- Forward/Backward 各使用一个单位时隙；
- 所有 Microbatch 使用同一版本权重；
- 一个 Step 的所有 Microbatch 完成后才执行 Optimizer Step；
- 讨论的是同步 Flush 型流水线，而不是允许权重陈旧的异步 PipeDream。

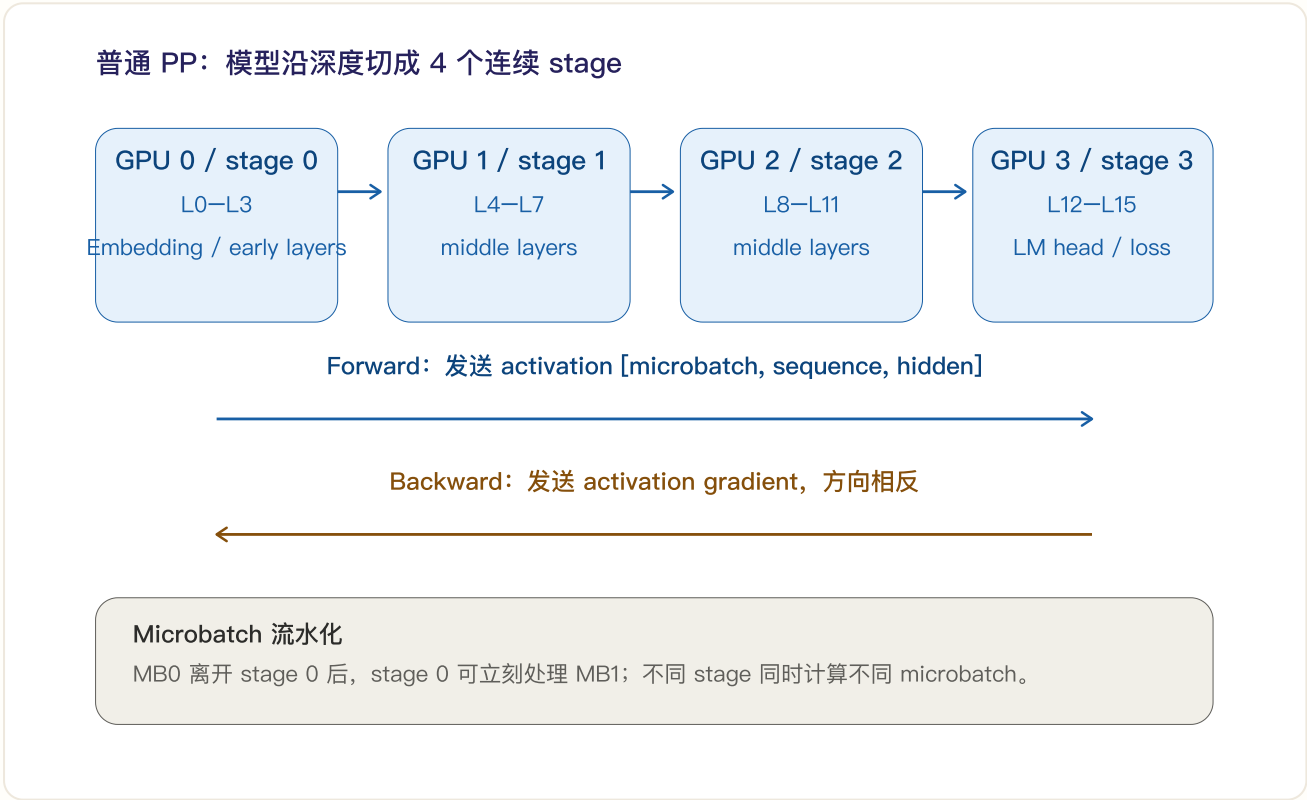
这些公式首先描述**调度上限**。真实时间还要加入：

```
暴露的 P2P 通信
+ Stage 不均衡
+ Kernel Launch / 调度开销
+ Stream 同步
+ 通信与计算资源争抢
```

二、普通 Pipeline Parallelism 的结构

16 层 Transformer 使用 `p=4` 做普通 PP：

```
GPU 0 / Stage 0: L0–L3
GPU 1 / Stage 1: L4–L7
GPU 2 / Stage 2: L8–L11
GPU 3 / Stage 3: L12–L15
```



一个 Microbatch 的 Forward：

```
Input
  ↓
Stage 0: L0–L3
  ↓ 发送 Activation
Stage 1: L4–L7
  ↓ 发送 Activation
Stage 2: L8–L11
  ↓ 发送 Activation
Stage 3: L12–L15 + Loss
```

Backward 方向相反：

```
Stage 3 计算输入梯度
  ↓ 发送 Activation Gradient
Stage 2
  ↓
Stage 1
  ↓
Stage 0
```

Stage 间通常不发送模型参数，而是发送边界 Activation 或对应的 Activation Gradient。典型张量 Shape：

```
[micro_batch_size, sequence_length, hidden_size]
```

普通 PP 的关键是把大 Batch 切成 `m` 个 Microbatches。MB0 离开 Stage 0 后，Stage 0 不必等待它跑完整个模型，可以立刻处理 MB1；于是不同 GPU 同时处理不同 Microbatch：

```
某一时刻：
Stage 0 在算 MB3
Stage 1 在算 MB2
Stage 2 在算 MB1
Stage 3 在算 MB0
```

这才形成流水线并行。

三、为什么会有 Pipeline Bubble

流水线开始时，后面的 Stage 没有输入：

```
time 0：只有 Stage 0 能算 MB0
Time 1：Stage 0 算 MB1，Stage 1 才拿到 MB0
Time 2：Stage 2 才第一次拿到 MB0
Time 3：Stage 3 才第一次拿到 MB0
```

因此，对 `p=4`：

```
Stage 3 在第一次开始 Forward 前等待了 p−1 = 3 个 Forward 时隙
```

这叫 **Pipeline Fill / Warm-up Bubble**。

结束时情况反过来。最后一个 Stage 可以先完成 Backward，但梯度还要逐级返回：

```
Stage 3 完成最后一个 Backward
→ Stage 2 才能继续对应 Backward
→ Stage 1
→ Stage 0 最后完成
```

这叫 **Pipeline Drain / Cool-down Bubble**。

所以普通同步 PP 的 Bubble 来自依赖链两端：

```
开始时：需要 p−1 个额外 Forward Stage 间隔
结束时：需要 p−1 个额外 Backward Stage 间隔
```

四、GPipe：All-Forward-All-Backward

最直观的调度是 GPipe 风格：

```
先完成所有 Microbatch 的 Forward
F0, F1, F2, ..., F7

再完成所有 Microbatch 的 Backward
B7, B6, B5, ..., B0
```

在平衡假设下：

```
Forward Makespan = (m + p − 1) × t_f
Backward Makespan = (m + p − 1) × t_b
```

总时间：

```
T_total_gpipe
= (m+p−1)t_f + (m+p−1)t_b
= (m+p−1)(t_f+t_b)
```

每个 Stage 真正必须做的有效计算只有：

```
m 次 Forward + m 次 Backward
```

因此：

```
T_useful = m(t_f+t_b)
```

两者相减就是 Bubble：

```
T_bubble
= T_total − T_useful
= (m+p−1)(t_f+t_b) − m(t_f+t_b)
= (p−1)(t_f+t_b)
```

GPipe 的主要问题不是 Bubble 比 1F1B 更大，而是 Activation 生命周期很长：在开始 Backward 前，已经做完所有 `m` 个 Forward，所以早期 Stage 需要保留接近 `m` 份尚未反向的 Activation。

五、1F1B 到底如何运行

1F1B = One Forward, One Backward。

它不是从第一个时隙开始所有 Stage 都严格 `F,B,F,B`。完整调度分三段：

1. **Warm-up**：先做若干 Forward，把流水线填起来；
2. **Steady State**：交替执行一个 Forward 和一个更早 Microbatch 的 Backward；
3. **Cool-down**：没有新 Forward 后，完成剩余 Backward。

对物理 Stage `s`（从 0 开始），经典非交错 1F1B 的 Warm-up Forward 数量近似为：

```
w_s = p − s − 1
```

在 `p=4` 时：

Stage	Warm-up Forward 数	为什么
Stage 0	3	要先把足够多的 Microbatch 推向后面，才能等到第一个 Backward 返回
Stage 1	2	比 Stage 0 少一跳
Stage 2	1	离最后 Stage 只差一跳
Stage 3	0	它算出 Loss 后就能立刻开始该 Microbatch 的 Backward

`p=4、m=8` 时，各 Stage 的逻辑任务顺序是：

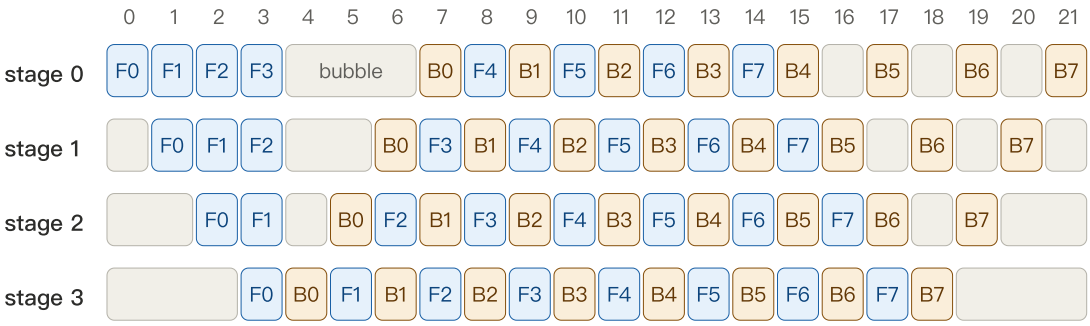
```
Stage 0:
F0 F1 F2 | F3 B0 F4 B1 F5 B2 F6 B3 F7 B4 | B5 B6 B7

Stage 1:
F0 F1    | F2 B0 F3 B1 F4 B2 F5 B3 F6 B4 F7 B5 | B6 B7

Stage 2:
F0       | F1 B0 F2 B1 F3 B2 F4 B3 F5 B4 F6 B5 F7 B6 | B7

Stage 3:
          F0 B0 F1 B1 F2 B2 F3 B3 F4 B4 F5 B5 F6 B6 F7 B7
```

竖线只是标识 Warm-up、Steady State、Cool-down 的逻辑分段；真实时间轴中还要满足前后 Stage 的依赖，因此会出现等待空槽。



蓝色 F：前向　　橙色 B：反向　　灰色：bubble / idle

每个 stage：16 个有效槽 + 6 个空闲槽；总长度 22。利用率 = 16/22 = 72.73%。

为什么 Stage 0 做完 F3 后不能立刻 B0?

Stage 0 的 B0 依赖完整的反向链：

```
MB0 必须先完成 Stage 3 的 F0 和 Loss
→ Stage 3 完成 B0
→ Stage 2 完成 B0
→ Stage 1 完成 B0
→ Stage 0 才能执行 B0
```

因此，前向波刚到达最后 Stage 后，反向波还要逐级返回。图中 Stage 0 在 F3 后出现 3 个灰色空槽，正是等待 B0 梯度返回。

1F1B 的核心收益是什么?

某个 Microbatch 完成当前 Stage 的 Backward 后，对应 Forward Activation 就可以释放。

因此 Outstanding Forward Activations 从 GPipe 的约 `m` 份下降到不超过约 `p` 份：

```
GPipe Activation Memory ≈ O(m)
1F1B Activation Memory ≈ O(p)
```

当 `m ≫ p` 时，显存差异非常大。

六、为什么普通 1F1B 的 Bubble 仍是 `(p−1)(t_f+t_b)`

1F1B 把 Backward 提前了，但没有消除两条物理依赖：

```
Forward 仍要从 Stage 0 逐级传播到 Stage p−1
```

```
Backward 仍要从 Stage p-1 逐级返回 Stage 0
```

所以：

```
Forward 端额外填充时间 = (p-1)t_f
Backward 端额外排空时间 = (p-1)t_b
```

相加：

```
T_bubble = (p-1)t_f + (p-1)t_b
         = (p-1)(t_f+t_b)
```

这也是为什么普通 GPipe 与普通 1F1B 在理想平衡模型下具有相同 Bubble：

- GPipe：先全 F 再全 B；
- 1F1B：把 F/B 在中间阶段交错；
- 但首端 Fill 和尾端 Drain 的依赖长度没有变化。

1F1B 主要优化的是 Activation 生命周期，而不是普通非交错 Pipeline 的理论 Bubble。

七、为什么 `T_useful = m(t_f+t_b)`

我们从**一张物理 GPU / 一个物理 Stage**的视角计数。

一个 Step 有 `m` 个 Microbatch。每个 Stage 必须对每个 Microbatch 做：

```
1 次 Forward，耗时 t_f
1 次 Backward，耗时 t_b
```

因此，一个 Stage 的有效工作：

```
T_useful
= m × t_f + m × t_b
= m(t_f+t_b)
```

这里没有乘 `p`，因为我们计算的是**单个 Stage 的时间线长度**。所有 Stage 并行运行，它们在平衡假设下都有相同的有效工作量。

如果改用整个集群的 GPU-Time 口径，则：

```
Cluster Useful GPU-Time = p × m(t_f+t_b)
```

实际总 GPU-Time：

```
Cluster Total GPU-Time = p × T_total
```

分子分母都有 `p`，因此 Bubble 比例不变。

八、普通 PP 的总时间和两种 Bubble 比例

普通 GPipe / 普通 1F1B：

```
T_useful = m(t_f+t_b)
T_bubble = (p-1)(t_f+t_b)

T_total
= T_useful + T_bubble
= (m+p-1)(t_f+t_b)
```

口径 1：Bubble 相对理想有效计算

```
Bubble / Useful
= T_bubble ÷ T_useful
= (p-1)(t_f+t_b) ÷ [m(t_f+t_b)]
= (p-1) ÷ m
```

这个比值可以大于 100%，因为它表示“额外浪费相当于理想计算的多少”，不是总时间占比。

口径 2：Bubble 占实际总时间

```
Bubble / Actual Total
= T_bubble ÷ (T_useful+T_bubble)
= (p-1) ÷ (m+p-1)
```

它一定在 0% 到 100% 之间。

Pipeline 利用率

```
Utilization
= T_useful ÷ T_total
= m ÷ (m+p-1)
```

并且：

```
Utilization + Bubble/Actual Total = 1
```

九、普通 1F1B 数值算例：p=4、m=8

代入：

```
p = 4
m = 8
t_f = 1 slot
t_b = 1 slot
```

有效计算：

```
T_useful
= 8 × (1+1)
= 16 slots
```

Bubble：

```
T_bubble
= (4−1) × (1+1)
= 3 × 2
= 6 slots
```

实际总时间：

```
T_total = 16 + 6 = 22 slots
```

两种比例：

```
Bubble / Useful
= 6 ÷ 16
= 37.5%
```

```
Bubble / Actual Total
= 6 ÷ 22
≈ 27.27%
```

利用率：

```
Utilization
= 16 ÷ 22
≈ 72.73%
```

这正对应时隙图：每个 Stage 在共享的 22-slot 时间轴中，都做了 16 个有效任务，并累计空闲 6 个 Slot。不同 Stage 的空闲位置不同，但总空闲时间相同。

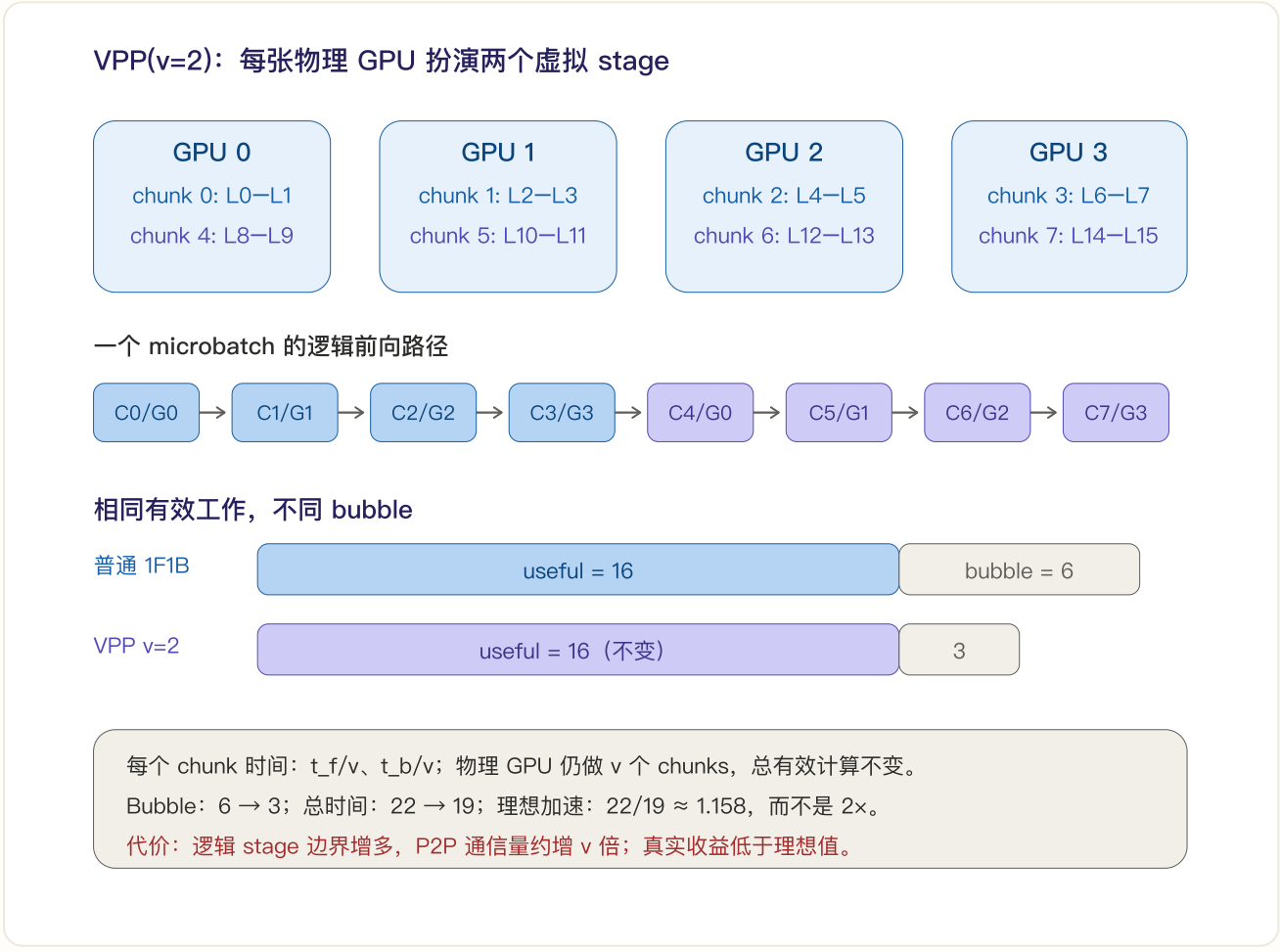
十、VPP 在普通 PP 基础上改变了什么

普通 PP：每张 GPU 持有一个连续大 Stage。

```
GPU 0: L0–L3
GPU 1: L4–L7
GPU 2: L8–L11
GPU 3: L12–L15
```

VPP `v=2`：把每张 GPU 的 4 层拆成两个 2 层 Chunk，并按全局层顺序交错映射：

```
GPU 0: chunk 0 = L0–L1,   chunk 4 = L8–L9
GPU 1: chunk 1 = L2–L3,   chunk 5 = L10–L11
GPU 2: chunk 2 = L4–L5,   chunk 6 = L12–L13
GPU 3: chunk 3 = L6–L7,   chunk 7 = L14–L15
```



一个 Microbatch 的逻辑 Forward 路径变成：

```
C0/GPU0 → C1/GPU1 → C2/GPU2 → C3/GPU3
        → C4/GPU0 → C5/GPU1 → C6/GPU2 → C7/GPU3
```

同一张 GPU 在不同时间处理不同 Chunk，所以它在调度上扮演多个虚拟 Stage。

十一、为什么 VPP 的有效计算不变

普通 PP 中，一个物理 Stage 对一个 Microbatch 的总计算时间：

```
t_f + t_b
```

拆成 `v` 个大小相近的 Chunk 后，每个 Chunk 的时间约为：

```
t_f_chunk ≈ t_f ÷ v
t_b_chunk ≈ t_b ÷ v
```

但是一张物理 GPU 需要执行自己持有的全部 `v` 个 Chunks。

因此，对一个 Microbatch：

```
每张 GPU 的有效计算
= v × (t_f/v + t_b/v)
= t_f + t_b
```

对 `m` 个 Microbatch：

```
T_useful_vpp
= m × v × (t_f/v + t_b/v)
= m(t_f+t_b)
```

所以 VPP 没有减少模型 FLOP，也没有减少每张 GPU 负责的层数。它改变的是任务粒度与时间排列。

十二、为什么 VPP Bubble 缩小约 v 倍

普通 PP 的 Fill/Drain 空档以“完整物理 Stage 计算时间”为粒度：

```
Forward 空档粒度 = t_f
Backward 空档粒度 = t_b
```

VPP 把大 Stage 切成 `v` 份后，Interleaved 1F1B 可以用其他 Chunk/Microbatch 的 Ready Task 填充原来的大空档。剩余不可消除的边缘空档以 Chunk 时间为粒度：

```
Forward Chunk 时间 ≈ t_f/v
Backward Chunk 时间 ≈ t_b/v
```

仍有 `p−1` 个物理 Rank 间隔需要填充和排空，因此理想 Bubble：

```
T_bubble_vpp
≈ (p-1)(t_f/v + t_b/v)
= (p-1)(t_f+t_b)/v
```

为什么不是 `(pv-1)(t_f+t_b)/v`？

`pv` 是逻辑虚拟 Stage 数，它影响**单个 Microbatch 穿过完整模型的延迟和通信边界数**。但这里计算的是多个 Microbatch 交错执行时，**每张物理 GPU 的批次吞吐时间线中的未填充空档**。

额外的虚拟 Stage 路径并不是全部串行暴露：它们会与其他 Microbatch、其他 Chunk 的计算交错。经典 Megatron Interleaved 调度的效果，就是把不可避免的 Pipeline Edge 空档从完整 Stage 时间压缩到 Chunk 时间。

因此要区分：

```
单 Microbatch 端到端延迟
≠ 一个 Batch 的 Pipeline Bubble
≠ 每张物理 GPU 的有效计算量
```

十三、VPP 两种 Bubble 比例

VPP 有效时间仍然是：

```
T_useful_vpp = m(t_f+t_b)
```

Bubble：

```
T_bubble_vpp = (p-1)(t_f+t_b)/v
```

实际总时间：

```
T_total_vpp
= m(t_f+t_b) + (p-1)(t_f+t_b)/v
= [m + (p-1)/v](t_f+t_b)
```

Bubble 相对 Useful

```
Bubble / Useful
= [(p-1)(t_f+t_b)/v] ÷ [m(t_f+t_b)]
= (p-1) ÷ (mv)
```

Bubble 占实际总时间

```
Bubble / Actual Total
= [(p−1)/v] ÷ [m+(p−1)/v]
= (p−1) ÷ (mv+p−1)
```

利用率

```
Utilization_vpp
= mv ÷ (mv+p−1)
```

十四、VPP 数值算例：p=4、m=8、v=2

Chunk 时间：

```
t_f_chunk = 1 ÷ 2 = 0.5 slot
t_b_chunk = 1 ÷ 2 = 0.5 slot
```

有效计算不变：

```
T_useful_vpp
= 8 × 2 × (0.5+0.5)
= 16 slots
```

Bubble：

```
T_bubble_vpp
= (4−1) × (1+1) ÷ 2
= 3 slots
```

总时间：

```
T_total_vpp = 16 + 3 = 19 slots
```

比例：

```
Bubble / Useful = 3 ÷ 16 = 18.75%
```

```
Bubble / Actual Total = 3 ÷ 19 ≈ 15.79%
```

```
Utilization = 16 ÷ 19 ≈ 84.21%
```

对比：

指标	普通 1F1B	VPP v=2	变化
Useful	16	16	不变
Bubble	6	3	减半
Total	22	19	减少 3
Bubble/Actual	27.27%	15.79%	下降 11.48 个百分点
利用率	72.73%	84.21%	上升 11.48 个百分点
理想吞吐提升	1×	22/19≈1.158×	约 15.8%

这说明：Bubble 减半不代表 Step Time 减半。因为 Useful 16 完全没有变，VPP 只减少了原来 6 个空闲 Slot 中的 3 个。

即使令 `v→∞`、Bubble 理想趋近 0，本例的绝对加速上限也只有：

```
Speedup_max
= 22 ÷ 16
= 1.375×
```

这还是忽略通信与调度开销的上限。

十五、VPP 为什么不是免费午餐

VPP 增加了逻辑 Stage 边界。普通 PP 的 Forward 路径：

```
GPU0 → GPU1 → GPU2 → GPU3
```

VPP `v=2`：

```
GPU0 → GPU1 → GPU2 → GPU3
     → GPU0 → GPU1 → GPU2 → GPU3
```

所以 P2P Send/Recv 次数和总通信量在经典模型中约增加 `v` 倍。

真实 Step Time 更接近：

```
T_real_vpp
≈ T_useful
 + T_bubble_vpp
 + T_extra_p2p_exposed
 + T_stage_imbalance
 + T_schedule_overhead
```

是否启用 VPP，应判断：

```
节省的 Bubble 时间
>
新增的暴露通信时间 + 调度开销
```

本例理论节省 3 Slots。如果新增 P2P 和调度暴露超过 3 Slots，VPP 反而更慢。

经典 Megatron Interleaved 1F1B 还要求 Microbatch 数满足调度约束；NVIDIA 文章给出的典型要求是：

```
m mod p = 0
```

不同 Megatron/NeMo 版本还可能有额外整除和模型切分限制，应以实际版本断言为准。

十六、1F1B 调度伪代码

下面是帮助理解的简化伪代码，不代表某个 Megatron 版本的逐行源码：

```
def schedule_1f1b(stage_id, p, microbatches):
    m = len(microbatches)
    warmup = min(p - stage_id - 1, m)

    # 1. Warm-up：只做 Forward
    for i in range(warmup):
        x = recv_forward(i)
        y = forward(i, x)
        send_forward(i, y)
        stash_activation(i)

    # 2. Steady state：一个较新的 Forward + 一个较早的 Backward
    for i in range(warmup, m):
        x = recv_forward(i)
        y = forward(i, x)
        send_forward(i, y)
        stash_activation(i)

        old = i - warmup
        dy = recv_backward(old)
        dx = backward(old, dy)
        release_activation(old)
        send_backward(old, dx)

    # 3. Cool-down：完成剩余 Backward
    for old in range(m - warmup, m):
        dy = recv_backward(old)
        dx = backward(old, dy)
        release_activation(old)
        send_backward(old, dx)
```

关键点：

- Forward 和 Backward 通常属于不同 Microbatch；
- Last Stage 的 Warm-up 为 0，所以可以 `F0→B0→F1→B1`；

- Stage 0 的 Warm-up 最长，因为第一个梯度需要走最远的返回路径；
- Optimizer Step 在整个 Microbatch Group Flush 后执行，避免不同 Microbatch 使用不同版本权重。

深度补充：1F1B 伪代码逐行解释与真实 Forward 调用

一句话结论

截图中的代码是为了表达调度思想而写的**教学伪代码**，不是 Megatron Core 可以直接运行的源码。它把生产实现中的四类对象压缩成了几个虚构函数：

```
通信：recv_forward / send_forward / recv_backward / send_backward
计算：forward / backward
状态：stash_activation / release_activation
调度：warm-up / steady state / cool-down 循环
```

在当前 Megatron Core 中，真实对应关系更接近：

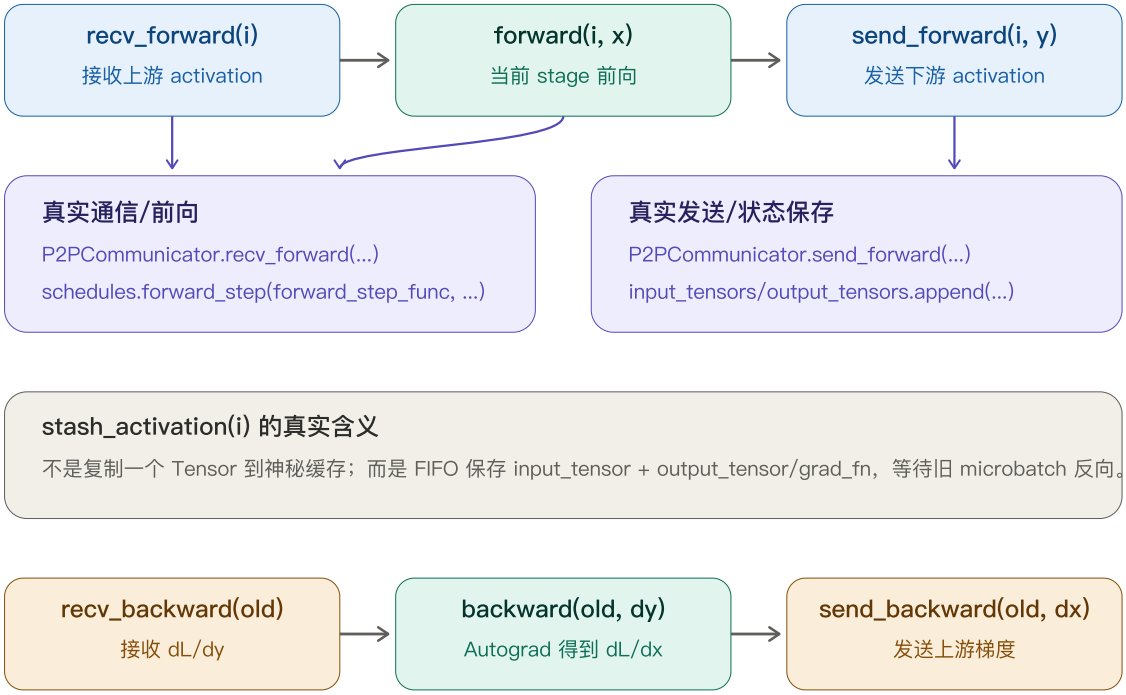
教学伪代码	Megatron Core 真实对应
`recv_forward(i)`	`P2PCommunicator.recv_forward(tensor_shapes, is_first_stage)`
`forward(i, x)`	`schedules.forward_step(forward_step_func, data_iterator, model, ..., input_tensor=x)`
`send_forward(i, y)`	`P2PCommunicator.send_forward(output_tensors, is_last_stage)`
`stash_activation(i)`	`input_tensors.append(input_tensor)` + `output_tensors.append(output_tensor)`；Autograd Graph 由 PyTorch 保存在 Tensor/`grad_fn` 中
`recv_backward(old)`	`P2PCommunicator.recv_backward(...)`；Steady State 中通常融合进 `send_forward_recv_backward(...)`
`backward(old, dy)`	`schedules.backward_step(input_tensor, output_tensor, output_tensor_grad, config)`
`release_activation(old)`	从 FIFO `pop(0)` 后执行 Backward，引用释放时计算图可回收；Megatron 还可能更早调用 `deallocate_output_tensor()` 伪释放已发送的输出数据
`send_backward(old, dx)`	`P2PCommunicator.send_backward(input_tensor_grads, is_first_stage)`；非最后一轮通常融合为 `send_backward_recv_forward(...)`

最重要的区别是：

> `stash_activation(i)` 并不是把 Activation 复制到某个隐藏仓库；真实实现只是在 FIFO 中保存当前 Microbatch 的 `input_tensor` 和 `output_tensor` 引用，而 PyTorch

> Autograd Graph 已经挂在 `output_tensor.grad_fn` 上。

教学伪代码是概念压缩；生产实现把状态、通信和计算拆成多个对象



Steady state 会组合通信：send_forward_recv_backward + send_backward_recv_forward，以减少串行等待。

一、先给伪代码加上行号

```
01  def schedule_1f1b(stage_id, p, microbatches):
02      m = len(microbatches)
03      warmup = min(p - stage_id - 1, m)

04      # 1. Warm-up：只做 Forward
05      for i in range(warmup):
06          x = recv_forward(i)
07          y = forward(i, x)
08          send_forward(i, y)
09          stash_activation(i)

10      # 2. Steady state：一个较新的 Forward + 一个较早的 Backward
11      for i in range(warmup, m):
12          x = recv_forward(i)
13          y = forward(i, x)
14          send_forward(i, y)
15          stash_activation(i)

16          old = i - warmup
17          dy = recv_backward(old)
18          dx = backward(old, dy)
19          release_activation(old)
20          send_backward(old, dx)
```

```
    # 3. Cool-down：完成剩余 Backward
    for old in range(m - warmup, m):
        dy = recv_backward(old)
        dx = backward(old, dy)
        release_activation(old)
        send_backward(old, dx)
```

下面逐行解释。

二、第 1 – 3 行：计算当前 Stage 要 Warm-up 几次

第 1 行

```
def schedule_1f1b(stage_id, p, microbatches):
```

教学含义：为某一个 Pipeline Stage 执行一个训练 Step 的全部 Microbatch 调度。

参数：

参数	含义
`stage_id`	当前物理 Pipeline Stage 的编号，从 0 开始
`p`	物理 Pipeline Stage 总数
`microbatches`	当前训练 Step 被拆分后的 Microbatch 集合

真实 Megatron Core 不会只传这三个参数。非交错 1F1B 入口是：

```
forward_backward_pipelining_without_interleaving(
    forward_step_func=...,
    data_iterator=...,
    model=...,
    num_microbatches=...,
    seq_length=...,
    micro_batch_size=...,
    forward_only=False,
    p2p_communicator=...,
    ...
)
```

VPP/Interleaved 1F1B 使用：

```
forward_backward_pipelining_with_interleaving(...)
```

真实调度器还要知道：

- 当前 Stage 对应的模型或 Model Chunks；
- 数据迭代器；

- P2P Process Group；
- 通信 Tensor Shape；
- 是否 First/Last Stage；
- 是否 Forward Only；
- Activation Checkpoint 策略；
- 梯度同步策略；
- Virtual Pipeline Rank。

第 2 行

```
m = len(microbatches)
```

表示本 Step 一共有多少个 Microbatch。

在 Megatron 中通常不把一个 Python List `microbatches` 直接传给调度器，而是传：

```
num_microbatches=m
data_iterator=data_iterator
```

每次 `forward_step_func` 被调用时，First/Last Stage 根据需要从 `data_iterator` 中读取当前 Microbatch。

第 3 行

```
warmup = min(p - stage_id - 1, m)
```

含义：当前 Stage 在第一次能够执行 Backward 之前，需要先做多少个 Forward。

若：

```
p = 4
```

则：

`stage_id`	`p-stage_id-1`	Warm-up 数量
0	3	3
1	2	2
2	1	1
3	0	0

为什么 Last Stage 为 0？因为它执行完一个 Microbatch Forward 并计算 Loss 后，就能立即从 Loss 开始该 Microbatch 的 Backward。

为什么要和 `m` 取最小值？如果 Microbatch 总数比所需 Warm-up 还少，不能执行超过 `m` 次 Forward。

真实源码更接近：

```
num_warmup_microbatches = (
    p2p_communicator.total_stages
    - p2p_communicator.current_stage
    - 1
)
num_warmup_microbatches = min(
    num_warmup_microbatches,
    num_microbatches,
)
```

生产代码使用 `P2PCommunicator` 维护的 Stage 信息，而不是把 `stage_id`、`p` 随意传进函数。

三、第 4 – 9 行：Warm-up 只做 Forward

第 5 行

```
for i in range(warmup):
```

`i` 是当前 Forward Microbatch 编号。

如果 Stage 0 的 `warmup=3`：

```
i = 0, 1, 2
```

这三个 Microbatch 的 Forward 结果会进入“尚未 Backward”的队列。

第 6 行

```
x = recv_forward(i)
```

含义：取得当前 Stage 的 Forward 输入。

不同 Stage 的行为不同：

Stage	`x` 从哪里来
First Stage	不从前一个 Pipeline Rank 接收；输入来自本地 Dataset 的 Tokens/Embedding
Middle Stage	从前一个 Pipeline Rank 接收 Hidden States
Last Stage	同样从前一个 Rank 接收 Hidden States，同时本地取得 Labels/Loss Mask

真实 API：

```
input_tensor = p2p_communicator.recv_forward(
    recv_tensor_shapes,
    p2p_communicator.is_pp_first_stage,
)
```

First Stage 时，Communicator 知道没有前驱，一般返回 `None` 或与模型接口一致的空输入；模型会从本地 `tokens` 开始。

`i` 通常不会直接发送给远端。通信双方通过相同的调度顺序，知道当前收发的是哪个 Microbatch。若 Variable Sequence Length 开启，还可能先通信 Tensor Shape。

第 7 行

```
y = forward(i, x)
```

教学上表示：当前 Stage 对 Microbatch `i` 执行本地层的 Forward。

真实调用分两层：

```
Megatron 调度器的 forward_step(...)
    ↓
用户/训练脚本提供的 forward_step_func(data_iterator, model)
    ↓
model(...)
```

真实调度层调用大致是：

```
output_tensor, num_tokens = forward_step(
    forward_step_func,
    data_iterator,
    model,
    num_microbatches,
    input_tensor,
    forward_data_store,
    config,
    cp_group_size,
    current_microbatch=i,
    is_last_stage=p2p_communicator.is_pp_last_stage,
)
```

`forward_step()` 在调用用户函数之前，会把接收到的 Pipeline Activation 注入模型，概念上相当于：

```
model.set_input_tensor(input_tensor)
output_tensor, loss_func = forward_step_func(data_iterator, model)
```

对于 Last Stage，它还会调用 `loss_func(output_tensor)` 得到 Loss 和日志数据。

第 8 行

```
send_forward(i, y)
```

含义：把当前 Stage 的输出 Activation 发给下一个 Stage。

真实 API：

```
p2p_communicator.send_forward(
    output_tensor,
    p2p_communicator.is_pp_last_stage,
)
```

- 非 Last Stage：发送 Hidden States；
- Last Stage：没有下一 Stage，不发送；它计算 Loss。

输出典型 Shape：

```
[sequence_length, micro_batch_size, hidden_size]
```

或框架内部采用的其他 Layout。若启用 Sequence/Context Parallel，通信 Shape 可能相应变化。

第 9 行

```
stash_activation(i)
```

这是最容易误解的一行。

它不是：

```
把 y 复制一份到额外的 Activation Cache
```

真实需要保留的是执行 Backward 所必需的计算图上下文。Megatron 非交错 1F1B 维护两个 FIFO：

```
input_tensors = []
output_tensors = []

input_tensors.append(input_tensor)
output_tensors.append(output_tensor)
```

`output_tensor` 本身带有 PyTorch Autograd 的：

```
grad_fn
```

它间接引用本 Stage Forward 所需的计算图和保存张量。

Megatron 在把输出发送给下一 Stage 后，还可能调用：

```
deallocate_output_tensor(
    output_tensor,
    config.deallocate_pipeline_outputs,
)
```

这个函数会把已经发送出去的 `output_tensor.data` 伪释放成很小的 Tensor，但保留 `grad_fn`，降低 Pipeline Output 占用。它不等于释放整个 Autograd Graph。

Warm-up 完成后，若 `warmup=3`：

```
input_tensors  = [input_0, input_1, input_2]
output_tensors = [output_0, output_1, output_2]
```

这些都是等待 Backward 的 Outstanding Microbatches。

四、第 10 – 20 行：Steady State 为什么同时处理两个 Microbatch

第 11 行

```
for i in range(warmup, m):
```

继续对尚未 Forward 的 Microbatch 执行循环。

例如：

```
warmup = 3
m = 8
```

则：

```
i = 3, 4, 5, 6, 7
```

每轮做两件不同的事：

```
较新的 Microbatch i: Forward
```

```
较早的 Microbatch old: Backward
```

第 12 – 15 行

```
x = recv_forward(i)
y = forward(i, x)
send_forward(i, y)
stash_activation(i)
```

与 Warm-up 相同，对新 Microbatch 做 Forward 并放入 Outstanding FIFO。

但生产实现通常不会把发送 Forward、接收 Backward 写成两个完全串行调用，而会组合成：

```
output_tensor_grad = (
    p2p_communicator.send_forward_recv_backward(
        output_tensor,
        send_tensor_shapes,
        p2p_communicator.is_pp_last_stage,
    )
)
```

它同时做：

```
向下一 Stage 发送当前新 Microbatch 的 Output
从下一 Stage 接收某个旧 Microbatch 的 dL/dOutput
```

第 16 行

```
old = i - warmup
```

教学代码通过算术计算当前应 Backward 的旧 Microbatch。

例如：

```
warmup = 3

i=3 → old=0
i=4 → old=1
i=5 → old=2
```

生产代码通常不依赖这个整数公式保存状态，而是使用 FIFO：

```
input_tensor = input_tensors.pop(0)
output_tensor = output_tensors.pop(0)
```

队首天然就是最早尚未完成 Backward 的 Microbatch。

FIFO 更可靠，因为真实实现还要处理：

- Virtual Pipeline Model Chunks；
- Activation Checkpoint；
- 多输入/多输出 Tensor；
- Variable Sequence Length；
- Encoder-Decoder；
- Multimodule Pipeline。

第 17 行

```
dy = recv_backward(old)
```

`dy` 是：

```
dy = ∂Loss / ∂y
```

即 Loss 对当前 Stage 输出 Activation 的梯度，由下一个 Stage 计算后发送回来。

真实 API：

```
output_tensor_grad = p2p_communicator.recv_backward(
    send_tensor_shapes,
    p2p_communicator.is_pp_last_stage,
)
```

Steady State 中常被融合进：

```
send_forward_recv_backward(...)
```

Last Stage 没有下一个 Stage，它的 Backward 从本地 Loss 开始，因此：

```
output_tensor_grad = None
```

第 18 行

```
dx = backward(old, dy)
```

当前 Stage 使用保存的旧 Microbatch 计算图和下游梯度 `dy` 执行反向：

```
dy = ∂L/∂y
当前 Stage: y = f(x; θ)

得到:
```

```
dx = ∂L/∂x
以及本 Stage 参数梯度 ∂L/∂θ
```

真实调用：

```
input_tensor_grad = backward_step(
    input_tensor,
    output_tensor,
    output_tensor_grad,
    config,
)
```

`backward_step()` 的职责：

1. 对 `output_tensor` 运行 PyTorch Autograd；
2. 累积当前 Stage 参数梯度；
3. 读取 `input_tensor.grad`；
4. 返回 `input_tensor_grad`，发送给前一 Stage。

如果启用了 `deallocate_pipeline_outputs`，Megatron 会使用 `custom_backward()` 直接调用 C++ Autograd Engine，因为 `output_tensor.data` 已经被伪释放为标量大小，普通 `torch.autograd.backward` 的 Shape 棠查可能不适用。

第 19 行

```
release_activation(old)
```

教学含义：旧 Microbatch Backward 已完成，它保存的 Activation/Autograd Graph 不再需要。

真实代码没有一个完全同名的单独调用。资源释放来自：

1. `input_tensors.pop(0)` 和 `output_tensors.pop(0)` 移除 FIFO 引用；
2. `backward_step()` 消耗该计算图；
3. 局部 Python 引用离开作用域；
4. PyTorch/CUDA Allocator 把不再引用的显存块回收到缓存池。

必须区分两个时刻：

时刻	释放什么
Forward Output 已发送后	`deallocate_output_tensor()` 可提前缩小 `output_tensor.data`，但保留 `grad_fn`
Backward 完成后	该 Microbatch 的完整 Autograd 上下文可以回收

第 20 行

```
send_backward(old, dx)
```

把：

```
dx = ∂L/∂x
```

发送给前一个 Pipeline Stage。

真实 API：

```
p2p_communicator.send_backward(
    input_tensor_grad,
    p2p_communicator.is_pp_first_stage,
)
```

First Stage 没有前驱，不需要发送。

Steady State 非最后一轮常组合成：

```
input_tensor = (
    p2p_communicator.send_backward_recv_forward(
        input_tensor_grad,
        recv_tensor_shapes,
        p2p_communicator.is_pp_first_stage,
    )
)
```

同时：

```
向前一个 Stage 发送旧 Microbatch 的 dx
从前一个 Stage 接收下一个新 Microbatch 的 Forward Input
```

这比教学伪代码中完全串行的 Send/Recv 更接近生产实现。

五、第 21 – 26 行：Cool-down 为什么只剩 Backward

第 22 行

```
for old in range(m - warmup, m):
```

Steady State 已经完成了：

```
m - warmup
```

个旧 Microbatch 的 Backward，仍有 `warmup` 个 Outstanding Microbatch 留在 FIFO 中。

例如：

```
m = 8
warmup = 3
```

Steady State 完成 B0–B4，剩余：

```
B5、B6、B7
```

第 23 行

```
dy = recv_backward(old)
```

从下一 Stage 接收该旧 Microbatch 的输出梯度。

真实 Cool-down 顺序实际上先从 FIFO 取出对应状态：

```
input_tensor = input_tensors.pop(0)
output_tensor = output_tensors.pop(0)
output_tensor_grad = p2p_communicator.recv_backward(...)
```

第 24 行

```
dx = backward(old, dy)
```

使用对应的 `input_tensor`、`output_tensor`、`dy` 执行当前 Stage Backward，得到 `dx` 和本 Stage 参数梯度。

第 25 行

```
release_activation(old)
```

Backward 完成，该 Microbatch 的 Autograd 上下文可以释放。

第 26 行

```
send_backward(old, dx)
```

将输入梯度发送给前一个 Stage。Cool-down 不再接收新的 Forward Input，因为所有 `m` 个 Forward 已经完成。

六、真实 Megatron 非交错 1F1B 的核心骨架

下面不是完整源码复制，而是保留真实函数名和 FIFO 行为后的等价骨架：

```
input_tensors = []
output_tensors = []

num_warmup = min(
    p2p_communicator.total_stages
    - p2p_communicator.current_stage
    - 1,
    num_microbatches,
)
num_remaining = num_microbatches - num_warmup

# 1. Warm-up
for microbatch_id in range(num_warmup):
    input_tensor = p2p_communicator.recv_forward(
        recv_tensor_shapes,
        p2p_communicator.is_pp_first_stage,
    )

    output_tensor, num_tokens = forward_step(
        forward_step_func,
        data_iterator,
        model,
        num_microbatches,
        input_tensor,
        forward_data_store,
        config,
        cp_group_size,
        current_microbatch=microbatch_id,
        is_last_stage=p2p_communicator.is_pp_last_stage,
    )

    p2p_communicator.send_forward(
        output_tensor,
        p2p_communicator.is_pp_last_stage,
    )

    input_tensors.append(input_tensor)
    output_tensors.append(output_tensor)

    deallocate_output_tensor(
        output_tensor,
        config.deallocate_pipeline_outputs,
    )

# Steady State 首个 Forward Input
if num_remaining > 0:
    input_tensor = p2p_communicator.recv_forward(
        recv_tensor_shapes,
        p2p_communicator.is_pp_first_stage,
    )

# 2. Steady State
for i in range(num_remaining):
    current_microbatch = i + num_warmup
    last_iteration = i == num_remaining - 1

    output_tensor, num_tokens = forward_step(
```

```
            forward_step_func,
            data_iterator,
            model,
            num_microbatches,
            input_tensor,
            forward_data_store,
            config,
            cp_group_size,
            current_microbatch=current_microbatch,
            is_last_stage=p2p_communicator.is_pp_last_stage,
        )

        output_tensor_grad = (
            p2p_communicator.send_forward_recv_backward(
                output_tensor,
                send_tensor_shapes,
                p2p_communicator.is_pp_last_stage,
            )
        )

        input_tensors.append(input_tensor)
        output_tensors.append(output_tensor)
        deallocate_output_tensor(
            output_tensor,
            config.deallocate_pipeline_outputs,
        )

        old_input = input_tensors.pop(0)
        old_output = output_tensors.pop(0)

        input_tensor_grad = backward_step(
            old_input,
            old_output,
            output_tensor_grad,
            config,
        )

        if last_iteration:
            p2p_communicator.send_backward(
                input_tensor_grad,
                p2p_communicator.is_pp_first_stage,
            )
        else:
            input_tensor = (
                p2p_communicator.send_backward_recv_forward(
                    input_tensor_grad,
                    recv_tensor_shapes,
                    p2p_communicator.is_pp_first_stage,
                )
            )

    # 3. Cool-down
    for _ in range(num_warmup):
        old_input = input_tensors.pop(0)
        old_output = output_tensors.pop(0)

        output_tensor_grad = p2p_communicator.recv_backward(
            send_tensor_shapes,
            p2p_communicator.is_pp_last_stage,
        )

        input_tensor_grad = backward_step(
            old_input,
            old_output,
            output_tensor_grad,
            config,
        )

        p2p_communicator.send_backward(
            input_tensor_grad,
            p2p_communicator.is_pp_first_stage,
        )
```

真实源码还包含梯度同步开关、Loss 收集、Token 计数、Activation Checkpoint、Multimodule Pipeline、动态 Shape 和计时器等逻辑。

七、最简单的真实 Forward：先不使用多进程

先用两个真实的 `nn.Module` 模拟两个 Pipeline Stage，完整展示 Activation 如何进入下一 Stage。

```
import torch
import torch.nn as nn


class Stage0(nn.Module):
    """模拟前半段模型。"""

    def __init__(self, hidden=8):
        super().__init__()
        self.layer = nn.Sequential(
            nn.Linear(hidden, hidden),
            nn.ReLU(),
        )

    def forward(self, x):
        return self.layer(x)


class Stage1(nn.Module):
    """模拟后半段模型。"""

    def __init__(self, hidden=8):
        super().__init__()
        self.layer = nn.Linear(hidden, 1)
        self.input_tensor = None

    def set_input_tensor(self, x):
        # 对应 Pipeline Scheduler 将上游 Activation 注入本 Stage。
        self.input_tensor = x

    def forward(self):
        # 中间/最后 Stage 不需要重新从 Tokens 开始，
        # 而是消费上一个 Stage 产生的 Hidden States。
        return self.layer(self.input_tensor)


torch.manual_seed(0)

stage0 = Stage0(hidden=8)
stage1 = Stage1(hidden=8)

# 一个最小 Microbatch：[micro_batch=2, hidden=8]
x = torch.randn(2, 8, requires_grad=True)
target = torch.randn(2, 1)

# Stage 0 Forward
h = stage0(x)
print("stage0 output:", h.shape)       # [2, 8]

# 模拟跨 Pipeline Rank 传输边界：
# 真正分布式通信后，接收端拿到的是新的叶子 Tensor。
h_recv = h.detach().requires_grad_(True)

# Scheduler 将接收到的 Activation 注入 Stage 1。
stage1.set_input_tensor(h_recv)
```

```
# Stage 1 Forward
prediction = stage1()
print("stage1 output:", prediction.shape)  # [2, 1]

loss = ((prediction - target) ** 2).mean()
print("loss:", float(loss))

# Last Stage 从 Loss 开始 Backward。
loss.backward()

# h_recv.grad 就是要发送回 Stage 0 的 dy = dLoss/dh。
dh = h_recv.grad
print("activation grad:", dh.shape)    # [2, 8]

# Stage 0 使用下游返回的 dh 继续反向。
h.backward(dh)
print("x grad:", x.grad.shape)         # [2, 8]
```

这段代码的关键不是模型本身，而是这两行：

```
h_recv = h.detach().requires_grad_(True)
stage1.set_input_tensor(h_recv)
```

为什么要 `detach()`？

真实跨进程 Send/Recv 不会把 Stage 0 的 Python Autograd Graph 自动传到 Stage 1。接收端的 Activation 是一个新的 Tensor，需要：

```
requires_grad_(True)
```

这样 Stage 1 Backward 后才能得到：

```
h_recv.grad = ∂Loss/∂h
```

然后把这个梯度发送回 Stage 0，Stage 0 执行：

```
h.backward(dh)
```

这正对应伪代码：

```
Forward: send y
Backward: recv dy → backward → send dx
```

八、最简单的两进程 Forward 通信示例

下面只演示 Forward 的真实跨进程通信，不实现完整 1F1B 调度。运行要求：两张 GPU，使用 `torchrun --nproc_per_node=2`。

```python
# minimal_pipeline_forward.py
import os
import torch
import torch.distributed as dist
import torch.nn as nn


def main():
    dist.init_process_group(backend="nccl")

    rank = dist.get_rank()
    torch.cuda.set_device(rank)
    device = torch.device(f"cuda:{rank}")

    micro_batch = 2
    hidden = 8

    if rank == 0:
        # Pipeline Stage 0
        stage0 = nn.Sequential(
            nn.Linear(hidden, hidden),
            nn.ReLU(),
        ).to(device)

        x = torch.randn(
            micro_batch,
            hidden,
            device=device,
        )

        # 当前 Stage 真实 Forward。
        activation = stage0(x)
        print("rank0 activation:", activation.shape)

        # Forward P2P：发送给下一个 Pipeline Rank。
        # send 只传 Tensor 数据，不传 Python Autograd Graph。
        dist.send(activation.detach(), dst=1)

    elif rank == 1:
        # Pipeline Stage 1
        stage1 = nn.Linear(hidden, 1).to(device)

        # 接收端必须提前分配正确 Shape/Dtype 的 Tensor。
        activation = torch.empty(
            micro_batch,
            hidden,
            device=device,
        )

        # Forward P2P：从前一个 Pipeline Rank 接收。
        dist.recv(activation, src=0)
        activation.requires_grad_(True)

        output = stage1(activation)
        print("rank1 output:", output.shape)

    dist.barrier()
    dist.destroy_process_group()


if __name__ == "__main__":
    main()
```

运行：

```
torchrun --standalone --nproc_per_node=2 minimal_pipeline_forward.py
```

这个例子对应：

```
Rank 0: forward → send_forward
Rank 1: recv_forward → forward
```

生产 Megatron 不直接在业务代码里写裸 `dist.send/dist.recv`，而是通过：

```
P2PCommunicator.recv_forward(...)
P2PCommunicator.send_forward(...)
```

原因包括：

- 管理 Pipeline Process Group；
- First/Last Stage 特殊处理；
- Tensor Shape 协议；
- Batched P2P；
- NCCL Request 等待；
- 与反向收发组合；
- 通信重叠配置；
- 多 Tensor 输入输出。

九、Megatron 风格的用户 `forward_step_func`

Megatron 把“调度”与“模型业务 Forward”分开。

调度器大致这样调用：

```
forward_backward_func = get_forward_backward_func()

losses = forward_backward_func(
    forward_step_func=forward_step_func,
    data_iterator=data_iterator,
    model=model,
    num_microbatches=num_microbatches,
    seq_length=seq_length,
    micro_batch_size=micro_batch_size,
    forward_only=False,
)
```

用户提供的 `forward_step_func` 通常返回：

```
模型输出 + Loss Reduction Callback
```

最小教学版：

```
from functools import partial
import torch


def loss_func(target, output_tensor):
    loss = torch.nn.functional.mse_loss(
        output_tensor,
        target,
    )

    # 第二项是记录/日志元数据。
    return loss, {"mse": loss.detach()}


def forward_step_func(data_iterator, model):
    batch = next(data_iterator)

    # First Stage 需要真正的模型输入。
    # Middle Stage 的真正输入已由 scheduler 通过
    # model.set_input_tensor(input_tensor) 注入。
    model_input = batch.get("input")
    target = batch.get("target")

    output_tensor = model(model_input)

    # 绑定当前 Microbatch 的 target。
    return output_tensor, partial(loss_func, target)
```

真实 GPT 示例更接近：

```
def forward_step(data_iterator, model):
    (
        tokens,
        labels,
        loss_mask,
        attention_mask,
        position_ids,
    ) = get_batch(data_iterator)

    output_tensor = model(
        tokens,
        position_ids,
        attention_mask,
        labels=labels,
        loss_mask=loss_mask,
    )

    return output_tensor, partial(
        loss_func,
        loss_mask,
        model=model,
    )
```

不过 Pipeline 情况下要理解两类输入：

输入类型	来源	示例
Dataset-Side Input	`data_iterator/get_batch`	Tokens、Labels、Loss Mask、Position IDs
Pipeline Activation Input	前一个 PP Rank，经 P2P 接收并 `set_input_tensor`	Hidden States

First/Middle/Last Stage 的差异：

Stage	Dataset Input	Pipeline Activation	输出
First	Tokens 等	无前驱 Activation	Hidden States，发送给下一 Stage
Middle	通常只需少量元数据或全 `None`	从上一 Stage 接收	新 Hidden States
Last	Labels、Loss Mask 等	从上一 Stage 接收	Logits/Per-Token Loss，计算最终 Loss

因此，中间 Stage 即使：

```
tokens = None
```

仍能运行，因为真正网络输入来自：

```
recv_forward
→ model.set_input_tensor(received_hidden_states)
→ model.forward(...)
```

十、伪代码中还隐藏了哪些生产细节

1. 通信 Shape

教学代码只写：

```
recv_forward(i)
```

真实代码必须知道接收 Tensor 的 Shape/Dtype。固定序列长度时可预先计算；变长序列时可能先交换 Shape。

2. First/Last Stage 分支

- First Stage：不 `recv_forward`；
- Last Stage：不 `send_forward`，也不 `recv_backward`；
- Last Stage Backward 从 Loss 开始；
- First Stage 不 `send_backward`。

3. 组合通信

Steady State 常用：

```
send_forward_recv_backward(...)
send_backward_recv_forward(...)
```

而不是四个完全分离、串行的调用。

4. 梯度同步

Megatron 通常在 Microbatch Accumulation 期间暂时关闭 DDP 梯度同步，仅在本 Stage 最后一个 Backward 恢复同步，避免每个 Microbatch 都触发一次全局梯度通信。

5. Activation Checkpoint

不同 Microbatch 可以选择 Partial/Full Activation Checkpoint，改变 FIFO 中计算图保存量与重算代价。

6. VPP Model Chunk

VPP 模式下 `model` 不再只是单个 `nn.Module`，而可能是多个 Model Chunks 的列表；调度器还要维护 Virtual Pipeline Rank，选择当前时刻执行哪个 Chunk。

7. Output Pseudo-Deallocation

输出发送后可执行：

```
deallocate_output_tensor(...)
```

减少输出数据存储，但保留 `grad_fn`。稍后反向可能使用 `custom_backward()` 绕过普通 Shape 检查。

8. Loss 只在 Last Stage 真正产生

非 Last Stage 的 `output_tensor` 是传给后续 Stage 的 Activation；Last Stage 才将模型输出交给 Loss Callback。

十一、完整数据生命周期

以某个 Microbatch `MB0` 为例：

```
1. Stage 0 从 Dataset 读取 Tokens
2. Stage 0 Forward 得到 h0
3. h0 发送到 Stage 1
4. Stage 0 把 input0/output0 引用放入 FIFO
5. Stage 1 接收 h0，设为 requires_grad 的 Pipeline Input
6. Stage 1 Forward 得到 h1
7. 继续传到最后 Stage
8. Last Stage 计算 Loss
9. Last Stage Backward 得到 dL/dh_prev
10. 梯度逐 Stage 向前发送
11. Stage 0 从 FIFO 取出 MB0 的 input/output
12. Stage 0 使用收到的 dy 执行 backward_step
```

```
13. 累积本 Stage 参数梯度
14. MB0 的 Autograd 上下文可回收
```

1F1B 的关键就是：当 MB0 在向后 Backward 时，较新的 MB3/MB4 等可能正在 Forward。

十二、最终记忆方式

把伪代码压缩成四个状态变换：

```
Forward Input
    ↓ recv_forward / 本地 Dataset
Current Stage Forward
    ↓
Forward Output
    ├─ send_forward 给下游
    └─ 保存 input/output/grad_fn 等待反向

下游返回 output gradient dy
    ↓
backward_step(old_input, old_output, dy)
    ├─ 累积本 Stage 参数梯度
    └─ 得到 input gradient dx
           ↓
       send_backward 给上游
```

对应 1F1B：

```
Warm-up: 只增加 Outstanding FIFO
Steady: 每轮入队一个新 Forward、出队一个旧 Backward
Cool-down: 只清空 Outstanding FIFO
```

这就是截图中全部代码真正表达的机制。

可选自测

1. 在 Stage 1 的 Steady State 中，当前正在做 `F4`，同时做 `B1`。`send_forward_recv_backward()` 发出的和收到的分别属于哪个 Microbatch?
2. 为什么跨 Rank 收到的 Activation 需要成为 `requires_grad=True` 的新 Tensor？为什么不能把 Stage 0 的 Autograd Graph 自动跨进程传过去?
3. `deallocate_output_tensor()` 为什么可以缩小 `output_tensor.data`，却仍然能在之后执行 Backward？它保留了什么?

事实来源与版本边界

本文对真实调用的描述依据截至 2026-07-29 的 NVIDIA Megatron Core 官方 API 文档，以及 NVIDIA/Megatron-LM `main` 分支中的 `megatron/core/pipeline_parallel/schedules.py`、`p2p_communication.py` 和 `pretrain_gpt.py`。具体参数、Multimodule 支持、组合通信与 VPP 调度细节会随版本演进；生产排查应以实际安装版本源码为准。

十七、最常见的五个误区

误区 1：1F1B 把普通 PP 的 Bubble 消掉了

没有。普通 1F1B 主要降低 Activation 存储，经典理想 Bubble 与 GPipe 相同。

误区 2：`(p-1)/m` 是 Bubble 占总时间百分比

不严格。它是 Bubble 相对 Ideal/Useful 时间的比值。真正占实际总时间的是：

```
(p-1)/(m+p-1)
```

误区 3：VPP 有 `pv` 个逻辑 Stage，所以 Useful 应乘 v

不对。每个 Chunk 只有原 Stage 约 `1/v` 的计算量，`v` 个 Chunk 加起来仍是原来的 Stage 工作量。

误区 4：Bubble 减半，所以训练速度翻倍

不对。只减少空闲部分，有效计算不变。本例理想速度只提升约 15.8%。

误区 5：VPP 的 v 越大越好

不对。Bubble 约按 `1/v` 下降，但通信量约按 `v` 增加，还会增加调度复杂度、Kernel Launch 和 Buffer 压力。

十八、最终心智模型

```
普通 PP:
每张 GPU 一个连续大 Stage
→ Microbatch 形成流水线
→ 开头需要 Fill，结尾需要 Drain
→ Bubble=(p-1)(t_f+t_b)

普通 1F1B:
Warm-up 后交替 Forward/Backward
→ 更早释放 Activation
→ 显存从约 O(m) 降至 O(p)
```

```
→ 但 Fill/Drain 依赖仍在，Bubble 不变

VPP:
每张 GPU 的大 Stage 切成 v 个小 Chunks
→ 同一 GPU 扮演多个虚拟 Stage
→ 不可避免的空档粒度从 Stage 时间缩短到 Chunk 时间
→ Bubble 理想缩小约 v 倍
→ Useful 不变，P2P 通信约增加 v 倍
```

判断 VPP 是否值得使用，只看一个净收益式：

```
VPP 净收益
≈ 节省的 Bubble
 - 新增的暴露 P2P 通信
 - 新增调度/Launch 开销
 - 负载不均放大项
```

可选自测

1. 当 `p=8、m=32、t_f=2 ms、t_b=3 ms` 时，普通 1F1B 的 `T_useful`、`T_bubble`、`T_total`、Bubble/Actual 分别是多少?
2. 在上题基础上使用 `v=4`，理想 VPP 的 Bubble、Total 和理想加速比是多少?
3. 为什么普通 1F1B 能显著减少 Activation Memory，却不能像 VPP 那样把理论 Bubble 缩小？请从依赖链和任务粒度解释。

通信代价与 overlap

普通 PP 的每个物理 stage 边界，每个 microbatch 前向发送 activation、反向发送 activation gradient。VPP 把总逻辑 stage 数从 p 增加到 p×v，边界也增加，因此 NVIDIA 给出的结论是总 P2P 通信量约增加 v 倍。消息 tensor shape 通常仍是 [micro_batch, sequence, hidden]，不是每条消息缩小 v 倍；增加的是边界/消息次数。

可以 overlap：调度器可用异步 P2P send/recv，在某个 chunk 通信时执行本 GPU 另一个 chunk 或另一个 microbatch 的计算。但是否真重叠取决于 NCCL/P2P backend、CUDA stream、网络拓扑和 SM/显存带宽争用。若通信暴露时间超过节省的 bubble，继续增大 v 反而变慢。必须用 Nsight Systems/Megatron timers 看 timeline。

显存影响

VPP 不会像 FSDP 那样把参数、梯度、优化器状态降到 1/N；每张 GPU 仍持有与普通 PP 大致相同总层数，只是拆成多个不连续 chunks。参数显存基本不变。activation 显存由调度中同时在飞的 microbatches、chunk 数、recompute 策略决定：VPP 不是天然的显存优化，某些配置下 bookkeeping/通信 buffer/保存 activation 反而更复杂。1F1B 的核心内存优势是把 outstanding forward activations 从 O(m) 限制到与 pipeline 深度同阶；实际 VPP 峰值应以框架实现和 profiler 为准。

Megatron 配置示例

```
# Megatron Core / NeMo Bridge 风格（当前官方文档）
model_config = GPTModelProvider(
    num_layers=16,
    pipeline_model_parallel_size=4,
    virtual_pipeline_model_parallel_size=2,  # 每个物理 rank 2 个 chunks
)

# Megatron-LM 某些 CLI 版本使用“每个虚拟 stage 的层数”表达
--pipeline-model-parallel-size 4 \
--num-layers-per-virtual-pipeline-stage 2

# 本例：v = L / (p × layers_per_virtual_stage)
#       = 16 / (4 × 2) = 2

# 版本敏感：不同 Megatron/NeMo 版本参数名与约束可能不同，
# 以当前安装版本的官方 config/API 为准。
```

实现约束与调优方法

常见约束：层数要能被物理 stage 数与 virtual chunk 粒度整除；Megatron 某些版本要求 microbatch 数能被 pipeline parallel size 整除；部分旧实现对 p=2 的 interleaved schedule 有限制。调优建议：1) 先保证物理 stage 计算均衡；2) 固定 p,m，从 v=2 开始；3) 比较 bubble time、P2P exposed time、activation peak、step time；4) 只有 bubble 节省大于额外通信时才提高 v；5) TP 放机内 NVLink，PP/VPP 边界尽量按拓扑映射，避免高频跨慢链路往返。

与相邻技术的区别

普通 PP：每 GPU 一个连续 stage；VPP：每 GPU 多个不连续 chunks，interleaved 调度，bubble 更小但通信更多。
1F1B：是一类调度策略，普通 PP 与 VPP 都可用；VPP 常指 interleaved 1F1B。

Zero-Bubble Pipeline：通常进一步拆分 backward 的 input-gradient(B) 与 weight-gradient(W) 并重排，以逼近零 bubble；VPP 主要靠把 stage 切成 chunks，并不等于 zero-bubble。
Tensor Parallel：切单个算子张量；VPP 仍是按层深度切分。
FSDP/DP：切数据和训练状态；与 VPP 正交，可组成 TP×PP/VPP×DP/FSDP。

什么时候应该用

适合：已经必须使用 PP；普通 1F1B 的 profiler 显示 bubble 明显；microbatch 数受 global batch 或显存限制不能继续增大；P2P 链路足够快；模型层数足够多且能均匀切 chunk。
不适合：p 很小或 m≫p，原 bubble 已很低；网络慢、P2P 已是瓶颈；层数不能均匀切；stage 本身严重不均衡；显存极紧且 VPP 调度增加 activation/buffer 峰值。此时先解决负载均衡、拓扑映射、recompute 或减少通信。

常见误区

1) VPP 不是创建更多 GPU，也不是让一张 GPU 真正同时跑两个大 GEMM；它创建多个逻辑 stage，并通过时间调度提高利用率。
2) v 越大不一定越快：bubble 下降约 1/v，但通信量约增 v，存在最优点。
3) VPP 不等于显存分片：每卡总参数基本不变。
4) 不要混淆两种 bubble 百分比口径。
5) 理论公式忽略通信和不均衡；最终决策必须看真实 step time 和 timeline。

关键总结

VPP = PP 的更细粒度 stage 切分 + interleaved 1F1B。它让每张 GPU 持有 v 个 model chunks，用更短的 chunk 计算填补流水线等待；理想 bubble 缩小 v 倍，但 P2P 通信约增加 v 倍。选择 v 的本质是用更多通信换更少空闲时间，工程上必须测量 bubble 节省是否超过通信与调度开销。

可选自测

1. 因果题：为什么 VPP 的 bubble 理想上缩小约 v 倍，但总通信量反而约增加 v 倍?
2. 计算题：p=8、m=16、v=4 时，分别计算普通 1F1B 与 VPP 的 bubble/useful 和 bubble/total。

3. 工程题：若 Nsight 显示普通 PP bubble 只有 5%，但 P2P 已占 step time 的 18%，是否应该启用 v=2？说明理由。

继续学习

1. 前置：Pipeline Parallel 与 1F1B 的完整时间线
2. 同层对比：Zero-Bubble Pipeline Parallelism
3. 下游应用：TP × VPP × DP/FSDP 的 3D 并行拓扑布局

事实来源与版本边界

核心公式和“通信量随 v 增加”的结论来自 NVIDIA Megatron 官方技术文章；配置字段参考 Megatron Core 0.16 与 NeMo Megatron Bridge 文档。截至 2026-07-27，不同 Megatron/NeMo 版本的 CLI 参数名和约束可能变化，生产配置应核对实际安装版本。

06

torch、Megatron、CUDA、Triton：概念、关联与整块知识地图

> **原始问题：**torch， Megatron，cuda，triton 这些概念分别是什么？之间有什么关联？有没有其他相关的概念和知识点，把这一块整体的内容都给我列出来，清晰地讲解透彻

提问：2026/07/27 16:15　更新：2026/07/27 16:50　重要度：5/5

---

torch、Megatron、CUDA、Triton：概念、关联与整块知识地图

先说明一个容易混淆的点：**Triton 有两个完全不同的东西同名**。本文默认讲的是 **OpenAI Triton（GPU kernel 语言与编译器）**，因为它和 torch/CUDA 是同一条技术链路；文末会单独澄清另一个 **NVIDIA Triton Inference Server**。

一句话结论

这四者不是并列关系，而是**一条从上到下的调用栈**：

- **Megatron** —— 大模型**分布式训练框架层**：决定模型怎么切到成百上千张 GPU 上（TP/PP/DP/EP 等）。
- **PyTorch（torch）** —— **深度学习框架层**：提供 Tensor、自动微分、nn.Module、优化器、分布式原语，是写模型的主语言。
- **CUDA** —— NVIDIA 的**并行计算平台与编程模型**：是 GPU 上一切计算的底座，提供 kernel、内存、stream、通信的基础设施。
- **Triton（OpenAI）** —— 一种**写 GPU kernel 的语言与编译器**：让你用 Python 风格写出接近手写 CUDA 性能的算子，是 `torch.compile` 默认的 GPU 代码生成后端。

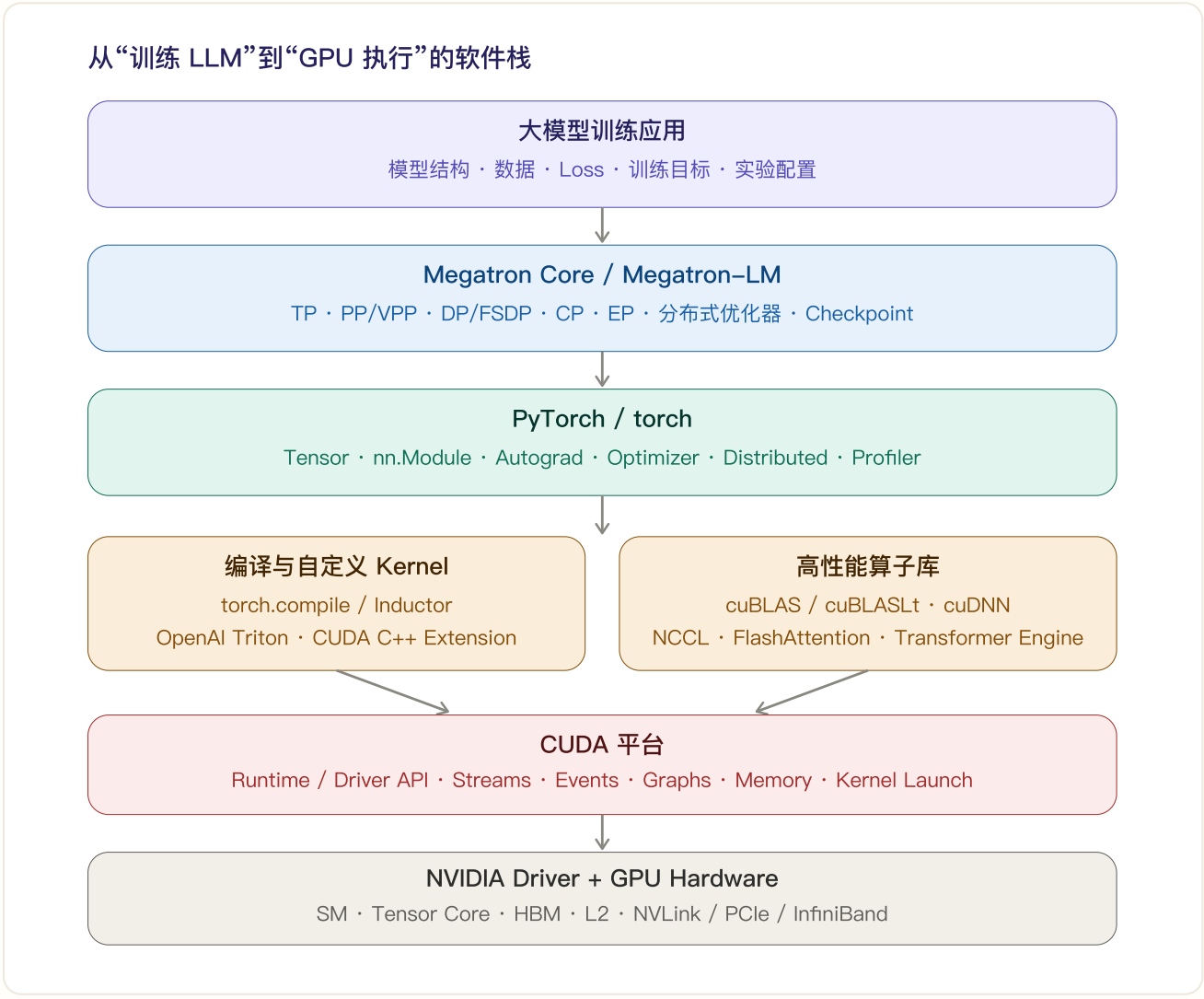
一句话串起来：

> **你用 PyTorch 写模型，Megatron 负责把它切到多卡上训练，真正在 GPU 上跑的是 CUDA kernel，而这些 kernel 越来越多地由 Triton 生成或手写。**

一、先建立总览：它们在软件栈的哪一层

请看下面的分层图。从上到下的调用关系是：

```
大模型训练应用
    ↓
Megatron Core / Megatron-LM     ← 分布式训练框架（切分与调度）
    ↓
PyTorch / torch                 ← 深度学习框架（张量与自动微分）
    ↓
编译器与算子层
  torch.compile / Inductor
  OpenAI Triton                 ← 生成/编写 GPU kernel
  cuBLAS / cuDNN / NCCL / FlashAttention
    ↓
CUDA 平台                       ← 并行计算平台（kernel/内存/stream）
    ↓
NVIDIA Driver + GPU 硬件        ← SM / Tensor Core / HBM / NVLink
```



理解这张图，就理解了 80% 的关系：**上层是“描述要算什么”，下层是“真正在 GPU 上怎么算”。**Megatron 最上、CUDA 最底、PyTorch 在中间、Triton 在“框架”和“CUDA”之间的编译层。

二、CUDA：一切的底座

定义

CUDA（Compute Unified Device Architecture） 是 NVIDIA 的**并行计算平台和编程模型**。它让开发者能把计算密集的任务放到 GPU 上执行。它不是一个库，而是一整套：编程模型 + 运行时 + 驱动 + 工具链 + 一堆官方库。

编程模型：kernel / thread / block / grid

CUDA 的核心抽象是 **kernel**：一个在 GPU 上被成千上万个线程并行执行的函数。

线程的层级组织（务必记住这个层级）：

```
grid（网格）
  └── block（线程块，同一个 block 内的线程能共享 shared memory、能同步）
        └── thread（线程，最小执行单位）
```

一次 kernel launch 会启动一个 grid，grid 里有很多 block，每个 block 里有很多 thread。硬件上，block 被调度到 **SM（Streaming Multiprocessor）** 上执行，thread 以 **warp（32 个线程）** 为单位被调度。

内存层级（性能的关键）

```
寄存器 register      — 最快，每线程私有
shared memory        — 快，block 内共享，手动管理
L2 cache             — 全 GPU 共享
global memory / HBM  — 最大但最慢，跨 kernel 持久
```

几乎所有 GPU 性能优化，本质都是“尽量让数据待在寄存器/shared memory，少读写 HBM”。后面讲 Triton 的融合优化就是这个道理。

Stream / Event / Graph

- **Stream（流）**：一串按顺序执行的 GPU 操作；不同 stream 之间可并行。这是我们之前聊的 **overlap（计算通信重叠）** 的底层机制。
- **Event**：跨 stream 的同步点。
- **CUDA Graph**：把一串 kernel launch 录制成一个图，一次性重放，消除反复 launch 的 CPU 开销。

Runtime / Driver / Toolkit 的区别

初学者最容易混：

```
CUDA Driver     — 随显卡驱动安装，最底层，向后兼容较强
CUDA Runtime    — 应用链接的运行时库（libcudart），API 更友好
CUDA Toolkit    — 开发套件：nvcc 编译器 + 库 + 头文件 + 工具
```

`nvidia-smi` 显示的是**驱动**支持的最高 CUDA 版本；`nvcc --version` 显示的是你装的 **Toolkit** 版本，二者可以不同。

官方库

CUDA 之上还有一层高性能库，PyTorch 大量依赖它们：

- **cuBLAS / cuBLASLt**：矩阵乘（GEMM）。
- **cuDNN**：卷积、部分 attention、归一化等深度学习原语。
- **NCCL**：多卡集合通信（AllReduce / AllGather / ReduceScatter / All-to-All）——你之前学的通信原语就由它实现。

> 检查点：你现在应该能说清 grid / block / thread 的层级，以及为什么"减少 HBM 读写"是优化核心。

三、PyTorch（torch）：写模型的主语言

定义

PyTorch 是一个面向 GPU/CPU 的深度学习框架。官方定义就是一句话：一个为深度学习优化的、支持 GPU 的张量库。它的价值在于让你用 Python 描述模型，而不用手写 GPU 代码。

核心组成

组件	作用
`torch.Tensor`	多维数组，可放在 CPU 或 GPU（`.cuda()`）
`autograd`	自动微分，自动构建反向计算图并求梯度
`nn.Module`	组织网络层与参数的容器
`optim`	SGD / Adam 等优化器

组件	作用
`torch.distributed`	DDP、FSDP、集合通信、`torch.distributed.pipelining` 等
`torch.cuda`	管理 GPU、stream、显存
`torch.profiler`	性能分析
`torch.compile`	2.x 引入的编译入口（下一节详解）

关键机制：Autograd

你只写前向；PyTorch 在前向时记录算子，反向时自动沿计算图求导。这就是“define-by-run”（动态图）。

关键机制：Eager vs Compiled

- **Eager 模式（默认）**：每个 torch 算子立即在 GPU 上启动一个（或几个）预编译的 CUDA kernel。灵活、好调试，但每个算子都要 kernel launch + 读写 HBM。
- **Compiled 模式（`torch.compile`）**：把一段代码捕获成图，交给编译器融合优化后再执行，通常快很多。

PyTorch 与 CUDA 的关系

PyTorch **不自己实现 GPU 计算**，它把算子分派（dispatch）到底层实现：GPU 上就是调用 cuBLAS / cuDNN / NCCL 或自带的 CUDA kernel。所以：

> **PyTorch 是“指挥”，CUDA 是“执行”。** 一句 `a @ b` 在 GPU 上最终变成一次 cuBLAS GEMM kernel。

最小示例

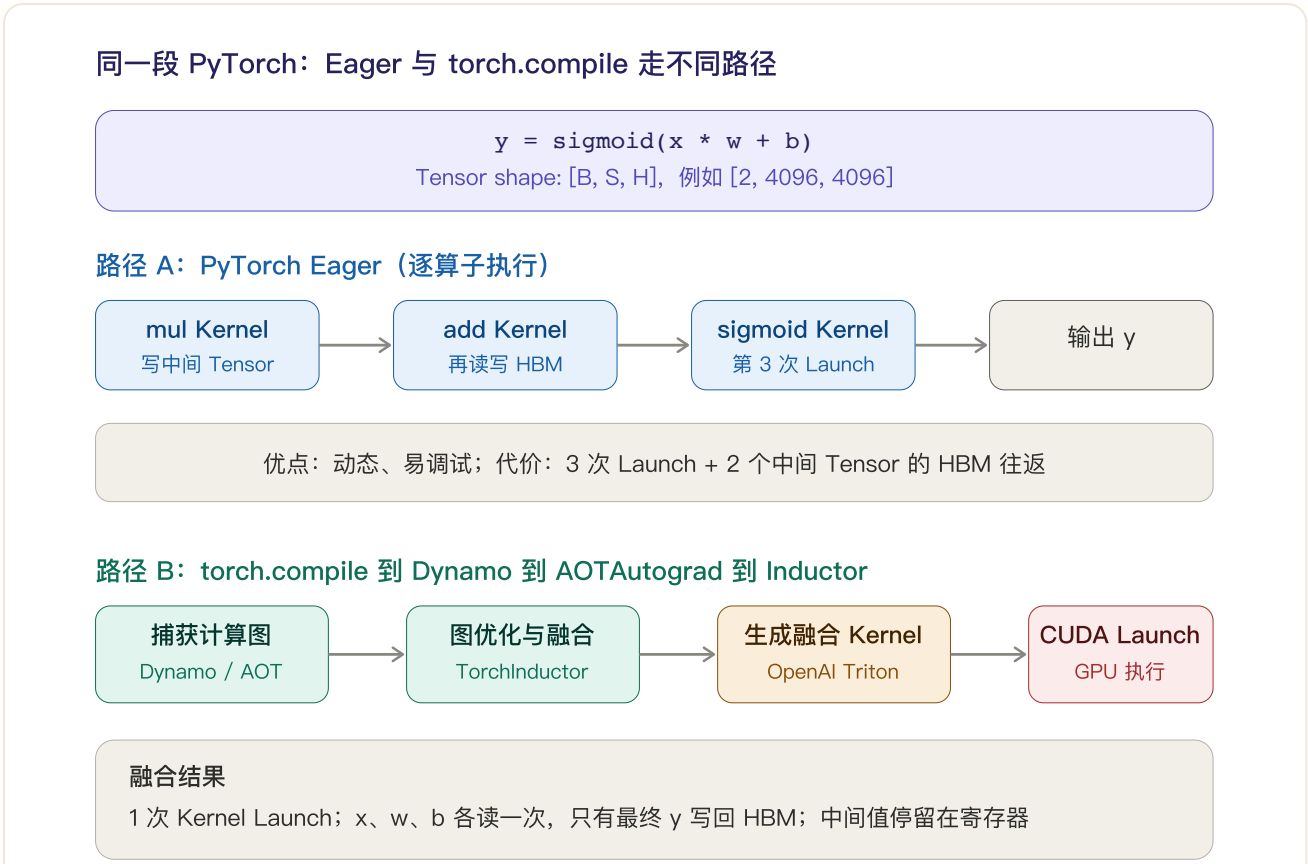
```
import torch

x = torch.randn(2, 4096, 4096, device="cuda")  # Tensor 放到 GPU
w = torch.randn_like(x)
b = torch.randn_like(x)

def f(x, w, b):
    return torch.sigmoid(x * w + b)   # 3 个逐元素算子

y = f(x, w, b)          # Eager：约 3 次 kernel launch
print(y.shape)          # torch.Size([2, 4096, 4096])
```

下面这张图展示同一段 PyTorch 代码在 Eager 与 `torch.compile` 下如何走到 GPU Kernel：



> 检查点：你应该能解释为什么 PyTorch 本身不算“跑在 GPU 上的代码”，真正跑的是它调用的 CUDA kernel。

四、Triton（OpenAI）：写 GPU kernel 的新方式

定义

OpenAI Triton 官方定义是：一个用于并行编程的**语言和编译器**，目标是提供一个基于 Python 的环境，让你高效写出能在现代 GPU 上跑满吞吐的自定义 DNN 计算 kernel。

一句话：**Triton 让你用 Python 语法写 GPU kernel，而不用写 CUDA C++。**

它解决什么问题

手写 CUDA 很强但门槛高：要手动管理 shared memory、线程索引、bank conflict、向量化等。Triton 把很多这类细节交给编译器：

- 你按 **block（program）** 粒度写代码，而不是按单个 thread；
- 编译器负责内存合并访问、shared memory 分配、指令调度等。

对照 CUDA 的心智模型差异：

```
CUDA：你写“每个 thread 做什么”
Triton：你写“每个 program（一块数据）做什么”，thread 级细节交给编译器
```

最小示例（向量加）

```
import triton
import triton.language as tl

@triton.jit
def add_kernel(x_ptr, y_ptr, out_ptr, n, BLOCK: tl.constexpr):
    pid = tl.program_id(0)                # 当前 program 处理第几块
    offs = pid * BLOCK + tl.arange(0, BLOCK)
    mask = offs < n
    x = tl.load(x_ptr + offs, mask=mask)   # 从 HBM 读一块
    y = tl.load(y_ptr + offs, mask=mask)
    tl.store(out_ptr + offs, x + y, mask=mask)  # 写回一块
```

注意：这里操作的是“一块数据（BLOCK 个元素）”，不是单个元素，这就是 Triton 的 block 编程模型。

Triton 与 PyTorch / CUDA 的关系（最关键）

在 `torch.compile` 路径中：

1. `torch.compile` 用 **TorchDynamo** 捕获计算图；
2. **AOTAutograd** 连反向图一起捕获；
3. **TorchInductor**（默认编译器）做算子融合等优化；
4. 对 NVIDIA/AMD/Intel GPU，Inductor **默认用 Triton 生成 GPU kernel**；
5. 生成的 Triton kernel 最终仍通过 **CUDA** 编译、launch 到 GPU 执行。

所以关系是：

```
Triton 是 CUDA 之上的一层 kernel 语言/编译器
Triton 生成的 kernel 最终还是跑在 CUDA 上
torch.compile 默认用 Triton 做 GPU 代码生成后端
```

Triton 不替代 CUDA，它是“更好写的 CUDA kernel 生成方式”。 著名的 FlashAttention 就有 Triton 实现版本。

为什么融合能加速（呼应 CUDA 内存层级）

Eager 下 `x*w+b` 再 sigmoid 是 3 个 kernel、2 个中间 Tensor 反复读写 HBM；Triton 融合成 1 个 kernel 后，中间结果留在寄存器，只读 3 个输入、写 1 个输出，HBM 流量减半、launch 从 3 次降到 1 次。这正是"减少 HBM 读写"原则的直接体现。

> 检查点：你应该能说清 Triton 在 `torch.compile` 链路里出现在哪一步，以及它和 CUDA 是"生成者与执行平台"的关系。

五、Megatron：大模型分布式训练框架

定义

Megatron-LM / Megatron Core 是 NVIDIA 开源的**大规模 Transformer 训练框架**。Megatron Core 是把并行能力模块化后的库，可被 NeMo 等更上层框架复用。它解决的核心问题是：**一个模型大到单卡、单机都放不下、算不动时，如何高效切到成百上千张 GPU 上训练。**

它提供什么（都是"系统层"能力）

能力	作用	你之前学过的关联
Data Parallelism (DP)	数据切分	AllReduce
Tensor Parallelism (TP)	切单层算子内张量	机内 NVLink
Pipeline Parallelism (PP)	按层切 stage	P2P、1F1B
Virtual Pipeline (VPP)	PP 的细粒度交错调度	减少 bubble
Context Parallelism (CP)	切序列维度	长上下文
Expert Parallelism (EP)	切 MoE 专家	All-to-All
分布式优化器 / FSDP	切优化器状态	ZeRO

还有：CUDA Graph 集成、激活重计算/offload、MoE、Transformer Engine（FP8）、checkpoint 等工程能力。

Megatron 与 PyTorch / CUDA 的关系

Megatron **构建在 PyTorch 之上**：它的模型仍是 `nn.Module`、张量仍是 `torch.Tensor`、通信仍走 `torch.distributed`/NCCL、kernel 仍是 CUDA。Megatron 的价值不在"重写计算"，而在"把计算和通信在多卡上编排得高效"。

```
Megatron 决定：模型怎么切、通信怎么排、流水线怎么调度
PyTorch 决定：单卡上张量和自动微分怎么算
CUDA/NCCL 决定：kernel 和通信在硬件上怎么执行
```

> 检查点：你应该能说清 Megatron 不替代 PyTorch，而是“多卡编排层”。

六、四者关系总表

维度	CUDA	PyTorch (torch)	OpenAI Triton	Megatron
类别	并行计算平台 + 编程模型	深度学习框架	GPU kernel 语言/编译器	分布式训练框架
层次	最底层（软件底座）	中间层	编译/算子层	最上层（多卡编排）
你用它做什么	写/调度 GPU kernel、管理内存与 stream	写模型、自动微分、训练	写高性能自定义算子	把大模型切到成百上千卡训练
语言	C++（也有 Python 接口）	Python	Python 风格 DSL	Python（基于 PyTorch）
归属	NVIDIA	PyTorch 基金会/Meta 起源	OpenAI	NVIDIA
是否直接碰硬件	是	否，靠 CUDA	生成 CUDA 代码，间接	否，靠 PyTorch+CUDA
典型触发	kernel、stream、显存	Tensor、nn.Module、autograd	torch.compile、自定义融合算子	TP/PP/DP、千卡训练

一条最简调用链，帮助你记忆：

```
Megatron（切模型到多卡）
  → PyTorch（写模型、算梯度）
    → Triton / cuBLAS / cuDNN（生成或提供 kernel）
      → CUDA（编译、launch、管理 stream）
        → NVIDIA GPU（真正执行）
```

七、这一整块还应该知道的相关概念

你问“还有没有其他相关概念”。下面按层归类，方便你后续逐个深入（这些都会进你的知识图谱和在线笔记本）。

1. 硬件层

- **GPU 架构**：SM、Tensor Core、Hopper/Blackwell 等代际
- **显存**：HBM、显存带宽、显存墙

- **互联**：NVLink、NVSwitch、PCIe、InfiniBand / RoCE
- **精度**：FP32 / TF32 / FP16 / BF16 / FP8

2. CUDA 生态层

- **计算库**：cuBLAS、cuBLASLt、cuDNN
- **通信库**：NCCL
- **机制**：CUDA Stream、Event、CUDA Graph、Unified Memory
- **底层语言**：PTX（虚拟指令集）、SASS（真实机器码）

3. 编译器 / Kernel 层

- **torch.compile 组件**：TorchDynamo、AOTAutograd、TorchInductor
- **Kernel 编写**：OpenAI Triton、CUDA C++ Extension、CUTLASS
- **推理编译**：TensorRT、TensorRT-LLM、torch-tensorrt
- **其他编译栈**：XLA、TVM、MLIR

4. 框架 / 算子层

- **框架**：PyTorch、JAX（对照理解）
- **明星算子**：FlashAttention、PagedAttention、Fused LayerNorm/RMSNorm
- **混合精度组件**：AMP、Transformer Engine（FP8）

5. 分布式 / 训练系统层

- **并行策略**：DP、TP、PP、VPP、CP、EP、3D/4D 并行
- **显存优化**：ZeRO、FSDP、激活重计算、offload
- **训练框架**：Megatron-LM/Core、DeepSpeed、NeMo、torchtitan
- **通信原语**：AllReduce、AllGather、ReduceScatter、All-to-All

6. 推理 / Serving 层（区别于训练）

- **推理引擎**：vLLM、SGLang、TensorRT-LLM
- **关键机制**：Continuous Batching、KV Cache、PagedAttention
- **指标**：TTFT、TPOT、吞吐、MFU

一张“从代码到硅”的记忆链：

```
分布式策略(Megatron/DeepSpeed)
 → 框架(PyTorch/JAX)
  → 编译器(torch.compile/Inductor/XLA)
   → Kernel 语言(Triton/CUDA C++/CUTLASS)
    → 计算/通信库(cuBLAS/cuDNN/NCCL)
     → CUDA 平台
      → 驱动 + GPU
```

八、常见误区

1. **“Triton 要取代 CUDA”**：不准确。Triton 生成的 kernel 最终仍编译并运行在 CUDA 之上；它替代的是“手写 CUDA C++ kernel 的繁琐”，不是 CUDA 平台本身。
1. **把两个 Triton 混为一谈**：OpenAI Triton 是 GPU kernel 语言/编译器；NVIDIA Triton Inference Server 是模型推理服务器，二者毫无继承关系，只是重名。
1. **“Megatron 和 PyTorch 是竞品”**：不是。Megatron 建在 PyTorch 之上，是多卡编排层，不替代张量与自动微分。
1. **“PyTorch 直接在 GPU 上算”**：PyTorch 是分派者，真正执行的是 CUDA kernel（cuBLAS/cuDNN/Triton 生成的 kernel 等）。
1. **“torch.compile 就是普通 JIT”**：它是“图捕获 + 融合 + Triton 代码生成”的完整编译栈，核心收益来自算子融合减少 HBM 往返和 kernel launch，而不仅是即时编译。
1. **CUDA 版本混淆**：`nvidia-smi` 显示驱动支持的最高版本，`nvcc --version` 显示 Toolkit 版本，二者可以不同。

九、关键总结

记住这条核心链路：

```
Megatron  = 把大模型切到多卡训练（系统编排层）
PyTorch   = 用张量和自动微分写模型（框架层）
Triton    = 用 Python 写高性能 GPU kernel（编译/算子层）
CUDA      = GPU 计算的底座（平台层）

四者是自上而下的调用栈，不是竞争关系：
你用 PyTorch 写模型 → Megatron 切到多卡 → 计算落成 CUDA kernel
→ 其中越来越多 kernel 由 Triton 生成 → 全部跑在 CUDA + GPU 上
```

一句话记忆：**Megatron 管“切”，PyTorch 管“写”，Triton 管“生成算子”，CUDA 管“执行”。**

可选自测

1. 因果题

为什么说 `torch.compile` 的加速主要来自“算子融合”，而这又和 CUDA 的内存层级直接相关？请用 `x*w+b` 后接 sigmoid 的例子说明。

2. 层次题

把下列组件按调用栈从上到下排序，并各用一句话说明职责：NCCL、Megatron、Triton、torch.Tensor、CUDA Stream、TorchInductor。

3. 辨析题

有人说“我们要用 Triton 替换掉 CUDA，并用 Triton Inference Server 部署”。这句话里有两个概念错误，请分别指出并纠正。

继续学习

1. **前置：CUDA 编程模型与 GPU 内存层级** —— 理解 kernel/thread/block、寄存器与 HBM，是理解上层一切性能优化的基础。
2. **同层对比：torch.compile 编译栈（Dynamo / AOTAutograd / Inductor / Triton）** —— 深入一句 `torch.compile` 背后到底发生了什么。
3. **下游应用：3D 并行（TP × PP/VPP × DP/FSDP）在 Megatron 中的落地** —— 把“分布式编排层”和你已学的通信/并行知识连起来。

回复 1、2 或 3 继续；也可以说“稍后”或“不感兴趣”。

CUDA 编程模型与 GPU 内存层级：从 thread 到 HBM

> **原始问题：**继续深入讲解这部分：前置：CUDA 编程模型与 GPU 内存层级 —— 理解 kernel/thread/block、寄存器与 HBM，是理解上层一切性能优化的基础。

提问：2026/07/27 17:18　更新：2026/07/27 17:18　重要度：5/5

CUDA 编程模型与 GPU 内存层级：从 thread 到 HBM

一句话结论

CUDA 性能优化可以压缩成两条主线：

1. **执行层级决定“谁做哪份工作”**：CPU 发起 kernel；一个 kernel 形成 grid；grid 由多个 block 组成；block 由多个 thread 组成；硬件真正以 32 个 thread 构成的 warp 为调度单位，把 block 分配到 SM 上执行。
2. **内存层级决定“数据搬运有多贵”**：register 和 shared memory 在片上、快但容量小；L2 是全 GPU 共享缓存；global memory 物理上通常是 HBM，容量大但访问延迟高。大量 AI kernel 的优化本质是减少 HBM 往返并提高片上复用。

必须同时记住一个反直觉：**片上资源并非用得越多越好**。每线程寄存器和每 block shared memory 用量过大，会减少一个 SM 上能同时驻留的 block/warp，降低隐藏延迟的能力；强行压寄存器又可能导致 register spill，把数据落到慢得多的 local memory。

一、系统位置：为什么学上层框架仍必须懂 CUDA

在 PyTorch、Triton、Megatron 下面，最终都落到 CUDA 执行模型：

```
Megatron：决定模型如何分布到多卡
    ↓
PyTorch：组织 Tensor、Autograd 和算子
```

```
    ↓
Triton / cuBLAS / cuDNN / 自定义 CUDA kernel：决定单个算子如何实现
    ↓
CUDA：kernel launch、线程层级、内存、stream、同步
    ↓
GPU：SM、warp scheduler、register file、shared memory、L2、HBM
```

理解 CUDA 后，你才能解释这些上层现象：

上层现象	CUDA 层根因
`torch.compile` 融合算子后更快	减少 kernel launch 和中间 Tensor 的 HBM 读写
FlashAttention 比普通 Attention 省显存、更快	用 tiling 在 shared memory/register 中复用，避免完整 attention matrix 往返 HBM
Triton 的 `BLOCK_SIZE` 影响性能	决定一个 program/block 的工作粒度、访问合并、寄存器压力和 occupancy
NCCL 与计算 overlap 不充分	通信 kernel、计算 kernel、HBM 带宽和 SM 可能互相争用
同一个模型换 shape 后性能突变	grid/block 数量、tile 尺寸、边界 mask、数据复用率和 kernel 选择发生变化

二、统一例子：用 SAXPY 串起整篇

全文先使用一个统一、可手算的例子：

```
z[i] = a × x[i] + y[i]
```

参数：

```
N = 1,048,576 = 2²⁰ 个元素
数据类型 = float32 = 4 Byte/元素
blockDim.x = 256 threads/block
gridDim.x = ceil(N ÷ 256) = 4096 blocks
warp size = 32 threads
每个 block = 256 ÷ 32 = 8 warps
```

每个 thread 负责一个元素：

```
global_idx = blockIdx.x × blockDim.x + threadIdx.x
```

总逻辑线程数：

```
4096 blocks × 256 threads/block = 1,048,576 threads
```

这恰好覆盖全部元素。真实工程通常仍保留 `if (i < N)`，这样 `N` 不能整除 block size 时也安全。

每个元素的有用工作量：

```
读取 x[i]：4 Byte
读取 y[i]：4 Byte
写入 z[i]：4 Byte
合计 HBM 有用流量：12 Byte/元素
计算：1 次乘法 + 1 次加法 ≈ 2 FLOP/元素
```

整个 kernel：

```
有用数据量 = 1,048,576 × 12 Byte
           = 12,582,912 Byte
           = 12 MiB

计算量 ≈ 1,048,576 × 2 FLOP
       = 2,097,152 FLOP
       ≈ 2.10 MFLOP
```

算术强度：

```
AI = FLOP ÷ HBM Byte
   ≈ 2 ÷ 12
   ≈ 0.167 FLOP/Byte
```

这是非常低的算术强度，因此 SAXPY 通常是 **memory-bound**：核心问题不是 GPU 会不会乘加，而是能否高效地把 x、y 从 HBM 搬进来，再把 z 写回去。

三、CUDA 执行模型：host、device 与 kernel

3.1 Host 与 Device

- **Host**：通常指 CPU 及其内存。负责分配资源、准备数据、发起拷贝和 kernel launch。
- **Device**：GPU 及其显存。负责执行 kernel。
- **Kernel**：在 GPU 上被大量线程并行执行的函数。

典型控制流：

```
CPU 分配 host/device 内存
→ 把输入复制到 device
→ CPU 发起 kernel launch
→ GPU 异步执行 kernel
→ 必要时同步
→ 把结果复制回 host
```

Kernel launch 对 CPU 通常是异步的：CPU 把任务放进 stream 后可以继续提交后续任务。这是计算/通信 overlap、数据预取和 CUDA Graph 的基础。

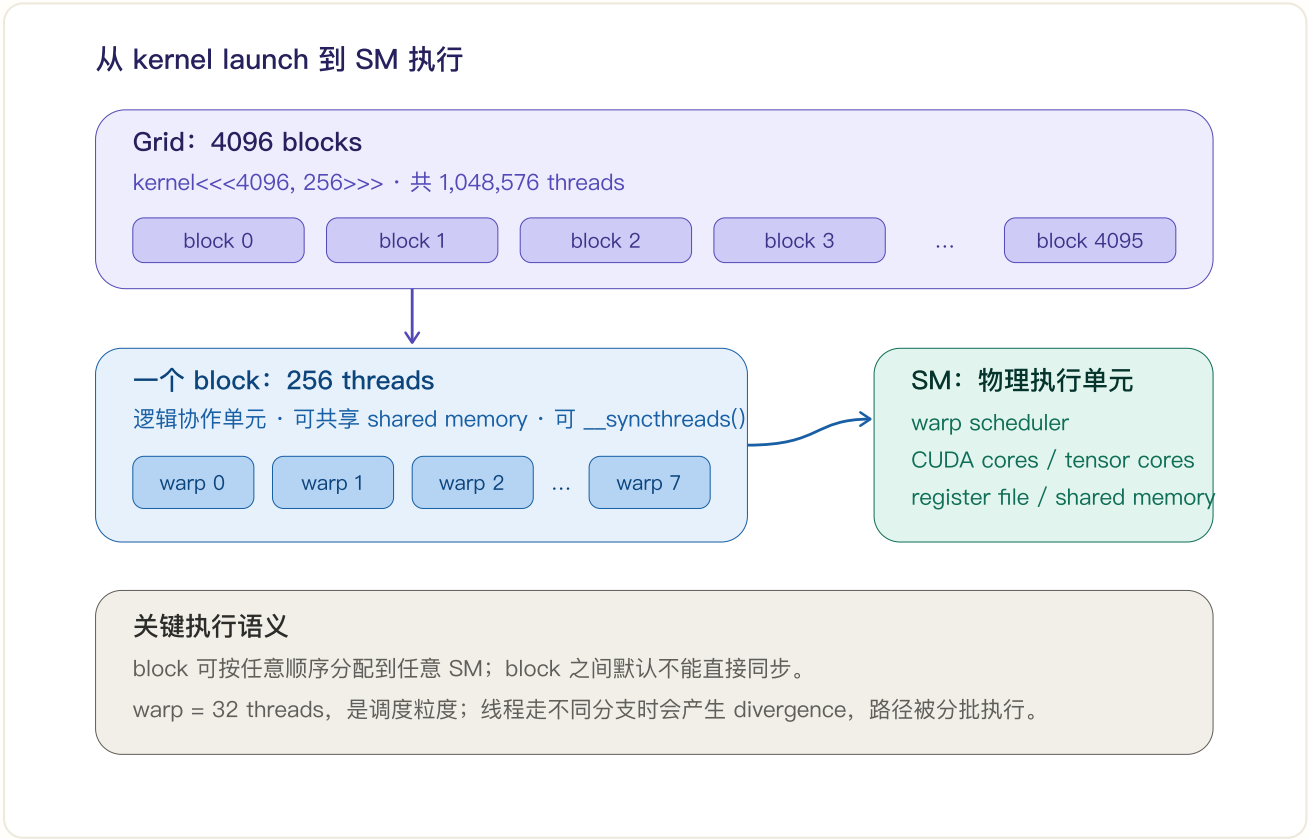
3.2 Grid、Block、Thread

层级关系：

```
一次 kernel launch
└── 一个 grid
    ├── block 0
    │   ├── thread 0
    │   ├── thread 1
    │   └── ...
    ├── block 1
    └── ...
```

三者职责：

层级	软件语义	硬件映射与约束
Thread	处理一个或多个数据元素	拥有私有寄存器和逻辑 local memory
Block	一组可协作的 thread	整个 block 驻留在一个 SM；可共享 shared memory，可做 block 内同步
Grid	本次 kernel 的全部 block	block 可按任意顺序分配到任意 SM；普通 kernel 中 block 间默认无直接同步



3.3 为什么 Block 必须彼此独立

CUDA 要让同一个 kernel 能在不同规模 GPU 上运行，block 的调度顺序不能成为正确性的前提：

- 某个 block 可能先执行，也可能后执行；
- 多个 block 可能同时驻留在不同 SM；
- grid 中 block 数可以远多于 SM 数，GPU 会分批执行；
- 普通 kernel 中不能假设“block 0 已经执行完，block 1 才开始”。

因此，若算法要求全 grid 同步，常见做法是：

1. 拆成两个 kernel；前一个 kernel 完成就是天然的全局边界；
2. 使用 cooperative groups 等特殊机制，但必须满足额外启动与驻留约束；
3. 用原子操作/计数器实现特定协议，但要非常谨慎，避免死锁和内存可见性错误。

四、Warp 与 SIMT：代码看似按 thread 写，硬件按 warp 执行

4.1 Warp 是什么

一个 warp 固定包含 32 个 thread，是 NVIDIA GPU 的基本调度粒度。以 `blockDim.x=256` 为例：

```
warp 0: threadIdx.x =   0..31
warp 1: threadIdx.x =  32..63
...
warp 7: threadIdx.x = 224..255
```

软件让每个 thread 有独立索引和寄存器状态，但 warp scheduler 每次选择一个 ready warp 发射指令。这种模型叫 **SIMT（Single Instruction, Multiple Threads）**。

4.2 Warp divergence

若同一 warp 内线程走不同分支：

```
if (threadIdx.x % 2 == 0) {
    path_a();
} else {
    path_b();
}
```

硬件不能让同一 warp 同一时刻真正执行两条不同指令流，通常需要用不同 active mask 分批执行：

```
先让偶数线程有效，执行 path_a
再让奇数线程有效，执行 path_b
```

于是两条路径都要付时间，未激活线程在对应阶段空转。关键不是“代码里有 if 就慢”，而是：

> **同一 warp 内是否发生分歧，以及各路径工作量是否显著。**

若分支条件按 warp 划分，例如 `warp_id < 4`，每个 warp 内所有线程走同一路径，通常不会产生同样的 divergence 代价。

4.3 Warp 如何隐藏延迟

HBM load 发出后往往不能立刻返回。GPU 不让当前 warp 阻塞整个 SM，而是切换到另一个 ready warp：

```
warp A 发出 HBM load → 等待
warp scheduler 改发 warp B 的算术指令
再发 warp C
等 A 数据就绪后继续 A
```

这就是 **latency hiding**。它要求 SM 上有足够多的 ready warps，所以 occupancy 有价值；但 ready warp 多不等于计算一定快，后面会讲为什么 100% occupancy 不是目标本身。

五、SM：Block 真正驻留和执行的地方

SM（Streaming Multiprocessor） 是 GPU 的主要计算单元。一个 SM 通常包含：

- warp scheduler / dispatch 单元；
- FP/INT 执行单元；
- Tensor Core；
- register file；
- shared memory / L1 相关片上资源；
- load/store、special function 等单元。

一个 block 一旦分配给某个 SM，它在生命周期内通常不会迁移到另一个 SM。一个 SM 可以同时驻留多个 block，但数量受到以下资源共同限制：

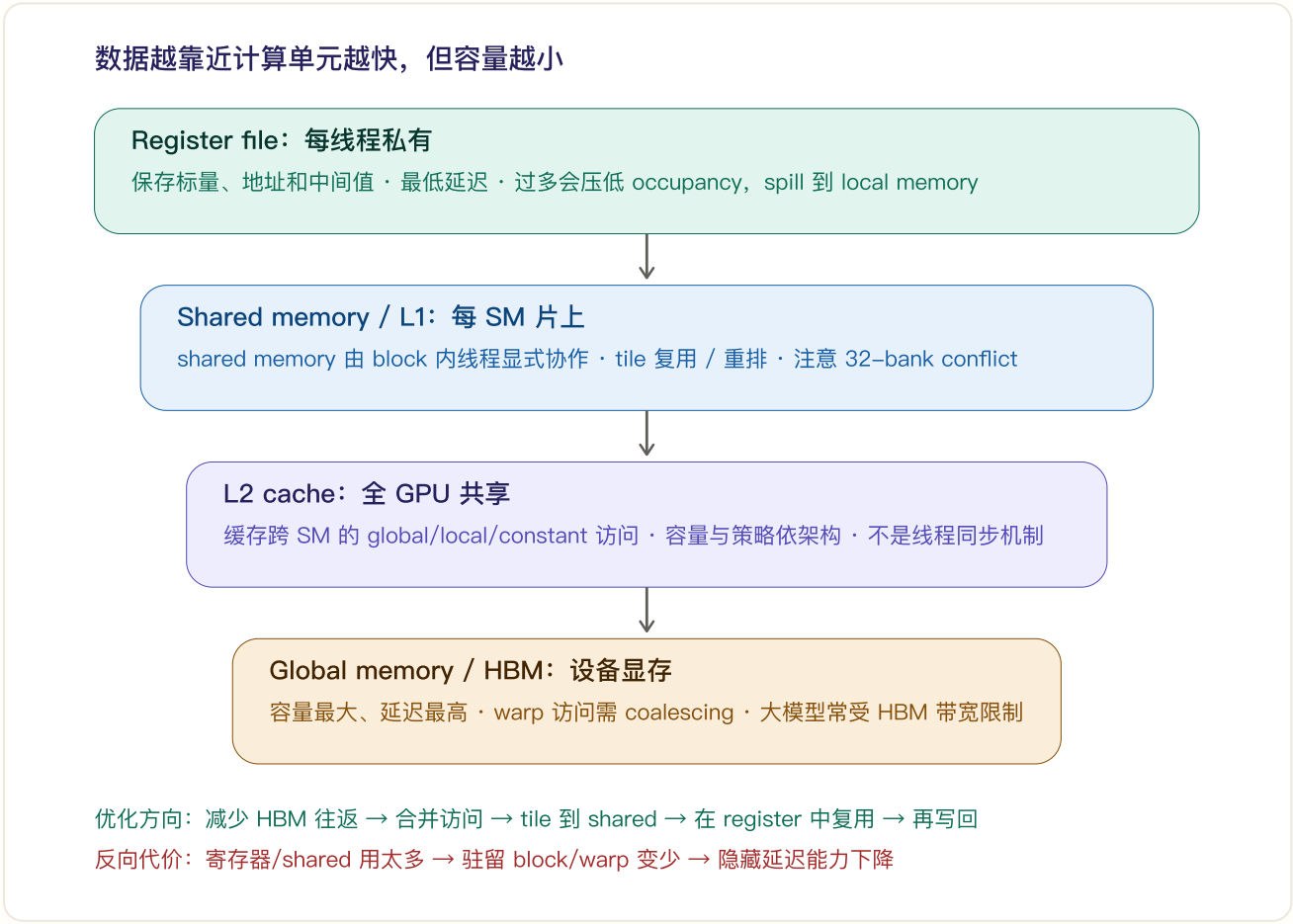
```
每 SM 最大 blocks
每 SM 最大 threads / warps
register file 总量
shared memory 总量
架构规定的其他限制
```

因此，配置 block size 时不是“线程越多越好”，而是要考虑：

```
并行度
× warp 对齐
× 寄存器压力
× shared memory 压力
× 数据访问模式
```

所有具体上限都依赖 GPU compute capability，不应把某一代 GPU 的数字当成 CUDA 永久规则。

六、GPU 内存层级：作用域、速度与管理方式



6.1 总览表

存储	典型作用域	物理位置/管理	典型用途	主要风险
Register	单 thread	每 SM 片上，编译器分配	地址、标量、中间累加值	用量过高降低 occupancy；spill
Local memory	逻辑上单 thread	通常位于设备显存，经 cache 访问	寄存器放不下的变量、动态索引数组、spill	名字含 local 但并不快
Shared memory	单 block	每 SM 片上，程序显式管理	tile、跨线程复用、数据重排	bank conflict；用量过高减少驻留 block
L1 cache	单 SM 附近	硬件管理，细节依架构	缓存部分 global/local 访问	命中率不稳定，不能代替正确同步
L2 cache	全 GPU	硬件管理	跨 SM 缓存 global/local 等流量	容量有限；不能假设数据总在 L2
Global memory / HBM	全 GPU	片外设备显存	模型参数、激活、KV、输入输出	高延迟；访问不合并会浪费带宽
Constant memory	全 grid 只读	设备内存 + 专用缓存	warp 内线程读取同一常量	访问地址高度分散时收益下降

6.2 Register：最快，但不是无限

源代码中的局部标量通常优先放寄存器：

```
float acc = 0.0f;
```

每个 thread 有自己的 `acc`。寄存器访问极快，也是 GEMM/Attention 把 tile 中间结果保存在片上的关键。

但寄存器消耗按 thread 乘开：

```
每 block 寄存器需求
≈ registers_per_thread × threads_per_block
```

如果每 thread 使用太多寄存器，一个 SM 能同时驻留的 block 就会减少。如果编译器无法分配足够寄存器，变量可能 spill 到 local memory；local memory 物理上通常在 HBM，代价远高于 register。

6.3 Shared memory：程序员管理的片上缓存

Shared memory 对同一个 block 的 thread 可见，典型模式：

```
所有 thread 合并读取 HBM 的一块 tile
→ 写入 shared memory
→ __syncthreads()
```

```
→ 每个 thread 多次复用 tile
→ 计算结果写回 HBM
```

这正是 tiled GEMM、卷积和 FlashAttention 的基础思想。

6.4 L1 / L2：硬件缓存，不等于共享变量

Cache 可以减少重复访问 HBM，但不能把“缓存可见”误解成“线程已经同步”。线程间正确通信仍需要：

- 正确的内存作用域；
- 原子操作或同步原语；
- 对应的 memory ordering / visibility 语义。

6.5 Global memory / HBM

CUDA 代码里的 global memory 是逻辑地址空间；在离散 GPU 上，其主要物理承载通常是 GDDR/HBM 设备显存。大模型中的参数、激活、梯度、优化器状态和 KV cache 大多长期驻留于这里。

HBM 有很高的吞吐带宽，但单次访问延迟仍高。GPU 依靠三种方式缓解：

1. 大量 warps 隐藏延迟；
2. warp 合并访问，提高每次事务的有效字节比例；
3. 用 register/shared/cache 提高数据复用，减少访问次数。

七、Global Memory Coalescing：相邻线程应访问相邻地址

7.1 SAXPY 的理想访问

一个 warp 的 32 个线程访问：

```
thread 0  → x[i+0]
thread 1  → x[i+1]
...
thread 31 → x[i+31]
```

float32 每元素 4 Byte，因此一个 warp 请求：

```
32 × 4 Byte = 128 Byte
```

对 compute capability 6.0 及之后的常见规则，可把它理解为覆盖若干个 32-Byte segment。若起始地址对齐，连续 128 Byte 理想情况下覆盖 4 个 segment：

```
4 segments × 32 Byte = 128 Byte
```

请求字节和实际搬运字节接近，效率高。

7.2 Stride 访问为什么浪费

若每个线程访问 `x[2 × i]`：

```
thread 0 → x[0]
thread 1 → x[2]
thread 2 → x[4]
...
```

相邻线程地址相差 8 Byte，但每次只使用其中 4 Byte。一个 warp 的 128 Byte 有用数据会散布在约 256 Byte 地址范围，理想效率约：

```
128 Byte useful ÷ 256 Byte transferred ≈ 50%
```

更大的 stride 可能让每个线程落到不同 segment，事务数急剧增加。

7.3 Requested 与 Actual Bandwidth

```
Load/Store Efficiency
≈ Requested bytes ÷ Actual transferred bytes
```

- Requested：程序真正用到的字节；
- Actual：内存系统为了完成请求实际搬运的字节。

两者差距大，通常意味着访问未合并、未对齐或读取了大量无用 cache line。

7.4 常用修复

- 让连续 thread 访问连续元素；
- 让最内层连续维映射到 `threadIdx.x`；
- 数据布局在 AoS 与 SoA 之间选择时，以 warp 访问模式为准；
- 对矩阵列访问/转置，先合并读入 shared memory，再在片上重排；

- block size 常选 32 的倍数，但最终仍需 profile。

八、Shared Memory Bank Conflict

Shared memory 被划分为多个 bank。对较新的常见 NVIDIA 架构，可用"32 个 bank、连续 32-bit word 映射到连续 bank"建立心智模型；具体模式仍应查对应架构文档。

对 float32 的简化映射：

```
bank_id = (address ÷ 4 Byte) mod 32
```

8.1 无冲突

warp 的 32 个线程访问 32 个不同 bank，可并行服务。

8.2 n-way conflict

多个线程访问同一个 bank 中的不同地址，访问需要拆分，吞吐可能近似降为原来的 `1/n`。注意：多个线程读取完全相同地址通常可广播，不按普通冲突处理。

8.3 经典转置例子

```
__shared__ float tile[32][32];
```

按列读取时，线程间步长是 32 个 float：

```
32 mod 32 = 0
```

不同线程可能都命中同一 bank，形成 32-way conflict。经典 padding：

```
__shared__ float tile[32][33];
```

此时：

```
33 mod 32 = 1
```

线程依次落到不同 bank，冲突消失。多出来的一列不是存业务数据，而是改变地址映射。

九、Occupancy：用于隐藏延迟，不是最终 KPI

Occupancy 定义：

```
Occupancy = active warps per SM ÷ maximum warps per SM
```

一个 block 的 warp 数：

```
warps_per_block = ceil(threads_per_block ÷ 32)
```

以 256 threads/block 为例：

```
warps_per_block = 256 ÷ 32 = 8 warps
```

教学近似下，能驻留的 block 数由最紧资源决定：

```
active_blocks_per_SM
≈ min(
    架构 block 上限,
    floor(SM thread 上限 ÷ threads_per_block),
    floor(SM register 总量 ÷ 每 block register 需求),
    floor(SM shared memory 总量 ÷ 每 block shared memory 需求)
  )
```

真实硬件会按特定分配粒度取整，准确结果应使用编译器报告、CUDA Occupancy API 或 Nsight Compute。

为什么 100% Occupancy 不保证最快

- memory-bound kernel 在已有足够 warps 隐藏延迟后，再提高 occupancy 可能无收益；
- 为提高 occupancy 强行减少寄存器，可能导致 spill，反而增加 HBM 流量；
- 减少 shared memory 可能失去数据复用，虽然 occupancy 上升但访问 HBM 更多；
- compute-bound kernel 可能更受 Tensor Core 利用率、指令依赖和 tile 形状影响。

正确目标是：

> 在不牺牲数据复用和制造 spill 的前提下，提供足够 ready warps 隐藏延迟。

十、Roofline：判断该优化计算还是优化内存

算术强度：

```
AI = FLOP ÷ Byte
单位：FLOP/Byte
```

Roofline 的简化上界：

```
可达到性能
≤ min(峰值计算吞吐，可达到内存带宽 × AI)
```

转折点：

```
AI_ridge = 峰值计算吞吐 ÷ 可达到内存带宽
```

- `AI < AI_ridge`：更可能 memory-bound；
- `AI > AI_ridge`：更可能 compute-bound。

SAXPY 的 `AI≈0.167 FLOP/Byte` 很低，因此优先优化：

- coalescing；
- 减少额外读写；
- 批量处理与融合；
- 避免不必要 dtype 转换；
- 检查真实 HBM throughput。

而大 GEMM 通过 tiling 让输入 tile 被反复使用，AI 随问题规模和 tile 复用提升，更可能进入 compute-bound 区域，此时重点变成 Tensor Core、矩阵 shape、数据类型、流水线与指令吞吐。

十一、有效带宽：如何量化 memory-bound kernel

有效带宽：

```
BW_effective = (Bytes_read + Bytes_written) ÷ kernel_time
```

单位检查：

```
Byte ÷ second = Byte/s
```

对本例：

```
有用数据量 = 12,582,912 Byte
```

若 CUDA Event 测得 kernel 时间为 `50 μs = 0.00005 s`：

```
BW_effective
= 12,582,912 Byte ÷ 0.00005 s
= 251,658,240,000 Byte/s
≈ 251.7 GB/s（十进制）
```

这个数字是“有用字节带宽”，不是实际 DRAM 搬运量。如果访问不合并，actual throughput 可能更高但有效带宽很低——说明总线在搬很多程序没用到的字节。

计时要注意：kernel launch 异步，不能只用普通 CPU 时钟包住 launch 而不做同步。优先使用 CUDA Event；教学测试也可在边界调用同步，但不要把大量同步留在生产热路径。

十二、同步：同步谁、保证什么

12.1 `__syncthreads()`

作用范围是一个 block：

- block 内所有参与线程到达 barrier 后才继续；
- 可用于 shared memory 的生产者/消费者阶段切换；
- 不能同步其他 block。

危险模式：只有部分线程进入 `__syncthreads()`，其他线程不进入，可能造成死锁或未定义行为。

12.2 Warp 级同步

Warp shuffle、vote、`__syncwarp()` 等用于 warp 内协作。现代 GPU 不应依赖“同 warp 天然锁步”来省略所有同步；使用 warp-level primitive 时仍要遵循其 mask 和内存可见性要求。

12.3 Kernel 边界

同一 stream 中，前一个 kernel 完成后后一个 kernel 才开始，因此拆 kernel 是常见的全 grid 同步方式。代价是额外 launch 和可能的中间数据写回 HBM。

12.4 Stream 与 Event

- 同一 stream 内按提交顺序执行；
- 不同 stream 的任务在无依赖时可以并发；
- Event 用来表达跨 stream 依赖；
- 不同 stream 能否真正重叠，还取决于资源是否冲突。

例如计算 kernel 和 NCCL kernel 虽在不同 stream，仍可能争抢 SM 或 HBM 带宽，所以时间线上有交叠不等于性能线性相加。

十三、最小 CUDA C++ 示例：SAXPY

```
#include <cuda_runtime.h>
#include <cstdio>

__global__ void saxpy(float a, const float* x, const float* y,
                      float* z, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        z[i] = a * x[i] + y[i];
    }
}

int main() {
    constexpr int N = 1 << 20;          // 1,048,576
    constexpr int threads = 256;
    const int blocks = (N + threads - 1) / threads;  // 4096
    const size_t bytes = N * sizeof(float);

    float *x, *y, *z;
    cudaMalloc(&x, bytes);
    cudaMalloc(&y, bytes);
    cudaMalloc(&z, bytes);

    saxpy<<<blocks, threads>>>(2.0f, x, y, z, N);
    cudaDeviceSynchronize();

    std::printf("blocks=%d threads/block=%d total_threads=%d\n",
                blocks, threads, blocks * threads);

    cudaFree(x);
    cudaFree(y);
    cudaFree(z);
}
```

这段代码展示的是线程映射，不是完整 benchmark：没有初始化输入、错误检查和 CUDA Event 计时。生产代码还应检查每个 CUDA API 和 kernel launch 的错误。

关键索引解释

```
blockIdx.x   当前 block 在 grid 中的编号
blockDim.x   每个 block 的 thread 数
threadIdx.x 当前 thread 在 block 内的编号
```

所以：

```
i = blockIdx.x × blockDim.x + threadIdx.x
```

把二维层级压成一维全局索引。

十四、PyTorch 视角：同一计算最终仍落到 CUDA Kernel

```
import torch

N = 1 << 20
torch.manual_seed(0)

x = torch.randn(N, device="cuda", dtype=torch.float32)
y = torch.randn(N, device="cuda", dtype=torch.float32)
a = 2.0

z = a * x + y
print(z.shape, z.dtype, z.device)
```

Eager 模式可能把乘法和加法分成多个 kernel，并产生中间 Tensor；`torch.compile`/Inductor 可能把它融合成一个 Triton/CUDA kernel。融合收益来自：

```
未融合：读 x → 写临时值 → 再读临时值和 y → 写 z
融合：读 x 和 y → register 中乘加 → 写 z
```

核心不是“少了一次 Python 调用”，而是**少了中间值的 HBM 往返和 kernel launch**。

十五、从 SAXPY 到矩阵乘：为什么 Shared Memory 能改变瓶颈

SAXPY 中每个输入只用一次，shared memory 很难创造复用；硬搬进 shared 反而多一道操作。

矩阵乘 `C = A × B` 不同。一个 A tile 和 B tile 会参与多次乘加：

```
合并从 HBM 读取 A/B tile
→ 放进 shared memory
→ block 内多个 thread 反复读取 tile
→ 每个 thread 在 register 中累加 C 的局部结果
→ 最终写回 HBM
```

这形成三层复用：

层级	保存内容	复用范围
HBM/L2	完整 A、B、C	全 GPU
Shared memory	当前 A/B tile	一个 block
Register	当前 thread 的累加器/fragment	单 thread

FlashAttention 也遵循相同总原则：通过 tile 和在线归一化把 Q/K/V 的局部块留在片上，避免把完整 `S×S` attention score matrix 写入 HBM。

十六、工程调优顺序：不要一上来就猜 block size

推荐顺序：

1. **确认正确性和 shape**：边界条件、对齐、dtype、索引是否正确。
2. **用 Profiler 定位热点**：先确认瓶颈确实在这个 kernel。
3. **计算 AI 和有效带宽**：判断 memory-bound 还是 compute-bound。
4. **检查 global access**：requested/actual throughput、coalescing、stride、对齐。
5. **检查片上复用**：是否可以用 register/shared 减少 HBM 流量。
6. **检查 shared bank conflict**：尤其是转置和二维 tile。
7. **检查资源压力**：register、shared memory、active blocks/warps、spill。
8. **检查 warp stall reason**：等待内存、执行依赖、同步还是指令供给。
9. **扫参数而不是猜参数**：block size、tile size、num warps、pipeline stages。

10. **回到端到端指标**：单 kernel 快不代表训练 step 一定快；还要看通信、CPU launch、同步和其他 kernel。

常用工具：

- Nsight Systems：看 CPU/GPU 时间线、stream、通信与计算 overlap；
- Nsight Compute：看单 kernel 的内存事务、occupancy、stall、吞吐；
- PyTorch Profiler：从框架算子关联到 CUDA kernel；
- 编译器报告：寄存器用量、spill、shared memory 使用量。

十七、常见误区

误区 1：Thread 是 GPU 的真实调度单位

不是。CUDA 暴露 thread 语义，但 NVIDIA GPU 以 warp 为基本调度单位。性能分析必须看 warp 的分支和访存模式。

误区 2：Local memory 在片上，所以很快

错误。Local 表示“线程私有的逻辑地址空间”，通常物理上位于设备显存，经 L1/L2 缓存访问；register spill 到 local memory 可能很贵。

误区 3：Shared memory 一定比 L1 快，所以都应该手动搬

错误。只有存在复用、重排或协作价值时才值得使用 shared。SAXPY 输入只用一次，额外搬运和同步可能使性能更差。

误区 4：Occupancy 越高越快

错误。Occupancy 是隐藏延迟的手段。若为提高 occupancy 导致寄存器 spill 或失去 shared tile 复用，性能会下降。

误区 5：Kernel launch 的 thread 数就是同时运行的物理线程数

错误。Grid 可以包含数百万逻辑 thread，但硬件只让受资源限制的一部分 block/warp 同时驻留，其他排队分批执行。

误区 6：有两个 stream 就一定能 overlap

错误。只有任务无数据依赖、设备支持并发、并且 SM/HBM/copy engine 等资源没有完全冲突时，才会出现有收益的真实 overlap。

十八、关键总结

把整篇压缩为一条因果链：

```
Kernel 把问题展开成大量逻辑 threads
→ threads 按 block 组织，block 被分配到 SM
→ 硬件以 warp 为调度单位执行
→ warp 遇到 HBM 延迟时切换到其他 ready warp
→ coalescing 决定一次内存事务有多少字节真正有用
→ shared/register 提高片上复用，减少 HBM 往返
→ 但 register/shared 用量过大会降低驻留 warp 数
→ 所以最终性能是“数据复用、访存效率、并行度、资源压力”的平衡
```

你应记住四个判断问题：

1. **映射**：每个 thread/block 到底处理哪块数据?
2. **访问**：同一个 warp 是否访问连续、对齐的地址?
3. **复用**：数据能否在 register/shared 中多用几次，少去 HBM?
4. **资源**：寄存器/shared 用量是否压低了 active warps，或造成 spill?

可选自测

1. 索引与 warp 题

`N=1,000,000`，`blockDim.x=256`：

- 需要多少个 block?
- 最后一个 block 有多少个有效 thread?
- 每个完整 block 有多少个 warp?

2. 带宽与算术强度题

对 float32 SAXPY `z=a*x+y`，若 `N=2²⁰`、kernel 时间 `40 µs`：

- 有用数据量是多少 MiB?
- 有效带宽是多少 GB/s（十进制）？
- 算术强度是多少 FLOP/Byte?
- 它更可能 memory-bound 还是 compute-bound?

3. 工程决策题

一个矩阵转置 kernel 的 Nsight Compute 显示：global load 基本合并，但 shared memory 有严重 32-way bank conflict；修改 padding 后 occupancy 从 75% 降到 62.5%。你是否应该保留 padding？还需要看哪些指标才能决定?

继续学习

1. 前置：GPU 微架构与 SM/Tensor Core —— 深入理解 warp 指令如何落到执行管线。
2. 同层对比：Triton 编程模型 —— 对比 CUDA 的 thread/block 与 Triton 的 program/block abstraction。
3. 下游应用：FlashAttention 的 IO-aware tiling —— 把 HBM、shared memory、register 和在线 softmax 连成真实大模型算子。

回复 1、2 或 3 继续；也可以说“稍后”或“不感兴趣”。

事实来源与边界

本文依据 NVIDIA CUDA Programming Guide 与 CUDA C++ Best Practices Guide 的稳定概念整理。warp 大小、常见 bank 模型和 coalescing 示例按当前 NVIDIA CUDA 文档说明；每个 SM 的最大线程数、寄存器总量、shared memory 容量、cache 组织和异步拷贝能力均随 compute capability 变化，生产调优必须以目标 GPU、CUDA 版本和 profiler 实测为准。

MoE 架构详解：从稀疏路由到 Expert Parallel

> **原始问题：**详细讲解 MoE 架构

提问：2026/07/29 20:27　更新：2026/08/05 16:26　重要度：5/5

MoE 架构详解：从稀疏路由到 Expert Parallel

一句话结论

MoE（Mixture of Experts，混合专家）是在 Transformer 的 FFN 位置放置多个独立 MLP Experts，再让一个可学习 Router 为每个 Token 只选择少数 Top-k Experts。

它的核心价值是：

```
总参数量由全部 E 个 Experts 决定
每 Token 计算量只由激活的 k 个 Experts 决定
且通常 k ≪ E
```

因此 MoE 可以让模型拥有非常大的参数容量，而不让每个 Token 都执行全部参数。

但 MoE 不是“免费扩大模型”：

- 所有 Expert 参数仍要存储、优化和加载；
- Router 可能把大量 Token 挤到少数 Experts，产生负载不均；
- Expert Parallel 需要两次 Token All-to-All；
- 每个 Expert 收到的 Token 数动态变化，容易形成小 GEMM 和 GPU 低利用率；
- 训练需要处理负载均衡、容量溢出、Token Drop 和 Router 稳定性；
- 推理解码阶段 Batch 小、Token 少，MoE 的通信和专家权重访问更难摊薄。

一句话记忆：

> **Dense 模型让所有 Token 使用同一组 FFN 参数；MoE 让不同 Token 动态选择不同 FFN 参数。它用“条件计算”换取“更大的参数容量”。**

一、MoE 在 Transformer 中放在哪里

标准 Transformer Block 通常包含：

```
Input
→ Attention
→ Residual
→ Dense FFN / MLP
→ Residual
→ Output
```

MoE 通常不替换 Attention，而是把某些层的 Dense FFN 替换为 Sparse MoE FFN：

```
Input
→ Attention
→ Residual
→ Router + Experts + Dispatch/Combine
→ Residual
→ Output
```

Dense FFN 对所有 Token 使用同一套参数：

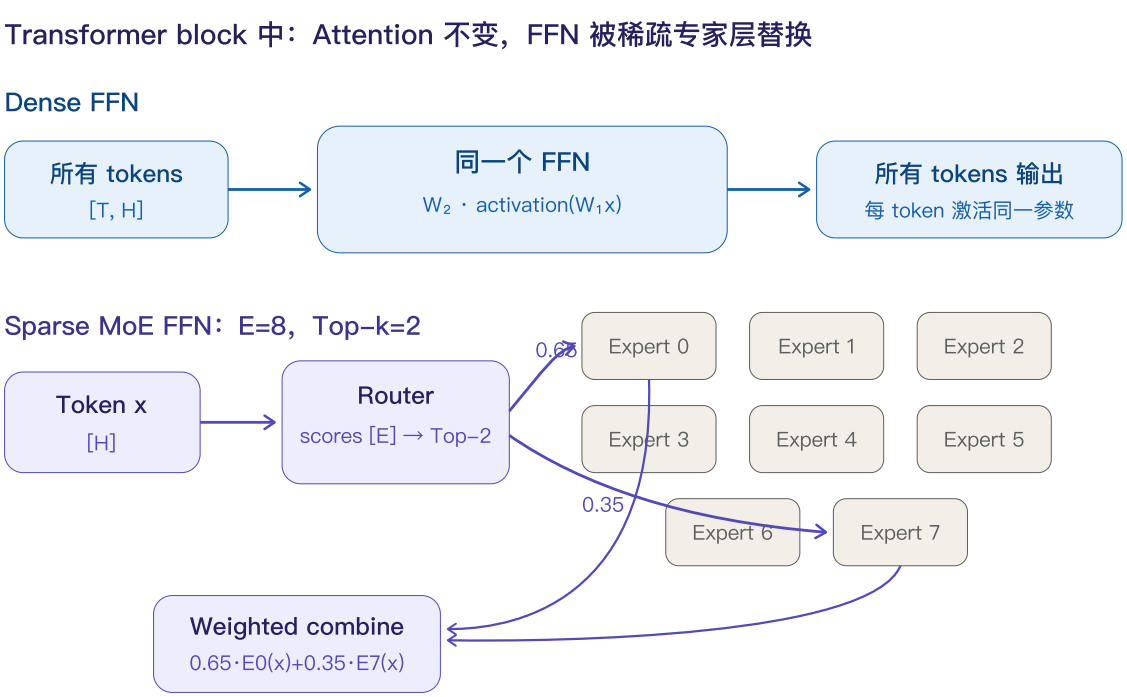
```
FFN(x) = W₂ · activation(W₁x)
```

MoE 对 Token `x_t` 先选择专家集合 `S_t`：

```
MoE(x_t) = Σ_{e∈S_t} g_{t,e} · Expert_e(x_t)
```

其中：

```
E        Expert 总数
k        每个 Token 激活的 Expert 数
S_t      Token t 选中的 Top-k Expert 集合
g_{t,e}  Token t 对 Expert e 的路由权重
```



总参数可含 8 个专家，但该 token 只执行 2 个专家：参数容量与每 token 计算量解耦。

MoE 不是什么

- 不是 Ensemble：不是把多个完整模型都跑一遍再投票；
- 不是 Data Parallel：不同 Experts 是不同参数，不是同一模型副本；
- 不是简单模型分片：Router 会按 Token 动态决定执行路径；
- 不是减少总参数显存：总 Expert 参数可能远大于 Dense 模型；
- 不是所有层都必须 MoE：常见架构会按固定频率插入 MoE 层，其他层仍为 Dense。

二、统一例子：全文共用的 Shape 与参数

为了让结构、公式和代码完全对应，使用一个小型可手算例子：

```
Batch B = 2
Sequence S = 3
Token 数 T = B × S = 6
Hidden Size H = 8
Expert 总数 E = 4
Top-k = 2
每个 Expert 的中间维度 F_expert = 16
对照 Dense FFN 的中间维度 F_dense = 32
Dtype = float32（代码示例）
```

输入 Hidden States：

```
X: [B, S, H] = [2, 3, 8]
Flatten 后: X_flat: [T, H] = [6, 8]
```

Router 权重：

```
W_r: [H, E] = [8, 4]
Router Logits: X_flat @ W_r = [6, 4]
```

每个 Token 选择 2 个 Experts：

```
Top-k Indices: [T, k] = [6, 2]
Top-k Weights: [T, k] = [6, 2]
```

总 Token-Expert Assignments：

```
A = T × k = 6 × 2 = 12
```

为什么设置 `F_dense=32`、`F_expert=16`？

简单两层 MLP 每 Token FLOP 近似与中间维度成正比。Top-2 会执行两个 `F=16` 的 Experts：

```
2 × 16 = 32
```

因此它与一个 `F_dense=32` 的 Dense FFN 具有接近的主 MLP 计算量，便于比较参数容量与激活计算。

三、Dense FFN 与 MoE 的参数量、激活参数和 FLOP

先使用最简单的两层 MLP，忽略 Bias：

```
Expert(x) = W₂ · activation(W₁x)
W₁: [H, F]
W₂: [F, H]
```

3.1 Dense FFN 参数量

```
P_dense = H×F_dense + F_dense×H
        = 2HF_dense
```

代入：

```
P_dense = 2 × 8 × 32
        = 512 params
```

3.2 MoE 全部 Expert 参数量

单个 Expert：

```
P_one_expert = 2HF_expert
             = 2 × 8 × 16
             = 256 params
```

4 个 Experts：

```
P_all_experts = E × 2HF_expert
              = 4 × 256
              = 1024 params
```

Router：

```
P_router = H × E
         = 8 × 4
         = 32 params
```

MoE 总参数：

```
P_moe_total = 1024 + 32 = 1056 params
```

3.3 每 Token 激活的参数量

Top-2 每 Token 只执行两个 Experts：

```
P_active_experts_per_token
= k × 2HF_expert
= 2 × 256
= 512 params
```

加上 Router 投影约 32 个参数参与计算：

```
P_active_per_token ≈ 544 params
```

因此这个例子中：

指标	Dense FFN	MoE FFN
总参数	512	1056
每 Token 主 MLP 激活参数	512	512
Router 参数	0	32
Expert 数	1	4
每 Token 激活 Experts	1	2

结论：**主 MLP 计算近似不变，但总参数容量约扩大 2 倍。**

如果保持每个 Expert 和 Dense FFN 同样宽，同时使用 Top-2，那么每 Token Expert 计算会约变成 Dense FFN 的 2 倍。MoE 是否“同计算量”取决于：

```
k × F_expert 是否接近 F_dense
```

不能只看 $k \ll E$ 就断言计算完全不变。

3.4 FLOP 估算

矩阵乘的 Multiply-Add 按 2 FLOP 计。两层简单 MLP 每 Token：

```
FLOP_one_expert ≈ 4HF_expert
```

Top-k：

```
FLOP_moe_experts_per_token ≈ 4kHF_expert
```

Router：

```
FLOP_router_per_token ≈ 2HE
```

代入：

```
Expert FLOP ≈ 4 × 2 × 8 × 16 = 1024 FLOP/token
Router FLOP ≈ 2 × 8 × 4 = 64 FLOP/token
```

Dense 对照：

```
FLOP_dense ≈ 4 × 8 × 32 = 1024 FLOP/token
```

因此：

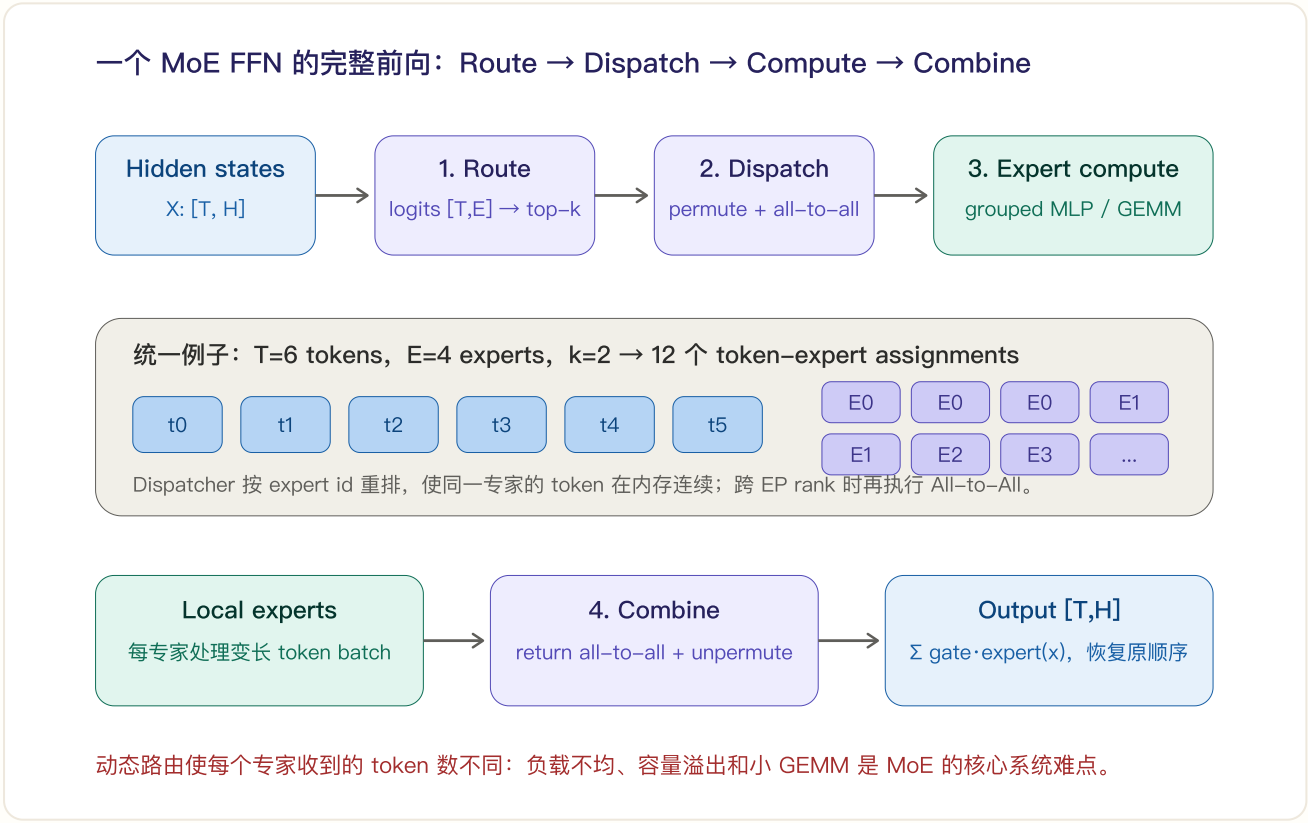
```
主 Expert MLP FLOP ≈ Dense MLP FLOP
MoE 额外增加 Router、Permutation 和通信成本
```

现代 Transformer 常使用 SwiGLU/Gated MLP，包含 3 个线性投影，参数量和 FLOP 系数会从 `2HF/4HF` 变为大约 `3HF/6HF`；但"总参数按 E 增长、激活计算按 k 增长"的关系不变。

四、MoE Layer 的四个阶段

一个 MoE Layer 的完整 Forward 可拆成：

```
1. Route
2. Dispatch
3. Expert Compute
4. Combine
```



4.1 Route：为每个 Token 选择 Experts

输入：

```
X: [T, H]
```

Router 是一个小型线性投影：

```
L = XW_r

X:   [T, H]
W_r: [H, E]
L:   [T, E]
```

将 Logits 转成 Scores。经典形式：

```
P = softmax(L, dim=-1)
```

然后选择 Top-k：

```
indices, weights = topk(P, k)
```

通常会对选中的 `k` 个权重重新归一化：

```
g_{t,e} = p_{t,e} ÷ Σ_{j∈S_t} p_{t,j}
```

于是：

```
Σ_{e∈S_t} g_{t,e} = 1
```

部分架构使用 Sigmoid Score、Group-Limited Routing、Expert Bias 或 Aux-Loss-Free Balancing；不能把所有 MoE 都等同于“全局 Softmax + Top-2”。

4.2 Dispatch：把 Token 发给目标 Experts

Router 产生的是 Token→Expert Assignment。Expert 计算要求同一个 Expert 的 Token 在内存中连续，因此先 Permute：

```
原顺序：t0, t1, t2, t3, t4, t5
按 Expert 分组后：
E0 的 tokens | E1 的 tokens | E2 的 tokens | E3 的 tokens
```

如果所有 Experts 在同一 GPU，只需本地重排。

如果 Experts 分布在不同 GPU，需要执行 Expert Parallel 通信：

```
Dispatch All-to-All
```

把每个 Token Assignment 发送给目标 Expert 所在的 Rank。

4.3 Expert Compute：每个 Expert 执行 MLP

Expert `e` 收到：

```
X_e: [T_e, H]
```

其中 `T_e` 是动态的，每个 Expert 不同。

执行：

```
Y_e = Expert_e(X_e)
```

由于 `T_e` 可能很小、差异很大，如果逐 Expert 启动独立 GEMM，会产生：

- 大量小 GEMM；
- Kernel Launch 开销；
- Tensor Core 利用率低；
- 最慢 Expert/Rank 决定整体延迟。

因此生产实现常使用 Grouped GEMM：把多个不同 `T_e` 的 Expert GEMM 组织在一次或少量 Kernel 中执行。

补充一：Grouped GEMM 到底怎样执行

先给结论：**Grouped GEMM 不是把所有 Expert 的权重拼成一个共享权重大矩阵，也不是让不同 Expert 互相混算；它是在一次或少量 GPU Kernel Launch 中，调度一组彼此独立、M 维不同但通常 K/N 相同的矩阵乘法。** 每个 Expert 仍使用自己的权重，输出也仍属于自己的 Token 段。

1. 每个 Expert 实际有哪两个 GEMM

以普通两层 Expert MLP 为例：

```
X_e      [T_e, H]
W1_e     [H, F]
A_e      [T_e, F] = activation(X_e × W1_e)
W2_e     [F, H]
Y_e      [T_e, H] = A_e × W2_e
```

如果是 SwiGLU，第一层通常包含 Gate/Up 两个投影；工程上可以把两个投影融合成一个更宽的 GEMM，再执行逐元素 `SiLU(gate) × up`，最后做 Down Projection。

这里最麻烦的是 `T_e` 动态变化。假设一张 GPU 上有 3 个本地 Experts：

```
T_0 = 2, T_1 = 5, T_2 = 1
H = 4, F = 8
```

```
GEMM 0: [2,4] × [4,8] → [2,8]
GEMM 1: [5,4] × [4,8] → [5,8]
GEMM 2: [1,4] × [4,8] → [1,8]
```

若逐 Expert 调用 3 次 GEMM，GPU 会看到 3 个很小的 M 维问题。每次都要付 Kernel Launch、参数装载、调度和尾块浪费，Tensor Core 很难吃满。

2. Grouped GEMM 前先做 Token Permutation

Router/Dispatcher 先把属于同一 Expert 的 Token 排到连续内存中：

```
原 assignment 顺序:
[t0→E1, t1→E0, t2→E1, t3→E2, t4→E1, t5→E0, ...]

按 expert_id 排序后:
[E0: t1,t5] [E1: t0,t2,t4,...] [E2: t3]

packed_X:       [Σ_e T_e, H]
expert_offsets: [0, T_0, T_0+T_1, T_0+T_1+T_2, ...]
```

`expert_offsets` 是前缀和。第 `e` 个 Expert 的输入视图为：

```
X_e = packed_X[expert_offsets[e] : expert_offsets[e+1], :]
```

因此不需要为每个 Expert 单独分配不规则 Tensor；一个大 Buffer 加一组 Offset 就能描述全部变长输入。

3. 一个 Grouped GEMM Kernel 内部如何调度

Kernel 启动前会准备每组 GEMM 的描述符：

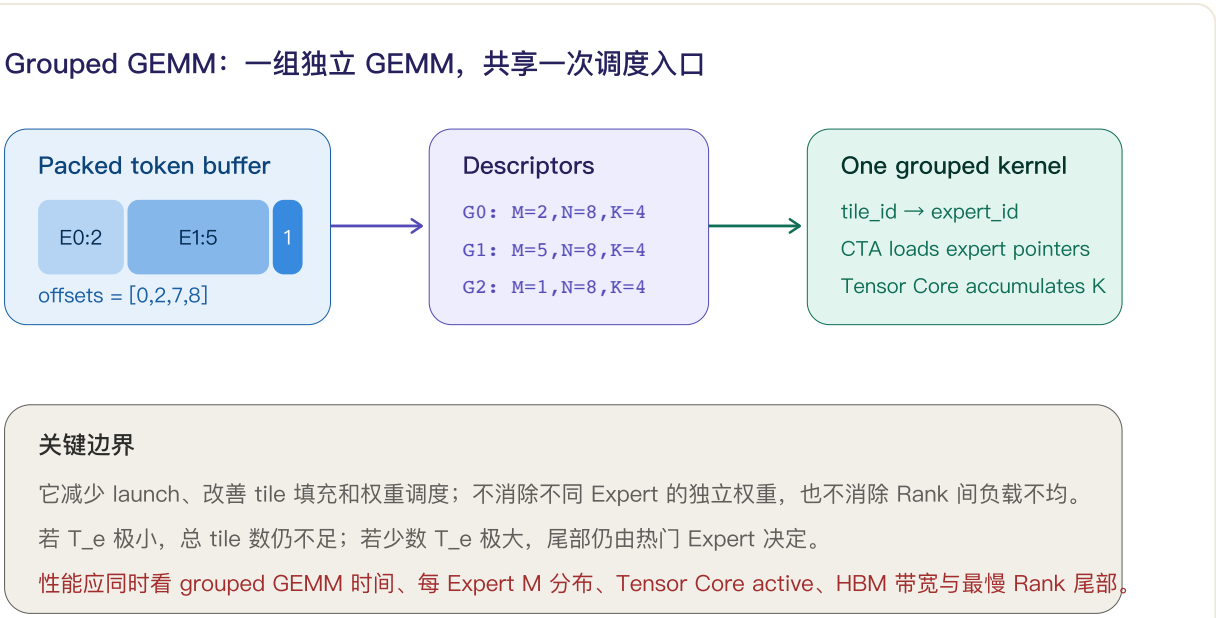
```
group e:
  A_ptr = X_e 首地址
  B_ptr = W_e 首地址
  C_ptr = Y_e 首地址
  M_e   = T_e
  N     = F
  K     = H
  lda / ldb / ldc
```

Kernel 不把三组矩阵数学上合并，而是把每组 GEMM 再切成 Tensor Core Tile，例如 `[M_tile, N_tile, K_tile]`。全局 Tile Scheduler 维护“第几个 Tile 属于哪一个 Expert”的映射：

```
Expert e 的输出 tile 数
tiles_e = ceil(T_e/M_tile) × ceil(F/N_tile)

prefix_tiles[e+1] = prefix_tiles[e] + tiles_e
```

一个 CTA/Thread Block 取得全局 `tile_id` 后，通过 `prefix_tiles` 找到所属 Expert，再加载该 Expert 的 `A_ptr/B_ptr/C_ptr` 和 shape，在自己的输出 Tile 上执行 K 维累加。Persistent Kernel 还会让同一个 CTA 连续领取多个 Tile，进一步摊薄 Launch 和调度开销。



4. 为什么它更快，以及它不能解决什么

Grouped GEMM 的主要收益：

- 将多次小 GEMM Launch 合并为一次或少量 Launch；
- 把多个 Expert 的 Tile 放入同一调度池，增加可并行 CTA 数；
- 统一处理指针、shape、epilogue，减少 Host/Device 调度开销；
- 对变长 `T_e` 不必 Pad 到统一 Capacity，减少无效 FLOP。

但它**不能**解决：

- Rank 0 总共收到 2000 Tokens、Rank 1 只收到 500 Tokens 的跨 Rank 不均衡；
- 热门 Expert 的 `T_e` 远大于其他 Expert，形成计算长尾；
- `T_e` 全都极小，总工作量仍不足以填满 GPU；
- Permute/Unpermute 占用 HBM 带宽；
- Grouped GEMM 与通信同时运行时争抢 SM、L2 和 HBM。

Backward 也有对应的 Grouped GEMM：用分组矩阵乘计算 `dX_e`、`dW1_e`、`dW2_e`。因此训练优化不能只看 Forward Kernel。

4.4 Combine：返回原 Rank，恢复顺序并加权

Expert 输出先通过第二次通信返回 Token 原始 Rank：

```
Combine All-to-All
```

然后 Unpermute 恢复原 Token 顺序，并按 Router 权重合并：

```
y_t = Σ_{e∈S_t} g_{t,e} · Expert_e(x_t)
```

最终输出 Shape 恢复为：

```
Y: [T,H] → [B,S,H]
```

五、手算一个完整 Top-2 Routing 例子

设 6 个 Token 对 4 个 Experts 的 Top-2 结果如下：

Token	Expert 1	Weight 1	Expert 2	Weight 2
t0	E0	0.65	E3	0.35
t1	E1	0.70	E0	0.30
t2	E2	0.60	E3	0.40
t3	E0	0.55	E2	0.45
t4	E3	0.80	E1	0.20
t5	E1	0.52	E2	0.48

每 Expert Token Assignment 数：

```
E0: t0, t1, t3 → 3
E1: t1, t4, t5 → 3
E2: t2, t3, t5 → 3
E3: t0, t2, t4 → 3
```

总 Assignment：

```
3 + 3 + 3 + 3 = 12 = T×k
```

Token `t0` 输出：

```
y_0 = 0.65·Expert_0(x_0) + 0.35·Expert_3(x_0)
```

同一个 Token 被复制成两个 Expert Assignments，但最终仍合并回一个 `[H]` 输出。

这个例子完全均衡。真实 Router 很容易出现：

```
E0: 6 assignments
E1: 3
E2: 2
E3: 1
```

此时 E0 变成热点，其他 Experts 空闲。

六、Expert Capacity、Capacity Factor 与 Token Dropping

6.1 平均容量

总 Assignment 数：

```
A = T×k
```

均匀分给 `E` 个 Experts，每 Expert 平均收到：

```
A_avg = T×k ÷ E
```

容量通常定义为：

```
Capacity C
= ceil(T×k×CapacityFactor ÷ E)
```

Megatron Core 文档给出的同类公式以每 Rank Token 数为口径：

```
capacity
= num_tokens_per_rank × topk × capacity_factor ÷ num_experts
```

具体是全局还是局部 Token 口径、是否向上取整、容量发生在通信前还是后，要以框架实现为准。

统一例子：

```
T=6, k=2, E=4, CapacityFactor=1.0
C=ceil(6×2÷4)=3
```

均衡路由时每 Expert 正好 3 个 Assignments，不溢出。

若 E0 收到 6 个：

```
overflow = 6-3 = 3
```

6.2 Token Dropping

Capacity-Based MoE 会只保留容量内的 Token Assignments，超出的 Assignment 被 Drop，常按 Router Probability 优先保留高分项。

风险：

- 被 Drop 的 Expert 路径不执行；
- 若 Token 的所有路径都被 Drop，会造成信息损失；
- Capacity 太小会影响模型质量；
- Capacity 太大则增加 Padding、显存和通信。

6.3 Padding to Capacity

为了得到固定 GEMM Shape，可以把不足容量的 Expert 输入补齐到 `C`：

```
Expert 真实收到 1 个 Token
Capacity = 3
→ Padding 到 3
```

优点：

- Shape 固定；
- Kernel 更规则；
- CUDA Graph 更容易；
- 集合通信尺寸可预测。

缺点：

- 计算 Padding；
- 浪费显存与带宽；
- Capacity Factor 越大，浪费越明显。

6.4 Dropless MoE

Dropless 实现不丢 Token，也不强制全部 Padding 到统一容量。每个 Expert 处理真实动态 Token 数。

优点：保留所有路由信息。

代价：

- 动态 Shape；
- Expert Token 数高度不均；
- Grouped GEMM 和通信实现更复杂；
- CUDA Graph、内存规划和负载均衡更难。

七、为什么 Router 会塌缩：负载均衡问题

如果某个 Expert 在训练早期偶然更擅长一些 Token，Router 会给它更高分；它获得更多数据，更新更充分，于是进一步变强：

```
更高路由概率
→ 更多 Token
→ 更多梯度和训练机会
→ Expert 更强
→ 更高路由概率
```

这会形成正反馈，导致：

- 少数 Experts 过载；
- 其他 Experts 几乎没有训练数据；
- All-to-All Rank 不均衡；
- 最慢 Rank 拖住全组；
- 容量溢出和 Token Drop 增加；
- Expert Specialization 退化。

八、Load Balancing Auxiliary Loss

一种经典思路同时约束：

1. 每个 Expert 实际分到的 Assignment 比例；
2. Router 给每个 Expert 的平均概率质量。

定义：

```
f_e = Expert e 实际接收的 Assignment 数 ÷ (T×k)
p_e = (1/T) × Σ_t P_{t,e}
```

补充二：`p_e` 是怎样算出来的

先给结论：`p_e` **是一个 Batch 内，Router 在 Softmax 后分给 Expert** `e` **的平均概率质量；它不是 Expert 的实际 Token 个数。** `f_e` 是离散硬路由后的实际负载，`p_e` 是可微的软倾向。

常见定义为：

```
P_{t,e} = softmax(router_logits_t)[e]
p_e     = (1/T) × Σ_t P_{t,e}
```

这里通常使用 **Top-k 截断之前、对全部 E 个 Experts 的 Softmax 概率**。因此：

```
对每个 token t: Σ_e P_{t,e} = 1
对整个 batch:   Σ_e p_e = 1
```

一个 4 Token、3 Expert 的简单例子

假设 Router Softmax 后的概率是：

Token	P(E0)	P(E1)	P(E2)	Top-1 选择
t0	0.70	0.20	0.10	E0
t1	0.60	0.30	0.10	E0
t2	0.20	0.70	0.10	E1
t3	0.10	0.60	0.30	E1

逐列求平均：

```
p_0 = (0.70 + 0.60 + 0.20 + 0.10) ÷ 4 = 0.40
p_1 = (0.20 + 0.30 + 0.70 + 0.60) ÷ 4 = 0.45
p_2 = (0.10 + 0.10 + 0.10 + 0.30) ÷ 4 = 0.15

检查：p_0 + p_1 + p_2 = 1.00
```

Top-1 硬选择结果是 `[E0, E0, E1, E1]`，所以：

```
f_0 = 2/4 = 0.50
f_1 = 2/4 = 0.50
f_2 = 0/4 = 0.00
```

这说明 `f` 与 `p` 的含义不同：E2 没有真正接到 Token，所以 `f_2=0`；但 Router 仍给过 E2 一些概率，所以 `p_2=0.15`。

代入常见辅助损失的归一化主体：

```
E × Σ_e f_e p_e
= 3 × (0.50×0.40 + 0.50×0.45 + 0×0.15)
= 3 × 0.425
= 1.275
```

完全均匀时该主体为 1；这里大于 1，表示实际负载与概率质量共同偏向 E0/E1。最终损失还要乘 `α`。

为什么同时需要 `f_e` 和 `p_e`

- 只看 `f_e`：它来自 Top-k/argmax，离散、几乎不可导，难以直接训练 Router。
- 只看 `p_e`：概率看似均匀，不代表 Top-k 后的硬分配真的均匀。
- 二者相乘：把“实际拥挤程度”作为系数，惩罚 Router 继续给拥挤 Expert 分配概率质量。

在把硬计数 `f_e` 视为当前 Batch 的常量时，`L_aux` 对 `p_e` 的系数与 `f_e` 成正比：越拥挤的 Expert，继续增加其概率的代价越大。Softmax 的耦合梯度会把概率质量转向负载较低的 Experts。

Top-2 时怎样改

若每个 Token 选择 `k=2`，总 Assignment 数为 `T×k`，所以常见硬负载定义是：

```
f_e = count(assignments to e) ÷ (T×k)
```

但不同论文/框架可能使用：Top-k 前概率、Top-k 后重新归一化权重、按 Sequence/Group 局部统计，或给每个 k 槽位分别算损失。阅读实现时必须确认统计轴和分母，不能只看变量名 `p_e`。

一个常见形式：

```
L_aux = α × E × Σ_e f_e p_e
```

其中：

```
α  Auxiliary Loss 系数
E  Expert 数
```

若完全均匀：

```
f_e = 1/E
p_e = 1/E

E × Σ_e (1/E × 1/E)
= E × E × 1/E²
= 1
```

训练时通常把它加到主 Loss：

```
L_total = L_task + L_aux + L_z + ...
```

不同论文和框架对 Top-k、归一化和比例系数的具体定义可能不同，不能看到“Aux Loss”就假设公式完全一致。

Aux Loss 的副作用

Aux Loss 太小：负载不均控制不住。

Aux Loss 太大：Router 为了均匀而均匀，可能损害语义路由与 Expert Specialization。

所以负载均衡与专家专门化存在张力：

```
绝对均匀不一定是质量最优
严重不均一定是系统灾难
```

现代系统还会使用：

- Expert Bias 动态调节；
- Aux-Loss-Free Balancing；
- Group-Limited Routing；
- Node-Limited Routing；
- Sinkhorn/S-BASE 等方法。

九、Router Z-Loss 与数值稳定性

Router Logits 可能持续增大，使 Softmax 饱和、梯度和低精度训练不稳定。

常见 Router Z-Loss：

```
L_z = λ_z × mean_t[(log Σ_e exp(l_{t,e}))²]
```

它鼓励 Logits 不要无界增大。

还可能使用：

- Router 使用 FP32 计算；
- Input Jitter/Noise；

- Router Weight 初始化控制；
- Logit Clipping；
- 更稳定的 Softmax/Fused Kernel。

Router 参数很少，但它决定每个 Token 走哪条计算路径，因此数值问题可能放大成整个系统的路由异常。

补充三：Router Logits、Softmax 饱和与 Z-Loss

1. “Logits 持续增大”要拆成两种情况

这句话需要精确理解：

1. **所有 Logits 一起加同一个常数**：`l_t ← l_t + c`。Softmax 完全不变，因为 Softmax 具有平移不变性。
2. **Logit 之间的差值持续放大**：例如 `[2,1,0] → [20,10,0]`。Softmax 会越来越接近 One-hot，这才是通常说的“饱和”。

```
softmax(l + c) = softmax(l)
```

因此主任务 Loss 可能根本不在意共同偏移量 `c`，Router Logits 的绝对值可以漂移；同时路由竞争又可能拉大专家之间的差值。两类现象都会让数值尺度变差，但只有第二类直接让概率饱和。

2. Softmax 饱和为什么让梯度变小

Softmax Jacobian 为：

```
∂p_i/∂l_j = p_i × (δ_ij − p_j)
```

若某个 Expert 概率 `p_i≈1`：

```
p_i(1−p_i) ≈ 0
```

其他 Experts 的 `p_j≈0`，对应梯度也很小。于是 Router 很难把一个已经极度自信的错误选择拉回来。

数值例子：

```
logits = [2, 1, 0]
softmax ≈ [0.665, 0.245, 0.090]
最大项局部斜率 ≈ 0.665×0.335 = 0.223

logits = [20, 10, 0]
```

```
softmax ≈ [0.999955, 0.000045, 0.000000002]
最大项局部斜率 ≈ 0.000045
```

后者梯度约小了 5000 倍。Router 一旦早期偏向某个 Expert，就更容易形成“越选越强、越强越难纠正”的正反馈，最终出现 Expert Collapse 或负载长尾。

3. 为什么低精度训练更容易不稳定

Softmax 要计算 `exp(logit)`。朴素实现对大正数可能上溢、对大负数可能下溢。稳定实现会先减去最大值：

```
m = max_e l_e
logsumexp(l) = m + log(Σ_e exp(l_e − m))
```

这能避免大部分直接 Overflow，但不能消除所有问题：

- BF16/FP16 的有效尾数有限，小概率可能舍入到 0；
- 饱和后 Top-k 边界对微小舍入误差敏感，临界 Experts 的顺序可能翻转；
- Fused Softmax/Top-k/Noise/Mask 的中间值若使用低精度，误差会积累；
- Router 的微小数值差异会变成离散路径差异，进一步改变每个 Rank 的 Token 数和通信形状；
- 路由异常会被 All-to-All、Capacity、Token Drop 和尾延迟放大为系统问题。

这就是“Router 参数很少，但数值问题可能放大成整个系统异常”的含义。

4. Z-Loss 公式逐项解释

对 Token `t`，定义：

```
z_t = logsumexp(l_t)
    = log(Σ_e exp(l_{t,e}))

L_z = λ_z × mean_t(z_t²)
```

变量含义：

```
l_{t,e}  Token t 对 Expert e 的 Router Logit，无量纲
z_t      该 Token 的 Log-Partition，衡量全部 exp(logit) 的总尺度
λ_z      Z-Loss 系数，无量纲
T        本次统计的 Token 数
```

`z_t` 不是熵，也不是最大概率。它是 Softmax 分母的对数。对全部 Logits 加常数 `c` 时：

```
logsumexp(l_t + c) = logsumexp(l_t) + c
```

所以虽然 Softmax 概率不变，Z-Loss 却会变化。这正好约束了主路由目标看不见的“共同偏移自由度”。

5. 一个最能说明 Z-Loss 的例子

```
A: logits = [2,1,0]
   softmax ≈ [0.665,0.245,0.090]
   z = log(exp(2)+exp(1)+exp(0)) ≈ 2.408
   z² ≈ 5.80

B: logits = [12,11,10] = A + 10
   softmax 与 A 完全相同
   z ≈ 12.408
   z² ≈ 153.96
```

对主路由概率而言 A/B 等价；对数值尺度而言 B 明显更差。Z-Loss 会强烈惩罚 B，把共同偏移压回来。

再看差值放大的情况：

```
C: logits = [20,10,0]
   softmax 几乎 One-hot
   z ≈ 20.000
   z² ≈ 400
```

C 同时具有很大的绝对尺度和饱和概率，也会受到很强惩罚。

6. Z-Loss 的梯度到底在做什么

若 Batch 有 `T` 个 Tokens：

```
∂L_z/∂l_{t,e}
= (2λ_z/T) × z_t × softmax(l_t)_e
```

因为：

```
∂logsumexp(l_t)/∂l_{t,e} = softmax(l_t)_e
```

含义是：

- 当 `z_t>0` 时，梯度下降会把 Logits 向下推，概率越大的 Expert 推得越多；
- 当 `z_t<0` 时，会把 Logits 向上推；
- 最终倾向让 `logsumexp(l_t)` 靠近 0，使 `Σ exp(logit)` 的尺度靠近 1。

注意：若所有 Logits 都是 0，则 `z=log(E)>0`，因此 Z-Loss 会把它们整体推向负方向；这不会改变 Softmax 的均匀分布，只是在选择一个数值更温和的绝对标尺。

7. Z-Loss 能做什么，不能做什么

Z-Loss 主要控制 **Logit 的绝对尺度/Log-Partition**，不是负载均衡 Loss 的替代品。它不能单独保证：

- 每个 Expert 收到相同 Token 数；
- Router 保持足够熵；
- 不发生 Top-k 边界抖动；
- 所有低精度 Kernel 都数值稳定。

因此常配合：Router/Softmax 用 FP32、合理初始化、稳定 `logsumexp`、必要时 Logit Clipping、Input Jitter，以及单独的负载均衡策略。`λ_z` 太大会过度限制 Router 表达能力，必须结合 `max|logit|`、`logsumexp`、Router Entropy、Aux/Z Loss 占比和训练质量一起调。

十、Expert Parallelism：Experts 如何分到多张 GPU

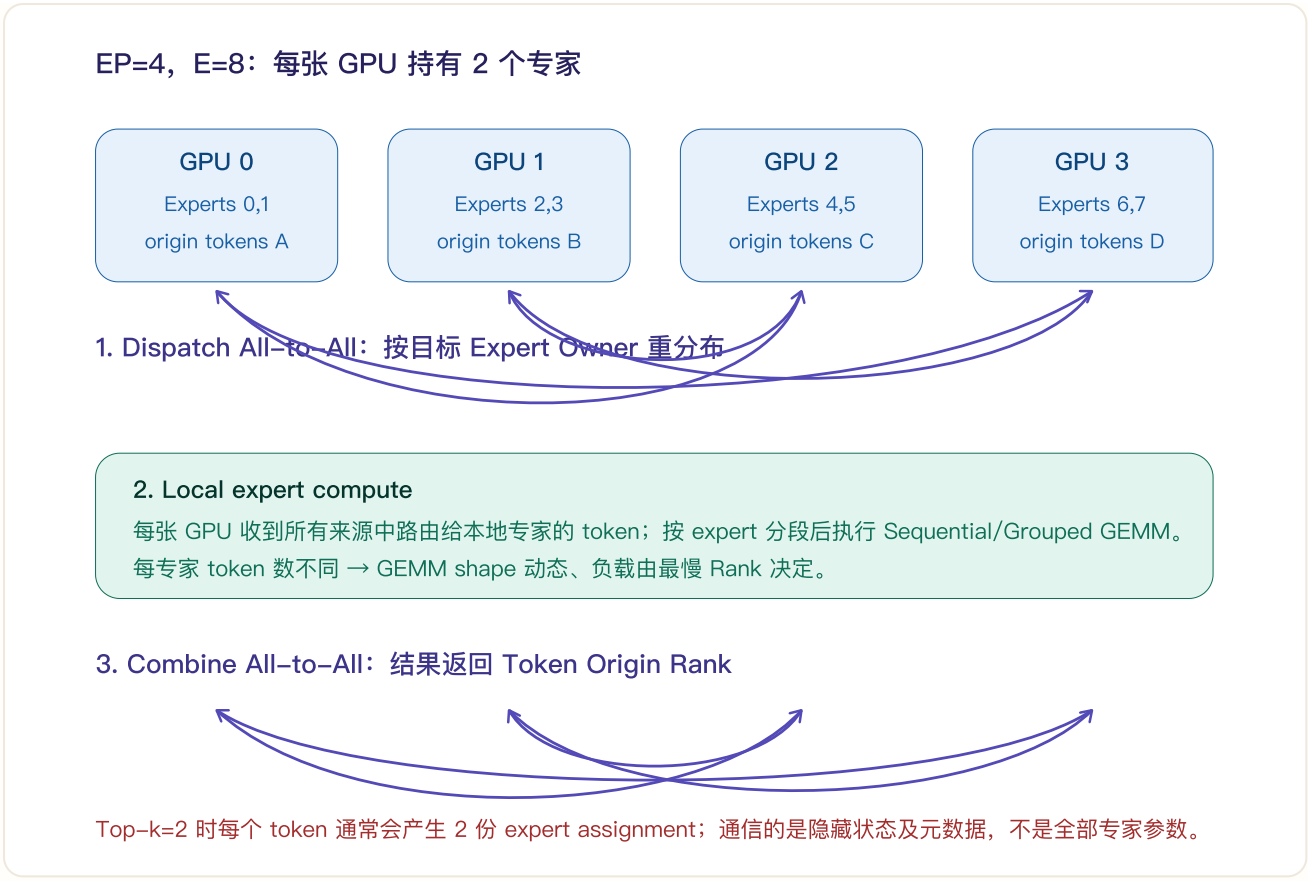
当 Expert 总参数无法或不应复制到每张 GPU 时，使用 Expert Parallel（EP）。

统一并行例子：

```
E = 8 Experts
EP = 4 GPUs
每 GPU Local Experts = E ÷ EP = 2
```

映射：

```
GPU 0: Experts 0,1
GPU 1: Experts 2,3
GPU 2: Experts 4,5
GPU 3: Experts 6,7
```



一个 Rank 上的本地 Token 可能被 Router 选到任意 Expert。因此 Forward 需要：

```
本地 Tokens
→ Router
→ 按 Expert Owner 分组
→ Dispatch All-to-All
→ Local Expert Compute
→ Combine All-to-All
→ 恢复原 Token 顺序
```

10.1 第一次 All-to-All：Dispatch

每个 Rank 把 Token Hidden State 发送到目标 Expert 所在 Rank。

发送的是：

- Hidden State；
- 路由权重；
- Token/Expert 索引或恢复顺序所需元数据。

不是把全部 Expert 参数来回发送。

10.2 第二次 All-to-All：Combine

Expert 计算完成后，结果返回 Token 原始 Rank，再进行 Unpermute 与 Weighted Sum。

Backward 会沿相反的数据依赖执行对应的通信与梯度计算，因此训练通信量比 Forward-only 更大。

补充四：两次 All-to-All、Backward 与通信计算 Overlap

先给结论：**MoE 的两次 All-to-All 是 Token Assignment 的“去程”和 Expert 输出的“回程”。Forward 有 Dispatch + Combine 两次；Backward 沿数据依赖再执行两次对应通信，因此训练时每个 MoE Layer 通常是 4 次大 Payload 交换。** 真正的 Overlap 优化不是简单把通信放到另一条 Stream，而是要拆出独立 Chunk、明确事件依赖，并控制通信 Kernel 与 GEMM 对 SM/HBM 的争用。

1. 第一次 All-to-All：Dispatch 的完整数据流

每个 Rank 初始拥有本地 Tokens：

```
X_local: [T_local, H]
```

Top-k 路由产生：

```
expert_id:   [T_local, k]
gate_weight: [T_local, k]
assignments: A_local = T_local × k
```

Dispatcher 对每个 Assignment 计算：

```
dst_rank      = owner_rank(expert_id)
local_expert  = expert_id 在目标 Rank 内的编号
origin_token  = 原 Token 行号
slot          = Top-k 的第几个槽位
```

然后执行五步：

1. 统计发给每个 Peer 的 Assignment 数 `send_counts[peer]`；
2. 对 Counts 做 Prefix Sum，得到每个 Peer 在 Send Buffer 中的区间；
3. Permute/Pack Hidden State，使发往同一 Peer 的数据连续；
4. 若是 Dropless/变长协议，交换 Split Sizes，使接收方知道 `recv_counts`；
5. 执行 All-to-All/All-to-All-v，把 Hidden States 发到 Expert Owner Rank。

Payload 的主项是：

```
send_hidden: [A_local, H]
```

元数据是否一起发送取决于实现：`local_expert` 和恢复映射通常需要；`gate_weight` 可以随 Assignment 发送，也可以留在 Origin Rank，等结果返回后再加权。不能把某个框架的选择当成所有实现的固定规则。

目标 Rank 收到来自所有 Peers 的 Token 后，还要按 `local_expert` 再分段，形成：

```
X_0 [T_0,H], X_1 [T_1,H], ...
```

随后才进入 Grouped GEMM。

2. 第二次 All-to-All：Combine 的完整数据流

本地 Experts 计算得到：

```
Y_e: [T_e,H]
packed_Y: [Σ_e T_e,H]
```

回程按 `origin_rank` 重新 Pack，然后执行第二次 All-to-All，把每个 Assignment 的 Expert 输出送回 Token 原始 Rank。

Origin Rank 收到后使用保存的逆映射：

1. `Unpermute`：恢复到 `(origin_token, topk_slot)`；
2. 乘 Router 权重 `g_{t,e}`；
3. 对同一 Token 的 k 个 Expert 输出做 `scatter_add/weighted_sum`。

```
y_t = Σ_{e∈S_t} g_{t,e} × Expert_e(x_t)
```

Top-2 时一个 Token 去程复制成 2 个 Assignments，回程也会收到 2 个 Expert 输出，最后再合并回 1 个 `[H]` 向量。

3. Backward 为什么还要两次通信

Forward 的依赖是：

```
Origin X
→ Dispatch A2A
→ Expert Forward
→ Combine A2A
→ Origin Y
```

Backward 反向传播：

```
dY at Origin
→ 按 Top-k 权重拆成 Assignment Gradients
→ A2A 发回 Expert Owner
```

```
→ Expert Backward，得到 dX_assignment 与 dW_expert
→ A2A 把 dX_assignment 返回 Origin
→ 对同一 Token 的梯度累加，并计算 Router Gradient
```

所以可把 Backward 两次通信理解为：先反向穿过 Forward Combine，再反向穿过 Forward Dispatch。Expert 参数梯度通常留在 Expert Owner 上；若 Expert 参数还有数据并行副本，则可能另外发生其 DP/FSDP 通信。

4. 一条可落地的 Chunk Pipeline

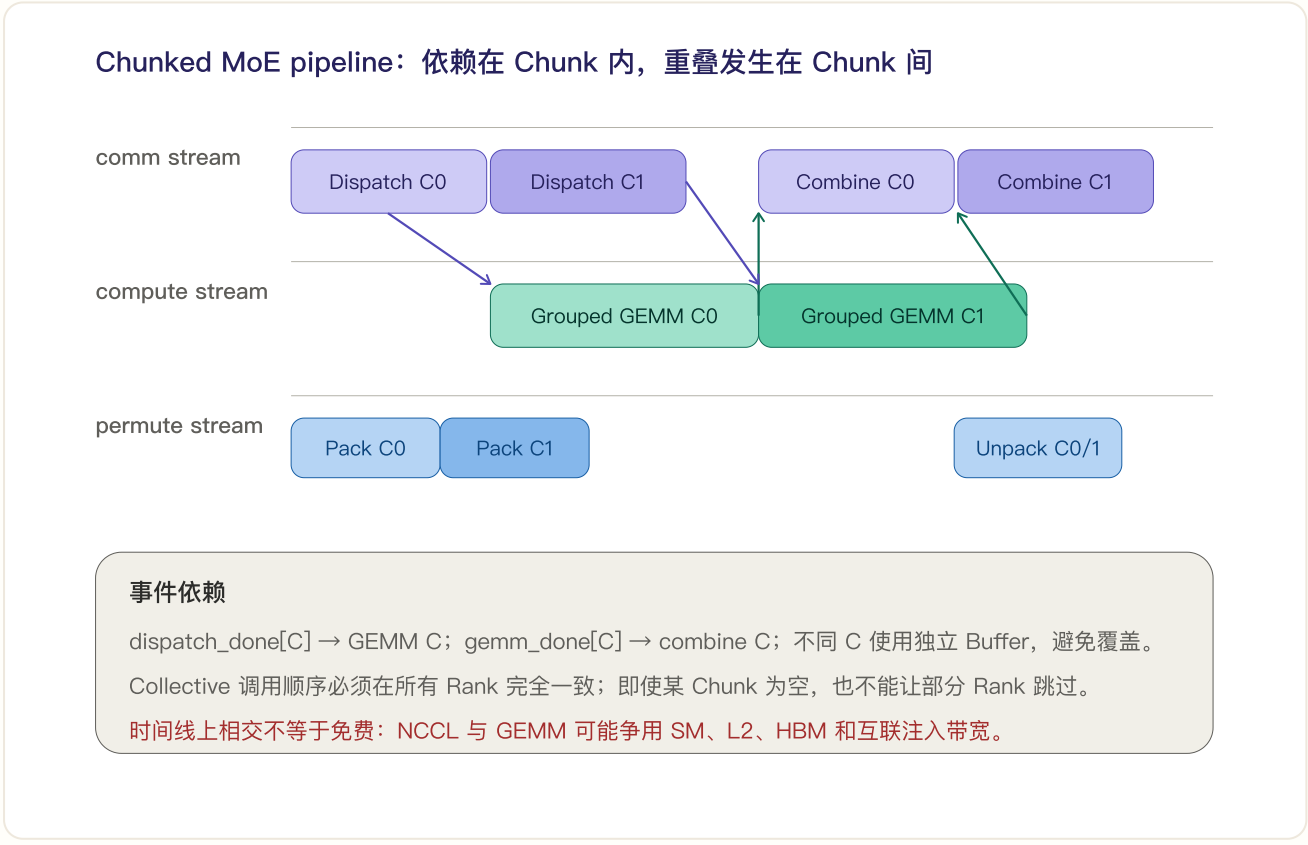
假设把 Assignment 切成 `C0`、`C1` 两个 Chunk，并至少有三条执行流：

```
permute_stream  负责 Pack/Unpack
comm_stream     负责 Dispatch/Combine
compute_stream  负责 Grouped GEMM
```

理想时序：

```
时间 →
comm:     Dispatch C0 | Dispatch C1 | Combine C0 | Combine C1
compute:              | GEMM C0---------------| GEMM C1---------------|
permute:  Pack C0 | Pack C1       | Unpack C0       | Unpack C1
```

更准确地说，`GEMM C0` 必须等待 `Dispatch C0` 的接收完成；`Combine C0` 必须等待 `GEMM C0` 完成。不同 Chunk 之间若 Buffer 和映射独立，就可以并行。



5. 一个两 Chunk 的时间算例

不切 Chunk 时：

```
Pack/Unpack = 2 ms
Dispatch    = 4 ms
GEMM        = 8 ms
Combine     = 4 ms
串行总时间   = 18 ms
```

切成 2 个 Chunk 后，由于 Launch 变多，每 Chunk 假设：

```
Dispatch = 2.3 ms
GEMM     = 4.2 ms
Combine  = 2.3 ms
```

三阶段流水线：

时间区间	通信流	计算流
0–2.3 ms	Dispatch C0	空闲
2.3–4.6 ms	Dispatch C1	GEMM C0 开始
4.6–6.5 ms	等待	GEMM C0 继续
6.5–8.8 ms	Combine C0	GEMM C1 开始
8.8–10.7 ms	等待	GEMM C1 继续
10.7–13.0 ms	Combine C1	完成

核心路径约为 13 ms，而不是 18 ms。但若重叠导致 GEMM 从 4.2 ms 膨胀到 5.5 ms、通信从 2.3 ms 膨胀到 3.0 ms，收益会明显缩水。这也是为什么不能只看 Trace 中有重叠色块，必须比较 Kernel Duration 与端到端 Step Time。

6. 三类最实用的 Overlap 机会

6.1 Routed Dispatch/Combine 与 Shared Expert MLP 重叠

Shared Expert 对所有 Tokens 计算，通常不依赖 Routed Dispatch 的结果：

```
路径 A: Route → Dispatch → Routed Experts → Combine
路径 B: Shared Expert MLP
```

因此 Shared Expert 是很好的“通信遮挡物”。前提是 Shared GEMM 不把全部 SM/HBM 吃满，否则 NCCL 进展会变慢。

6.2 Routed Path 内部做 Chunk Pipeline

把 Tokens/Assignments 切 Chunk，让 `Dispatch(C1)` 与 `GEMM(C0)`、`Combine(C0)` 与 `GEMM(C1)` 重叠。难点是标准 Collective 往往要等整次接收完成；若要做到更细粒度的“收到某 Expert 的一段就开算”，通常需要 All-to-All-v/P2P、专用 Dispatcher 或支持 Progressive Receive 的后端。

6.3 Backward 中通信与 Expert 梯度计算重叠

Backward 可按 Chunk/Expert 流水化：某个 Chunk 的 `dY` 一到 Expert Owner 就开始计算该 Chunk 的 Expert Backward，同时其他 Chunk 仍在通信。`dW` 聚合、`dX` 回程和其他独立梯度计算之间也可能形成重叠窗口。

7. Chunk Size 为什么是首要调参旋钮

通信时间可近似为：

```
T_comm(n) ≈ α + n ÷ BW_effective
```

```
α             一次通信/Kernel 的固定启动延迟，单位：秒
n             Payload 字节数，单位：Byte
BW_effective  有效带宽，单位：Byte/s
```

Chunk 太小：

- `α` 被重复支付；
- NCCL Kernel/网络包效率差；
- Grouped GEMM 的 M 更小，Tensor Core 利用率下降；
- Event、Pack、Prefix Sum 和 Kernel Launch 增多。

Chunk 太大：

- Pipeline 填充/排空时间长；
- 可重叠窗口减少；
- 最后一个 Chunk 形成暴露尾部；
- 热门 Expert/Rank 的长尾更明显。

所以最佳 Chunk 不是“越小越好”，而是同时满足：通信达到带宽区间、Grouped GEMM 有足够 M、又能形成至少 2–4 个有效流水段。

8. CUDA Stream/Event 的正确依赖

概念伪代码：

```
for c in chunks:
    with stream(permute_stream):
        pack[c] = permute(assignments[c])
        pack_done[c].record()

    comm_stream.wait_event(pack_done[c])
    with stream(comm_stream):
        recv[c] = all_to_all_async(pack[c])
        dispatch_done[c].record()

    compute_stream.wait_event(dispatch_done[c])
    with stream(compute_stream):
        expert_out[c] = grouped_gemm(recv[c])
        gemm_done[c].record()

    comm_stream.wait_event(gemm_done[c])
    with stream(comm_stream):
        returned[c] = all_to_all_async(expert_out[c])
        combine_done[c].record()

    permute_stream.wait_event(combine_done[c])
    with stream(permute_stream):
        output.add_(unpermute_and_weight(returned[c]))
```

生产实现还必须处理：

- Buffer 生命周期不能在异步操作完成前被复用；
- Stream-aware Allocator/ `record_stream`，避免缓存分配器提前回收；
- 所有 Ranks 的 Collective 调用顺序一致，否则可能 Hang；
- 变长 Split Size 必须两端一致；
- 避免隐式 Default Stream 同步、Host `wait()` 或调试 API 把流水线串行化；
- 多条通信 Stream 不代表网络能无限并发，可能反而增加拥塞。

9. 为什么“放到两条 Stream”仍可能没有收益

NCCL GPU 通信通常也要运行 Kernel，会占一部分 SM；Pack/Unpack、Grouped GEMM 和通信都可能访问 HBM/L2。因此存在四类资源冲突：

资源	冲突表现	常见手段
SM	NCCL CTA 与 GEMM CTA 互抢	调 NCCL channels/CTA、通信流优先级、限制计算占满程度
HBM/L2	Permute、GEMM、NCCL 同时搬数据	融合 Permute、减少中间 Buffer、低精度通信
网络	多 Chunk/多 Rail 注入拥塞	拓扑感知 EP Group、Node-Limited Routing、合理 Chunk
启动/调度	Chunk 太多导致 Kernel 碎片	Persistent/Fused Kernel、批量提交、CUDA Graph（形状允许时）

流优先级也不是越高越好：高优先级通信可能减少暴露通信尾部，却让 GEMM 被抢占/延迟；最终应优化 Critical Path，而不是单个通信 Timer。

10. 针对你的优化工作，建议这样测

至少同时采集：

```
1. raw_dispatch / raw_combine
   在不重叠或隔离环境下的通信基线

2. exposed_dispatch / exposed_combine
   真正阻塞依赖消费者的可见通信时间

3. grouped_gemm_duration_overlap / grouped_gemm_duration_solo
   判断 overlap 是否让 GEMM 膨胀

4. per-rank send/recv bytes、send_counts 矩阵
   判断热点 Peer 和负载偏斜

5. tokens_per_expert、max/mean、P95/P99 T_e
   判断尾部由哪一个 Expert/Rank 造成

6. SM active、Tensor Core active、HBM throughput、NVLink/IB throughput
   判断争用资源

7. pipeline bubble 与最后一个 Chunk tail
   判断填充/排空是否吞掉收益
```

可定义近似隐藏率：

```
hidden_fraction = 1 − exposed_comm ÷ raw_comm
```

但要注意：如果 overlap 时通信本身因争用从 4 ms 膨胀到 7 ms，只用旧的 `raw_comm=4 ms` 会误导。应同时报告“隐藏了多少等待”和“通信/计算各自膨胀了多少”。最终裁决指标始终是 MoE Layer Latency 或 Step Time。

11. 最容易踩的正确性与性能坑

- **Collective 顺序不一致**：某 Rank 因 Chunk 为空跳过一次 All-to-All，其他 Rank 没跳，直接 Hang。
- **过早开 Expert GEMM**：只收到部分 Peer 数据，却误以为该 Expert 的输入已完整。
- **逆映射错误**：Combine 后 Token/Top-k Slot 对不上，数值可能不报错但训练悄悄劣化。
- **异步 Buffer 被覆盖**：下一 Chunk 复用 Send/Recv Buffer，前一次通信尚未完成。
- **只测 NCCL Timer**：通信变短但 Permute、GEMM 或尾部变长，端到端没有收益。
- **只看平均 Tokens/Expert**：P99 热门 Expert 才决定同步点。
- **Chunk 过细**：通信和 GEMM 都从带宽/算力受限退化为 Launch-Latency 受限。

关键总结

1. Grouped GEMM 用一个 Tile Scheduler 调度多组独立变长 GEMM；它减少 Launch 和小 GEMM 碎片，但不消除负载长尾。

2. `p_e` 是 Router 全概率的 Batch 均值，`f_e` 是 Top-k 后硬 Assignment 比例；二者结合才能既看真实拥挤又给 Router 可微梯度信号。
3. Softmax 对共同偏移不敏感、对差值放大却会饱和；Z-Loss 通过惩罚 $\text{logsumexp}(\text{logits})^2$ 固定绝对数值尺度，不能替代负载均衡。
4. MoE Forward 是 Dispatch 去程 + Combine 回程，Backward 再来两次；Overlap 的核心是 Chunk、异步执行、事件依赖和资源争用控制。
5. 优化时要同时报告暴露通信、通信/GEMM 膨胀、负载分布和端到端 Critical Path，不能只看时间线上是否出现重叠。

可选自测

1. 若 `T=4,E=3`，概率矩阵如补充二，改成 Top-2 后应如何定义 `f_e` 的分母?
2. 为什么 `[2,1,0]` 与 `[12,11,10]` 的 Softmax 相同，但 Z-Loss 差异巨大?
3. 若两个 Chunk 的通信与 GEMM 已在不同 Stream 重叠，但 Step Time 反而变长，你会优先检查哪三个资源或指标?

继续学习

1. 前置：CUDA Stream/Event 与 NCCL 异步语义 —— overlap 正确性的底层基础。
2. 同层对比：All-to-All-v、P2P Dispatcher 与 DeepEP 类后端 —— 不同动态通信执行方式的差异。
3. 下游应用：MoE Chunk Pipeline 性能建模与 Nsight Systems 定位 —— 把本节方法落到真实优化实验。

十一、All-to-All 通信量手算

设某 EP Rank 原始拥有：

```
T_local = 4096 Tokens
H = 4096
Dtype = BF16 = 2 Byte
k = 2
EP = 8
```

总 Token Assignments：

```
A_local = T_local × k
        = 4096 × 2
        = 8192
```

若路由均衡，目标 Expert 在本 Rank 的概率约为 `1/EP`，需要发往远端的比例约：

```
(EP-1)/EP = 7/8
```

一次 Dispatch 的远端 Hidden State 发送量近似：

```
Bytes_dispatch_send
≈ T_local × k × H × dtype_bytes × (EP-1)/EP

= 4096 × 2 × 4096 × 2 × 7/8
= 58,720,256 Byte
= 56 MiB
```

Combine 方向近似再发送一次：

```
Forward 两次 All-to-All 发送 Payload
≈ 56 MiB + 56 MiB
= 112 MiB / Rank / MoE Layer
```

这还没有包括：

- 元数据；
- 对齐/Padding；
- 接收流量；
- Backward 通信；
- 不均衡导致的额外等待；
- 网络协议与实际带宽折损。

因此 MoE 常出现：参数 FLOP 看起来很省，但 All-to-All 却成为 Step Time 的主要瓶颈。

十二、为什么 All-to-All 很难优化

AllReduce 中每个 Rank 的消息量和形状通常可预测；MoE All-to-All 的消息量由本批 Token 路由结果动态决定：

```
Rank 0 可能向 Rank 1 发很多 Token
Rank 2 可能几乎不向 Rank 1 发
下一批又完全不同
```

难点包括：

- 每个 Peer 消息尺寸不同；

- 网络热点随 Batch 变化；
- 最慢 Rank 决定全组完成时间；
- 跨节点带宽远低于机内 NVLink；
- Token Permutation/Unpermutation 也消耗 HBM 带宽；
- 通信和 Expert GEMM 可能同时争用 HBM；
- Top-k 越大，Assignments 与通信近似线性增加。

常见优化：

- 尽量把 EP Group 放在高速互联域；
- 使用优化 Dispatcher，如 All-to-All/Flex/DeepEP 类后端；
- Token Permutation Fusion；
- Dispatch/Combine 与 Shared Expert 或其他计算 Overlap；
- Node-Limited Routing，减少跨节点目的地；
- 低精度通信；
- 调整 EP、TP、DP、CP 的映射；
- Batch-Level Overlap。

十三、MoE 能否与计算 Overlap

可以，但比 DDP AllReduce Overlap 更复杂。

一个典型机会：

```
Routed Experts 路径：Route → Dispatch A2A → Expert Compute → Combine A2A
Shared Expert 路径：Shared Expert MLP
```

如果两条路径没有直接依赖，可以：

```
Dispatch All-to-All 与 Shared Expert Compute 重叠
Expert Compute 与其他通信阶段流水化
Combine 与后续可独立任务重叠
```

也可以把 Token 分 Chunk，形成：

```
Chunk 0 通信
→ Chunk 0 计算，同时 Chunk 1 通信
```

```
→ Chunk 0 返回，同时 Chunk 1 计算
```

但实际 Overlap 受限于：

- NCCL/通信 Kernel 是否占 SM；
- Expert GEMM 和通信是否争抢 HBM；
- 消息是否足够大；
- Chunk 太小导致 Launch 开销；
- 路由结果是否提前可用；
- CUDA Stream/Event 依赖；
- 跨节点带宽与拓扑。

因此“时间线上重叠”不等于“通信完全隐藏”。要用 Nsight Systems 和框架 Timer 看：

```
exposed dispatch time
exposed combine time
expert GEMM utilization
per-expert token histogram
per-rank send/recv bytes
```

十四、Expert Compute 为什么需要 Grouped GEMM

假设本地有 4 个 Experts，Token 数分别为：

```
T_0=128, T_1=64, T_2=19, T_3=173
```

若逐个执行：

```
GEMM Expert 0
GEMM Expert 1
GEMM Expert 2
GEMM Expert 3
```

问题：

- 4 次或更多 Kernel Launch；
- 小 `M=T_e` GEMM 难以占满 GPU；
- 动态 Shape 多；
- Expert 2 只有 19 个 Token，效率很低。

Grouped GEMM 把多个不同 `M`、相同或兼容 `K/N` 的 GEMM 组织起来：

```
一次 Grouped GEMM
处理多个 Expert 的 Token 分段和权重分段
```

收益：

- 减少 Launch；
- 更好地调度 Tensor Core 工作；
- 提升多个小 Expert 的整体利用率；
- 与 FP8/低精度 Expert Kernel 结合。

但 Grouped GEMM 不能消除负载不均：如果某个 Rank 收到远多于其他 Rank 的 Token，其他 Rank 仍要等待。

十五、Shared Experts 与 Routed Experts

传统 MoE 所有知识都由 Routed Experts 承担，可能产生：

- 不同 Experts 重复学习通用知识；
- Router 为常见模式浪费路由容量；
- Routed Expert Specialization 不够纯粹。

Shared Expert 对所有 Token 始终执行：

```
Y = SharedExpert(x) + Σ_{e∈S_t} g_{t,e} RoutedExpert_e(x)
```

作用：

- Shared Expert 学习通用知识；
- Routed Experts 更专注差异化知识；
- 减少 Routed Experts 之间的冗余；
- Shared 路径可作为通信 Overlap 的计算来源。

代价：

- Shared Expert 是 Dense Compute，所有 Token 都执行；
- 增加每 Token FLOP；

- 需要设计共享与路由输出如何融合；
- 可能再次成为计算瓶颈。

十六、Fine-Grained Expert Segmentation

DeepSeekMoE 提出的核心思想之一是把较大的 Experts 细分为更多、更小的 Experts。

传统：

```
N 个大 Experts，激活 K 个
```

细粒度方案：

```
mN 个小 Experts，激活 mK 个
```

若每个小 Expert 大约是原 Expert 的 `1/m` 宽度，则激活总计算可保持接近，但组合方式更多：

```
C(N,K) 种粗粒度组合
→ C(mN,mK) 种更灵活的细粒度组合
```

目标：让专家知识分解得更细，Token 可以组合多个更专门的能力单元。

代价：

- Experts 更小，单个 GEMM 更碎；
- Router 输出维度更大；
- Assignment 数更多；
- Dispatch 元数据和通信更复杂；
- 更依赖 Grouped GEMM 和高效 Dispatcher。

因此 Fine-Grained Experts 是模型表达能力和系统效率的共同设计，而不是单纯增加 Expert 数。

十七、训练阶段与推理阶段有什么不同

17.1 训练

训练需要：

- Router 梯度；
- Expert 参数梯度；
- Load Balancing Aux Loss；
- Router Z-Loss；
- Forward 与 Backward All-to-All；
- Optimizer State；
- Expert 参数分片/复制；
- Activation 保存或重计算。

训练的 Token 数通常较大，Grouped GEMM 和通信更容易形成大批量，但 All-to-All 也很重。

17.2 Prefill 推理

Prompt Prefill 一次有较多 Token：

```
Batch × Prompt Length
```

Expert GEMM 相对较大，GPU 利用率通常比 Decode 好；但路由不均仍可能造成 Rank 热点。

17.3 Decode 推理

每条序列每步只产生一个新 Token。若 Batch 不大：

```
T 很小
→ 每 Expert 收到的 Token 更少
→ GEMM 极小
→ All-to-All 延迟难摊薄
```

同时所有 Expert 权重仍需驻留在 GPU 集群内，容易出现：

- Memory Capacity 压力；
- Expert Weight 访问不连续；
- 跨 GPU 通信主导 TPOT；
- 某些热门 Experts 形成尾延迟；

- Expert Cache/Offload 命中率问题。

所以“MoE 训练吞吐高”不代表“MoE 单请求推理延迟低”。

十八、MoE 与 Dense 模型的核心对比

维度	Dense Transformer	Sparse MoE Transformer
FFN 参数	每层一套	每层多个 Routed/Shared Experts
每 Token 执行路径	固定	Router 动态选择
总参数与计算关系	基本同步增长	总参数按 E 增长，激活计算按 k 增长
通信	TP/DP/PP 等	额外 Token Dispatch/Combine All-to-All
GEMM Shape	相对规则	每 Expert Token 数动态，易产生小 GEMM
负载均衡	主要是静态切分	Router 动态负载，需均衡策略
显存	参数与优化器状态可预测	Expert 总参数很大，分片与优化器更复杂
容错/调试	路径固定，较容易	动态路由、Token Drop、通信和质量耦合
推理解码	Batch GEMM 相对稳定	小 Batch 时 Expert GEMM 与 A2A 难摊薄
优势	简单、稳定、硬件友好	大参数容量、条件计算、潜在专家专门化

十九、MoE 与各种并行策略的关系

MoE 不是替代其他并行策略，而是增加一个新的并行维度：Expert Parallel。

```
DP：复制模型/分片训练数据
TP：切单个 Dense/Expert 内部矩阵
PP/VPP：沿层深度切模型
CP：切序列上下文
EP：把不同 Experts 放到不同 Rank
```

常见组合：

```
TP × PP/VPP × DP/FSDP × EP × CP
```

关键拓扑原则：

- TP 每层频繁通信，优先放高速 NVLink 域；
- EP All-to-All 对跨节点网络非常敏感；
- EP 太小，每卡 Expert 参数多、显存压力高；
- EP 太大，All-to-All 范围大、每 Expert Token 少、GEMM 更碎；
- DP/FSDP 如何处理 Expert 参数，需要区分 Dense 参数组和 Expert 参数组；
- 某些框架要求 EP 与 TP 组合时启用 Sequence Parallel；
- Attention 与 MoE 的最优并行布局可能不同，因此出现 Parallel Folding 等解耦映射。

二十、最小 PyTorch MoE 实现

下面代码是单 GPU、Dropless、Top-2 的教学实现。它保留真实 Router、Top-k、按 Expert 分组计算和加权合并，但没有分布式 All-to-All、容量限制和高性能 Grouped GEMM。

```
import torch
import torch.nn as nn
import torch.nn.functional as F


class Expert(nn.Module):
    def __init__(self, hidden_size: int, ffn_size: int):
        super().__init__()
        self.w1 = nn.Linear(hidden_size, ffn_size, bias=False)
        self.w2 = nn.Linear(ffn_size, hidden_size, bias=False)

    def forward(self, x):
        # x: [tokens_for_this_expert, H]
        return self.w2(F.relu(self.w1(x)))


class SimpleTopKMoE(nn.Module):
    def __init__(
        self,
        hidden_size=8,
        ffn_size=16,
        num_experts=4,
        top_k=2,
    ):
        super().__init__()
        self.hidden_size = hidden_size
        self.num_experts = num_experts
        self.top_k = top_k

        self.router = nn.Linear(
            hidden_size,
            num_experts,
            bias=False,
        )

        self.experts = nn.ModuleList([
            Expert(hidden_size, ffn_size)
            for _ in range(num_experts)
        ])

    def forward(self, x):
```

```
        # x: [B, S, H]
        batch, seq, hidden = x.shape
        flat = x.reshape(-1, hidden)       # [T, H]
        tokens = flat.shape[0]

        # 1. Route
        logits = self.router(flat)         # [T, E]
        probs = torch.softmax(logits, dim=-1)

        top_weights, top_indices = torch.topk(
            probs,
            k=self.top_k,
            dim=-1,
        )                                  # both [T, k]

        # 只在选中的 k 个 Experts 内重新归一化。
        top_weights = top_weights / top_weights.sum(
            dim=-1,
            keepdim=True,
        )

        # 2~4. Dispatch、Expert Compute、Combine
        output = torch.zeros_like(flat)    # [T, H]
        tokens_per_expert = []

        for expert_id, expert in enumerate(self.experts):
            # selected: [T, k]，表示每个 Token 的每个 Top-k Slot
            # 是否选择了当前 Expert。
            selected = top_indices == expert_id

            # 找出所有选择当前 Expert 的 Token 和对应 Top-k Slot。
            token_idx, slot_idx = torch.where(selected)
            tokens_per_expert.append(token_idx.numel())

            if token_idx.numel() == 0:
                continue

            expert_input = flat[token_idx]             # [T_e, H]
            expert_output = expert(expert_input)        # [T_e, H]

            gate = top_weights[token_idx, slot_idx]     # [T_e]
            weighted = expert_output * gate.unsqueeze(-1)

            # 多个 Expert 对同一个 Token 的贡献累加。
            output.index_add_(0, token_idx, weighted)

        output = output.reshape(batch, seq, hidden)

        aux = {
            "router_logits": logits,
            "router_probs": probs,
            "top_indices": top_indices,
            "top_weights": top_weights,
            "tokens_per_expert": torch.tensor(
                tokens_per_expert,
                device=x.device,
            ),
        }
        return output, aux


torch.manual_seed(0)

model = SimpleTopKMoE(
    hidden_size=8,
    ffn_size=16,
    num_experts=4,
    top_k=2,
)

x = torch.randn(2, 3, 8)  # [B=2, S=3, H=8]
y, aux = model(x)
```

```
print("input shape:", x.shape)
print("output shape:", y.shape)
print("top indices shape:", aux["top_indices"].shape)
print("top weights shape:", aux["top_weights"].shape)
print("tokens per expert:", aux["tokens_per_expert"].tolist())
print("assignments:", int(aux["tokens_per_expert"].sum()))

# 最小反向验证：Router 和被选 Experts 都能收到梯度。
loss = y.square().mean()
loss.backward()

print("router grad exists:", model.router.weight.grad is not None)
```

预期 Shape：

```
input shape: [2,3,8]
output shape: [2,3,8]
top indices shape: [6,2]
top weights shape: [6,2]
assignments: 12
```

这段代码说明了什么

- Router 对每个 Token 产生 `[E]` Scores；
- Top-k 产生 `[T,k]` Expert IDs 和 Weights；
- 同一个 Token 会被多个 Experts 处理；
- `index_add_` 把多个 Expert 的加权输出合并回原 Token；
- Router 的 Top-k Index 是离散选择，但选中权重仍可对 Router 反向传播；
- 未被选中的 Expert 本批可能没有梯度。

它没有说明什么

生产实现还需要：

- Expert Capacity/Token Drop；
- EP All-to-All；
- 高性能 Permute/Unpermute；
- Grouped GEMM；
- Router Aux Loss/Z-Loss；
- Shared Experts；
- FP8/BF16；
- TP/DP/PP/CP 组合；
- 通信与计算 Overlap；

- 分布式 Checkpoint；
- 动态 Shape 与 CUDA Graph。

二十一、训练中应该监控哪些指标

Router/负载

```
Tokens per Expert Histogram
Max / Mean Tokens per Expert
Expert Utilization
Router Entropy
Top-k Probability
Aux Loss
Z-Loss
Drop Rate / Overflow Rate
```

负载不均衡比：

```
Imbalance Ratio = max_e(T_e) ÷ mean_e(T_e)
```

越接近 1 越均匀，但也不能只追求绝对均匀而牺牲语义路由。

通信

```
Dispatch All-to-All Time
Combine All-to-All Time
Exposed A2A Time
Per-Rank Send/Recv Bytes
Cross-Node Traffic
Permutation/Unpermutation Time
```

Expert Compute

```
Grouped GEMM Time
Per-Expert GEMM M Dimension
Tensor Core Utilization
Kernel Launch Count
Expert Compute Imbalance
```

端到端

```
Step Time
Tokens/s
MFU
```

```
Peak Memory
Optimizer Memory
Activation Memory
Checkpoint Time
```

不要只看“总 FLOP 少了多少”。MoE 的性能可能被通信、Permutation 和小 GEMM 完全主导。

二十二、常见失败模式

失败 1：Expert Collapse

少数 Experts 获得大部分 Token，其他 Experts 不训练。

排查：Tokens-per-Expert、Router Entropy、Aux Coeff、Input Distribution。

失败 2：Capacity 太小，Drop Rate 高

模型质量下降，部分 Token 路径丢失。

排查：Capacity Factor、Top-k、Router Skew、Drop Policy。

失败 3：Capacity 太大，Padding 浪费

固定 Shape 方便，但大量 Padding 吃掉计算和通信。

失败 4：EP Group 跨慢网络

All-to-All 暴露时间过高，训练吞吐下降。

失败 5：Experts 太细，Grouped GEMM 仍很小

模型表达更灵活，但 GPU Tensor Core 吃不满。

失败 6：Aux Loss 太强

路由很均匀，但专家无法形成有意义的专门化。

失败 7：推理 Batch 太小

每 Expert 只有少量 Token，Decode TPOT 被通信和 Kernel Launch 主导。

失败 8：只优化通信，不看 HBM 与 Permutation

All-to-All 变快后，Permutation、Unpermutation 或 Expert GEMM 成为新瓶颈。

二十三、什么时候适合使用 MoE

适合：

- 希望在相近每 Token FLOP 下扩大参数容量；
- 训练 Token 规模足够大，能让 Experts 获得充足数据；
- 集群有高速互联，能承受 EP All-to-All；
- 有成熟的 Router、Grouped GEMM、Dispatcher 和并行框架；
- 目标是大规模预训练/高吞吐 Serving；
- 可以接受更高工程复杂度和参数显存。

不适合：

- 模型规模不大，Dense 已能满足容量需求；
- 单请求低延迟是绝对优先，Batch 很小；
- 跨节点网络差；
- 训练数据不足，Experts 难以专门化；
- 团队缺少路由、通信、Checkpoint 和可观测性能力；
- 显存只能容纳 Active 参数，却无法容纳/分片全部 Expert 参数。

二十四、常见误区

误区 1：MoE 只加载被选中的 Experts，所以总参数不占显存

错误。训练时全部 Expert 参数及其梯度/优化器状态必须存在于某些设备或存储层中；EP 只是把它们分散，不是让未选 Experts 消失。

误区 2：Top-2 表示同时运行两个完整模型

错误。通常只是同一 Transformer Block 的两个 Expert FFN，被选结果在该层加权合并。

误区 3：MoE 总参数增加 E 倍，计算一定不变

错误。每 Token 计算取决于 `k×F_expert`、Router、Shared Experts、Padding、通信和实现开销。

误区 4：负载越均匀，模型质量一定越好

错误。系统需要避免严重热点，但过强均衡可能损害专家专门化。

误区 5：All-to-All 是唯一瓶颈

错误。Permutation、动态小 GEMM、内存、Router、Checkpoint 和 Decode 小 Batch 都可能成为主要瓶颈。

误区 6：MoE 更省训练显存

不一定。Active FLOP 较少不代表总参数、梯度和优化器状态少；MoE 的参数显存常显著高于相近计算量的 Dense 模型。

二十五、关键总结

MoE 的完整因果链：

```
Dense FFN 只能通过增大同一 MLP 扩大参数
→ 每 Token 计算与参数一起增长

MoE 放置 E 个 Experts
→ Router 为每 Token 选择 k 个，k≪E
→ 总参数由 E 决定，激活计算由 k 决定
→ 参数容量与每 Token 计算解耦

但动态路由产生不均匀 Token Assignment
→ 需要负载均衡与容量策略
→ Experts 分布到多 GPU 后需要两次 All-to-All
→ 形成通信、动态小 GEMM、显存和可观测性难题
```

分析任何 MoE 系统时固定问六个问题：

1. **Architecture**：多少 Experts？Top-k？Expert 宽度？是否 Shared Experts？

2. **Routing**：Softmax/Sigmoid？Aux Loss？Capacity？Dropless？
3. **Compute**：每 Expert 收到多少 Token？Grouped GEMM 是否足够大？
4. **Communication**：EP 多大？All-to-All 是否跨节点？暴露时间多少？
5. **Memory**：全部 Expert 参数、梯度、优化器状态如何分布？
6. **Serving**：Prefill/Decode Batch 多大？专家权重和通信能否摊薄？

可选自测

1. Shape 与容量题

```
B=4, S=2048, E=64, k=2, CapacityFactor=1.25
```

请计算：

- `T`；
- 总 Assignments；
- 每 Expert 平均 Assignments；
- Capacity `C`。

2. 参数与 FLOP 题

一个 Dense FFN 使用 `H=4096, F_dense=14336`；MoE 使用 `E=8, k=2`。如果希望主 Expert FLOP 近似 Dense FFN，应如何估算 `F_expert`？MoE 全部 Expert 参数约是 Dense FFN 的多少倍？

3. 工程决策题

某 MoE 训练中：

```
All-to-All = 35% Step Time
Grouped GEMM = 25%
Permutation = 15%
Max/Mean Expert Tokens = 2.8
Drop Rate = 4%
```

你会按什么顺序优化？哪些优化是在减少通信量，哪些是在减少暴露时间，哪些是在改善负载与计算效率？

继续学习

1. 前置：MoE Router 与负载均衡损失 —— 深入 Top-k、Aux Loss、Z-Loss 和 Aux-Loss-Free Balancing。
2. 同层对比：Dense FFN、Switch、Mixtral、DeepSeekMoE —— 对比 Top-1/Top-2、Shared Experts 与细粒度专家。
3. 下游应用：Expert Parallel 与 DeepEP —— 深入 All-to-All Dispatcher、通信 Overlap 和跨节点拓扑。

回复 1、2 或 3 继续；也可以说“稍后”或“不感兴趣”。

事实来源与版本边界

本文的稳定原理基于 Sparsely-Gated MoE、GShard、Switch Transformer 与 DeepSeekMoE 的公开论文，以及截至 2026-07-29 的 NVIDIA Megatron Core MoE 官方文档。不同模型与框架在 Router Score、Top-k 归一化、Capacity、Token Drop、Aux Loss、Shared Expert、Dispatcher 和并行映射上存在差异；具体配置、函数名和性能应以目标版本源码与实测为准。

Data Parallel · FSDP · Tensor Parallel · Pipeline Parallel · Context Parallel · Expert Parallel · Multi-Dimensional Parallelism

09

大模型并行策略全解：DP、FSDP、TP、PP/VPP、CP、EP 与多维组合

> **原始问题：**详细讲解不同的并行策略：PP,EP,DP,CP,TP等，每一种都讲透彻，并且列出各自的异同，适合的场景，具体应用例子，然后全部串起来，举一个简单的数据例子，把各种并行策略都叠加使用，完整的展现每种并行策略的实际应用。

提问：2026/07/29 20:49　更新：2026/07/29 20:49　重要度：5/5

大模型并行策略全解：DP、FSDP、TP、PP/VPP、CP、EP 与多维组合

一句话结论

这些并行策略并不是互相替代，而是在切分不同维度：

```
DP / DDP：切 Batch，复制模型
FSDP / ZeRO：在 DP 语义下进一步切参数、梯度和优化器状态
TP：切单层内部的矩阵、Hidden 或 Attention Heads
PP：切模型深度，把不同 Layers 放到不同 Stage
VPP：把每个 PP Stage 再切成多个 Chunks，优化流水线调度
CP：切 Sequence / Context，让每张 GPU 只保存一部分长序列激活
EP：切 MoE Experts，把不同专家放到不同 GPU
```

选择原则：

> **模型并行主要解决“单卡放不下”；数据并行主要解决“吞吐扩展”。先使用尽可能小的 TP/PP/CP/EP 让模型和激活放得下，再把剩余 GPU 尽量用于 DP。**

所有并行都会降低某类显存或扩大可训练规模，但同时引入通信：

```
内存节省不是免费午餐
= 更复杂的数据布局
+ 更多通信
+ 更严格的拓扑映射
+ 更难的调试与 Checkpoint
```



一、先建立统一坐标系：每种并行到底切什么

假设一次 Transformer 训练的核心对象包括：

```
数据：[B, S]
激活：[B, S, H]
Attention：[B, S, Heads, HeadDim]
模型深度：L Layers
MoE Experts：E
训练状态：参数、梯度、优化器状态
```

符号：

```
B       Batch Size
S       Sequence Length
H       Hidden Size
L       Transformer Layer 数
E       Expert 数
TP      Tensor Parallel Size
PP      Pipeline Parallel Size
CP      Context Parallel Size
EP      Expert Parallel Size
DP      Data Parallel Size
```

并行维度总览：

策略	主要切分对象	每卡主要保留什么	主要通信	最直接解决的问题
DP/DDP	Batch	完整模型副本 + 本地 Batch	Gradient AllReduce	扩大数据吞吐
FSDP/ZeRO	Batch + 模型状态	参数/梯度/优化器状态分片	Parameter AllGather + Gradient ReduceScatter	训练状态放不下
TP	Layer 内 Tensor	矩阵/Heads/Hidden 的分片	每层 AllReduce/AllGather/ReduceScatter	单层或大矩阵放不下
PP	Layers / 深度	一段连续 Layers	P2P Activation/Gradient	模型层数多、整体放不下
VPP	PP Stage 内 Chunks	多个不连续 Layer Chunks	更多 P2P	减少 Pipeline Bubble
CP	Sequence / Context	一段 Token 与激活	KV Ring/AllGather，dKV ReduceScatter	长上下文激活 OOM
EP	MoE Experts	一部分 Experts	Dispatch/Combine All-to-All	MoE Expert 参数放不下

另外还有 **Sequence Parallelism（SP）**：它通常依附 TP，只切 LayerNorm、Dropout 等非 Attention 区域的 Sequence 激活；CP 则贯穿网络输入和全部激活，并专门解决 Attention 的跨 Token 依赖。SP 不是 CP 的同义词。

二、Data Parallelism：DP、DDP 与 FSDP

2.1 普通 DP/DDP 的核心

Data Parallel 的逻辑是：

```
每个 DP Rank 持有一份相同模型
每个 Rank 处理不同的数据子 Batch
Backward 后同步梯度
所有 Rank 用相同全局梯度更新参数
```

设：

```
Global Batch = 8
DP = 4
Gradient Accumulation = 1
```

则每个 DP Replica 的 Local Batch：

```
B_local = B_global ÷ DP
        = 8 ÷ 4
        = 2
```

四个 Rank：

```
DP0: 样本 0,1
DP1: 样本 2,3
DP2: 样本 4,5
DP3: 样本 6,7
```

各自计算本地梯度：

```
g_0, g_1, g_2, g_3
```

然后执行 Gradient AllReduce：

```
g_global = (g_0 + g_1 + g_2 + g_3) ÷ 4
```

所有 Rank 都得到相同的 `g_global`，因此参数保持一致。

Global Batch 公式

```
Global Batch
= Micro Batch Per GPU
× Gradient Accumulation Steps
× DP Size
```

例如：

```
Micro Batch = 2
Accumulation = 4
DP = 8

Global Batch = 2 × 4 × 8 = 64
```

TP、PP、CP、EP 通常共同组成一个 Model Replica，它们不会像 DP 一样自动扩大独立样本数。

2.2 DDP 的通信与 Overlap

PyTorch DDP 通常将梯度分 Bucket：

```
后层梯度先计算完成
→ 对该 Bucket 异步 AllReduce
```

```
→ 同时继续计算前层 Backward
```

因此 DDP 的 Gradient AllReduce 很适合与 Backward Overlap。

通信发生在每个 Step 的 Backward 阶段，频率比 TP 的“每层通信”低，因此 DP 往往更适合跨节点扩展。

2.3 普通 DDP 的显存问题

每个 DP Rank 都保存完整：

- 参数；
- 梯度；
- Optimizer States；
- 混合精度的 Master Weights；
- 本地 Activation。

假设混合精度 Adam 经典估算约为：

```
FP16/BF16 参数       2 Byte
FP16/BF16 梯度       2 Byte
FP32 Master Weight   4 Byte
FP32 Adam m          4 Byte
FP32 Adam v          4 Byte
--------------------------------
合计约              16 Byte/参数
```

一个 10B 模型仅训练状态约：

```
10×10⁹ × 16 Byte = 160 GB
```

普通 DDP 在每张卡上复制这 160GB，无法训练。

2.4 FSDP / ZeRO：DP 语义下分片模型状态

FSDP 保留“不同 Rank 处理不同数据”的 DP 语义，但不再让每卡永久保存完整训练状态。

策略	分片内容	对应 ZeRO
Shard Optimizer	Optimizer States	ZeRO-1
Shard Gradients + Optimizer	Gradients、Optimizer	ZeRO-2

策略	分片内容	对应 ZeRO
Shard Parameters + Gradients + Optimizer	全部训练状态	ZeRO-3 / Full Shard

Full Shard 中：

```
平时：每卡只保存 1/DP 参数分片
计算某个 FSDP Unit 前：Parameter AllGather
Backward 后：Gradient ReduceScatter
Optimizer：每卡只更新自己的状态分片
```

理想每卡训练状态：

```
M_state_per_rank ≈ M_total_state ÷ DP
```

10B 模型、DP=8：

```
160 GB ÷ 8 = 20 GB/Rank
```

注意这不包含：

- Activation；
- AllGather 临时完整参数峰值；
- Communication Buffer；
- CUDA Workspace；
- Fragmentation。

2.5 DP/FSDP 适用场景

适合普通 DDP：

- 模型完整训练状态能放入单卡；
- 主要目标是扩大吞吐；
- 希望实现简单、计算效率高；
- 可以使用大 DP Group。

适合 FSDP/ZeRO：

- 模型训练状态超过单卡；
- 单个 Layer 仍可在单卡或 TP Group 内计算；

- 有足够通信带宽；
- 可以通过 Prefetch/Overlap 隐藏 AllGather/ReduceScatter。

不解决：

- 单个巨大 Layer 本身放不下，仍需 TP；
- 长序列 Activation OOM，仍需 CP/Checkpoint；
- 模型太深的 Pipeline 显存，仍可用 PP。

三、Tensor Parallelism（TP）

3.1 TP 切的是什么

TP 将一个 Layer 内部的大矩阵乘切到多张 GPU。

例如 Linear：

```
Y = XW

X: [T,H]
W: [H,F]
Y: [T,F]
```

设：

```
H = 16
F = 64
TP = 2
```

Column Parallel

沿输出维 `F` 切权重：

```
W = [W_0 | W_1]
W_0: [16,32]
W_1: [16,32]
```

每个 TP Rank：

```
Y_0 = XW_0  → [T,32]
```

```
Y_1 = XW_1  → [T,32]
```

逻辑完整输出：

```
Y = concat(Y_0,Y_1)  → [T,64]
```

如果下一算子能直接消费分片输出，可以不立刻 AllGather。

Row Parallel

沿输入维切：

```
X = [X_0 | X_1]
W_0: [32,H_out]
W_1: [32,H_out]
```

各 Rank 得到部分和：

```
Z_0 = X_0W_0
Z_1 = X_1W_1
```

完整结果：

```
Z = Z_0 + Z_1
```

需要 AllReduce，或在 Sequence Parallel 中使用 ReduceScatter 保持激活分片。

3.2 Transformer 中的典型 TP

Attention

设：

```
Q Heads = 4
HeadDim = 4
H = 16
TP = 2
```

每个 TP Rank 计算：

```
2 个 Q Heads
```

若 KV Heads=2，也可每 Rank 持有 1 个 KV Head。

各 Rank 独立计算部分 Heads，然后输出投影通过 Row Parallel 归约。

MLP

```
W₁: [16,64] → 每 Rank [16,32]
W₂: [64,16] → 每 Rank [32,16]
```

典型安排：

```
Column Parallel W₁
→ Activation 本地执行
→ Row Parallel W₂
→ AllReduce / ReduceScatter
```

这样中间非线性之前不需要额外通信。

3.3 TP 的收益

- 单层参数约降到 `1/TP`；
- 大矩阵计算量分到多卡；
- Attention Heads 可以分片；
- 配合 SP 可降低部分 Activation；
- 单层超过单卡显存时不可替代。

3.4 TP 的代价

TP 通信发生在几乎每个 Transformer Layer，且下一计算常立即依赖通信结果，难以完全隐藏。

因此：

> TP 通信频率最高、依赖最紧，优先放在单机 NVLink/NVSwitch 域内。

TP 过大还会导致：

- 每卡 GEMM 变小，Tensor Core 利用率下降；
- Collective Latency 占比上升；
- Hidden/Head/FFN 维度整除限制；
- Kernel Launch 与通信难 Overlap。

3.5 TP 适用场景

- 单个 Layer、Embedding 或 FFN 矩阵放不进单卡；
- Hidden Size 很大；
- Attention Heads 能整除 TP；
- 同一节点内有高速互联；
- 与 PP/DP 组合训练大模型。

实际例子：一个大模型的 64 个 Attention Heads 使用 TP=8，每卡计算 8 个 Heads；FFN 中间维 32768 每卡持有 4096。

四、Pipeline Parallelism（PP）与 VPP

4.1 PP 切模型深度

假设 8 层 Transformer：

```
PP = 2
Stage 0: Layers 0–3
Stage 1: Layers 4–7
```

Stage 0 Forward 输出 Activation，通过 P2P 发送给 Stage 1：

```
[B_local,S,H]
```

Backward 时 Stage 1 将 Activation Gradient 发回 Stage 0。

4.2 Microbatch 与流水线

不拆 Microbatch 时：

```
Stage 0 完成整个 Batch
→ Stage 1 才开始
```

GPU 利用率低。

将 Batch 拆成 m 个 Microbatches 后，不同 Stage 同时处理不同 Microbatch：

```
Stage 0: MB3 Forward
Stage 1: MB2 Forward / 其他 MB Backward
```

1F1B 调度分为：

```
Warm-up → Steady 1F1B → Cool-down
```

普通平衡 PP 的经典 Bubble：

$$T_{bubble} = (PP-1)(t_f+t_b)$$

相对 Useful：

$$Bubble / Useful = (PP-1) \div m$$

Bubble 占实际总时间：

$$Bubble / Total = (PP-1) \div (m+PP-1)$$

4.3 VPP

VPP 将每张物理 Stage 再切成多个 Model Chunks，使同一 GPU 扮演多个虚拟 Stage。

若每卡 v 个 Chunks：

$$T_{bubble_vpp} \approx (PP-1)(t_f+t_b) \div v$$

收益：减少 Bubble。

代价：

- P2P 边界增加；
- Pipeline 通信量约增 v 倍；
- 调度和 Activation 管理更复杂；
- Chunk 太细会增加 Kernel/P2P Launch 开销。

4.4 PP 适用场景

- 模型 Layer 很多，单个 Stage 装不下全模型；
- 希望跨节点扩展，而不扩大高频 TP Group；
- Stage 可以做到计算均衡；
- 有足够 Microbatch 填充流水线。

不适合：

- 模型层数少；
- Global Batch 太小，Microbatch 数不足；
- Stage 严重不均衡；
- Activation 边界特别大且网络慢。

实际例子：96 层模型用 PP=8，每 Stage 12 层；跨节点主要传 Activation，而把 TP 限制在单机内。

五、Context Parallelism（CP）

5.1 CP 切 Sequence

输入：

```
X: [B,S,H]
```

设：

```
B = 2
S = 8
H = 16
CP = 2
```

每个 CP Rank 保存：

```
X_local: [2,4,16]
```

除 Attention 外，Linear、LayerNorm、MLP 通常按本地 Token 独立工作，不需要看到其他 Sequence Chunk。

5.2 为什么 Attention 特殊

本地 Query 只对应 `S/CP` 个 Token，但每个 Query 需要关注完整序列的 Keys/Values：

```
Q_local: [B,S/CP,Heads,HeadDim]
K/V_global: [B,S,KVHeads,HeadDim]
```

所以 CP 需要交换 KV：

- 概念上可看作 KV AllGather；
- 高性能实现常将其转成 Ring P2P，边收 KV Chunk 边计算 Attention；
- Backward 对 dK/dV 使用对应 ReduceScatter 或 Ring 归约。

每卡仅长期保存本地 KV Chunk，Backward 需要时可重新 Gather/交换，以降低 Activation Memory。

5.3 CP 与 SP 区别

维度	Sequence Parallel（SP）	Context Parallel（CP）
所属	通常依附 TP	独立 CP Process Group
切分范围	LayerNorm/Dropout 等非 TP 区域	网络输入和几乎全部激活
Attention	通常仍依赖 TP 布局	专门处理跨 Context KV 依赖
主要目的	减少 TP 重复激活	长上下文 Activation OOM
是否需要 TP>1	通常需要	可以 TP=1

5.4 CP 收益与代价

收益：

```
Activation Memory ≈ 原来的 1/CP
Local Token Compute ≈ 原来的 1/CP
```

代价：

- 每个 Attention Layer 都要交换 KV；
- Causal Attention 需要负载均衡，避免某些 Rank 计算更多有效块；
- CP 太大时每 Rank Q Chunk 太小；
- 通信与 FlashAttention 计算的 Overlap 变难；
- Sequence Length 必须满足分片/打包约束。

GQA/MQA 的 KV Heads 少，因此可显著降低 CP 的 KV 通信量。

5.5 CP 适用场景

- 长上下文，例如 8K、32K、128K；
- Activation Memory 成为主瓶颈；
- 不希望继续增大 TP 或使用昂贵 Full Recomputation；
- Attention Kernel 支持 CP/Ring 计算；
- 能与 TP、PP、DP 组合。

实际例子：S=128K、CP=8，每 Rank 长期处理约 16K Query Tokens，并按 Ring 流式接收其他 KV Chunks。

六、Expert Parallelism（EP）

6.1 EP 切 Experts

MoE Layer 有：

```
E = 4 Experts
EP = 2
```

每个 EP Rank 保存：

```
E_local = E ÷ EP = 2 Experts
```

Router 可把任意 Token 路由到任意 Expert，所以 Token 必须移动到 Expert 所在 Rank。

6.2 两次 All-to-All

Forward：

```
Router Top-k
→ Permute Tokens by Expert
→ Dispatch All-to-All
→ Local Expert / Grouped GEMM
→ Combine All-to-All
→ Unpermute + Weighted Sum
```

Top-k=2 时，每个 Token 产生两个 Token-Expert Assignments。

6.3 EP 收益

- Expert 参数显存约降到 `1/EP`；
- 可扩展到大量 Experts；
- 相比在单个 Expert 内增大 TP，EP 往往能保持更大的本地 GEMM；
- 适合 MoE 架构。

6.4 EP 代价

- 每个 MoE Layer 两次 All-to-All；
- Router 负载不均导致 Rank 热点；
- 每 Expert Token 数动态变化；
- EP 太大时每 Expert GEMM 太小；
- 跨节点 All-to-All 昂贵；
- 需要 Grouped GEMM、Permutation Fusion、负载均衡和专用 Dispatcher。

当 EP 与 TP 组合时，Megatron 类实现通常要求启用 Sequence Parallel，以保证 Token/Hidden 布局可组合。

6.5 EP 适用场景

- 模型存在 MoE Layers；
- Expert 总参数超过单卡；

- All-to-All 网络足够快；
- 每 Expert 能获得足够多 Token；
- 有成熟的 Router 与 Grouped GEMM。

实际例子：64 Experts、EP=8，每 Rank 保存 8 Experts；Top-2 Token 经 All-to-All 到对应 Rank。

七、六类并行的完整异同比较

维度	DP/DDP	FSDP/ZeRO	TP	PP/VPP	CP	EP
切分对象	Batch	Batch + 模型状态	Layer 内 Tensor	Layers/Chunks	Sequence	Experts
是否复制模型参数	是	分片	分片	每 Stage 只持有部分 Layers	通常复制当前模型分片	只持有部分 Experts
每卡计算量	约不变	约不变	约 `1/TP`	约 `1/PP`	约 `1/CP`，Attention含通信	只算路由到本地 Experts
参数显存	不降	约 `1/DP`	约 `1/TP`	约 `1/PP`	通常不降	Expert 参数约 `1/EP`
Activation 显存	Local Batch	Local Batch	SP时部分约 `1/TP`	Stage内；VPP可能更复杂	约 `1/CP`	动态，依路由
主要通信	Gradient AR	Param AG + Grad RS	每层 AR/AG/RS	P2P Activation	KV Ring/AG + dKV RS	两次 A2A
通信频率	每 Step/Bucket	每Unit前后向	每Layer多次	每Microbatch边界	每Attention Layer	每MoE Layer
Overlap难度	较容易	可Prefetch	较难	依调度	与Attention计算流水	复杂且动态
最适合链路	可跨节点	高带宽更好	机内NVLink	可跨节点	高带宽域优先	NVLink/高速网络
核心目标	扩吞吐	省训练状态显存	切大层	切深模型	长上下文	切MoE专家
主要风险	状态复制	通信/峰值	小GEMM、高频通信	Bubble/不均衡	KV通信	A2A/负载不均

口诀：

```
Batch 太大要扩吞吐 → DP
训练状态放不下 → FSDP
单层放不下 → TP
层数太多/跨节点扩模型 → PP
长序列 Activation OOM → CP
MoE Experts 放不下 → EP
```

八、统一组合例子：64 卡五维并行

8.1 模型与数据

使用一个教学型 MoE Transformer：

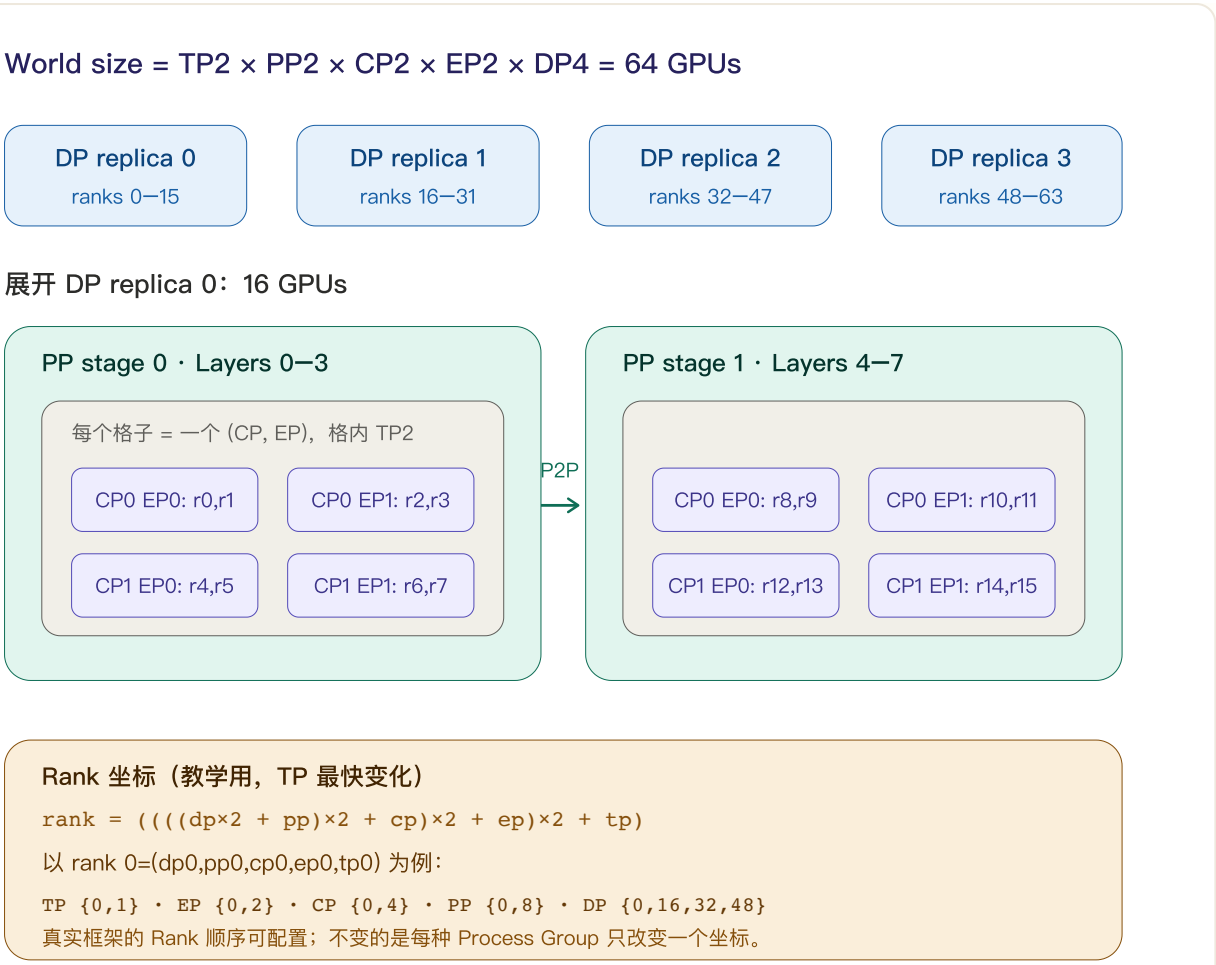
```
World Size = 64 GPUs
Layers L = 8
Hidden H = 16
Q Heads = 4
KV Heads = 2
HeadDim = 4
Dense FFN F = 64
MoE Experts E = 4
MoE Top-k = 2
Global Batch = 8
Sequence Length S = 8
Gradient Accumulation = 1
```

并行配置：

```
TP = 2
PP = 2
CP = 2
EP = 2
DP = 4
```

World Size：

```
2 × 2 × 2 × 2 × 4 = 64
```



每个 DP Replica 需要完整覆盖一个逻辑模型，因此占用：

```
TP × PP × CP × EP
= 2 × 2 × 2 × 2
= 16 GPUs
```

共有 4 个 DP Replicas：

```
Replica 0: ranks 0–15
Replica 1: ranks 16–31
Replica 2: ranks 32–47
Replica 3: ranks 48–63
```

8.2 Rank 五维坐标

教学用 Rank 排列，TP 最快变化：

```
rank = ((((dp×PP + pp)×CP + cp)×EP + ep)×TP + tp)
```

代入各维 Size=2/4：

```
rank = ((((dp×2 + pp)×2 + cp)×2 + ep)×2 + tp)
```

Rank 0 坐标：

```
(dp=0, pp=0, cp=0, ep=0, tp=0)
```

固定其他坐标，只改变一个维度，得到 Rank 0 所属通信组：

```
TP Group: {0,1}
EP Group: {0,2}
CP Group: {0,4}
PP Group: {0,8}
DP Group: {0,16,32,48}
```

真实框架 Rank Ordering 可以配置，例如 `tp-cp-ep-dp-pp`；不变的是：每个 Process Group 只沿一个逻辑坐标变化。

8.3 PP 切 Layers

```
PP Stage 0: Layers 0–3
PP Stage 1: Layers 4–7
```

每个 Stage 内部再由 TP×CP×EP=8 张 GPU 协作。

若启用 VPP=2，可进一步变成：

```
GPU Group for Stage 0: Chunk 0 + Chunk 2
GPU Group for Stage 1: Chunk 1 + Chunk 3
```

具体 Layer 分配取决于框架和整除条件。

8.4 DP 切 Batch

```
Global Batch = 8
DP = 4
```

每个 Replica：

```
B_local = 8 ÷ 4 = 2
```

所以 Replica 0 的逻辑输入：

```
X_replica0: [2,8,16]
```

四个 DP Replicas 处理不同的 2 个样本。

8.5 CP 切 Sequence

```
CP = 2
S_local = 8 ÷ 2 = 4
```

每个 CP Rank：

```
X_cp_local: [2,4,16]
```

Attention 中，本地 Query 只对应 4 个 Token：

```
Q_local: [2,4,4 Heads,4]
```

再考虑 TP=2，每个 TP Rank 持有 2 个 Q Heads：

```
Q_local_tp: [2,4,2,4]
```

KV Heads=2，TP=2 时每 TP Rank 可持有 1 个 KV Head：

```
K/V_local_tp: [2,4,1,4]
```

但本地 Query 必须与完整 Sequence 的 KV 交互，因此 CP Group 交换 KV，逻辑可见长度恢复为 8：

```
K/V_visible: [2,8,1,4]
```

高性能实现可以不物化完整 AllGather，而是以 Ring 方式逐 Chunk 计算 Attention。

8.6 TP 切 Dense Attention/MLP

Attention：

```
4 Q Heads ÷ TP2 = 2 Heads/Rank
2 KV Heads ÷ TP2 = 1 KV Head/Rank
```

Dense MLP：

```
W₁ global: [16,64]
W₁ local:  [16,32]

W₂ global: [64,16]
W₂ local:  [32,16]
```

每个 TP Rank 计算部分中间维，W_2 后通过 AllReduce 或 ReduceScatter 归并部分和。

8.7 EP 切 MoE Experts

```
E = 4
EP = 2
```

每个 EP Rank：

```
2 Local Experts
```

当前 DP Replica、CP Chunk 的逻辑 Token 数：

```
T_local = B_local × S_local
        = 2 × 4
        = 8 Tokens
```

Top-2：

```
Assignments = 8 × 2 = 16
```

Router 将这 16 个 Assignments 按 Expert Owner 分组：

```
Dispatch All-to-All across EP2
→ 每 Rank 处理路由到两个 Local Experts 的 Tokens
→ Combine All-to-All
```

如果 TP 也用于 Expert 内矩阵，Expert MLP 权重还会沿 TP2 分片。Megatron 类实现中 EP+TP 通常要求启用 SP，以保持 Token/Hidden 布局可组合。

九、组合并行中一个训练 Step 的完整数据流

统一例子：B_global=8，S=8，H=16，Layers=8，E=4，Top-2

1. DP4 切 Batch

每个 DP replica 获得 B_local=8/4=2 个样本；四个 replica 处理不同数据。

↓

2. CP2 切 Sequence

[2,8,16] → 每 CP rank [2,4,16]；本地 Q 长度4，KV 通过 Ring/AG覆盖完整长度8。

↓

3. TP2 执行 Attention / Dense MLP

4个Q Heads → 每TP rank 2 Heads；W_1[16,64] → [16,32]，W_2[64,16] → [32,16]。

Column/Row Parallel通过 AllReduce 或 ReduceScatter/AllGather组合恢复逻辑输出。

↓

4. EP2 执行 MoE Layer

E=4 → 每EP rank 2 Experts；本地逻辑Tokens=2×4=8，Top-2 → 16 Assignments。

Dispatch All-to-All → Local Experts/Grouped GEMM → Combine All-to-All。

↓

5. PP2 / VPP 传递 Stage 边界

Stage 0 Layers 0–3 → P2P Activation → Stage 1 Layers 4–7；Backward 反向传 dActivation。

↓

6. Backward 后 DP / FSDP 同步模型状态

DDP：相同(tp,pp,cp,ep)分片在dp0..3间AllReduce梯度。

FSDP：ReduceScatter梯度、分片Optimizer更新；下一次用参数前AllGather。

同一时刻并非所有通信都串行：TP/CP/EP/DP通信可在不同依赖窗口与计算重叠，但会争用SM、HBM和互联。

9.1 数据读取与 DP

DP Replica 0 读取样本 0–1，Replica 1 读取 2–3，依此类推。

每个 Replica 内 First PP Stage 获取 Tokens：

```
[2,8]
```

Embedding 后：

```
[2,8,16]
```

9.2 CP 分片

按 Sequence 切成两个 Chunk：

```
CP0: Tokens 0–3 → [2,4,16]
CP1: Tokens 4–7 → [2,4,16]
```

所有后续 Activation 主要保持 CP 分片。

9.3 TP Attention

每个 CP Rank 内，TP0/TP1 分别处理一半 Heads。

本地 Query：

```
[2,4,2,4]
```

CP Ring 交换 KV Chunk：

```
Round 0: 计算本地 Q × 本地 KV
Round 1: 计算本地 Q × 远端 KV
```

在线 Softmax 合并两个 KV Chunk 的 Attention 结果。

TP Output Projection 后归约部分结果，使逻辑 Hidden 输出保持 `[2,4,16]`，物理上可继续以 SP/TP 兼容布局保存。

9.4 Dense MLP 的 TP

```
[2,4,16]
→ flatten [8,16]
→ Column Parallel W₁, 每TP Rank [8,32]
→ Activation
→ Row Parallel W₂, 部分输出 [8,16]
→ ReduceScatter/AllReduce
```

9.5 MoE Layer 的 EP + TP

Router 对逻辑 `[8,16]` Token Hidden States 计算 Top-2，得到 16 Assignments。

EP：

```
Dispatch A2A → 目标 Expert Rank
```

TP：

```
每个 Expert 内的大矩阵继续由 TP2 切分
```

本地 Experts 使用 Grouped GEMM，随后：

```
Combine A2A → Token Origin EP Rank
→ Unpermute
→ Weighted Sum
```

9.6 PP Stage 边界

Stage 0 完成 Layers 0–3 后，将 Activation 发到对应 Stage 1 Rank：

```
rank(dp,pp=0,cp,ep,tp)
→ rank(dp,pp=1,cp,ep,tp)
```

例如：

```
rank 0 → rank 8
rank 1 → rank 9
...
rank 7 → rank 15
```

通信 Tensor 保留与 CP/TP/EP 布局兼容的分片，而不是必须先在单卡上拼成全局 `[2,8,16]`。

9.7 Backward

反向顺序大致相反：

```
Stage 1 Backward
→ PP P2P 发送 dActivation 给 Stage 0
→ MoE Combine/Expert/Dispatch 的反向通信
→ TP Collectives
→ CP dKV ReduceScatter/Ring
→ 各 Rank 得到本地参数梯度
```

9.8 DP/DDP 或 FSDP 更新

对于 Dense 参数的某个相同模型分片：

```
固定 pp,cp,ep,tp
改变 dp
```

例如 Rank 0 对应 DP Group：

```
{0,16,32,48}
```

DDP：

```
Gradient AllReduce across DP Group
→ 每个 Replica 的该分片参数保持一致
```

FSDP：

```
Gradient ReduceScatter
→ 每个 DP Rank 只保留梯度分片
→ 本地 Optimizer 更新状态分片
→ 下次计算该 Unit 前 Parameter AllGather
```

CP Rank 之间通常复制相同权重、处理不同 Sequence Chunk，因此权重梯度还必须在适当的 DP-with-CP 组中归约。MoE Expert 参数则使用“拥有同一 Expert Shard 的 Expert-Data-Parallel Group”同步，而不是盲目跨所有 EP Rank 同步。

十、一个 Rank 到底存什么：以 Rank 0 为例

Rank 0 坐标：

```
(dp0, pp0, cp0, ep0, tp0)
```

它负责：

```
DP: Global Batch中的样本0–1
PP: Layers 0–3
CP: 每个样本的Sequence前4个Token
EP: Experts 0、1
TP: Attention Heads前半、Linear矩阵前半Shard
```

Rank 0 不拥有：

- 样本 2–7，它们属于其他 DP Replicas；
- Layers 4–7，它们属于 PP Stage 1；
- Sequence 后 4 个 Token 的长期 Activation，它们属于 CP1；
- Experts 2、3，它们属于 EP1；
- TP 的另一半矩阵/Heads，它们属于 TP Rank 1。

但在执行过程中，Rank 0 会通过通信短暂看到：

- CP1 的 KV Chunk；
- 被其他 EP Rank 路由到本地 Experts 的 Token；
- TP Collective 的部分和；
- FSDP AllGather 的临时参数；
- PP 上下游的 Activation 或 Gradient。

这就是区分 **常驻 Shard** 与 **通信时临时可见数据** 的关键。

十一、进程组与通信不会混在一起

一个物理 Rank 同时属于多个 Process Groups：

```
TP Group：层内矩阵协作
PP Group：相邻 Stage
CP Group：序列上下文协作
EP Group：专家 Token 路由
DP Group：不同数据副本的梯度/状态同步
```

每个 Collective 必须使用正确 Group：

```
TP AllReduce 不能错误地包含 DP Rank
EP All-to-All 不能发送给不相关 PP Stage
DP AllReduce 不能混合不同 TP Shard
CP KV Exchange 只在相同模型分片位置进行
```

分布式 Hang 和 Silent Wrong Result 很多都来自 Process Group、Rank Mapping、Collective 顺序或 Tensor Shape 不一致。

十二、如何把 64 卡映射到真实硬件拓扑

假设每节点 8 张 NVLink GPU，共 8 个节点。

本例每 PP Stage 内：

```
TP2 × CP2 × EP2 = 8 GPUs
```

一种自然映射：

```
一个节点 = 一个 DP Replica 的一个 PP Stage
```

于是：

- TP/CP/EP 通信限制在节点内 NVLink；
- PP 在节点间走较少频率的 P2P；
- DP Group 跨多个节点进行梯度同步；
- 4 个 DP Replicas × 2 个 PP Stages = 8 Nodes。

这符合通用原则：

```
高频、强依赖通信（TP/EP/CP）放快链路
较低频或可Overlap通信（PP/DP）允许跨节点
```

但如果 EP All-to-All 的消息很大，可能需要特殊 Dispatcher 或跨节点 EP；实际布局应以节点拓扑、模型 Shape 和 Profiler 为准。

十三、组合策略的配置示例

Megatron-LM 风格的教学配置：

```
--tensor-model-parallel-size 2 \
--pipeline-model-parallel-size 2 \
--context-parallel-size 2 \
--expert-model-parallel-size 2 \
--num-experts 4 \
--moe-router-topk 2 \
--sequence-parallel \
--moe-grouped-gemm \
--moe-token-dispatcher-type alltoall \
--data-parallel-sharding-strategy optim_grads_params \
--use-megatron-fsdp
```

概念含义：

- TP2：切 Layer 内矩阵；
- PP2：切 Layers；
- CP2：切 Sequence；
- EP2：切 Experts；
- SP：支持 TP/EP 激活布局；
- Grouped GEMM：合并本地 Expert 小 GEMM；
- All-to-All Dispatcher：路由 Token；
- FSDP Full Shard：在 DP4 上切训练状态。

具体参数名和约束随 Megatron Core 版本变化，生产环境必须以实际版本文档和源码为准。

十四、如何选择并行配置

14.1 第一步：先解决显存可行性

按顺序问：

1. 单层是否放得下？放不下先增加 TP。
2. 总 Layers 是否放得下？放不下增加 PP。
3. 长序列 Activation 是否 OOM？增加 CP 或使用 Checkpoint。
4. MoE Expert 参数是否放得下？增加 EP。
5. 参数/梯度/Optimizer 是否放得下？使用 FSDP/ZeRO。

14.2 第二步：最小化模型并行

模型并行越大，通常通信越多、单卡 GEMM 越小。

因此在满足显存后：

```
尽量减小 TP、PP、CP、EP
尽量增大 DP
```

但不能机械执行：MoE Expert Layer 有时增加 EP 反而比增加 TP 更高效；长序列增加 CP 可能比扩大 TP 或 Full Recompute 更好。

14.3 第三步：拓扑感知

- TP 优先机内；
- EP/CP 尽量机内或高速域；
- PP 适合跨节点；
- DP/FSDP 可跨节点并与 Backward Overlap；
- Process Group 必须对应物理链路。

14.4 第四步：用 Profiler 调整

监控：

```
Step Time / Tokens per Second / MFU
TP Collective Exposed Time
PP Bubble / Stage Imbalance
CP KV Communication
EP Dispatch/Combine A2A
DP Gradient Sync
FSDP Parameter AllGather
Per-GPU Peak Memory
GEMM Shape 与 Tensor Core Utilization
```

优化一个通信维度可能把瓶颈转移到另一个维度，所以必须端到端测量。

十五、常见误区

误区 1：总 GPU 数永远等于 TP×PP×CP×EP×DP

这是常见的笛卡尔积教学模型，也是 Megatron Guide 的基础公式；但高级 MoE Parallel Folding、Expert Tensor Parallel、Hierarchical DP 等可能让 Attention 与 MoE 使用不同进程组布局。真实 World Size 和 Group 关系应以框架配置为准。

误区 2：CP 和 SP 是同一个东西

不是。SP 是 TP 内部的激活优化，CP 是完整 Context/Sequence 维度并行，Attention 需要额外 KV 通信。

误区 3：FSDP 能替代 TP

不能。FSDP 在计算 Layer 前会 AllGather 参数；若单层本身放不下，仍需要 TP。

误区 4：PP 会把单层切开

不会。PP 切 Layers；TP 才切单层 Tensor。

误区 5：EP 只切参数，不通信 Token

错误。动态 Router 让 Token 路由到远端 Experts，Forward 通常需要 Dispatch/Combine 两次 All-to-All。

误区 6：并行度越大越快

错误。并行度提高会减少每卡工作，但增加通信、同步和小 GEMM。最优点取决于模型 Shape 与硬件拓扑。

误区 7：所有通信都必须串行执行

错误。DDP Bucket AllReduce、FSDP Prefetch、CP Ring、EP A2A 和 PP 调度都有 Overlap 机会；但它们可能争用 SM、HBM 与网络，不能简单相加。

十六、关键总结

把所有策略串成一条决策链：

```
先看一个 Layer:
放不下 → TP

再看模型深度:
全部 Layers 放不下 → PP / VPP

再看 Sequence Activation:
长上下文 OOM → CP

再看 MoE Experts:
全部 Experts 放不下 → EP

再看训练状态:
参数/梯度/Optimizer 放不下 → FSDP / ZeRO

模型已经能放下后:
剩余 GPU → DP 扩吞吐
```

组合并行不是让数据被随意切碎，而是为每种逻辑维度建立独立 Process Group：

```
TP 组负责层内矩阵
PP 组负责层间 Activation
CP 组负责 Context KV
EP 组负责 Expert Token
DP 组负责不同数据副本的模型一致性
```

一个 Rank 同时属于多个组，每个通信操作只在正确组内执行。

可选自测

1. World Size 与 Rank Group

配置：

```
TP=4, PP=2, CP=2, EP=1, DP=8
```

- World Size 是多少?
- 一个 Model Replica 占多少 GPU?
- 每个 DP Rank 的 Local Batch 如何由 Global Batch 计算?

2. Shape 题

```
B_global=32, DP=4, S=32768, CP=8, H=8192,
Q Heads=64, TP=8
```

计算每个 Rank 的：

- `B_local`；
- `S_local`；
- Q Heads 数；
- 逻辑 Q Shape；
- CP Attention 需要看到的 KV Sequence Length。

3. 工程选型题

一个 70B Dense 模型训练表现：

```
TP=8 已在单机
PP=8 跨8节点
DP=2
Sequence=128K
Activation OOM, 需要 Full Recomputation
TP Collective 已占 Step 20%
```

你会优先增加 TP、PP、CP 还是 DP？为什么？如何重新映射拓扑？

继续学习

1. 前置：Process Group 与 Rank Mapping —— 彻底理解多维并行如何建立通信组和避免 Collective Hang。
2. 同层对比：Megatron 3D/5D Parallel 与 DeepSpeed/FSDP —— 对比框架在状态分片和调度上的实现。
3. 下游应用：大模型并行配置自动搜索 —— 用显存模型、通信模型和拓扑约束寻找最优 TP/PP/CP/EP/DP。

回复 1、2 或 3 继续；也可以说“稍后”或“不感兴趣”。

事实来源与版本边界

本文基于截至 2026-07-29 的 NVIDIA Megatron Core Parallelism Guide、Context Parallelism 文档、MoE 文档，以及 PyTorch DDP/FSDP 的稳定语义整理。不同框架在 Rank Ordering、Sequence Parallel、Expert Data Parallel、FSDP Unit、VPP Schedule、Parallel Folding 和通信后端上存在差异；具体参数、整除约束、Process Group 和性能应以实际版本源码、模型 Shape 与硬件 Profiler 为准。

大模型数值精度全解：FP32、BF16、FP16、FP8、INT8、INT4 与 FP4

> **原始问题：**详细讲解不同精度：bf32, 16, 8等等，有什么区别，作用在哪里，如何影响训练的性能和显存等指标

提问：2026/08/02 21:09　更新：2026/08/02 21:16　重要度：5/5

大模型数值精度全解：FP32、TF32、BF16、FP16、FP8、INT8、INT4 与 FP4

先纠正一个术语：`BF32` 通常不是你真正想问的格式

在主流 PyTorch、CUDA、NVIDIA Tensor Core 与大模型训练语境中，常见的是：

- `FP32`：32-bit IEEE single precision；
- `TF32`：TensorFloat-32，NVIDIA 的矩阵计算模式；
- `BF16`：bfloat16，16-bit Brain Floating Point；
- `FP16`：IEEE half precision；
- `FP8`：常见 E4M3、E5M2；
- `INT8 / INT4`：整数定点量化；
- `FP4`：例如 Blackwell 上的 NVFP4 E2M1。

`BF32` **不是当前 PyTorch/CUDA 大模型训练栈里的统一主流标准名称。**有些非权威资料会自行使用这个词，但不能把它当作像 BF16、FP32 那样稳定且通用的格式。结合你的表达“bf32、16、8”，本文按你想系统比较 **FP32、16-bit、8-bit 及更低精度** 来讲。

一句话结论

1. **位宽越低，单个元素占用的显存、HBM 流量和通信 payload 越小，支持该格式的 Tensor Core 吞吐通常越高。**
2. **位宽越低，能够表示的数值范围或相邻数密度越差，更容易发生溢出、下溢、舍入误差和累加误差。**
3. **大模型训练几乎从来不是“全模型只用一种精度”，而是混合精度：**

- GEMM/卷积等大计算使用 BF16、FP16、FP8；
- 累加、Softmax、归一化、损失和部分归约使用 FP32 或其他较高精度；
- Adam 的一阶/二阶状态通常仍保留 FP32；
- FP16 常配 Dynamic Loss Scaling；
- FP8/FP4 还需要逐张量或逐块 Scaling。

1. **BF16 是现代大模型训练的通用稳健默认值；FP16 范围小、通常需要 Loss Scaling；FP8 是更激进的训练优化；INT8/INT4 主要用于推理量化；FP4 训练属于新硬件与专用 Recipe 驱动的前沿方案。**
2. **“理论 FLOPS 翻倍”不等于 Step Time 减半。**端到端收益取决于 Tensor Core 覆盖率、Memory-Bound 比例、非 GEMM 算子、通信、Cast/Scale 开销、Shape 对齐和数值回退。

一、浮点数究竟如何表示一个数

一个规格化二进制浮点数可以写成：

```
value = (−1)^sign × 2^exponent × significand
```

格式通常分成三部分：

```
Sign        符号位：决定正负
Exponent    指数位：决定数量级范围
Mantissa    尾数/小数位：决定同一数量级内能刻多细
```

直觉：

- **指数位多**：能表示特别大和特别小的数，不容易 Overflow/Underflow；
- **尾数位多**：相邻可表示数更密，舍入误差更小；

- **总位宽少**：必须在“范围”和“精细程度”之间做更激进的取舍。

位数怎么分，决定“装得多大”和“刻得多细”

指数位 → 动态范围；尾数位 → 相对精度；符号位 → 正负

格式	位布局	工程直觉
FP32	S | Exponent 8 | Mantissa 23	范围大，精度高，4 B
TF32	S | Exponent 8 | Mantissa 10 | 计算模式	FP32存储，GEMM截尾
BF16	S | Exponent 8 | Mantissa 7 | 总计16 bit	FP32范围，精度较粗
FP16	S | Exp 5 | Mantissa 10 | 总计16 bit	更细，但范围窄
FP8 E4M3	S | E4 | M3 | 需scale，通常前向	更精细，最大约448
FP8 E5M2	S | E5 | M2 | 需scale，常用于梯度	范围更大，最大57344

BF16与FP16：同为2 Byte；BF16保范围，FP16保更多有效数字

1.1 范围与精度不是同一件事

假设都在 `1.0` 附近：

```
FP32 下一个可表示数约为：1 + 2^-23
FP16 下一个可表示数约为：1 + 2^-10
BF16 下一个可表示数约为：1 + 2^-7
```

所以：

```
1.0 + 0.0001
```

在 FP32 中可以产生可见变化，但在 FP16、BF16 中可能仍被舍入回 `1.0`。这就是“小更新被大数吃掉”。

然而 BF16 虽然比 FP16 尾数更少，却拥有与 FP32 一样宽的 8-bit 指数，因此能覆盖接近 FP32 的数量级范围。

二、常见精度格式总表

下表中的动态范围和 `epsilon near 1` 用于建立量级直觉。具体硬件可能对 Subnormal、NaN/Inf、累加方式和舍入模式有额外行为。

格式	总位数	Sign / Exp / Mantissa	每元素字节	最大有限值（约）	1附近相对间隔（约）	核心用途
FP64	64	1 / 11 / 52	8 B	`1.8e308`	`2.22e-16`	科学计算、数值敏感求解
FP32	32	1 / 8 / 23	4 B	`3.4e38`	`1.19e-7`	稳定基线、累加、优化器状态
TF32	计算有效位约19	1 / 8 / 10	输入/输出通常仍以FP32存储	接近FP32	`9.77e-4` 量级	Ampere及以后加速FP32 GEMM/Conv
BF16	16	1 / 8 / 7	2 B	`3.39e38`	`7.81e-3`	现代大模型训练默认低精度
FP16	16	1 / 5 / 10	2 B	`65504`	`9.77e-4`	混合精度训练、旧硬件兼容
FP8 E4M3	8	1 / 4 / 3	1 B	`448`	很粗	常用于前向权重/激活
FP8 E5M2	8	1 / 5 / 2	1 B	`57344`	更粗	常用于范围更大的梯度
INT8	8	整数，无指数	1 B	有符号通常 `-128..127`	固定步长	推理权重/激活量化、部分通信
INT4	4	整数，无指数	0.5 B	有符号量化范围依方案	固定步长	Weight-Only推理量化
FP4 E2M1	4	1 / 2 / 1	0.5 B	原始值幅度约到 `6`	极粗	NVFP4等块缩放训练/推理

两个重要说明：

1. **TF32 不是通常意义上把模型权重存成 19-bit 的 dtype。**它主要是 Tensor Core 执行 FP32 GEMM/Conv 时对输入有效尾数做截短/舍入，输出与累加仍走 FP32 语义或较高精度路径。
2. **FP8/FP4 的原始范围不能脱离 Scaling 来看。**真实训练通过 Scale 把张量当前范围映射进很窄的低精度格式。

三、逐个讲清每种精度

3.1 FP64：大模型训练通常不需要，但数值科学需要

FP64 有约 16 位十进制有效数字，范围也极大。

适合：

- 数值线性代数；
- 高条件数问题；
- 分子动力学、计算流体、科学仿真；
- 需要严格误差控制的统计或优化算法。

不适合常规 LLM 训练的原因：

- 每元素 8 Byte；
- 参数、激活和带宽压力是 FP32 的 2 倍、BF16 的 4 倍；
- AI GPU 的 FP64 吞吐通常不是为大模型 Tensor Core 热路径设计的；
- Transformer 训练一般不需要如此高的精度。

但如果你做的是极其敏感的 `torch.linalg`、条件数很差的分解，FP64 可能是诊断工具。

3.2 FP32：稳定基线与“精度锚点”

FP32 长期是神经网络训练基线：

- 动态范围大；
- 约 7 位十进制有效数字；
- 数值调试相对容易；
- 适合作为累加器、优化器状态和敏感归约精度。

但它的代价是：

```
每个值 4 Byte
```

相对 BF16/FP16：

- 参数/激活原始存储翻倍；
- HBM 流量翻倍；
- 通信 payload 翻倍；
- 在支持低精度 Tensor Core 的 GPU 上，矩阵计算吞吐通常明显更低。

现代大模型很少“所有地方都 FP32”，而是让 FP32 留在最需要稳定性的地方。

3.3 TF32：FP32 接口下的 Tensor Core 快速矩阵模式

TF32 的正确心智模型：

```
存储：通常仍是FP32 Tensor
GEMM/Conv输入有效精度：约8-bit指数 + 10-bit尾数
乘加/输出：使用Tensor Core和FP32级累加路径
```

它保留 FP32 的指数范围，但有效尾数接近 FP16，因此：

- 比完整 IEEE FP32 GEMM 更快；
- 不需要把模型显式转成 `torch.float16`；
- **不会把参数显存从 4 Byte 降成 2 Byte**；
- 数值误差高于完整 FP32。

截至 PyTorch 2.13 的接口趋势：

```
torch.backends.cuda.matmul.fp32_precision = "tf32"
torch.backends.cudnn.conv.fp32_precision = "tf32"
```

需要完整 IEEE FP32 时：

```
torch.backends.cuda.matmul.fp32_precision = "ieee"
torch.backends.cudnn.conv.fp32_precision = "ieee"
```

TF32 适合：

- 想保留 FP32 Tensor API；
- 模型对 TF32 误差不敏感；
- 不急于节省参数/激活显存；
- 希望先以最小代码改动获得 Tensor Core 加速。

3.4 BF16：现代 LLM 训练最常见的低精度默认值

BF16 位布局：

```
1 Sign + 8 Exponent + 7 Mantissa
```

它的核心设计是：

```
牺牲尾数精度
换取接近FP32的动态范围
```

优点：

- 每元素 2 Byte；
- 指数范围接近 FP32；
- 大梯度、大激活不容易像 FP16 那样 Overflow；
- 小梯度也比 FP16 更不容易因指数范围而 Underflow；
- 大模型预训练通常不需要 Loss Scaling；
- Ampere 及以后 GPU 对 BF16 Tensor Core 支持成熟。

缺点：

- 只有 7 位显式尾数，`1.0` 附近间隔约为 `0.0078125`；
- 直接用 BF16 保存长期的小更新，可能被舍入掉；
- Reduction、Optimizer State、Softmax/Norm 等仍常需 FP32；
- 并非所有算子和硬件都同样高效。

一句话：

> BF16 选择“范围优先”，非常适合数值分布跨度大的深度学习训练。

3.5 FP16：尾数较细，但范围明显更窄

FP16 位布局：

```
1 Sign + 5 Exponent + 10 Mantissa
```

关键范围：

```
最大有限值：65504
最小正规正数：约6.10e-5
最小Subnormal：约5.96e-8
```

相对 BF16：

- FP16 在 `1.0` 附近更精细；
- 但指数范围窄很多；
- 大值容易 Overflow 为 Inf；
- 小梯度容易 Underflow 为 0。

因此 FP16 训练通常需要 Dynamic Loss Scaling。

适合：

- GPU 不支持 BF16、但支持 FP16 Tensor Core；
- 已验证模型在 FP16 下稳定；
- 需要较细的局部尾数精度；
- 有成熟 AMP/GradScaler 流程。

不适合直接把一个 BF16 预训练模型无脑转换成 FP16：该模型训练中形成的数值范围可能超过 FP16 的 `65504`。

3.6 FP8：不只是把 BF16 再砍一半

FP8 通常有两种互补格式：

```
E4M3：指数4位、尾数3位
      精度相对更好，范围更小，最大幅值约448

E5M2：指数5位、尾数2位
      精度更差，范围更大，最大幅值约57344
```

典型 Hybrid Recipe：

```
Forward Weight / Activation: E4M3
Backward Gradient: E5M2
Accumulator / Sensitive Ops: 更高精度
```

为什么 FP8 必须 Scaling?

不同层、不同张量的 `amax = max(abs(x))` 可能相差几个数量级。若直接转 FP8：

- Scale 太小：大值 Saturate/Overflow；
- Scale 太大：大多数小值量化成 0；
- 一个全局 Loss Scale 无法适配所有 Activation、Weight 和 Gradient。

概念上：

```
x_scaled = x × scale
x_fp8 = cast_to_fp8(x_scaled)
x_reconstructed ≈ cast_high(x_fp8) ÷ scale
```

常见 Scaling：

- **Delayed Scaling**：使用过去若干 Step 的 `amax history` 计算本 Step Scale；性能好，但对分布突变有滞后；
- **Current Scaling**：基于当前张量范围决定 Scale；更及时，但朴素实现可能需要额外遍历；
- **Block/Microscaling**：每 16/32 个值共享一个 Scale，降低 Outlier 对整张 Tensor 的影响。

FP8 的额外成本：

- 维护 Scale 与 `amax history`；
- Cast/Quantize/Dequantize；
- 可能需要 Scale 同步；
- Outlier、Saturation 和分布漂移监控；
- Shape/Kernel/硬件约束。

所以 FP8 是否更快取决于：

```
Tensor Core收益 + HBM/通信节省
>
量化、Scaling、同步和不支持算子的回退开销
```

3.7 INT8 与 INT4：量化，不是普通浮点缩窄

整数没有指数。要用 INT8/INT4 表示浮点张量，需要 Scale，可能还需要 Zero Point。

典型仿射量化：

```
q = clip(round(x ÷ scale) + zero_point)

x_reconstructed
≈ scale × (q − zero_point)
```

量化粒度：

- Per-Tensor：整张 Tensor 一个 Scale；
- Per-Channel：每个输出通道一个 Scale；
- Per-Group：每组 32/64/128 个权重一个 Scale；
- Per-Token：每个 Token/行一个 Scale；
- Per-Block：二维块共享 Scale。

INT8 常见用途：

- Weight + Activation 推理；
- PTQ/QAT；
- 部分低精度通信；
- KV Cache 量化。

INT4 常见用途：

- Weight-Only Quantization；
- LLM 推理中压缩模型权重；
- 让更大模型放入更少 GPU；
- 减少权重读取带宽。

但 INT4 推理可能从 GEMM Compute-Bound 变成：

```
反量化开销 + Scale读取 + 小Batch GEMV + Kernel效率
```

因此 4-bit 权重不保证端到端速度一定是 BF16 的 4 倍。

3.8 FP4 / NVFP4：前沿低精度训练，不是“裸 E2M1”

FP4 E2M1 原始可表示值非常稀疏，幅度约到 `±6`。单独使用几乎无法覆盖 LLM 的 Activation、Weight 和 Gradient 分布。

NVFP4 通过 Recipe 才变得可用：

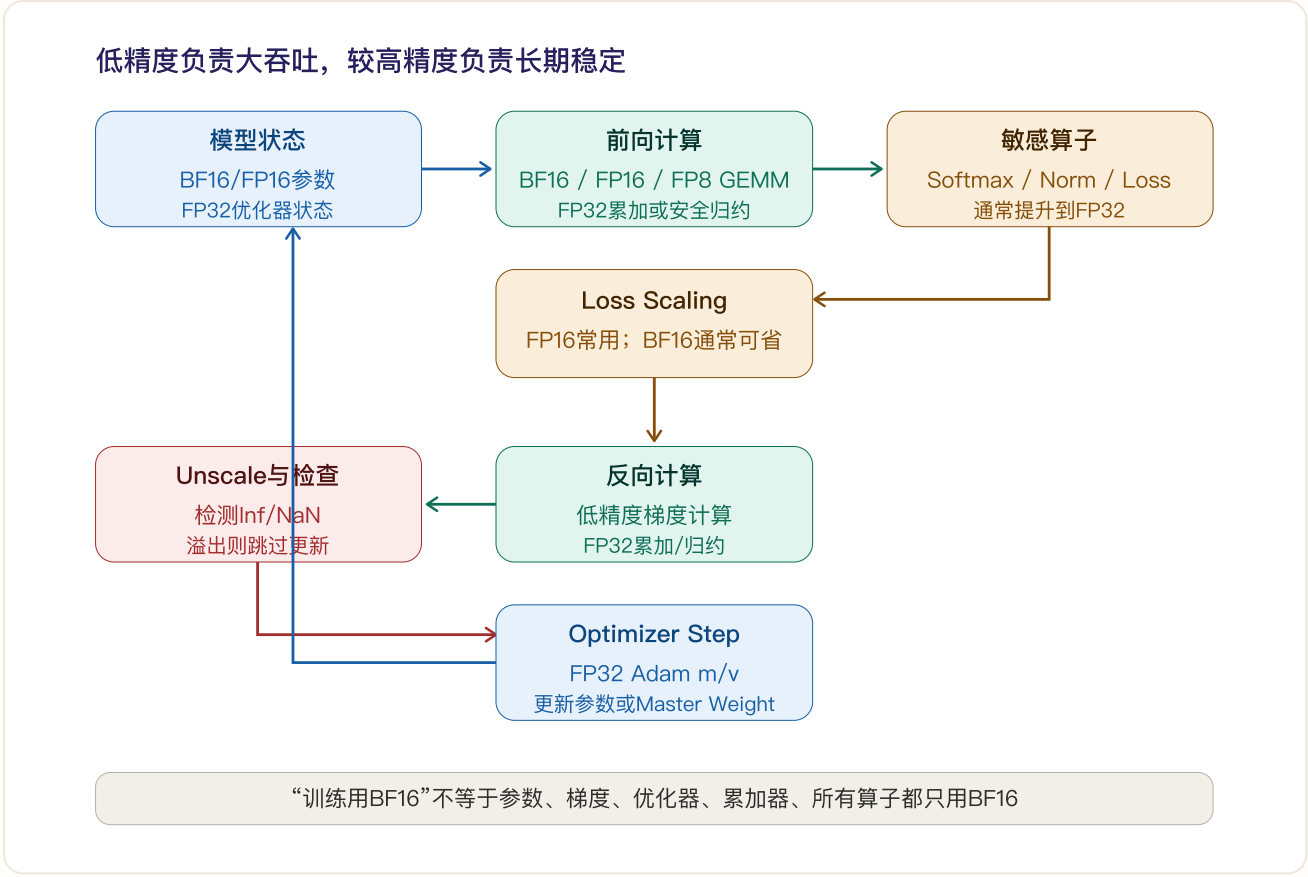
- 每 16 个值共享一个 E4M3 Block Scale；
- 额外使用 Per-Tensor FP32 Scale 防止全局 Overflow；
- Weight 可使用二维 16×16 Block Scaling；
- Gradient 使用 Stochastic Rounding 减少舍入偏差；
- 使用 Hadamard Transform 平滑 Outlier；
- 部分张量和敏感操作继续保留更高精度。

因此：

> “FP4 训练”不是所有状态都存成 4 bit，而是专用 Tensor Core、微块缩放、随机舍入、分布整形和混合精度共同组成的训练 Recipe。

截至 2026-08-02，这仍强依赖 Blackwell 级硬件与 Transformer Engine 等专用软件栈，不能把它当作任何 GPU 上的通用默认训练精度。

四、混合精度训练到底混合了什么



真实训练至少要区分以下精度：

对象/阶段	常见精度	为什么
GEMM输入权重	BF16/FP16/FP8	Tensor Core吞吐、减少HBM读取
激活	BF16/FP16，部分FP8	显存大户，低精度收益明显
GEMM乘积累加	FP32或硬件定义的较高精度	长Reduction易累积误差/溢出
Softmax输入与Reduction	通常提升FP32	Exp和求和数值敏感
LayerNorm/RMSNorm统计	常用FP32统计	均值/方差/平方和敏感
Loss	通常FP32	训练信号和归约稳定性
Gradient工作张量	BF16/FP16/FP8混合	省显存/通信，但需避免下溢/溢出
Gradient Accumulation Buffer	BF16或FP32，依实现	稳定性与显存取舍
Adam first/second moment	常见FP32	长期累计历史，需要精度与范围
Master Weight	FP32，是否存在依实现	保存小更新；某些BF16优化器布局可不单独保留
Checkpoint	BF16/FP16或混合状态	取决于恢复训练和部署需求
DP/TP/PP/EP通信	BF16/FP16/FP8等	降低通信量，但Collective精度需验证

4.1 “输入精度、累加精度、输出精度”必须分开

一个矩阵乘：

```
C = A × B
```

可能是：

```
A: BF16
B: BF16
乘法: BF16输入Tensor Core
Accumulator: FP32
C输出: BF16
```

也可能因性能开关，在长 Reduction 中允许部分 Reduced-Precision Reduction。PyTorch 为 FP16/BF16 GEMM 提供相关控制：

```
torch.backends.cuda.matmul.allow_fp16_reduced_precision_reduction = False
torch.backends.cuda.matmul.allow_bf16_reduced_precision_reduction = False
```

关闭可能提高稳定性，但也可能降低性能。不能只看 Tensor dtype 就断言内部每一步是什么精度。

五、FP16 为什么需要 Loss Scaling

设原始 Loss 为 `L`，参数为 `W`：

```
g = ∂L/∂W
```

若 `g` 很小，转成 FP16 可能变成 0。

把 Loss 乘以 Scale `S`：

```
L_scaled = L × S

g_scaled
= ∂(L×S)/∂W
= S × g
```

Backward 后再除回去：

```
g = g_scaled ÷ S
```

动态 Loss Scaling：

```
若Gradient出现Inf/NaN:
    跳过Optimizer Step
    降低Scale

若连续多个Step都有限:
    逐步提高Scale
```

PyTorch 典型写法：

```
import torch

model = model.cuda()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
scaler = torch.amp.GradScaler("cuda")

for x, target in dataloader:
    x = x.cuda(non_blocking=True)
    target = target.cuda(non_blocking=True)
    optimizer.zero_grad(set_to_none=True)

    with torch.autocast(
        device_type="cuda",
        dtype=torch.float16,
    ):
        output = model(x)
        loss = loss_fn(output, target)

    scaler.scale(loss).backward()

    # 若要做Gradient Clipping，先unscale
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(
        model.parameters(),
        max_norm=1.0,
    )

    scaler.step(optimizer)
    scaler.update()
```

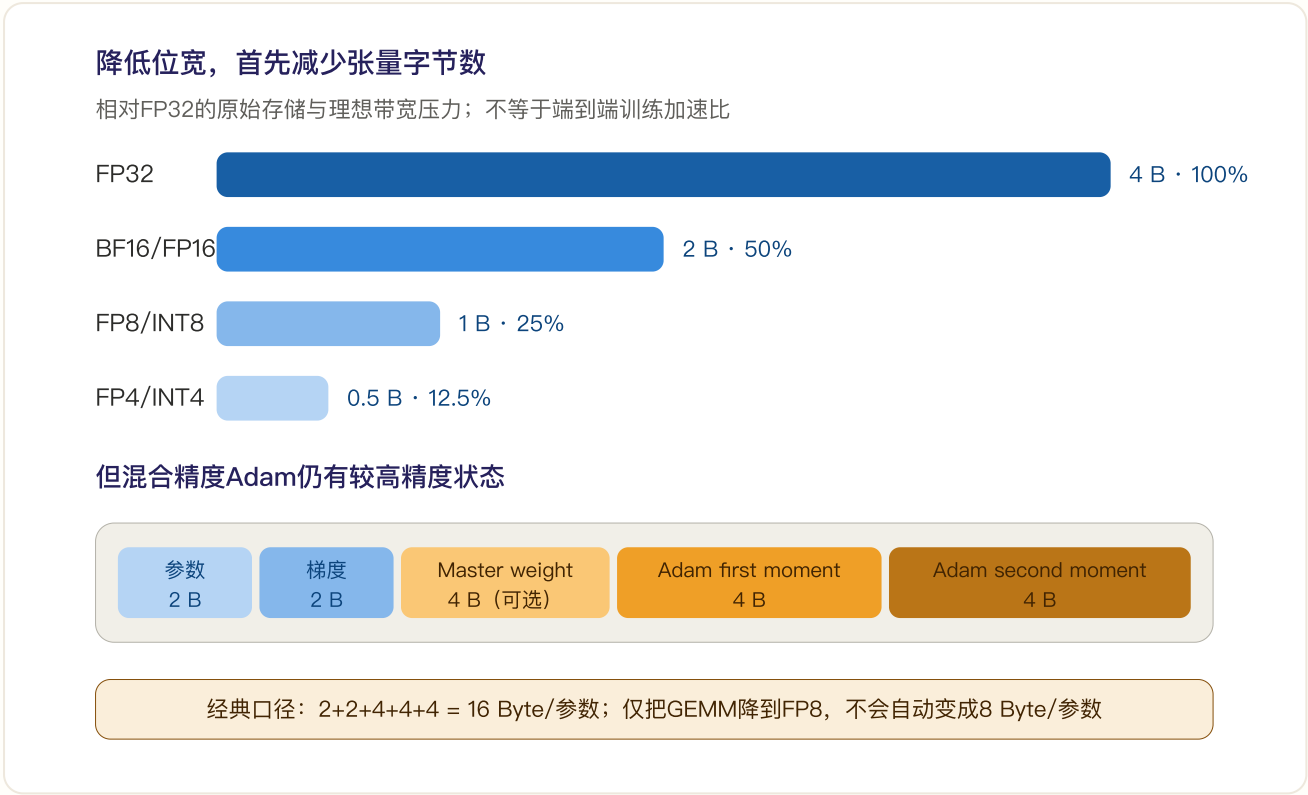
BF16 通常可以：

```
with torch.autocast(
    device_type="cuda",
    dtype=torch.bfloat16,
):
    output = model(x)
    loss = loss_fn(output, target)

loss.backward()
optimizer.step()
```

工程上仍应检查实际硬件、框架和模型；“BF16 通常不需要 GradScaler”不等于所有平台和自定义算子永远不需要任何稳定措施。

六、精度如何影响显存



6.1 只看模型权重：7B 参数

设：

```
P = 7×10^9 params
```

权重存储：

```
FP32: 7e9 × 4 Byte = 28 GB ≈ 26.08 GiB
BF16/FP16: 7e9 × 2 Byte = 14 GB ≈ 13.04 GiB
FP8/INT8: 7e9 × 1 Byte = 7 GB ≈ 6.52 GiB
INT4/FP4: 7e9 × 0.5 Byte = 3.5 GB ≈ 3.26 GiB
```

这是**权重本身**，不含：

- Scale/Zero Point；
- Padding/Alignment；
- KV Cache；
- Activation；
- CUDA Workspace；

- 框架元数据；
- 训练梯度和优化器状态。

6.2 训练状态：为什么 BF16 训练不是 2 Byte/参数

经典混合精度 Adam 口径：

```
低精度参数          2 Byte
低精度梯度          2 Byte
FP32 Master Weight  4 Byte
FP32 Adam m          4 Byte
FP32 Adam v          4 Byte
--------------------------
合计约              16 Byte/参数
```

7B：

```
7e9 × 16 Byte
= 112 GB
≈ 104.31 GiB
```

某些 BF16 训练布局不单独保留 FP32 Master Weight，则可能接近：

```
2 + 2 + 4 + 4 = 12 Byte/参数
```

但真实口径取决于：

- Optimizer 实现；
- 是否有 FP32 Parameter Copy；
- Gradient Accumulation dtype；
- ZeRO/FSDP 分片；
- 是否使用 8-bit Optimizer；
- Padding、Bucket、Flat Parameter 和临时 Buffer。

因此报告显存时必须说清“每参数包含哪些状态”。

6.3 Activation 示例

统一 Shape：

```
B = 2
S = 4096
```

```
H = 8192
Activation Shape = [B,S,H]
```

元素数：

```
N = 2 × 4096 × 8192
  = 67,108,864 elements
```

一个 Activation Tensor：

```
FP32: 67,108,864 × 4 B = 256 MiB
BF16/FP16: × 2 B = 128 MiB
FP8: × 1 B = 64 MiB
```

但完整训练 Activation 显存还乘以：

- 层数；
- 需要为 Backward 保存的中间张量；
- Microbatch 数；
- Attention/MLP 中间值；
- 是否 Activation Checkpoint；
- TP/SP/CP 分片；
- Fused Kernel 是否避免物化中间 Tensor。

七、精度如何影响 HBM 带宽与通信

7.1 HBM 带宽

一个 Memory-Bound Kernel 读取 `N` 个元素并写 `N` 个元素：

```
Bytes ≈ N × (read_count + write_count) × bytes_per_element
```

若从 FP32 降到 BF16：

```
每元素4 B → 2 B
理论流量减半
```

如果 Kernel 完全由 HBM 带宽限制、且没有额外 Cast/Scale，理论上时间可以接近减半。

但实际可能受限于：

- Kernel Launch；
- Index/Metadata；
- Cache 命中；
- 反量化；
- 不支持低精度的算子；
- 访存未合并；
- 小 Shape 无法打满带宽。

7.2 分布式通信

7B 梯度：

```
FP32 Gradient Payload = 28 GB
BF16/FP16 Payload = 14 GB
FP8 Payload = 7 GB（若通信与归约支持并且数值可接受）
```

Ring AllReduce 每 Rank 理想收发量：

```
Bytes_per_rank
≈ 2 × (N−1)/N × Payload
```

8 卡：

```
系数 = 2 × 7/8 = 1.75

FP32：1.75 × 28 GB = 49 GB/Rank
FP16：1.75 × 14 GB = 24.5 GB/Rank
```

这说明降低通信 dtype 可显著减少带宽压力。

但要注意：

- 梯度量化需要 Scale；
- AllReduce 是多次累加，误差会传播；
- Error Feedback、Chunk Scaling、FP32 Accumulation 可能需要额外 Buffer；
- 网络固定延迟不会因 payload 减半而消失；
- 通信与计算 Overlap 后，减少的可能是“隐藏时间”，不一定减少同等 Step Time。

八、精度如何影响计算性能

8.1 为什么低精度 Tensor Core 更快

更低位宽允许硬件在同样的：

- 晶体管预算；
- 数据通路宽度；
- 寄存器/共享内存带宽；
- Tensor Core 单元；

中并行处理更多元素。

因此在支持的硬件上，通常存在：

```
FP8 Tensor Core Throughput > BF16/FP16 > TF32 > IEEE FP32
```

但具体峰值倍数依 GPU 型号、稀疏模式、功耗和数据布局而变，不能脱离硬件规格给统一数字。

8.2 端到端速度为什么不会按位数同比增加

假设训练 Step 中：

```
70% 时间是可降精度GEMM
30% 时间是通信、归一化、调度、数据加载等
```

若 GEMM 加速 2 倍：

```
Speedup
= 1 ÷ (0.30 + 0.70 ÷ 2)
= 1 ÷ 0.65
≈ 1.54×
```

若 GEMM 加速 4 倍：

```
Speedup
= 1 ÷ (0.30 + 0.70 ÷ 4)
= 1 ÷ 0.475
≈ 2.11×
```

即使 GEMM 无限快，上限也只有：

```
Speedup_max = 1 ÷ 0.30 ≈ 3.33×
```

这是 Amdahl's Law。

低精度端到端收益取决于：

- GEMM 占比；
- Shape 是否满足 Tensor Core 对齐；
- GEMM 是否足够大；
- FP8/FP4 Scale 与 Cast 是否融合；
- HBM 或网络是否成为新瓶颈；
- CPU Launch/Data Loader 是否成为瓶颈；
- 是否因不稳定而回退或重跑。

九、精度如何影响模型质量与训练稳定性

9.1 四类主要数值风险

Overflow

值的绝对值超过最大有限值：

```
FP16中 |x| > 65504
→ Inf
```

后续常传播为 NaN。

Underflow

绝对值过小：

```
小Gradient → 0
```

该参数本 Step 不再获得有效更新。

Rounding

真实值被舍入到最近可表示值：

```
BF16：1.0 + 0.001
可能仍等于1.0
```

Accumulation Error

长 Reduction 的求和顺序、分块方式和 Accumulator dtype 都会影响结果：

```
(a + b) + c
不一定等于
a + (b + c)
```

这也是不同 Kernel、不同 Batch 实现、CPU/GPU 结果不保证 Bitwise Identical 的原因。

9.2 哪些算子更敏感

通常需要谨慎的区域：

- Softmax 的 `exp` 与归一化；
- Cross Entropy / LogSumExp；
- LayerNorm/RMSNorm Reduction；
- 大规模 Sum/Mean；
- Optimizer m/v 更新；
- 梯度裁剪范数；
- 概率极小或极大的路由；
- Ill-Conditioned Linear Algebra；
- 长序列 Attention 的在线归约。

AMP 的意义就是：

```
安全的大GEMM用低精度
敏感操作自动提升精度
```

十、训练、微调和推理分别怎么选

10.1 大模型预训练

优先顺序：

```
BF16：通用稳健默认
FP8：硬件/框架成熟时追求更高吞吐
FP16：BF16不可用或已验证稳定时
FP32/TF32：数值调试、敏感任务或基线
FP4：专用新硬件与成熟Recipe下评估
```

推荐实践：

- BF16 参数/激活；
- FP32 Optimizer State；
- FP32 Norm/Softmax/关键 Reduction；
- 观察 Loss、Grad Norm、Inf/NaN、Overflow、Amax；
- 低精度改变后必须做收敛对齐，而不只是吞吐 Benchmark。

10.2 Fine-Tuning

- 从 BF16 预训练 Checkpoint 继续训练，优先 BF16；
- LoRA/QLoRA 中 Base Weight 可为 INT4/NF4，Adapter/Optimizer 仍使用 BF16/FP32；
- 小数据微调对数值噪声和超参数可能更敏感；
- FP16 下重点观察 Overflow 与 GradScaler。

10.3 推理

常见路线：

```
BF16/FP16：质量稳、部署简单
FP8：权重+激活低精度，需硬件支持和校准/Scaling
INT8：成熟的Weight+Activation量化
INT4：常用于Weight-Only，显著压缩权重
FP4：新硬件的更激进低精度方案
```

推理要分别看：

- Prefill：大 GEMM，容易吃到 Tensor Core 收益；
- Decode：小 Batch/GEMV，常受权重带宽、KV Cache 和调度影响；
- KV Cache：精度降低可直接扩大并发和上下文，但需验证 Attention 质量；
- Logits/Softmax/Sampling：通常不应无脑降到最窄精度。

十一、一个可运行的 PyTorch 精度检查示例

```
import torch

for dtype in [
    torch.float32,
    torch.bfloat16,
    torch.float16,
]:
    info = torch.finfo(dtype)

    print("dtype:", dtype)
    print("bytes:", info.bits // 8)
    print("max:", info.max)
    print("smallest_normal:", info.smallest_normal)
    print("eps near 1:", info.eps)
    print()

# 观察大数吃小数
for dtype in [
    torch.float32,
    torch.float16,
    torch.bfloat16,
]: dtype=dtype)
    delta = torch.tensor(1e-4, dtype=dtype)

    print(
        dtype,
        "1 + 1e-4 =",
        (one + delta).item(),
    )
```

你应看到：

- FP32 可以保留 `1e-4` 级变化；
- FP16 在 `1.0` 附近的间隔约 `9.77e-4`；
- BF16 在 `1.0` 附近的间隔约 `7.81e-3`；
- 因此 FP16/BF16 都可能把 `1 + 1e-4` 舍入为 `1.0`。

但这不表示 BF16 不适合训练，因为大模型训练会使用：

- FP32 Accumulation；
- FP32 Optimizer State；

- 合适的学习率/更新路径；
- 混合精度算子策略。

十二、FP8 Transformer Engine 最小示意

以下示例表达 API 形态，具体 Recipe 和支持范围依 Transformer Engine/GPU 版本：

```
import torch
import transformer_engine.pytorch as te
from transformer_engine.common import recipe

layer = te.Linear(
    in_features=4096,
    out_features=16384,
    bias=False,
).cuda()

x = torch.randn(
    8,
    4096,
    device="cuda",
    dtype=torch.bfloat16,
)

fp8_recipe = recipe.DelayedScaling(
    fp8_format=recipe.Format.HYBRID,
    amax_history_len=16,
    amax_compute_algo="max",
)

with te.autocast(
    enabled=True,
    recipe=fp8_recipe,
):
    y = layer(x)
    loss = y.float().square().mean()

# backward通常在autocast作用域之外触发
loss.backward()

print(y.shape)
```

`Format.HYBRID` 的典型含义：

```
Forward: E4M3
Backward Gradient: E5M2
```

Transformer Engine 管理：

- FP8 Cast；
- Scale；
- Amax History；

- 支持的 FP8 Linear/Transformer Kernel；
- 必要的更高精度路径。

十三、如何选择精度：工程决策表

需求/约束	首选	原因	需要重点验证
现代LLM预训练稳健默认	BF16 Mixed Precision	FP32级范围、2Byte、通常无需Loss Scaling	收敛、FP32 Reduction、硬件吞吐
BF16硬件不可用	FP16 + GradScaler	Tensor Core成熟、2Byte	Overflow/Underflow、Scale变化、NaN
保持FP32存储且最小改动加速	TF32	FP32接口下加速GEMM/Conv	数值误差、PyTorch开关
Hopper/Blackwell追求训练吞吐	FP8 Recipe	1Byte输入、Tensor Core吞吐高	Amax、Saturation、Scale、回退算子
通用稳定基线/敏感归约	FP32	范围与精度较平衡	显存和吞吐成本
科学计算/病态线代	FP64	高精度	巨大性能/显存成本
推理兼顾质量与部署简单	BF16/FP16	无复杂量化或较少校准	显存容量、吞吐
推理压权重、扩大部署密度	INT4 Weight-Only	权重约0.5Byte/参数	反量化、Scale、质量、Kernel
推理Weight+Activation量化	INT8/FP8	降低权重和激活流量	Calibration、Outlier、算子覆盖
Blackwell前沿低精度训练	NVFP4	极低位宽+专用Recipe	Recipe成熟度、质量、硬件与软件版本

十四、低精度调优应该监控什么

数值稳定性

```
Loss Curve
Grad Norm
Inf/NaN Count
Dynamic Loss Scale
Skipped Optimizer Steps
Activation/Gradient Amax
FP8 Saturation Rate
Quantization Error
Per-Layer Output Error
```

性能

```
Step Time
Tokens/s
MFU
Tensor Core Utilization
GEMM Kernel Dtype
HBM Read/Write Throughput
Cast/Scale Kernel Time
Communication Exposed Time
Kernel Fallback Count
```

显存

```
Parameter Memory
Gradient Memory
Optimizer State
Master Weight
Activation Peak
KV Cache
Scale/Metadata
Communication Buffer
Workspace
Fragmentation
```

必须做两类对照：

1. **Numerical Parity**：相同数据/Seed 下，Loss、Grad、输出误差是否可接受；
2. **End-to-End Performance**：不是只看 GEMM Microbenchmark，而是看训练 Step 或 Serving 指标。

十五、常见误区

误区1：BF16 比 FP16“全面更精确”

错误。

```
BF16：范围更大，尾数更少
FP16：范围更小，尾数更多
```

BF16 更适合大模型训练，主要因为范围，而不是因为它每个局部数值都更精细。

误区2：模型用 BF16 训练，所有状态都是 BF16

错误。Optimizer State、Accumulation、Norm、Softmax、Loss 等经常保留 FP32。

误区3：FP8 相比 BF16 位数减半，显存一定再减半

只对真正以 FP8 常驻的张量成立。若参数本体、优化器、Checkpoint、部分激活仍为 BF16/FP32，总显存不会整体减半。

误区4：TF32 能把模型显存从 FP32 降一半

错误。TF32 主要是矩阵计算模式；Tensor 通常仍以 FP32 存储。

误区5：INT4 推理必然比 BF16 快4倍

错误。可能受反量化、Scale、GEMV、Kernel覆盖、Batch大小、KV Cache和调度限制。

误区6：累加器是FP32就绝对没有低精度误差

错误。输入量化已经丢失信息；Reduction 顺序、部分 Reduced-Precision Reduction、输出 Cast 仍会产生误差。

误区7：训练不出现NaN就说明精度安全

错误。还可能存在：

- 大量小梯度被舍入为0；
- 收敛速度变慢；
- 最终质量轻微下降；
- 某些层长期 Saturate；
- Router/Attention分布发生偏移。

十六、最终心智模型

```
指数位决定范围
尾数位决定精细程度
位宽决定存储、带宽和潜在计算密度

FP32：稳定锚点
TF32：FP32接口下的快速矩阵模式
BF16：范围优先的16位训练默认
FP16：精度较细但范围窄，需要Loss Scaling
FP8：需要逐张量/逐块Scaling的激进训练格式
INT8/INT4：主要通过量化服务推理
FP4：依赖微块Scaling与专用Recipe的前沿低精度
```

选精度时固定问六个问题：

1. **Storage**：参数、激活、梯度、KV Cache 分别以什么 dtype 常驻?
2. **Compute**：GEMM 输入、Accumulator、输出分别是什么精度?
3. **State**：Optimizer 与 Master Weight 是否保留 FP32?
4. **Scale**：是否需要 Loss Scaling、Per-Tensor Scale 或 Block Scale?
5. **Hardware**：GPU 是否原生支持该格式和高吞吐 Kernel?
6. **Quality**：是否完成训练收敛、下游指标和逐层误差对照?

真正的工程目标不是“使用最低位宽”，而是：

> **在目标质量不下降、训练不发散、运维可控的前提下，找到 Time-to-Train、显存、吞吐和成本最优的精度组合。**

可选自测

1. 格式辨析题

为什么 BF16 的局部尾数精度比 FP16 更低，却通常更适合 LLM 预训练？请从：

- 指数范围；
- Gradient Overflow/Underflow；
- Loss Scaling；

- FP32 Accumulation；

四个角度解释。

2. 显存计算题

一个 13B 模型：

```
参数与梯度：BF16
Master Weight：FP32
Adam m/v：FP32
```

请计算经典口径下参数相关训练状态的：

- Byte/参数；
- 总 GB；
- 总 GiB；
- 若 FSDP Full Shard 到 16 卡，理想每卡是多少？

不计 Activation、临时 AllGather 和 Workspace。

3. 工程决策题

某训练从 BF16 切到 FP8 后：

```
GEMM时间下降45%
Cast/Scale增加8% Step Time
AllReduce不变
某些层Amax偶发突增
最终Step Time只下降18%
```

请解释：

- 为什么端到端收益低于 GEMM 收益；
- Amax 突增可能带来什么风险；
- 应监控哪些指标；
- 你会尝试 Delayed Scaling、Current Scaling 还是更细粒度 Block Scaling，为什么？

继续学习

1. 前置：浮点舍入、Underflow/Overflow 与误差传播 —— 从数值分析解释训练为什么会发散。
2. 同层对比：BF16、FP16 与 FP8 混合精度训练源码路径 —— 深入 Autocast、GradScaler、Transformer Engine。
3. 下游应用：LLM INT8/INT4 量化 —— 深入 Per-Channel/Per-Group、GPTQ、AWQ、SmoothQuant 与 KV Cache 量化。

回复 1、2 或 3 继续；也可以说“稍后”或“不感兴趣”。

事实来源与版本边界

截至 2026-08-02，本文稳定原理与接口主要核对：

- PyTorch 2.13 Numerical Accuracy；
- PyTorch AMP 官方文档；
- NVIDIA Mixed Precision Training Guide；
- NVIDIA Transformer Engine FP8/FP4 Primer 与用户指南；
- NVIDIA CUDA Programming Guide。

具体 Tensor Core 峰值、支持格式、Shape 约束、PyTorch 开关与 Transformer Engine Recipe 会随 GPU 和软件版本变化。生产环境应以实际 GPU 架构、CUDA/PyTorch/Transformer Engine 版本及 Profiler 结果为准。

H2D、D2H 与 CPU – GPU 协同：计算、内存、通信和流水线优化

> **原始问题：**详细讲解D2H,H2D，以及GPU和CPU各自的运算和通信特点，还有两者之间的互相通信，什么时候应用，可以得到什么方面的优化提升

提问：2026/08/03 11:42　更新：2026/08/03 11:50　重要度：5/5

H2D、D2H 与 CPU – GPU 协同：计算、内存、通信和流水线优化

一句话结论

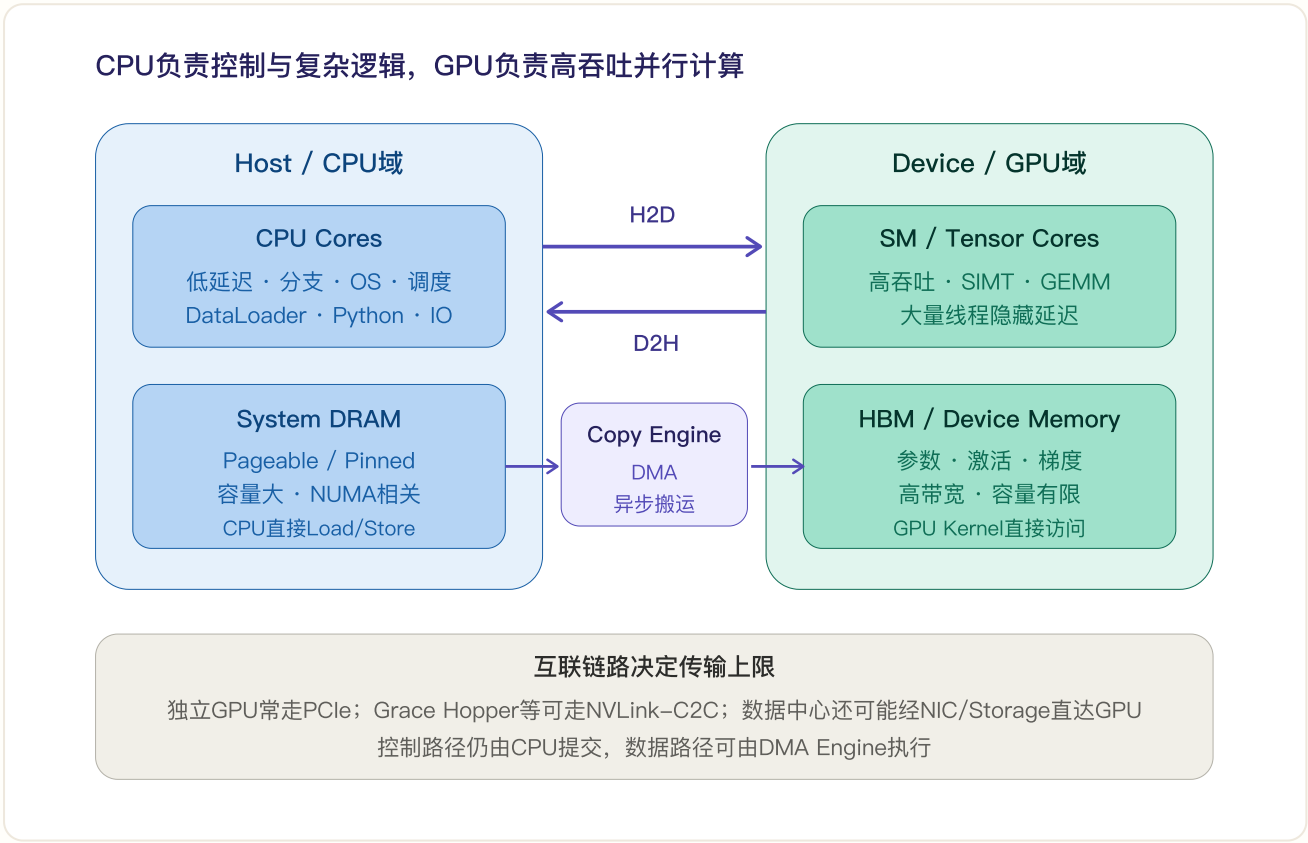
- **H2D（Host-to-Device）**：把数据从 CPU 侧系统内存搬到 GPU 侧设备内存；典型场景是把训练 Batch、推理输入、模型参数或临时 Buffer 送到 GPU。
- **D2H（Device-to-Host）**：把结果从 GPU 设备内存搬回 CPU 系统内存；典型场景是返回推理结果、保存 Checkpoint、CPU 后处理、日志和调试。
- 对独立 GPU，H2D/D2H 通常经过 PCIe 或类似 CPU–GPU 互联，由 GPU Copy Engine/DMA 搬运。**CPU 提交控制命令，不等于 CPU Core 逐字节执行整个传输。**
- CPU 擅长低延迟、复杂控制流、分支、OS/IO 和少量串行工作；GPU 擅长规则、高并行、高算术强度的大批量计算。
- 最有效的优化顺序通常是：

```
尽量不传
→ 数据留在GPU
→ 合并小传输
→ 使用提前准备好的Pinned Memory
→ non_blocking异步提交
→ 独立Stream + Event
→ 双缓冲/预取，将H2D与当前Batch计算重叠
→ 做NUMA和拓扑绑定
→ 特定场景考虑Unified Memory或GPUDirect
```

最重要的判断式：

```
把一个操作放到GPU是否值得
≈ GPU计算节省的时间
  是否大于
  H2D + D2H + Launch + Synchronization额外时间
```

一、系统位置：CPU 和 GPU 各自在哪里工作



1.1 Host 与 Device

CUDA 语境中：

```
Host   通常指CPU及其系统内存
Device 通常指GPU及其设备内存
```

独立 GPU 系统中，两套内存物理上分离：

```
CPU Core ↔ CPU Cache ↔ System DRAM
                       ↕ PCIe / NVLink-C2C等
GPU SM   ↔ L1/L2     ↔ GPU HBM
```

CPU 通常负责：

- Python/C++ 控制流；
- DataLoader、Tokenizer、解码和数据增强；
- 文件系统、网络和磁盘 IO；
- Kernel Launch；
- CUDA Stream/Event 编排；
- 分布式进程管理；
- GPU 不能高效执行的少量、分支密集任务。

GPU 通常负责：

- GEMM、Attention、卷积；
- 大规模逐元素运算；
- Reduction；
- Embedding、MoE Expert 等高并行算子；
- 前向、反向和优化器 Kernel；
- NCCL 通信 Kernel。

二、CPU 与 GPU 的计算特点

2.1 CPU：少量强核，优化单线程和复杂控制

CPU 的设计重点通常是：

- 高单线程性能；
- 大 Cache；
- 复杂分支预测；
- Out-of-Order Execution；
- 低延迟访问系统资源；
- OS、文件、网络、线程和中断处理；
- SIMD 向量指令处理适量数据并行。

适合 CPU 的任务：

- Tokenizer、JSON/文本解析；
- 动态数据结构；
- 分支密集算法；
- 非常小的 Tensor；
- 单次低延迟任务；
- 数据读取、解压、增强；
- 调度、RPC、Serving 控制面；
- 需要访问大量主机内存但计算复用很低的任务。

2.2 GPU：大量并行执行单元，优化吞吐

GPU 的设计重点通常是：

- 大量 SM 和 Warp；
- SIMT 执行；
- 高 HBM 带宽；
- Tensor Core；
- 用大量 Ready Warp 隐藏访存延迟；
- 用大批量规则计算摊薄 Kernel Launch 和调度开销。

适合 GPU 的任务：

- 大矩阵乘；
- Attention；
- 大 Batch 的向量化计算；
- 相同操作应用到大量元素；
- 高算术强度；
- 能够在 HBM、Shared Memory、Register 中复用数据的计算。

2.3 CPU 与 GPU 对比

维度	CPU	GPU
优化目标	低延迟、通用性	高吞吐、并行性

维度	CPU	GPU
核心数量	较少但强	大量轻量执行单元
控制流	擅长复杂分支	Warp分歧会损失效率
Cache	单核/共享Cache较强	更依赖HBM带宽和显式复用
内存容量	系统DRAM通常更大	HBM容量较小但带宽高
启动成本	函数执行直接	Kernel Launch有固定开销
最佳工作粒度	小任务、串行/控制	大任务、大Batch、规则计算
IO/OS	可直接处理	通常由CPU/驱动提交
典型LLM角色	数据、控制、Serving调度	模型计算和GPU通信

结论不是“GPU 永远比 CPU 快”，而是：

```
任务必须有足够计算量、并行度和数据复用
才能摊薄CPU↔GPU传输与Kernel Launch成本
```

三、H2D 和 D2H 的精确定义

3.1 H2D

```
Host Memory → Device Memory
```

CUDA C++：

```
cudaMemcpy(
    device_ptr,
    host_ptr,
    bytes,
    cudaMemcpyHostToDevice
);
```

PyTorch：

```
gpu_tensor = cpu_tensor.to("cuda")
```

典型用途：

- 训练 Batch；

- 推理输入；
- 初始化模型参数；
- CPU 预处理后的特征；
- CPU Offload 状态重新加载；
- Checkpoint 恢复。

3.2 D2H

```
Device Memory → Host Memory
```

CUDA C++：

```
cudaMemcpy(
    host_ptr,
    device_ptr,
    bytes,
    cudaMemcpyDeviceToHost
);
```

PyTorch：

```
cpu_tensor = gpu_tensor.to("cpu")
```

典型用途：

- 推理结果返回业务代码；
- GPU 结果交给 CPU 后处理；
- 保存模型或优化器状态；
- 日志、Metric、可视化；
- 调试；
- CPU Offload；
- GPU 生成数据写入普通 CPU IO 栈。

3.3 方向名是相对谁而言

```
H2D = CPU内存到GPU内存
D2H = GPU内存到CPU内存
```

```
D2D = GPU内存到GPU内存
H2H = CPU内存到CPU内存
```

在多 GPU 系统中，GPU↔GPU 还可能走：

- NVLink/NVSwitch；
- PCIe P2P；
- GPUDirect RDMA + NIC；
- 无 P2P 时退化为 GPU→Host→GPU Bounce Path。

四、传输到底由谁执行：DMA 与 Copy Engine

4.1 CPU 是控制面，DMA 是数据面

常见异步 H2D：

```
CPU调用cudaMemcpyAsync
→ CUDA Runtime/Driver建立传输描述
→ 命令进入CUDA Stream
→ GPU Copy Engine/DMA读取Pinned Host Memory
→ 数据经过PCIe/NVLink-C2C
→ 写入GPU HBM
→ Event标记完成
```

CPU Core 不需要在整个传输期间逐字节循环复制，因此提交后可以继续：

- 处理下一 Batch；
- 发起其他 CUDA 操作；
- 执行业务逻辑；
- 准备其他 GPU 的输入。

但对 Pageable Memory，驱动通常需要额外的 CPU 内存复制到 Pinned Staging Buffer，CPU/DRAM 仍会参与。

4.2 Copy Engine 与 Compute Engine

GPU 可能拥有独立 Copy Engine：

- H2D Copy Engine；
- D2H Copy Engine；
- SM/Tensor Core 负责计算。

某些设备允许：

```
一个H2D
+ 一个D2H
+ GPU Kernel
```

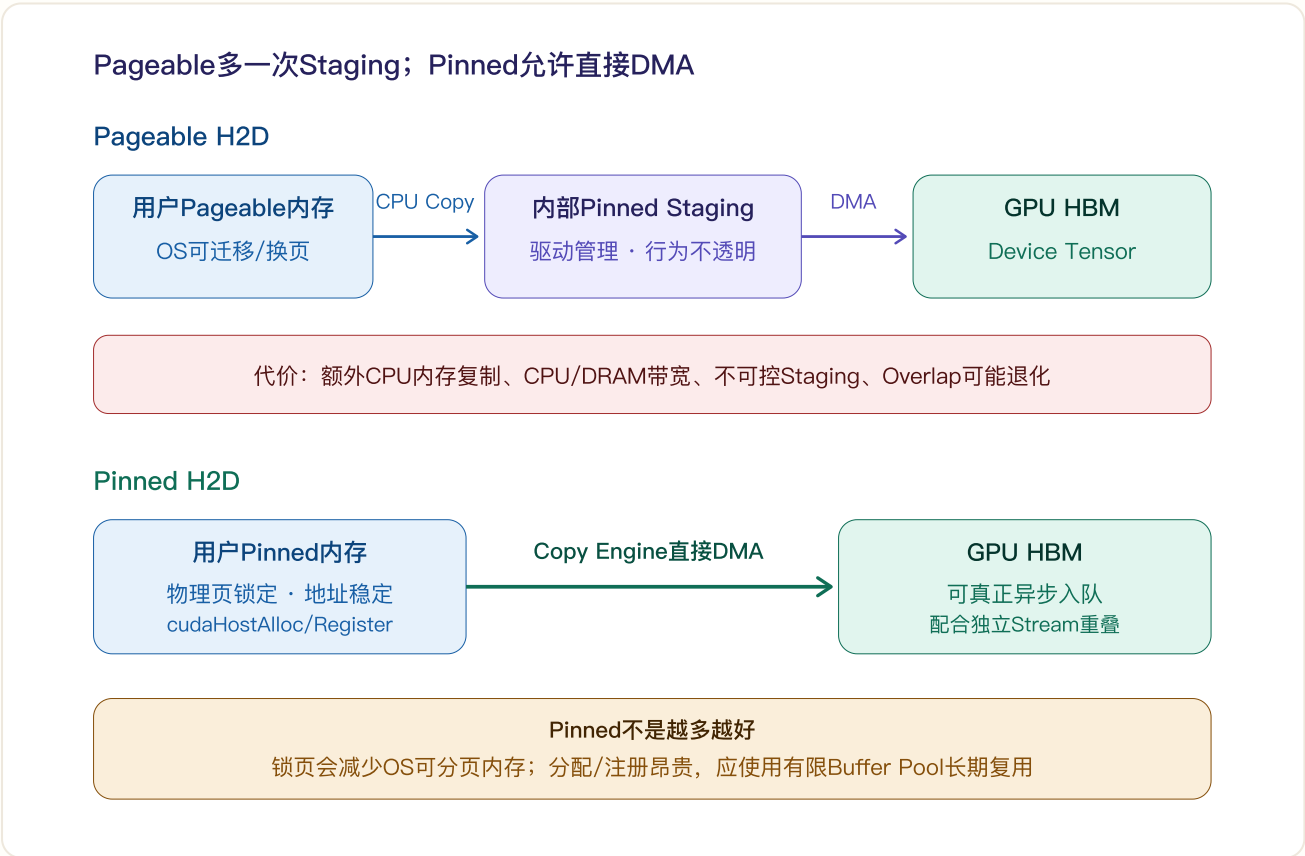
同时推进。

但能否并发必须查询并实测：

```
cudaDeviceProp prop;
cudaGetDeviceProperties(&prop, device);
printf("asyncEngineCount=%d\n", prop.asyncEngineCount);
```

`asyncEngineCount` 表示硬件能力，不保证应用自动获得重叠。

五、Pageable Memory 与 Pinned Memory



5.1 Pageable Memory

普通 `malloc` 或普通 CPU Tensor 通常位于 Pageable Memory。

OS 可以：

- 迁移物理页；
- 换页；
- 改变虚拟页到物理页的映射。

GPU DMA 需要传输期间物理页稳定，因此 Pageable H2D 常见路径为：

```
Pageable User Buffer
→ CPU memcpy到内部Pinned Staging
→ DMA到GPU HBM
```

代价：

- 多一次 CPU Memory Copy；
- 消耗 System DRAM 带宽；
- 增加 CPU 开销；
- 延迟更不稳定；
- 真正异步和 Copy–Compute Overlap 可能退化。

具体 Staging Buffer 的大小、复用策略和同步时机属于驱动实现细节，应用不应依赖。

5.2 Pinned / Page-Locked Memory

Pinned Memory 的物理页被锁定，传输期间不会被 OS 换出或迁移。

CUDA API：

```
cudaHostAlloc(&ptr, bytes, flags);
```

或注册已有内存：

```
cudaHostRegister(ptr, bytes, flags);
```

PyTorch：

```
x = torch.empty(
    shape,
    pin_memory=True,
)
```

或者：

```
loader = DataLoader(
    dataset,
    pin_memory=True,
)
```

优势：

- Copy Engine 可直接将其作为 DMA 源/目标；
- H2D/D2H 带宽更高且更可预测；
- 是可靠 Copy–Compute Overlap 的关键条件；
- 避免驱动内部 Pageable→Pinned Staging。

5.3 Pinned Memory 的代价

Pinned Memory 不是越多越好：

- 不能被 OS 正常换页；
- 占用宝贵物理 RAM；
- 大量锁页会影响整个系统；
- `cudaHostAlloc`、`cudaHostRegister` 和取消注册相对昂贵；
- 频繁 Pin/Unpin 可能比直接 Pageable 复制更慢。

最佳实践：

```
建立有限大小Pinned Buffer Pool
→ 长期复用
→ 不在每个Batch热路径里临时Pin/Unpin
```

5.4 为什么 `x.pin_memory().to("cuda")` 可能更慢

它包含两步：

```
Pageable x
→ CPU复制出一个新的Pinned x_pinned
```

```
→ H2D复制到GPU
```

总时间：

```
T_total
= T_pin_copy + T_H2D_pinned
```

只有满足：

```
T_pin_copy + T_H2D_pinned
< T_H2D_pageable
```

才有单次收益。

通常更合理的是：

```
DataLoader后台线程提前Pin下一Batch
+ 主线程non_blocking传输当前Batch
```

这样 Pinning 开销可以被 CPU 数据准备和 GPU 计算隐藏。

六、同步复制、异步提交和真正重叠是三件不同的事

6.1 同步复制

```
gpu = cpu.to("cuda")
```

从调用者视角，PyTorch 通常会在合适位置等待，使调用安全地完成后再继续。

优点：

- 简单；
- 正确性直观。

缺点：

- CPU 主线程等待；
- 难以形成流水线；
- 每个小 Tensor 都同步时，固定开销很高。

6.2 非阻塞提交

```
gpu = cpu.to(
    "cuda",
    non_blocking=True,
)
```

这首先意味着：

> CPU 不在每次复制后立刻等待 CUDA Stream 完成。

它可以减少：

- Host 线程阻塞；
- 每 Tensor 的 `cudaStreamSynchronize`；
- 多个小复制之间的提交间隙。

但 `non_blocking=True` 不自动保证：

- H2D 与 Kernel 在 GPU 上同时执行；
- Pageable Memory 不需要 Staging；
- 跨 Stream 数据依赖自动正确；
- D2H 结果可以被 CPU 立即读取。

6.3 真正的 Copy – Compute Overlap

通常需要同时满足：

1. Host Buffer 是 Pinned Memory；
2. 使用 `cudaMemcpyAsync` / `non_blocking=True`；
3. Copy 与独立计算位于不同非默认 Streams；
4. GPU 有可用 Copy Engine；
5. Copy 与 Compute 没有直接数据依赖；
6. 使用 Event/Stream Wait 表达必要依赖；
7. 数据块足够大，能摊薄提交开销；
8. PCIe、DRAM、HBM 和 SM 没有形成更严重的共享瓶颈。

七、统一数值算例：256 MiB Batch

统一输入：

```
B = 8
S = 4096
H = 4096
Dtype = BF16 = 2 Byte

Shape = [B,S,H]
```

元素数：

```
N
= 8 × 4096 × 4096
= 134,217,728 elements
```

数据量：

```
Bytes
= 134,217,728 × 2
= 268,435,456 Byte
= 256 MiB
≈ 0.268 GB
```

假设实测有效 H2D 带宽：

```
BW_H2D = 24 GB/s
```

则：

```
T_H2D
= 0.268 GB ÷ 24 GB/s
≈ 0.01118 s
≈ 11.18 ms
```

变量与单位：

```
Bytes     传输字节数        单位：GB
BW_H2D    H2D有效带宽       单位：GB/s
T_H2D     传输时间          单位：s

量纲：GB ÷ (GB/s) = s
```

假设该 Batch 的 GPU Compute 为：

```
T_compute = 8 ms
```

7.1 串行

```
T_serial
= T_H2D + T_compute
= 11.18 + 8
= 19.18 ms/batch
```

稳态吞吐：

```
Throughput_serial
≈ 1000 ÷ 19.18
≈ 52.1 batch/s
```

7.2 完美双缓冲

```
T_pipeline_steady
≈ max(T_H2D, T_compute)
= max(11.18, 8)
= 11.18 ms/batch
```

稳态吞吐：

```
Throughput_overlap
≈ 1000 ÷ 11.18
≈ 89.4 batch/s
```

理想吞吐提升：

```
Speedup
≈ 19.18 ÷ 11.18
≈ 1.72×
```

这里 GPU 仍有约：

```
11.18 − 8 = 3.18 ms
```

等待下一 Batch，因为 H2D 比 Compute 慢。

如果 Compute 改为 15 ms：

```
T_steady ≈ max(11.18,15) = 15 ms
```

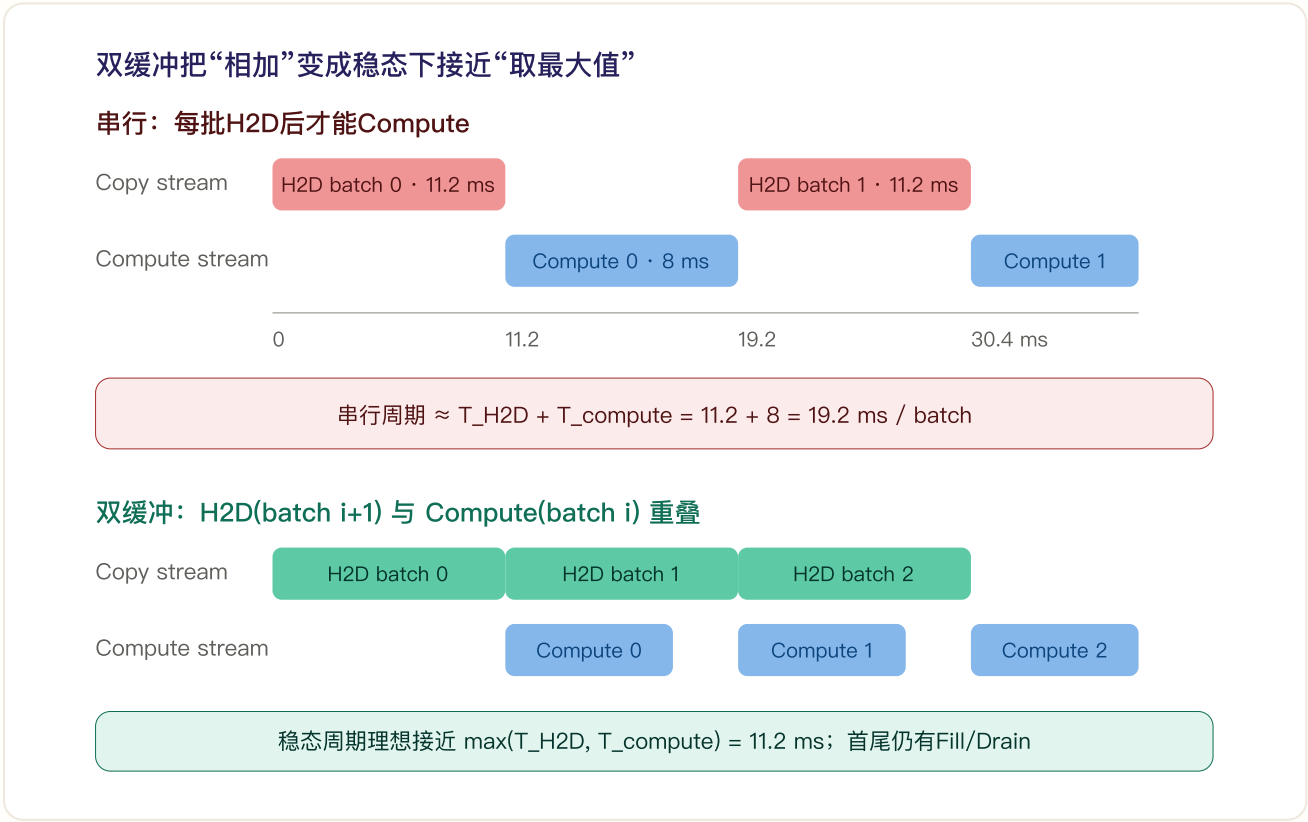
此时 H2D 可被计算完全隐藏。

7.3 为什么不是每一批都只要 11.18 ms

完整流水线还有：

- 第一个 Batch 的 Fill；
- 最后一个 Batch 的 Drain；
- Event/Launch 开销；
- DataLoader 抖动；
- Buffer 轮换；
- PCIe 与其他设备竞争；
- Pageable→Pinned、预处理和数据解码。

因此 `max(T_copy,T_compute)` 是长流水线的理想稳态下界，不是所有 Batch 的精确延迟。



八、H2D 和 D2H 的正确同步规则

8.1 Pinned H2D 后，CPU 不能立即修改源 Buffer

```
cpu = torch.ones(
    1024,
    pin_memory=True,
)

gpu = cpu.to(
    "cuda",
    non_blocking=True,
)

# 危险：DMA可能还在读取cpu
cpu.zero_()
```

正确做法是等对应复制 Event/Stream 完成后再复用 Pinned Buffer。

8.2 同一个 CUDA Stream 中，后续 GPU 操作自动等待

```
gpu = cpu.to(
    "cuda",
    non_blocking=True,
)

# 若在同一Stream提交，matmul按序等待H2D完成
output = model(gpu)
```

不需要在两者之间立刻全局同步。

8.3 跨 Stream 使用需要显式依赖

```
copy_stream = torch.cuda.Stream()
compute_stream = torch.cuda.current_stream()

with torch.cuda.stream(copy_stream):
    gpu = cpu.to(
        "cuda",
        non_blocking=True,
    )
    copy_done = copy_stream.record_event()

compute_stream.wait_event(copy_done)
output = model(gpu)
```

`wait_event()` 建立 GPU 侧顺序，不阻塞 CPU。

8.4 D2H 后 CPU 读取前必须确认完成

```
cpu_result = gpu_result.to(
    "cpu",
    non_blocking=True,
)

# 不能假定这里已经完成D2H
# print(cpu_result) 可能过早读取

torch.cuda.synchronize()
print(cpu_result)
```

更细粒度做法是同步负责 D2H 的 Event，而不是全设备同步。

8.5 `wait_stream()` 和 `record_stream()` 解决不同问题

API	解决的问题
`wait_stream()` / `wait_event()`	数据和执行依赖：消费者何时可以运行
`record_stream()`	内存生命周期：Caching Allocator何时可以复用底层Storage

如果 Tensor 在非创建 Stream 上使用，应考虑：

```
tensor.record_stream(other_stream)
```

否则原 Python 引用消失后，Caching Allocator 可能过早复用仍被异步操作使用的内存。

九、PyTorch DataLoader 的标准 H2D 路径

```
import torch
from torch.utils.data import DataLoader

loader = DataLoader(
    dataset,
    batch_size=8,
    num_workers=8,
    pin_memory=True,
    persistent_workers=True,
    prefetch_factor=2,
)

model = model.cuda()
```

```
for batch in loader:
    inputs, labels = batch

    inputs = inputs.to(
        "cuda",
        non_blocking=True,
    )

    labels = labels.to(
        "cuda",
        non_blocking=True,
    )

    output = model(inputs)
    loss = loss_fn(output, labels)
```

各参数作用：

```
num_workers
→ 多进程并行读取、解码、增强

pin_memory=True
→ 后台Pin Memory线程将Batch放入Pinned Host Memory

persistent_workers=True
→ Epoch之间不反复创建Worker

prefetch_factor
→ Worker预先准备多个Batch

non_blocking=True
→ 主线程提交H2D后不立即等待
```

注意：这能改善 CPU Pipeline，但不保证 H2D 一定与当前 GPU Compute 重叠。要确保重叠，通常还需要显式预取到 Side Stream。

十、PyTorch 双缓冲预取器

下面展示核心机制；生产代码要扩展到 Nested Dict/List、多 GPU、异常处理和 CUDA Graph。

```
import torch


def move_to_cuda(batch, device, stream):
    with torch.cuda.stream(stream):
        if isinstance(batch, torch.Tensor):
            return batch.to(
                device,
                non_blocking=True,
            )

        if isinstance(batch, (list, tuple)):
            return type(batch)(
                move_to_cuda(x, device, stream)
                for x in batch
            )

        if isinstance(batch, dict):
```

```
            return {
                k: move_to_cuda(v, device, stream)
                for k, v in batch.items()
            }

        return batch


def record_on_stream(batch, stream):
    if isinstance(batch, torch.Tensor):
        batch.record_stream(stream)

    elif isinstance(batch, (list, tuple)):
        for x in batch:
            record_on_stream(x, stream)

    elif isinstance(batch, dict):
        for x in batch.values():
            record_on_stream(x, stream)


class CUDAPrefetcher:
    def __init__(self, loader, device="cuda"):
        self.loader = iter(loader)
        self.device = torch.device(device)
        self.copy_stream = torch.cuda.Stream(
            device=self.device,
        )

        self.next_batch = None
        self._preload()

    def _preload(self):
        try:
            cpu_batch = next(self.loader)
        except StopIteration:
            self.next_batch = None
            return

        self.next_batch = move_to_cuda(
            cpu_batch,
            self.device,
            self.copy_stream,
        )

    def next(self):
        current = torch.cuda.current_stream(
            self.device,
        )

        # 当前Compute Stream等待下一Batch H2D完成
        current.wait_stream(self.copy_stream)

        batch = self.next_batch

        if batch is None:
            return None

        # 告诉Caching Allocator：当前Stream仍在使用这些Tensor
        record_on_stream(batch, current)

        # 立即在Side Stream预取再下一批
        self._preload()

        return batch


prefetcher = CUDAPrefetcher(loader)

while True:
    batch = prefetcher.next()

    if batch is None:
```

```
        break

    inputs, labels = batch

    # 计算当前Batch时，下一Batch正在copy_stream做H2D
    output = model(inputs)
    loss = loss_fn(output, labels)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
```

时间关系：

```
H2D(batch 0)
→ Compute(batch 0) || H2D(batch 1)
→ Compute(batch 1) || H2D(batch 2)
→ ...
```

十一、CUDA C++ 双缓冲示例

```
#include <cuda_runtime.h>

constexpr int NUM_BUFFERS = 2;

float* h_buf[NUM_BUFFERS];
float* d_buf[NUM_BUFFERS];
cudaStream_t streams[NUM_BUFFERS];

for (int i = 0; i < NUM_BUFFERS; ++i) {
    cudaHostAlloc(
        &h_buf[i],
        bytes,
        cudaHostAllocDefault
    );

    cudaMalloc(
        &d_buf[i],
        bytes
    );

    cudaStreamCreate(
        &streams[i]
    );
}

for (int batch = 0; batch < num_batches; ++batch) {
    int slot = batch % NUM_BUFFERS;

    // 复用slot前，确保它上一轮的copy和kernel完成
    cudaStreamSynchronize(streams[slot]);

    fill_cpu_batch(
        h_buf[slot],
        batch
    );

    cudaMemcpyAsync(
        d_buf[slot],
        h_buf[slot],
        bytes,
        cudaMemcpyHostToDevice,
```

```
        streams[slot]
    );

    kernel<<<grid, block, 0, streams[slot]>>>(
        d_buf[slot]
    );
}

for (int i = 0; i < NUM_BUFFERS; ++i) {
    cudaStreamSynchronize(streams[i]);
    cudaStreamDestroy(streams[i]);
    cudaFree(d_buf[i]);
    cudaFreeHost(h_buf[i]);
}
```

同一个 Stream 内：

```
H2D(slot i)
→ Kernel(slot i)
```

严格有序。

不同 Stream：

```
Stream 0 Kernel(batch 0)
可与
Stream 1 H2D(batch 1)
重叠
```

真正的生产实现通常使用 Event 而不是在 CPU 填充前粗粒度 `cudaStreamSynchronize()`，以缩小等待范围。

十二、D2H 的高效模式

12.1 不要每 Step 把大量 Tensor 拉回 CPU

错误模式：

```
for step in range(num_steps):
    huge = activation.cpu()
    save_or_log(huge)
```

问题：

- 强制 D2H；
- 可能同步整个当前 Stream；

- 阻塞训练主循环；
- 占用 PCIe 和 CPU DRAM；
- 破坏 GPU Pipeline。

更好的方式：

- 只复制必要的标量或聚合结果；
- 在 GPU 上先 Reduction；
- 降低日志频率；
- 异步 D2H 到 Pinned Buffer；
- 用后台 CPU 线程处理完成的 Buffer；
- Checkpoint 使用分片和后台写盘。

12.2 异步 D2H 到 Pinned Destination

```
copy_stream = torch.cuda.Stream()
compute_stream = torch.cuda.current_stream()

# GPU结果y由compute_stream产生
copy_stream.wait_stream(compute_stream)

cpu_out = torch.empty(
    y.shape,
    dtype=y.dtype,
    pin_memory=True,
)

with torch.cuda.stream(copy_stream):
    cpu_out.copy_(
        y,
        non_blocking=True,
    )

    # y在copy_stream完成前不能被Allocator复用
    y.record_stream(copy_stream)
    done = copy_stream.record_event()

# CPU可以处理别的事
prepare_next_request()

# CPU第一次读取cpu_out之前等待
# 只同步这个Event，不全局同步整个GPU
done.synchronize()
consume_on_cpu(cpu_out)
```

十三、何时应把数据留在 GPU，何时送回 CPU

13.1 尽量留在 GPU

适合：

- 训练 Forward→Loss→Backward→Optimizer；
- 多个连续 GPU 算子；
- GPU 上可以完成 Reduction；
- 推理后续步骤仍在 GPU；
- GPU 上能直接执行 Sampling；
- 后续需要 NCCL 通信。

原则：

```
中间Tensor不要因为“CPU代码好写”就频繁D2H/H2D往返
```

13.2 适合 H2D

- DataLoader Batch；
- CPU Tokenizer 输出；
- CPU 数据增强结果；
- Checkpoint Restore；
- CPU Offload 参数或优化器状态回载；
- CPU/NVMe 数据到 GPU 的传统路径。

13.3 适合 D2H

- 最终业务结果需要 CPU；
- 保存 Checkpoint；
- CPU 编码、压缩或网络协议；
- 少量 Metric；

- GPU 不适合的分支密集后处理；
- Offload；
- 调试。

13.4 不适合 GPU 的“小计算”

设一个 CPU 操作只需：

```
T_cpu = 20 μs
```

若 Offload 到 GPU：

```
H2D = 15 μs
Kernel Launch + Compute = 10 μs
D2H = 15 μs
总计 = 40 μs
```

即使 GPU Kernel 本身更快，总流程反而更慢。

所以 GPU Offload 的收益边界：

```
T_cpu
>
T_H2D + T_kernel + T_D2H + T_sync
```

如果输出可继续留在 GPU，去掉 D2H 后，收益条件会更容易满足。

十四、CPU – GPU 通信优化能改善哪些指标

优化	主要改善	原理	不一定改善
减少往返传输	Latency、吞吐、CPU占用	直接删除PCIe和同步成本	GPU显存容量
合并小传输	Latency、吞吐	摊薄每次API/DMA固定开销	总Payload字节数
Pinned Memory	H2D/D2H带宽、抖动	允许直接DMA	GPU计算时间
`non_blocking=True`	CPU提交效率、吞吐	减少Host同步	单次GPU复制本身一定变快
Side Stream	吞吐	Copy与独立Compute并发	单请求关键路径延迟一定下降
双缓冲/预取	稳态吞吐、GPU利用率	下一批传输隐藏在当前计算下	首批Latency、尾部Drain
降低传输dtype	带宽、Latency	Payload减少	数值质量、固定延迟

优化	主要改善	原理	不一定改善
NUMA绑定	带宽、尾延迟	避免跨Socket DRAM流量	单Socket系统
Unified Memory Prefetch	易用性、迁移可预测性	提前迁移页面	所有访问模式
GPUDirect RDMA/GDS	CPU占用、Latency、带宽	NIC/Storage直达GPU	不支持的拓扑/文件系统

十五、NUMA：多路 CPU 服务器的隐藏瓶颈

双 Socket 服务器中：

```
GPU 0靠近NUMA Node 0
但DataLoader Worker和Pinned Buffer在NUMA Node 1
```

传输路径可能变成：

```
Node 1 DRAM
→ CPU互联UPI/Infinity Fabric
→ Node 0 PCIe Root Complex
→ GPU 0
```

结果：

- H2D 带宽下降；
- CPU 互联被占用；
- 尾延迟和抖动增加；
- 多 GPU 同时加载时拥塞。

建议：

1. 用 `nvidia-smi topo -m` 查看 GPU/CPU/NIC 拓扑；
2. 将负责 GPU 的 CPU Worker 绑到相近 NUMA Node；
3. 在绑定后分配并 First-Touch Host Buffer；
4. 再注册为 Pinned Memory；
5. 建立 GPU–CPU Worker–Pinned Buffer 局部性。

示意：

```
numactl \
  --cpunodebind=0 \
```

```
  --membind=0 \
  python train.py
```

不能只看 GPU Utilization；还要检查 CPU NUMA Miss、PCIe Throughput 和 DataLoader Wait。

十六、Unified Memory：统一地址不等于没有数据移动

CUDA Unified Memory：

```
cudaMallocManaged(&ptr, bytes);
```

CPU 和 GPU 可以使用同一指针，但底层页面仍可能在：

- CPU System Memory；
- GPU Memory；
- 两端之间迁移。

首次访问可能触发 Page Fault 和页面迁移：

```
GPU访问CPU驻留页面
→ Fault
→ Driver迁移/映射
→ Kernel继续
```

优点：

- 编程简单；
- 可超额使用 GPU 显存；
- 适合复杂、不规则数据结构；
- 某些 CPU–GPU 一致性平台更自然。

风险：

- Page Fault 延迟；
- Thrashing：CPU/GPU反复争用同一页面；
- 访问模式不规则时不可预测；
- 性能可能低于显式批量 H2D/D2H。

可使用 Prefetch：

```
cudaMemPrefetchAsync(
    ptr,
    bytes,
    gpu_id,
    stream
);
```

或预取回 CPU：

```
cudaMemPrefetchAsync(
    ptr,
    bytes,
    cudaCpuDeviceId,
    stream
);
```

关键心智模型：

> Unified Virtual Addressing/Managed Memory 统一的是寻址和管理体验，不会让 PCIe 物理搬运成本凭空消失。

在 Grace Hopper 等 CPU–GPU 高带宽一致性平台上，访问与迁移语义可能明显不同，必须按具体平台评估，不能套用普通离散 PCIe GPU 的结论。

十七、Mapped Pinned / Zero-Copy Memory

Pinned Host Memory 可以映射到 GPU 地址空间，GPU Kernel 直接访问 Host Memory。

这叫 Zero-Copy 的原因是：

```
不先显式复制整个Buffer到HBM
```

但每次 GPU Load/Store 仍可能穿过 PCIe，因此并不是“零成本”。

适合：

- 数据只访问一次；
- 稀疏、随机、小量读取；
- GPU 显存容量紧张；
- 集成 GPU 或高带宽一致性平台；
- 显式复制成本高于实际访问量。

不适合：

- 反复读取；
- 大规模 GEMM 输入；
- 高复用数据；
- 对带宽和延迟敏感的主训练热路径。

对于高复用 Tensor：

```
先H2D到HBM一次
→ GPU在HBM反复使用
```

通常比每次都远程访问 Host Memory 更好。

十八、GPUDirect：绕过CPU Bounce Buffer

18.1 GPUDirect RDMA

允许 NIC 或其他 PCIe Peer Device 直接读写 GPU Memory：

```
Remote GPU/NIC
→ RDMA
→ Local GPU HBM
```

避免：

```
NIC → CPU DRAM → GPU
```

收益：

- 降低 CPU 占用；
- 减少额外内存复制；
- 降低延迟；
- 提高网络到 GPU 的带宽。

常用于：

- NCCL 跨节点；

- 分布式训练；
- GPU Serving；
- 高性能网络数据流。

18.2 GPUDirect Storage

直接路径：

```
NVMe / Remote Storage
→ DMA
→ GPU Memory
```

避免 CPU Bounce Buffer。

适合：

- 大规模 Checkpoint；
- 数据集直接加载到 GPU；
- GPU 生成数据直接写存储；
- Storage 已成为训练瓶颈。

但控制路径仍由 CPU/驱动设置，且需要：

- 支持的 GPU/驱动；
- 支持的文件系统与存储；
- 正确拓扑；
- 内核模块/DMA-BUF；
- 对齐和注册条件。

如果条件不满足，系统可能回退到 CPU Bounce Path。

十九、训练中的典型 CPU – GPU 流水线

完整训练数据流可能是：

```
Storage
→ CPU Worker读取
→ 解码/Tokenizer/Data Augmentation
```

```
→ Pageable Batch
→ DataLoader Pin线程
→ Pinned Batch
→ H2D Copy Stream
→ GPU Input Buffer
→ Forward/Backward/Optimizer
→ 少量Metric D2H
→ CPU日志
```

理想并发：

```
CPU准备Batch i+2
||
H2D复制Batch i+1
||
GPU计算Batch i
```

这是三级 Pipeline：

1. CPU 数据准备；
2. CPU→GPU 搬运；
3. GPU 计算。

稳态周期接近三者最大值：

```
T_steady
≈ max(
    T_cpu_prepare,
    T_H2D,
    T_gpu_compute
  )
```

如果：

```
T_cpu_prepare = 20 ms
T_H2D = 11 ms
T_gpu_compute = 8 ms
```

即使 H2D 完美重叠，整体仍被 CPU DataLoader 的 20 ms 限制。

所以要同时看：

- DataLoader Queue Depth；
- CPU Worker Utilization；
- Pin Memory Thread；
- H2D Throughput；
- GPU Idle Gap。

二十、推理和 Serving 中的特点

20.1 输入 Token 通常很小

Token IDs：

```
[B,S] × int32/int64
```

与模型权重、KV Cache 和计算相比，输入 Token H2D 通常很小。

因此 LLM Serving 的主要 CPU–GPU 问题可能不是 Token H2D 带宽，而是：

- CPU 调度；
- Continuous Batching；
- Sampling；
- KV Cache 管理；
- 小 Kernel Launch；
- D2H Logits；
- 多请求同步。

20.2 不要把完整 Logits 无脑 D2H

假设：

```
Batch = 128
Vocab = 128K
FP16 Logits
```

数据量：

```
128 × 131072 × 2 Byte
= 32 MiB
```

每个 Decode Step 都 D2H 32 MiB，会产生巨大 PCIe 压力。

更好的方式：

```
GPU上完成Softmax/Top-k/Sampling
→ 只D2H每请求一个Token ID
```

假设每请求返回一个 int32：

```
128 × 4 Byte = 512 Byte
```

从 32 MiB 降到 512 Byte，差异约 65536 倍。

这就是"先在 GPU Reduction，再传最小结果"的典型优化。

二十一、常见失败模式

失败1：每个小Tensor单独 `.cuda()`

问题：

- 每次调用有固定开销；
- 多次 Stream 同步；
- DMA 传输过碎。

优化：

- 合并/打包 Tensor；
- 一次传大 Batch；
- 使用 Nested Batch 的统一异步搬运。

失败2：训练循环中临时 `pin_memory()`

问题：

- Pinning 本身需要 CPU Copy；
- 主线程阻塞；
- 频繁锁页/解锁。

优化：

- `DataLoader(pin_memory=True)`；

- Pinned Buffer Pool；
- 后台线程提前准备。

失败3：`non_blocking=True` 后立刻 `torch.cuda.synchronize()`

```
x = x.to("cuda", non_blocking=True)
torch.cuda.synchronize()
```

这会立刻消除异步收益。

只在：

- CPU 必须读取结果；
- 精确计时边界；
- 调试；

时同步。

失败4：Pinned H2D 后立即复用源Buffer

DMA 仍在读取，数据可能损坏。

失败5：D2H 后CPU立即 `.numpy()` 或打印

异步 D2H 尚未完成，必须先等待对应 Event。

失败6：Copy与Compute放在同一Stream却期待重叠

同一 Stream 严格按序执行：

```
Copy → Compute
```

需要 Side Stream 和双缓冲才能与另一个 Batch 的计算重叠。

失败7：忽略NUMA

Pinned 了也不代表 Host Buffer 在离 GPU 最近的 Socket。

失败8：把Unified Memory理解成“没有传输”

页面仍可能迁移，甚至发生 Thrashing。

失败9：只看API耗时，不看GPU时间线

`non_blocking=True` 的 Python 调用很快返回，不代表 DMA 已完成。

失败10：为了CPU处理，把大Tensor频繁D2H

应先在 GPU 做 Reduction、Filtering、Top-k 或 Sampling，再返回小结果。

二十二、如何用 Profiler 验证

22.1 PyTorch Profiler

观察：

- `aten::to`；
- `aten::_to_copy`；
- `cudaMemcpyAsync`；
- `cudaStreamSynchronize`；
- DataLoader 等待；
- CPU Launch Gap。

22.2 Nsight Systems

重点看 Timeline：

```
CPU Thread
CUDA Copy Streams
CUDA Compute Streams
H2D / D2H Memcpy
Kernel
NCCL
CUDA API
```

验证问题：

1. H2D 是否在 GPU Compute 期间发生?
2. Copy 是否位于独立 Stream?
3. 是否存在隐式同步?
4. GPU Compute 前是否长期等待数据?
5. H2D 是否达到预期有效带宽?
6. D2H 是否让 CPU 主线程或默认 Stream 停顿?
7. 是否有大量小 Memcpy?

22.3 正确计时

CUDA 操作异步，不能只用普通 CPU 时钟包住调用：

```
start = time.perf_counter()
y = x.to("cuda", non_blocking=True)
elapsed = time.perf_counter() - start
```

这主要测到提交时间。

可使用 Event：

```
start = torch.cuda.Event(enable_timing=True)
end = torch.cuda.Event(enable_timing=True)

start.record()
y = x.to("cuda", non_blocking=True)
end.record()

end.synchronize()
print(start.elapsed_time(end), "ms")
```

端到端测量则在合理边界同步：

```
torch.cuda.synchronize()
t0 = time.perf_counter()

run_step()

torch.cuda.synchronize()
print(time.perf_counter() - t0)
```

二十三、工程选型决策表

场景	推荐路径	原因
常规训练输入	DataLoader Pinned + non_blocking H2D	简单、成熟、可流水化
GPU计算比H2D长	双缓冲	H2D可被完全隐藏
H2D比计算长	降低输入字节、提高链路、增加计算批量	Overlap后仍由H2D决定周期
很多小Tensor	合并/打包	摊薄API与DMA固定成本
CPU后处理只需少量结果	GPU先Reduction/Top-k	减少D2H Payload
Checkpoint保存	分片D2H + 后台IO；条件允许用GDS	避免主训练流长时间阻塞
参数/Optimizer Offload	预取 + Pinned Pool + Overlap	隐藏H2D/D2H，但占PCIe
不规则超大数据结构	Unified Memory + Prefetch评估	编程方便，但需防Page Fault/Thrash
稀疏一次性Host访问	Mapped Pinned/Zero-Copy评估	避免复制未使用的大部分数据
跨节点GPU通信	GPUDirect RDMA/NCCL	避免CPU Bounce Buffer
Storage直接喂GPU	GPUDirect Storage	降低CPU占用和Bounce Copy
双Socket多GPU	NUMA绑定	避免跨SocketHost内存流量

二十四、最终心智模型

```
CPU:
擅长控制、分支、IO、数据准备和低延迟小任务

GPU:
擅长大规模、规则、高并行、高算术强度计算

H2D:
把CPU内存数据送到GPU HBM

D2H:
把GPU结果返回CPU内存
```

```
Pageable:
通常需要内部Pinned Staging

Pinned:
允许Copy Engine直接DMA，是可靠异步传输的基础

non_blocking:
让CPU不立即等待，不等于自动产生GPU侧Overlap

真正Overlap:
Pinned + Async Copy + Side Stream + Event + 无依赖 + Double Buffer
```

固定问七个问题：

1. **必要性**：这份数据必须跨 CPU–GPU 边界吗?
2. **大小**：Payload 是多少 Byte？能否压缩、降精度或先 Reduction?
3. **粒度**：是否有大量小复制，应不应该合并?
4. **内存类型**：Pageable、Pinned、Managed 还是 GPU Memory?
5. **依赖**：谁什么时候第一次读取结果？需要什么 Event?
6. **重叠**：是否有独立计算可隐藏 Copy?
7. **拓扑**：PCIe/NVLink-C2C、NUMA、NIC、Storage 路径是什么?

真正的目标不是“让 memcpy 更快”，而是：

> **尽量删除跨边界传输，并将剩余传输从关键路径移到并行流水线中。**

可选自测

1. 计算题

输入：

```
Shape = [4,8192,4096]
Dtype = BF16
H2D有效带宽 = 28 GB/s
GPU计算 = 20 ms
```

计算：

- Batch 数据量的 MiB；

- H2D 时间；
- 串行每 Batch 时间；
- 完美双缓冲的稳态周期；
- 理想吞吐加速比。

2. 因果题

为什么：

```
x.pin_memory().to("cuda", non_blocking=True)
```

可能比：

```
x.to("cuda", non_blocking=True)
```

更慢？什么情况下 Pinning 才值得？

3. 工程决策题

LLM Decode 每 Step 产生：

```
Batch = 256
Vocab = 128K
FP16 Logits
```

CPU 只需要每个请求最终采样的 Token ID。

请比较：

1. 把完整 Logits D2H 后在 CPU Sampling；
2. 在 GPU 完成 Top-k/Softmax/Sampling，只 D2H Token ID。

计算两种方案的 Payload，并说明对 TPOT、PCIe、CPU 占用和 Batch 扩展性的影响。

继续学习

1. 前置：PCIe、NVLink、NVSwitch 与 NUMA 拓扑——理解链路、Root Complex 和跨Socket瓶颈。

2. 同层对比：CUDA Stream、Event、Copy Engine 与真正的 Overlap——深入异步依赖和时间线调试。
3. 下游应用：ZeRO/FSDP CPU Offload 与 NVMe Offload——理解参数、梯度和优化器状态如何在 CPU/GPU/存储间流水。

回复 1、2 或 3 继续；也可以说“稍后”或“不感兴趣”。

事实来源与版本边界

截至 2026-08-03，本文稳定机制主要核对：

- NVIDIA CUDA C++ Programming Guide；
- NVIDIA CUDA C++ Best Practices Guide；
- PyTorch CUDA Semantics；
- PyTorch `pin_memory` / `non_blocking` 官方教程；
- NVIDIA GPUDirect RDMA / GPUDirect Storage 官方文档。

PCIe/NVLink 带宽、Copy Engine 数量、默认 Stream 行为、Unified Memory、一致性语义和 GPUDirect 支持依 GPU、CPU、驱动、操作系统、CUDA/PyTorch 版本与平台拓扑变化。生产环境必须以实际设备属性、`nvidia-smi topo -m`、CUDA Events 和 Nsight Systems Timeline 为准。

大模型训练优化器全解：作用、主流算法、显存代价与选型

> **原始问题：**详细解释训练中优化器的作用以及不同主流优化器的详细讲解，并且归纳总结不同优化器的异同和适用场景

提问：2026/08/07 15:24　更新：2026/08/07 15:34　重要度：5/5

大模型训练优化器全解：作用、主流算法、显存代价与选型

一句话结论

优化器（Optimizer）的工作不是“计算梯度”，而是**把反向传播产生的瞬时梯度，变成稳定、尺度合适、带有历史信息与正则化约束的参数更新**。

```
Forward 得到 Loss
→ Backward 得到梯度 g_t
→ Unscale / Gradient Clipping
→ Optimizer 用历史状态把 g_t 变成更新方向 u_t
→ Learning-Rate Schedule 给出步长 η_t
→ 更新参数 θ_t
→ 清梯度，进入下一 Step
```

对于 Transformer/LLM：

- **AdamW 是最稳妥、最常见的默认基线**：逐参数自适应缩放，面对不同层、稀疏梯度和非平稳训练通常比纯 SGD 更容易调；
- **代价是状态显存大**：标准 AdamW 为每个参数维护一阶矩 `m` 和二阶矩 `v`；若均为 FP32，仅这两份状态就是 `8 Byte/参数`；
- **Adafactor**、**8-bit AdamW**、**Lion**主要从不同方向降低状态显存，但优化几何和调参方式并不完全等价；
- **LARS/LAMB**主要针对极大 Global Batch 的 Layer-wise 更新尺度问题；
- **Shampoo/Muon**利用矩阵结构做更强预条件或正交化，可能提高样本效率，但计算、分布式实现和调参复杂度明显更高；

- **Fused AdamW、ZeRO/FSDP Optimizer Sharding、CPU Offload 不是新的优化数学算法，**而是同一算法的系统实现与状态布局优化。

一、优化器在完整训练链路中的位置

设模型参数为：

```
θ_t: 第 t 步开始时的全部参数
```

一个 Mini-batch 的目标函数为：

```
L_t(θ_t)
```

反向传播计算：

```
g_t = ∇_θ L_t(θ_t)
```

梯度只回答：

> 在当前 Mini-batch、当前参数点附近，哪个方向让 Loss 上升最快?

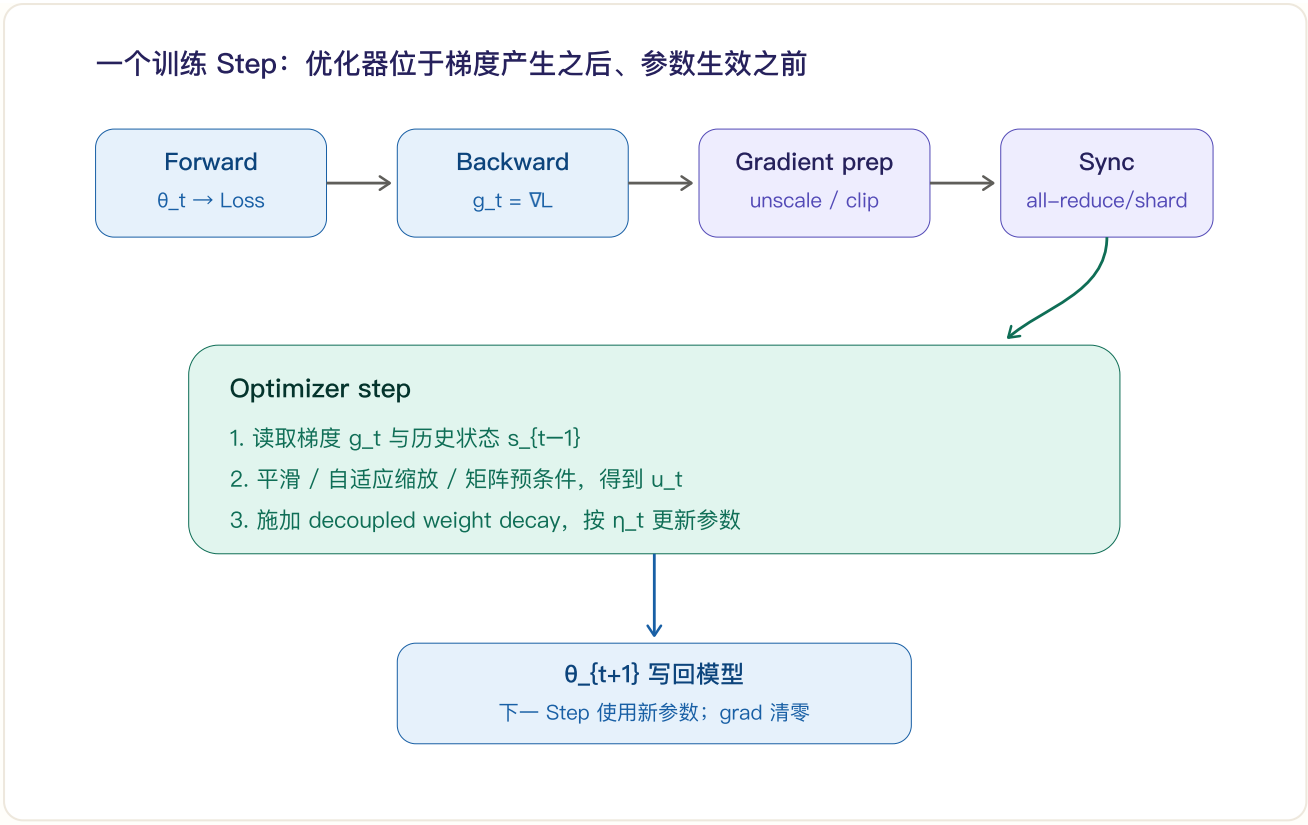
要做梯度下降，就沿相反方向移动。但真实训练中的 `g_t` 有四个问题：

1. **有噪声**：Mini-batch 梯度不是全量数据梯度；
2. **不同参数尺度差异大**：Embedding、Attention、MLP、Norm 的梯度统计不同；
3. **曲率不同**：某些方向很陡，另一些方向很平；同一个学习率可能一边震荡、一边爬得很慢；
4. **分布持续变化**：训练早期、稳定期、衰减期的梯度统计不同。

因此优化器通常实现一个状态机：

```
s_t = update_state(s_{t-1}, g_t)
u_t = transform(g_t, s_t, θ_t)
θ_{t+1} = θ_t − η_t × u_t
```

```
s_t   优化器状态，例如 Momentum、二阶矩、预条件矩阵
u_t   经过平滑、归一化或预条件后的更新方向
η_t   当前学习率，由 Scheduler 决定
```



优化器不负责什么

必须区分相邻概念：

机制	主要职责	是否属于优化器数学规则
Backward / Autograd	计算梯度 `g_t`	否
Loss Scaling	防止 FP16 梯度下溢	否，发生在梯度处理阶段
Gradient Clipping	限制异常梯度范数或数值	通常是 Step 前处理
Optimizer	将梯度及历史状态变成更新方向	是
LR Scheduler	决定每一步的全局步长 `η_t`	通常独立于优化器
Weight Decay	约束参数规模	可耦合进梯度，也可由 AdamW 解耦执行
ZeRO/FSDP	分片状态、梯度或参数	系统实现，不改变基础更新公式

二、为什么不能永远只用最朴素的 SGD

2.1 SGD

Stochastic Gradient Descent：

```
θ_{t+1} = θ_t − η_t × g_t
```

优点：

- 无额外状态；
- 更新规则透明；
- 每步计算和显存开销最低；
- 在某些视觉任务上经过充分调参可获得很好的泛化。

缺点：

- 对学习率和参数尺度敏感；
- 在狭长谷底中容易来回震荡；
- 面对 Transformer 不同模块的梯度尺度差异，需要更精细调参；
- Mini-batch 噪声直接进入更新。

2.2 Momentum SGD

一种常见记法：

```
v_t = μ × v_{t-1} + g_t
θ_{t+1} = θ_t − η_t × v_t
```

```
μ  Momentum 系数，常见量级接近 0.9
v  梯度的指数累积方向
```

不同框架可能在梯度项前乘 `1−μ`，对应状态尺度和有效学习率定义不同，但核心相同：**保留长期一致方向，抵消短期反向噪声。**

简单例子：

```
μ = 0.9
连续梯度 g = [1, 1, −0.2]

v_1 = 1
v_2 = 0.9×1 + 1 = 1.9
v_3 = 0.9×1.9 − 0.2 = 1.51
```

第三步梯度短暂变成 `−0.2`，Momentum 仍保持正向 `1.51`，不会立刻掉头。

2.3 Nesterov Momentum

Nesterov 的直觉是：

> 先按 Momentum 看一眼即将到达的位置，再根据那个位置附近的梯度修正方向。

不同库的等价公式写法较多。工程上应以框架实现为准，不要仅凭一行伪公式判断其更新顺序。

SGD 系适用场景

- CNN/视觉模型或已有成熟 SGD Recipe；
- 优化器显存极度敏感；
- 想要最简单、最容易分析的基线；
- Gradient Statistics 相对均匀，且有充足超参数搜索预算。

对于从零训练 Transformer/LLM，SGD 通常不是首选默认值，但它仍是理解其他优化器的基准坐标系。

三、从 AdaGrad、RMSProp 到 Adam：为什么要逐参数自适应

3.1 AdaGrad

为每个参数累计历史平方梯度：

```
V_t = V_{t-1} + g_t²
θ_{t+1} = θ_t − η × g_t ÷ (√V_t + ε)
```

逐元素执行。某个坐标历史梯度越大，后续有效步长越小。

优点：适合稀疏特征；出现次数少的参数不会被过度压小步长。

核心问题：`V_t` 只增不减，训练很长后有效学习率可能持续趋近 0。

3.2 RMSProp

用指数移动平均替代无限累积：

```
v_t = ρ × v_{t-1} + (1−ρ) × g_t²
θ_{t+1} = θ_t − η × g_t ÷ (√v_t + ε)
```

`ρ` 控制对历史平方梯度的遗忘速度。RMSProp 解决了 AdaGrad 的二阶统计永不遗忘问题，但没有 Adam 那套一阶矩与偏差修正的标准组合。

3.3 Adam

Adam 同时维护一阶矩和二阶矩：

```
m_t = β1 × m_{t−1} + (1−β1) × g_t
v_t = β2 × v_{t−1} + (1−β2) × g_t²
```

由于初始 `m_0=v_0=0`，早期估计偏向 0，需要 Bias Correction：

```
m_hat_t = m_t ÷ (1−β1^t)
v_hat_t = v_t ÷ (1−β2^t)
```

参数更新：

```
u_t = m_hat_t ÷ (√v_hat_t + ε)
θ_{t+1} = θ_t − η_t × u_t
```

直觉：

```
m_hat   回答“最近梯度平均朝哪里走”
v_hat   回答“这个坐标最近梯度幅度/波动多大”

m_hat ÷ √v_hat
= 方向一致性 ÷ 历史尺度
```

因此 Adam 对每个坐标形成不同的有效学习率。

3.4 一个可手算的 Adam 例子

设：

```
β1 = 0.9
β2 = 0.999
ε   暂时忽略
梯度 g_1=2, g_2=4
```

第 1 步：

```
m_1 = 0.9×0 + 0.1×2 = 0.2
v_1 = 0.999×0 + 0.001×2² = 0.004

m_hat_1 = 0.2 ÷ 0.1 = 2
v_hat_1 = 0.004 ÷ 0.001 = 4

u_1 = 2 ÷ √4 = 1
```

第 2 步：

```
m_2 = 0.9×0.2 + 0.1×4 = 0.58
v_2 = 0.999×0.004 + 0.001×16 = 0.019996

m_hat_2 = 0.58 ÷ (1−0.9²)
        ≈ 3.0526

v_hat_2 = 0.019996 ÷ (1−0.999²)
        ≈ 10.003

u_2 ≈ 3.0526 ÷ √10.003
    ≈ 0.965
```

虽然原始梯度从 2 增大到 4，经过历史尺度归一化后，更新量没有简单翻倍。这就是自适应缩放。

Adam 的适用与风险

适合：

- Transformer/LLM；
- 梯度稀疏或不同模块尺度差异大；
- 非平稳目标；
- 希望较少调参即可获得稳定基线。

风险：

- 两份逐参数状态显存大；
- `β2` 太大时二阶矩响应慢，遇到训练分布变化可能产生陈旧尺度；
- `ε` 不只是防除零，也影响小 `v` 坐标的有效步长；
- Adam 中直接加入 L2 项不等价于真正的 Weight Decay。

四、AdamW：为什么它是 LLM 默认基线

4.1 Adam 中的 L2 正则为什么有问题

如果把 L2 正则写进 Loss，梯度会变成：

```
g'_t = g_t + λ × θ_t
```

Adam 随后对整个 `g'_t` 做逐参数预条件：

```
u_t = AdamPrecondition(g_t + λθ_t)
```

于是正则项 `λθ_t` 也会被每个坐标的 `1/√v_hat` 缩放。不同参数获得不同的实际衰减强度。

对于普通 SGD，L2 正则与 Weight Decay 在合适缩放下等价；对于 Adam 这类自适应优化器，它们不等价。

4.2 AdamW 的 Decoupled Weight Decay

AdamW 将两件事分开：

```
自适应梯度更新：
θ ← θ − η_t × m_hat ÷ (√v_hat + ε)

参数衰减：
θ ← θ − η_t × λ × θ
```

合并写为：

```
θ_{t+1}
= (1−η_t λ) × θ_t
− η_t × m_hat_t ÷ (√v_hat_t + ε)
```

这样 Weight Decay 不再经过 Adam 的二阶矩预条件。

4.3 哪些参数通常不做 Weight Decay

常见 Recipe 会把参数分组：

```
Decay：Linear/Attention/MLP 的矩阵权重
No Decay：Bias、LayerNorm/RMSNorm Scale，有时还包括 Embedding
```

这是一种经验设计，不是数学定律。Embedding 是否衰减、是否对所有二维矩阵统一衰减，应以目标模型的已验证 Recipe 为准。

4.4 LLM 中的典型配套

AdamW 几乎不会单独工作：

```
AdamW
+ Warmup
+ Cosine / Linear / WSD Decay
+ Global Gradient Norm Clipping
+ BF16 或 FP16 Mixed Precision
+ FP32 Optimizer States
+ Fused Kernel
+ ZeRO/FSDP State Sharding
```

`β1/β2/ε/weight_decay/peak_lr/warmup` 是一组耦合超参数。更换模型规模、Global Batch、Token 数或精度后，不应机械复制。

五、Adafactor：怎样把二阶矩状态从 O(nm) 降到 O(n+m)

对矩阵参数：

```
W: [n,m]
G: [n,m]
```

Adam 为其保存完整二阶矩：

```
V: [n,m]
状态量 = n×m
```

Adafactor 不保存完整 `V`，而保存平方梯度统计的行、列摘要：

```
R: [n]
C: [m]
状态量 = n+m
```

再用外积近似完整二阶矩：

```
V_hat[i,j]
≈ R[i] × C[j] ÷ mean(R)
```

随后：

```
U = G ÷ (√V_hat + ε)
```

原论文还强调：

- Update Clipping：防止二阶矩衰减过慢时产生过大的更新；
- 随训练步数变化的二阶矩衰减率；
- Relative Step Size：根据参数 RMS 调整更新尺度；
- 可不使用一阶 Momentum，从而进一步减少状态。

数量级例子

假设矩阵：

```
W: [4096,16384]
元素数 = 67,108,864
```

FP32 完整二阶矩：

```
67,108,864 × 4 Byte
= 268,435,456 Byte
= 256 MiB
```

Adafactor 行列状态：

```
(4096 + 16384) × 4 Byte
= 81,920 Byte
≈ 80 KiB
```

这里只比较该矩阵的二阶矩，不包含一阶 Momentum、参数、梯度和临时 Buffer。

Adafactor 的边界

- 对二维大矩阵节省非常显著；
- 向量、小矩阵无法获得同等级因式分解收益；
- 因式分解是假设性近似，不等价于完整逐元素二阶矩；
- 不同库对 Relative Step、Warmup、Scale Parameter、Momentum 的默认值差异大；
- 从 AdamW 切换时不能直接照搬学习率。

适合：优化器状态成为硬瓶颈、T5/Transformer 类模型、参数矩阵很大，且愿意重新验证稳定性和超参数。

六、LARS 与 LAMB：极大 Batch 下的 Layer-wise Trust Ratio

6.1 为什么极大 Batch 会遇到更新尺度问题

Global Batch 增大后，梯度噪声下降，通常希望提高学习率以增加吞吐。但不同层的：

```
参数范数 ||θ_l||
梯度/更新范数 ||u_l||
```

差异可能很大。一个全局学习率可能让某些层更新过猛，另一些层几乎不动。

6.2 LARS

LARS 主要建立在 SGD/Momentum 更新上，按层计算 Trust Ratio：

```
r_l = ||θ_l|| ÷ (||g_l|| + λ||θ_l|| + ε)
```

更新近似：

```
θ_l ← θ_l − η_t × r_l × (g_l + λθ_l)
```

它在极大 Batch 的视觉训练中影响较大，但并不是 Transformer 的通用默认选择。

6.3 LAMB

LAMB 先得到 Adam 类方向：

```
a_l = m_hat_l ÷ (√v_hat_l + ε)
r_l = a_l + λθ_l
```

再计算 Layer-wise Trust Ratio：

```
trust_l = ||θ_l|| ÷ (||r_l|| + ε)
```

更新：

```
θ_l ← θ_l − η_t × trust_l × r_l
```

因此 LAMB 可以理解为：

```
Adam 的逐参数自适应
+ Layer-wise Update Norm 控制
```

LAMB 原始工作重点是极大 Batch 的 BERT/ResNet 训练。它解决的不是“模型太大放不下”，而是“大 Batch 提高学习率后，各层相对更新尺度失衡”。

适合：Global Batch 极大、已有 LAMB Recipe、希望保持 Layer-wise 更新比例。普通规模训练不应仅因为“模型大”就自动选择 LAMB。

七、Lion：用 Sign Momentum 换掉二阶矩

Lion 只维护一份 Momentum 状态：

```
c_t = β1 × m_{t−1} + (1−β1) × g_t
```

参数更新使用符号：

```
θ_t = θ_{t−1} − η_t × (sign(c_t) + λθ_{t−1})
```

再更新状态：

```
m_t = β2 × m_{t−1} + (1−β2) × g_t
```

核心差异：

```
AdamW: m_hat ÷ √v_hat，连续地使用梯度幅度
Lion: sign(c)，每个坐标主要保留方向，幅度约为全局学习率
```

优点：

- 只需一份状态，FP32 状态约 `4 Byte/参数`；
- 更新计算简单；
- 某些训练任务中可获得有竞争力的结果。

代价：

- 丢弃了逐坐标更新幅度；
- 学习率直接控制每个坐标的更新大小，通常比 AdamW 更敏感；
- Weight Decay 与学习率需要重新搜索；
- AdamW 的超参数不能原样迁移。

适合：状态显存重要、愿意做独立 LR/WD/Beta Sweep，并已有目标模型上的验证。若只允许一次高成本预训练，AdamW 往往仍是更保守的选择。

八、Shampoo：从逐元素缩放升级为矩阵预条件

Adam 的二阶矩是对角预条件：每个参数独立缩放，不建模同一矩阵不同行列之间的相关性。

对于二维梯度：

```
G_t: [n,m]
```

Shampoo 累计两个矩阵统计：

```
L_t = L_{t-1} + G_t G_t^T      [n,n]
R_t = R_{t-1} + G_t^T G_t      [m,m]
```

预条件后的梯度近似：

```
G_pre = L_t^(-1/4) × G_t × R_t^(-1/4)
```

为什么是 `-1/4`：左右各承担一部分矩阵二阶预条件，使组合效果对应更完整的曲率归一化。

优点：

- 利用参数 Tensor 的矩阵结构；
- 比 Adam 的对角缩放表达更丰富；
- 可能用更少 Step 达到目标 Loss。

代价：

- 状态从 `O(nm)` 变成 `O(n²+m²)`，对非常不方正的矩阵可能有利，但大维度仍可能很重；
- 需要计算逆矩阵根；
- 常通过 Block Partition、低频更新 Preconditioner、分布式 Preconditioner 等方式控制成本；
- 每步墙钟时间、通信和实现复杂度更高。

适合：训练成本由“需要多少 Step”主导，愿意以更复杂的单步计算换样本效率，并有成熟分布式实现。它不是随手替换 AdamW 的低风险选项。

九、Muon：矩阵 Momentum 的近似正交化

Muon 是面向二维矩阵参数的新兴方案。基本思路：

```
M_t = μ × M_{t-1} + G_t
O_t = Orthogonalize(M_t)
W_t = W_{t-1} − η_t × O_t
```

实践中常用 Newton–Schulz Iteration 近似计算矩阵的极分解方向，而不是精确 SVD。

直觉：

> AdamW 对每个元素单独缩放；Muon 把整个梯度矩阵视作一个线性映射，调整其奇异方向，使更新在矩阵空间中更均衡。

工程上通常是 Hybrid Optimizer：

```
二维 Hidden Weight Matrices → Muon
Embedding / LM Head / Bias / Norm / Scalar → AdamW
```

原因是 Muon 的正交化天然针对矩阵，向量和特殊参数不适用同一规则。

截至 2026-08，已有大规模 LLM/MoE 技术报告展示了分布式 Muon 实践，但应注意：

- 报告中的计算效率收益依赖模型、数据、Recipe 和对照调参质量，不能视为所有任务的固定倍数；
- Newton–Schulz 增加额外矩阵运算；
- FSDP/ZeRO 将矩阵展平或切片后，如何恢复完整矩阵语义并高效通信是系统难点；
- 更新尺度、Weight Decay 和矩阵长宽比需要专门处理；
- 生态成熟度和可复现实证仍弱于 AdamW。

适合：愿意投入研究工程预算、追求更高样本效率，并能进行严格 AdamW 对照实验的大规模预训练团队。默认生产基线仍应保留 AdamW。

十、8-bit AdamW、Fused AdamW、ZeRO：不要把实现优化误认成新算法

10.1 8-bit AdamW

核心更新仍是 AdamW，但 `m/v` 使用 8-bit 或块量化格式保存，更新时解量化、计算、再量化。

收益：显著减少状态显存和状态读写带宽。

风险：

- Quantization Scale/Block Size 影响误差；
- 小 Tensor、异常值和非常小的状态需要特殊处理；
- 并不等价于把 AdamW 的数学状态精确保存为 FP32。

如果目标是“尽量保留 AdamW 行为，同时降低显存”，8-bit AdamW 通常比直接换优化器家族更接近原基线。

10.2 Fused / Foreach AdamW

把大量逐 Tensor 操作合并成少量 GPU Kernel：

```
读取参数、梯度、m、v
→ 更新 m/v
→ Bias Correction
→ 参数更新与 Weight Decay
```

数学规则可相同，但减少 Kernel Launch 和 HBM 往返。Fused 是性能实现属性，不代表优化效果一定不同。

10.3 ZeRO-1 / FSDP Optimizer State Sharding

DDP 中每个 Rank 通常保留完整参数和完整优化器状态，状态存在 N 份副本。

ZeRO-1 将 Optimizer States 按 DP Rank 分片：

```
每个 Rank 只更新自己负责的参数分片
→ 更新后同步新参数分片
```

FSDP/ZeRO-2/3 还可以进一步分片梯度和参数。它们改变的是状态放在哪里、何时通信，不应改变理想数学更新。

十一、优化器状态到底占多少显存

假设：

```
参数量 P
BF16 参数       2 Byte/参数
BF16 梯度       2 Byte/参数
FP32 Master     4 Byte/参数
```

```
FP32 Momentum m 4 Byte/参数
FP32 Variance v 4 Byte/参数
```

典型 Mixed-Precision AdamW 静态模型状态近似：

```
2 + 2 + 4 + 4 + 4
= 16 Byte/参数
```

这不包含 Activation、临时 Buffer、通信 Bucket、Allocator 碎片和 Workspace。

7B 参数数值例子

```
P = 7×10⁹
```

AdamW 两份 FP32 Moments：

```
7×10⁹ × 8 Byte
= 56×10⁹ Byte
≈ 52.2 GiB
```

加 FP32 Master Weight：

```
7×10⁹ × 12 Byte
= 84×10⁹ Byte
≈ 78.2 GiB
```

再加 BF16 参数和梯度：

```
7×10⁹ × 16 Byte
= 112×10⁹ Byte
≈ 104.3 GiB
```

量纲检查：

```
参数 × Byte/参数 = Byte
GiB = Byte ÷ 2³⁰
```

若 8 个 DP Ranks 理想均匀分片全部这些状态：

```
104.3 GiB ÷ 8 ≈ 13.0 GiB/Rank
```

真实峰值还取决于参数 All-Gather、梯度 Reduce-Scatter、Prefetch、Checkpoint 和临时 Full Buffer。

状态显存对比

优化器	主要状态	FP32 额外状态量级	特点
SGD	无	0 B/参数	最省状态
Momentum SGD	`v`	4 B/参数	一阶平滑
Adam/AdamW	`m,v`	8 B/参数	一阶+逐参数二阶矩
Lion	`m`	4 B/参数	Sign Momentum
8-bit AdamW	量化 `m,v`	通常约 2 B/参数量级，另有 Scale/元数据	行为接近 AdamW，但有量化误差
Adafactor	行/列二阶矩，可选 Momentum	大矩阵二阶状态约 O(n+m)	节省取决于 Shape
LAMB	`m,v` + 临时范数	约 8 B/参数	Adam 类状态，加 Layer Trust Ratio
Shampoo	每维 Preconditioner	依 Shape/Block 而变	状态和计算不再按固定 B/参数描述
Muon	矩阵 Momentum；非矩阵常另用 AdamW	主矩阵约 4 B/参数，混合部分另算	额外正交化计算

十二、统一对比：这些优化器到底有什么相同和不同

优化器差异的三条轴：历史状态、缩放粒度、结构信息

输入都来自同一个 g_t；最终都产生参数更新 θ_{t+1}

差别在于如何过滤噪声、归一化尺度，以及是否利用 Layer/Matrix 结构

梯度与一阶历史

SGD：raw gradient

Momentum：EMA direction

Lion：sign(momentum)

逐元素自适应

AdaGrad / RMSProp

AdamW：$m / \sqrt{v}$

Adafactor：factored v

Layer / Matrix 结构

LARS/LAMB：trust ratio

Shampoo：matrix roots

Muon：orthogonalization

选型不是只看收敛曲线

算法轴：样本效率、稳定性、超参数敏感度、正则化行为

系统轴：状态显存、HBM 流量、额外 FLOP、通信、Checkpoint 体积

运营轴：实现成熟度、失败恢复、可观测性、Recipe 可迁移性

公平比较必须分别调 LR / WD / Warmup / Betas，不能照搬 AdamW 参数。

优化器	更新几何	状态成本	调参难度	典型优势	主要风险
SGD	原始梯度	最低	高	简单、低显存	对尺度和 LR 敏感
Momentum SGD	一阶平滑	低	中	降噪、加速一致方向	无逐参数自适应
AdamW	一阶矩/二阶矩逐元素预条件	高	中低	Transformer 稳定默认	状态显存和 HBM 流量大
Adafactor	因式分解二阶矩	低到中	中高	大矩阵状态极省	近似误差、实现默认值差异
LARS	SGD + Layer Trust Ratio	中低	高	极大 Batch 视觉训练	Transformer 不一定适合
LAMB	Adam + Layer Trust Ratio	高	高	极大 Batch Transformer/BERT	普通 Batch 收益不确定
Lion	Sign Momentum	中低	高	状态约为 AdamW 一半	LR/WD 敏感、丢失幅度信息
Shampoo	矩阵预条件	Shape 相关	很高	可能减少所需 Steps	逆根、状态和分布式复杂
Muon	矩阵 Momentum 正交化	混合	很高	新兴的样本效率潜力	生态和大规模复现尚在发展

所有优化器的共同点

- 都依赖同一个目标函数和 Backward 梯度；
- 都需要 Learning-Rate Schedule；
- 都可能被异常梯度、错误 Loss Scaling、数据问题破坏；
- 都必须与模型规模、Batch、精度和并行方式共同调优；
- 都需要保存状态以便 Checkpoint 精确续训，只有无状态 SGD 例外。

最根本的不同点

1. **是否记忆历史**：SGD 不记；Momentum/Lion 一阶；AdamW 一阶+对角二阶；
2. **缩放粒度**：全局、逐层、逐参数、逐矩阵；
3. **是否使用梯度幅度**：Lion 主要用 Sign，AdamW 使用连续幅度；
4. **状态精度与布局**：FP32、8-bit、Factored、Sharded、Offloaded；
5. **每步成本与所需 Step 数的权衡**：复杂优化器可能单步更贵但目标 Loss 所需 Step 更少。

十三、按场景给出选型建议

场景 A：第一次训练新的 Transformer/LLM

推荐：

```
AdamW + BF16 + FP32 States
+ Gradient Clipping
+ Warmup + 已验证的 LR Schedule
+ Fused Implementation
```

理由：资料、实现、调参经验和故障诊断最成熟。先建立可信基线，再比较其他优化器。

场景 B：AdamW 状态显存成为硬瓶颈

优先级通常是：

1. ZeRO-1/FSDP Shard Optimizer States；
2. 若通信/集群允许，继续优化 Sharding/Offload；
3. 8-bit AdamW，尽量保持 AdamW 更新行为；

4. Adafactor 或 Lion，但必须重新调参和验证质量。

原因：先改变状态布局，通常比直接更换优化算法风险低。

场景 C：极大 Global Batch，扩展后训练不稳定或效果下降

- 先检查 LR Scaling、Warmup、Gradient Noise Scale、数据重复和有效 Token Batch；
- Transformer/BERT 类可评估 LAMB；
- 视觉 CNN 可评估 LARS；
- 不要把 LAMB 当作任意大模型的默认选项。

场景 D：研究更高样本效率

可评估 Shampoo/Muon/SOAP/Sophia 等预条件或二阶近似方向，但公平实验必须：

```
同模型、同数据顺序、同 Token Budget
分别调最优 LR/WD/Warmup
同时报告 Validation Loss vs Tokens
和 Validation Loss vs Wall-clock / FLOPs
```

只报告“达到某 Loss 的 Step 更少”不够，因为每步可能更慢、通信更多。

场景 E：LoRA/PEFT 微调

可训练参数很少时，AdamW 的状态显存通常不再是主矛盾。优先选择稳定简单的 AdamW；只有在全参数微调或超大 Adapter 时，8-bit State 才更重要。

场景 F：Sparse Embedding 或极稀疏参数

AdaGrad 类方法仍可能有价值，因为低频坐标保留更大的有效学习率。具体需看框架是否支持稀疏梯度和目标参数类型。

十四、最小 PyTorch 示例：正确组织参数组和训练 Step

```
import torch
import torch.nn as nn

torch.manual_seed(0)
model = nn.Sequential(
    nn.Linear(8, 16),
    nn.LayerNorm(16),
    nn.GELU(),
```

```
    nn.Linear(16, 4),
).cuda().to(torch.bfloat16)

decay, no_decay = [], []
for name, p in model.named_parameters():
    if not p.requires_grad:
        continue
    if p.ndim >= 2 and "norm" not in name.lower():
        decay.append(p)
    else:
        no_decay.append(p)

optimizer = torch.optim.AdamW(
    [
        {"params": decay, "weight_decay": 0.1},
        {"params": no_decay, "weight_decay": 0.0},
    ],
    lr=3e-4,
    betas=(0.9, 0.95),
    eps=1e-8,
)

for step in range(10):
    x = torch.randn(32, 8, device="cuda", dtype=torch.bfloat16)
    target = torch.randn(32, 4, device="cuda", dtype=torch.bfloat16)

    optimizer.zero_grad(set_to_none=True)
    pred = model(x)
    loss = ((pred - target) ** 2).mean()
    loss.backward()

    grad_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()

    print(step, float(loss), float(grad_norm))
```

说明：

- 示例用 BF16，所以通常不需要 FP16 GradScaler；具体硬件与算子仍需验证；
- `set_to_none=True` 避免无意义清零写带宽；
- `p.ndim>=2` 是常见参数分组启发式，不是所有模型的绝对规则；
- 生产环境还应加入 LR Scheduler、Gradient Accumulation、Distributed Synchronization、Checkpoint 和 NaN/Inf 检查；
- `betas=(0.9,0.95)` 只是 LLM 中常见的一类配置，不是通用最优值。

十五、训练中应该监控什么

15.1 优化动态

```
Global Grad Norm
Update Norm
Parameter Norm
Update-to-Parameter Ratio = ||Δθ|| ÷ ||θ||
```

```
每层的 Grad/Update/Parameter Norm
Adam m/v 的 RMS 与最大值
Gradient Clipping 触发比例
NaN/Inf 与 Loss Scale
```

15.2 系统性能

```
optimizer.step wall time
optimizer kernel 数量与 launch 间隙
HBM read/write throughput
Fused kernel 是否生效
Optimizer State 总显存
State Sharding 的通信时间
Checkpoint 保存/加载耗时与体积
CPU Offload 的 PCIe/NVLink-C2C 流量
```

15.3 质量与公平比较

```
Training Loss vs Tokens
Validation Loss vs Tokens
Validation Loss vs Wall-clock
Validation Loss vs Training FLOPs
下游任务指标
不同随机种子的稳定性
```

若一个优化器每 Step 更快但需要更多 Steps，或者每 Step 更慢但样本效率更高，只看单一维度都会得出错误结论。

十六、常见误区

误区 1：优化器负责反向传播

错误。Backward 计算梯度；Optimizer 消费梯度并更新参数。

误区 2：Adam 的 L2 正则就是 AdamW

错误。Adam 中把 $\lambda\theta$ 加入梯度后会被二阶矩预条件；AdamW 把 Weight Decay 与自适应梯度更新解耦。

误区 3：状态更少的优化器一定更快

错误。Muon/Shampoo 虽状态结构可能有优势，但额外矩阵运算可能提高每步成本。8-bit State 还需要量化/解量化。

误区 4：Fused AdamW 是新的优化算法

错误。它通常是相同更新公式的 Kernel Fusion 实现。

误区 5：ZeRO 会改变 AdamW 的数学结果

理想情况下不会；它改变状态、梯度和参数的布局及通信。精度、Reduction Order、Offload 和异步执行可能产生微小数值差异，但不是新的优化器规则。

误区 6：换优化器可以直接复用 AdamW 的学习率

错误。Lion、Adafactor、LAMB、Muon 的更新尺度完全不同，必须独立调 LR、WD、Warmup 和内部系数。

误区 7：训练 Loss 降得最快就是最好优化器

错误。还要看 Validation、最终能力、Wall-clock、FLOPs、状态显存、失败率和调参成本。

十七、决策树式总结

```
是否缺少可信基线?
└ 是 → AdamW

AdamW 是否主要卡在状态显存?
├ 可以做分片 → ZeRO-1/FSDP Optimizer Sharding
├ 希望最小改变算法 → 8-bit AdamW
├ 大矩阵为主且愿意调参 → Adafactor
└ 可接受改变更新几何 → Lion

是否是极大 Global Batch?
├ Transformer/BERT 类 → 评估 LAMB
└ 视觉 CNN → 评估 LARS

是否有研究预算追求更少训练 Tokens/Steps?
├ 成熟矩阵预条件实现 → Shampoo
├ 二维权重为主、可做 Hybrid → Muon
└ 否 → 保持 AdamW，优化数据、Schedule 和系统实现
```

最重要的工程原则：

> **先把 AdamW 基线做到可信，再改变一个维度。优化器比较必须同时看算法质量和系统成本。**

可选自测

1. 数值题

一个 7B 模型使用 BF16 参数、BF16 梯度、FP32 Master Weight 和 FP32 AdamW `m/v`。不考虑 Activation 和临时 Buffer，为什么静态状态约为 `16 Byte/参数`？8 个 Ranks 完全分片后，每 Rank 理论值是多少?

2. 因果题

为什么把 `λθ` 直接加进 Adam 梯度，不等价于 AdamW 的 Decoupled Weight Decay?

3. 工程题

一个新 LLM 使用 AdamW 时 OOM，但训练质量和稳定性都很好。你会按照什么顺序评估 ZeRO/FSDP、8-bit AdamW、Adafactor 和 Lion？为什么?

继续学习

1. 前置：Learning-Rate Schedule 与 Warmup —— 优化器方向如何与全局步长共同决定稳定性。
2. 同层对比：AdamW、Muon 与 Shampoo 的矩阵更新几何 —— 对角预条件、正交化和矩阵逆根的本质差异。
3. 下游应用：ZeRO/FSDP Optimizer State Sharding —— 优化器状态如何在多 GPU 上分片、通信与恢复。

回复 1、2 或 3 继续；也可以说"稍后"或"不感兴趣"。

事实来源与版本边界

截至 2026-08-07，本讲解依据稳定算法原理及原始资料，包括：Adam（Kingma & Ba, 2015）、Decoupled Weight Decay/AdamW（Loshchilov & Hutter, 2019）、Adafactor（Shazeer & Stern, 2018）、LAMB（You et al., 2020）、Shampoo（Gupta et al., 2018）、Lion（Chen et al., 2023）及 Muon 大规模 LLM 技术报告（Liu et al., 2025）。

具体框架的公式约定、Bias Correction 顺序、Weight Decay、Fused Kernel、State Precision、默认超参数和分布式布局可能随 PyTorch、JAX、TensorFlow、DeepSpeed、Megatron Core 或目标训练栈版本变化。生产配置应以目标版本源码、Checkpoint 格式和实测为准；论文报告中的效率

收益不能跨模型、数据、Token Budget、硬件和调参预算直接外推。

反向传播全流程：从 Loss 到每一个参数梯度

> **原始问题：**从头到尾详细讲解一次反向传播的过程，并且要简单的例子来说明，包含具体数值和shape，使用图片来辅助我完整理解整个流程

提问：2026/08/13 11:14　更新：2026/08/13 11:24　重要度：5/5

反向传播全流程：从 Loss 到每一个参数梯度

一句话结论

反向传播（Backpropagation）不是一种独立于链式法则的神秘算法，而是：在前向计算形成的计算图上，从标量 Loss 开始，按照与前向相反的拓扑顺序，把上游梯度乘以每个算子的局部导数，并在分叉汇合处累加，最终得到每个参数的梯度。

完整训练 Step 可以压缩为：

```
前向：输入 X → 各层中间激活 → 预测 Y_hat → Loss L
建图：记录哪些 Tensor 由哪些算子产生，以及 backward 需要的中间值
反向：从 dL/dL = 1 出发，逐算子执行 Vector-Jacobian Product
累加：同一 Tensor 经多条路径影响 Loss 时，把各路径贡献相加
更新：Optimizer 使用 parameter.grad 修改参数
清理：梯度清零或置 None，释放不再需要的计算图和激活
```

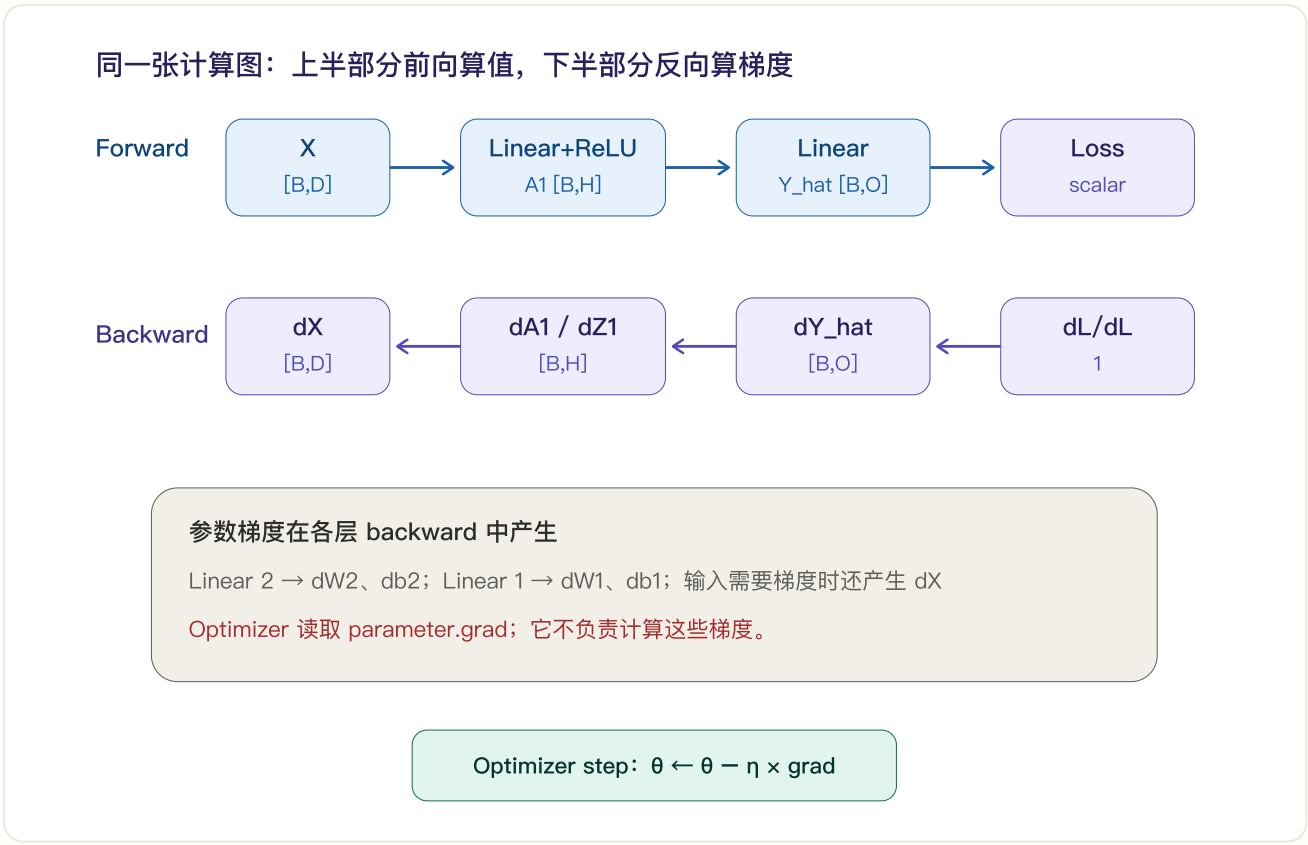
最重要的三个心智模型：

1. **梯度表示敏感度**：`∂L/∂x` 表示 `x` 发生一个微小变化时，Loss 会怎样变化；
2. **每个算子只做局部工作**：Backward 不需要一次构造整个网络的巨大 Jacobian；每个节点只接收上游梯度并返回对输入的梯度；
3. **反向传播的是梯度，不是原始激活**：但计算梯度通常需要读取前向保存的输入、输出、Mask 或统计量。

一、反向传播在训练流程中的位置

一次标准训练迭代包含：

```
1. 读取一个 Mini-batch
2. Forward：根据当前参数产生预测
3. Loss：比较预测与目标
4. Backward：计算 Loss 对所有可训练参数的梯度
5. Gradient Processing：Unscale、Clip、分布式同步
6. Optimizer Step：使用梯度更新参数
7. Zero Grad：清理梯度，进入下一次迭代
```



二、先用一个标量例子理解链式法则

设一个最小模型：

```
x = 2
w = 3
b = 1
y = 8
```

前向：

```
z = wx + b
  = 3×2 + 1
  = 7

L = 1/2 × (z−y)²
  = 1/2 × (7−8)²
  = 0.5
```

我们需要：

```
∂L/∂w
∂L/∂b
∂L/∂x
```

把计算拆成局部关系：

```
L = 1/2 × e²
e = z−y
z = wx+b
```

局部导数：

```
∂L/∂e = e = −1
∂e/∂z = 1
∂z/∂w = x = 2
∂z/∂b = 1
∂z/∂x = w = 3
```

从 Loss 反向：

```
∂L/∂L = 1

∂L/∂z
= ∂L/∂e × ∂e/∂z
= −1 × 1
= −1
```

再传播到 `w,b,x`：

```
∂L/∂w
= ∂L/∂z × ∂z/∂w
= −1 × 2
= −2

∂L/∂b
= ∂L/∂z × ∂z/∂b
= −1 × 1
= −1

∂L/∂x
= ∂L/∂z × ∂z/∂x
= −1 × 3
= −3
```

梯度符号的含义：

```
∂L/∂w = -2
```

表示 `w` 微增时，Loss 会下降。使用 SGD：

```
w_new = w - η × ∂L/∂w
```

若 `η=0.1`：

```
w_new = 3 - 0.1×(-2) = 3.2
```

负梯度导致参数增大，这与当前预测 `7` 小于目标 `8` 一致。

三、计算图、局部梯度和上游梯度

每个计算图节点只需要知道三件事：

```
1. 自己的输入和输出
2. 自己的局部导数
3. 从下游收到的上游梯度
```

假设：

```
a = f(x)
L = g(a)
```

反向公式：

```
∂L/∂x
= ∂L/∂a × ∂a/∂x
```

可以读成：

```
对 x 的总敏感度
= 下游 Loss 对 a 的敏感度
× 当前算子 a 对 x 的局部敏感度
```

为什么反向从标量 Loss 的 1 开始

Loss 是标量：

```
L ∈ R
```

它对自身的导数：

```
∂L/∂L = 1
```

这个 `1` 是反向传播的初始种子。之后每个节点把上游梯度转换成对自己输入的梯度。

为什么变量的梯度通常和变量 Shape 相同

若：

```
X: [B,D]
```

则：

```
dX = ∂L/∂X: [B,D]
```

`dX[i,j]` 表示 `X[i,j]` 微小变化时，标量 Loss 的变化率。

参数也一样：

```
W:  [D,H]
dW: [D,H]
```

这条规则是检查 Backward Shape 是否正确的第一道 sanity check。

四、统一数值例子：两层 MLP

全文使用同一组参数：

```
Batch Size      B = 2
Input Dimension D = 2
Hidden Size     H = 2
Output Size     O = 1
```

网络：

```
Z1    = XW1 + b1
A1    = ReLU(Z1)
```

```
Y_hat = A1W2 + b2
L     = (1/(2B)) × Σ(Y_hat−Y)²
```

Shape：

```
X      [B,D] = [2,2]
W1     [D,H] = [2,2]
b1     [H]   = [2]
Z1/A1  [B,H] = [2,2]
W2     [H,O] = [2,1]
b2     [O]   = [1]
Y_hat  [B,O] = [2,1]
Y      [B,O] = [2,1]
L      scalar
```

具体数值：

```
X = [[ 1, 2],
     [−1, 1]]

W1 = [[ 0.5, −1.0],
      [ 1.0,  0.5]]

b1 = [0.0, −0.5]

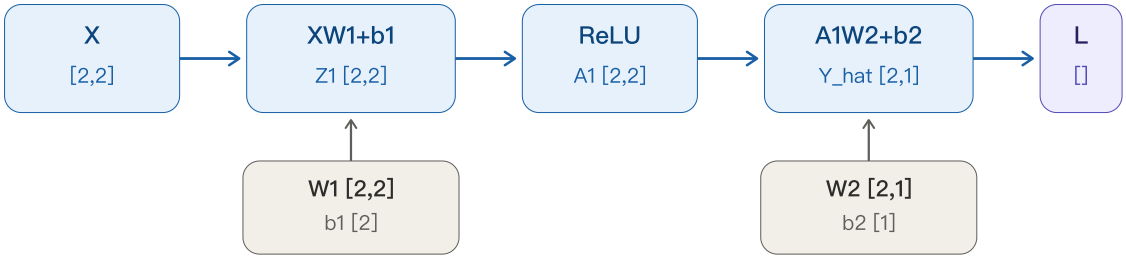
W2 = [[ 0.4],
      [−0.6]]

b2 = [0.1]

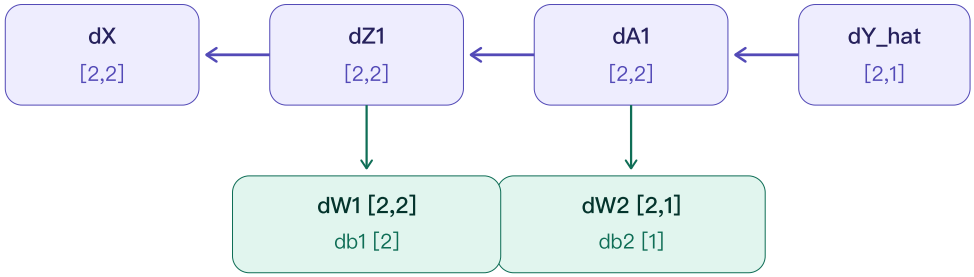
Y = [[1.0],
     [0.0]]
```

统一例子：B=2，D=2，H=2，O=1

Forward values



Backward gradients



每个梯度与被求导变量 Shape 相同；矩阵转置来自维度对齐。

五、前向传播：从 X 算到 Loss

5.1 第一层线性变换

```
Z1 = XW1 + b1
```

Shape 检查：

```
X   [2,2]
W1  [2,2]
XW1 [2,2]
b1  [2] → Broadcast 成每一行都加 [0,-0.5]
Z1  [2,2]
```

计算 `XW1`：

```
XW1
= [[ 1, 2],     [[ 0.5, -1.0],
   [-1, 1]]  ×   [ 1.0,  0.5]]
```

```
= [[1×0.5+2×1.0, 1×(−1.0)+2×0.5],
   [(−1)×0.5+1×1.0, (−1)×(−1.0)+1×0.5]]

= [[2.5, 0.0],
   [0.5, 1.5]]
```

加偏置：

```
Z1
= [[2.5, 0.0],
   [0.5, 1.5]]
+ [0.0, −0.5]

= [[2.5, −0.5],
   [0.5,  1.0]]
```

5.2 ReLU

```
A1 = ReLU(Z1) = max(0,Z1)
```

逐元素：

```
A1
= [[2.5, 0.0],
   [0.5, 1.0]]
```

注意第一行第二个值：

```
Z1[0,1] = −0.5
A1[0,1] = 0
```

它会在反向时把该位置的梯度截断为 0。

5.3 第二层线性变换

```
Y_hat = A1W2 + b2
```

Shape：

```
A1     [2,2]
W2     [2,1]
A1W2   [2,1]
b2     [1]
Y_hat  [2,1]
```

数值：

```
A1W2
= [[2.5,0.0],     [[ 0.4],
   [0.5,1.0]]  ×   [-0.6]]

= [[2.5×0.4 + 0×(-0.6)],
   [0.5×0.4 + 1×(-0.6)]]

= [[ 1.0],
   [-0.4]]
```

加 `b2=0.1`：

```
Y_hat
= [[ 1.1],
   [-0.3]]
```

5.4 MSE Loss

定义：

```
E = Y_hat - Y

L = (1/(2B)) × Σ E²
```

这里使用 `1/(2B)`，前面的 `1/2` 会与平方求导产生的 `2` 抵消。

误差：

```
E
= [[ 1.1],      [[1.0],
   [-0.3]]  -    [0.0]]

= [[ 0.1],
   [-0.3]]
```

Loss：

```
L
= 1/(2×2) × (0.1² + (-0.3)²)
= 1/4 × (0.01 + 0.09)
= 0.025
```

前向完成后，计算图至少需要为反向保留这些信息：

```
X、W1：计算 dW1 和 dX
Z1 或 ReLU Mask：计算 dZ1
A1、W2：计算 dW2 和 dA1
Y_hat、Y 或 E：计算 dY_hat
```

六、反向第 1 步：Loss 对预测的梯度

```
L = (1/(2B)) × Σ(Y_hat−Y)²
```

对 `Y_hat` 求导：

```
dY_hat
= ∂L/∂Y_hat
= (Y_hat−Y) ÷ B
= E ÷ B
```

因为 `B=2`：

```
dY_hat
= [[ 0.1],
   [−0.3]] ÷ 2

= [[ 0.05],
   [−0.15]]
```

Shape：

```
Y_hat   [2,1]
dY_hat  [2,1]
```

含义：

- 第一个样本预测偏高 `0.1`，梯度为正；梯度下降会尝试降低预测；
- 第二个样本预测偏低 `−0.3`，梯度为负；梯度下降会尝试提高预测。

七、反向第 2 步：第二个 Linear Layer

前向：

```
Y_hat = A1W2 + b2
```

收到上游梯度：

```
dY_hat: [B,O] = [2,1]
```

Linear Backward 的三个输出：

```
dW2 = A1^T × dY_hat
db2 = sum_batch(dY_hat)
dA1 = dY_hat × W2^T
```

7.1 为什么 dW2 要用 A1 转置

Shape：

```
A1^T       [H,B] = [2,2]
dY_hat     [B,O] = [2,1]
dW2        [H,O] = [2,1]
```

矩阵乘法收缩 Batch 轴：

```
dW2[h,o]
= Σ_b A1[b,h] × dY_hat[b,o]
```

每个样本对同一共享参数 `W2` 的梯度贡献要在 Batch 上求和。

数值：

```
dW2
= [[2.5,0.5],     [[ 0.05],
   [0.0,1.0]]  ×   [-0.15]]

= [[2.5×0.05 + 0.5×(-0.15)],
   [0.0×0.05 + 1.0×(-0.15)]]

= [[ 0.05],
   [-0.15]]
```

7.2 偏置梯度

`b2` 被 Broadcast 到每个样本，所以反向必须沿被 Broadcast 的 Batch 轴求和：

```
db2
= 0.05 + (-0.15)
= -0.10
```

Shape：

```
b2   [1]
db2  [1]
```

7.3 传回隐藏层的梯度

```
dA1 = dY_hat × W2^T
```

Shape：

```
dY_hat  [B,O] = [2,1]
W2^T     [O,H] = [1,2]
dA1      [B,H] = [2,2]
```

数值：

```
dA1
= [[ 0.05],     [[0.4, −0.6]]
   [−0.15]]  ×

= [[ 0.05×0.4,  0.05×(−0.6)],
   [−0.15×0.4, −0.15×(−0.6)]]

= [[ 0.02, −0.03],
   [−0.06,  0.09]]
```

八、反向第 3 步：ReLU

前向：

```
A1 = ReLU(Z1)
```

局部导数：

```
ReLU'(z) = 1, z>0
ReLU'(z) = 0, z<0
```

在 `z=0` 处数学上不可导，框架通常选择一个约定值，常见实现取 0。

本例：

```
Z1
= [[2.5, −0.5],
   [0.5,  1.0]]
```

Mask：

```
M = (Z1>0)
```

```
= [[1,0],
   [1,1]]
```

反向逐元素相乘：

```
dZ1 = dA1 ⊙ M
```

```
dZ1
= [[ 0.02, -0.03],     [[1,0],
   [-0.06,  0.09]]  ⊙   [1,1]]

= [[ 0.02, 0.00],
   [-0.06, 0.09]]
```

关键点：

```
dA1[0,1] = -0.03
```

但前向 `Z1[0,1] = -0.5` 被 ReLU 截断，因此：

```
dZ1[0,1] = 0
```

这就是 ReLU 的梯度门控。

九、反向第 4 步：第一个 Linear Layer

前向：

```
Z1 = XW1 + b1
```

Backward：

```
dW1 = X^T × dZ1
db1 = sum_batch(dZ1)
dX  = dZ1 × W1^T
```

9.1 计算 dW1

Shape：

```
X^T   [D,B] = [2,2]
dZ1    [B,H] = [2,2]
```

```
dW1    [D,H] = [2,2]
```

```
X^T
= [[1,-1],
   [2, 1]]
```

计算：

```
dW1
= [[1,-1],     [[ 0.02,0.00],
   [2, 1]]  ×   [-0.06,0.09]]
```

逐项：

```
dW1[0,0] = 1×0.02 + (-1)×(-0.06) = 0.08
dW1[0,1] = 1×0    + (-1)×0.09    = -0.09
dW1[1,0] = 2×0.02 + 1×(-0.06)    = -0.02
dW1[1,1] = 2×0    + 1×0.09       = 0.09
```

所以：

```
dW1
= [[ 0.08, -0.09],
   [-0.02,  0.09]]
```

9.2 计算 db1

沿 Batch 轴求和：

```
db1
= [0.02 + (-0.06), 0 + 0.09]
= [-0.04, 0.09]
```

9.3 计算 dX

如果 `X` 来自更前面的网络层，就必须继续传播 `dX`；若 `X` 是原始数据且不需要输入梯度，可以不保存或不返回它。

```
dX = dZ1 × W1^T
```

```
W1^T
= [[ 0.5, 1.0],
   [-1.0, 0.5]]
```

```
dX
= [[ 0.02,0.00],     [[ 0.5,1.0],
   [-0.06,0.09]]  ×   [-1.0,0.5]]
```

```
= [[ 0.01,  0.02],
   [-0.12, -0.015]]
```

Shape：

```
X   [2,2]
dX  [2,2]
```

十、全部梯度汇总

变量	Shape	数值
`dY_hat`	`[2,1]`	`[[0.05],[-0.15]]`
`dW2`	`[2,1]`	`[[0.05],[-0.15]]`
`db2`	`[1]`	`[-0.10]`
`dA1`	`[2,2]`	`[[0.02,-0.03],[-0.06,0.09]]`
`dZ1`	`[2,2]`	`[[0.02,0],[-0.06,0.09]]`
`dW1`	`[2,2]`	`[[0.08,-0.09],[-0.02,0.09]]`
`db1`	`[2]`	`[-0.04,0.09]`
`dX`	`[2,2]`	`[[0.01,0.02],[-0.12,-0.015]]`

Sanity Check：

```
梯度 Shape = 被求导变量 Shape

dW1.shape == W1.shape == [2,2]
db1.shape == b1.shape == [2]
dW2.shape == W2.shape == [2,1]
db2.shape == b2.shape == [1]
dX.shape  == X.shape  == [2,2]
```

十一、Optimizer 如何使用这些梯度

假设使用 SGD：

```
η = 0.1
```

```
θ_new = θ − η × dθ
```

更新 `W2`：

```
W2_old
= [[ 0.4],
   [−0.6]]

dW2
= [[ 0.05],
   [−0.15]]

W2_new
= W2_old − 0.1×dW2

= [[0.4−0.005],
   [−0.6−(−0.015)]]

= [[ 0.395],
   [−0.585]]
```

更新 `b2`：

```
b2_new
= 0.1 − 0.1×(−0.1)
= 0.11
```

更新 `W1`：

```
W1_new
= [[ 0.5,−1.0],
   [ 1.0, 0.5]]
− 0.1 ×
  [[ 0.08,−0.09],
   [−0.02, 0.09]]

= [[0.492,−0.991],
   [1.002, 0.491]]
```

更新 `b1`：

```
b1_new
= [0.0,−0.5] − 0.1×[−0.04,0.09]
= [0.004,−0.509]
```

若重新 Forward，新 Loss 应略微下降。反向传播只计算梯度；真正修改参数的是 Optimizer。

十二、为什么梯度在分叉处要累加

考虑：

```
y = x² + 3x
```

`x` 通过两条路径影响 `y`：

```
路径 1: x → x² → y
路径 2: x → 3x → y
```

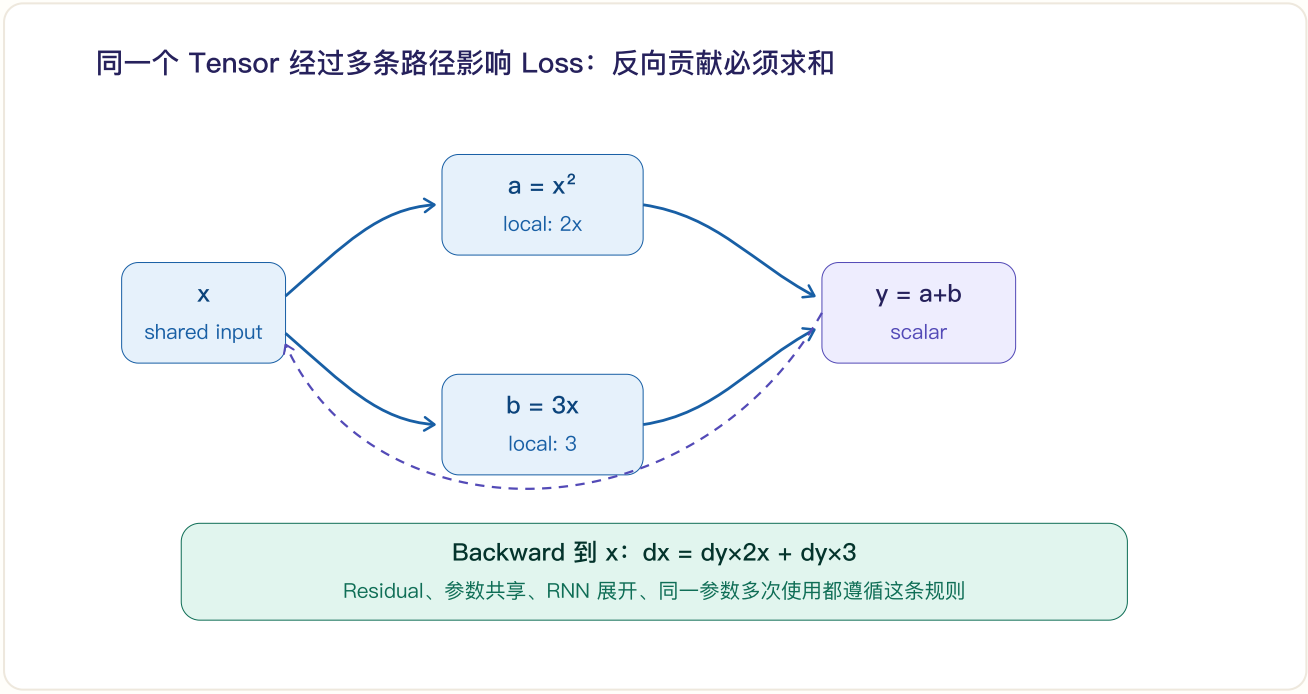
两条路径贡献：

```
路径 1: ∂y/∂x = 2x
路径 2: ∂y/∂x = 3
```

总梯度：

```
∂y/∂x = 2x + 3
```

不是二选一，而是相加。



神经网络中的典型例子：

- Residual：`y=x+F(x)`，`x` 同时经过 Identity 和 `F`；
- Weight Sharing：同一个参数在多个位置被使用；
- RNN：同一参数跨时间步共享；
- 一个 Tensor 同时供多个后续算子使用。

Autograd 会等所有下游贡献到达后，对同一叶子 Tensor 的 `.grad` 做累加。

十三、广播、Reduce 和反向 Shape

前向中的 Broadcast，反向通常对应 Reduce-Sum。

本例：

```
Z1 = XW1 + b1
```

`b1:[H]` 在前向被 Broadcast 到 `[B,H]`：

```
每个样本都加同一个 b1
```

反向：

```
db1[h] = Σ_b dZ1[b,h]
```

所以 Batch 轴消失。

反过来，前向 Reduce-Sum 的反向通常是 Broadcast：

```
s = Σ_i x_i
```

若上游梯度为 `ds`：

```
dx_i = ds
```

每个输入元素都收到同一个上游梯度。

理解 Broadcast/Reduce 的互逆关系，可以解释很多 Shape Bug。

十四、反向传播为何不显式构造完整 Jacobian

如果：

```
f: R^n → R^m
```

完整 Jacobian：

```
J_f: [m,n]
```

大型网络的 `m,n` 都可能极大，显式构造不可接受。

反向传播真正计算的是 Vector-Jacobian Product（VJP）：

```
v^T × J_f
```

其中 `v` 是从下游传来的上游梯度。

对标量 Loss：

```
L: R^m → R
```

我们只关心：

```
∂L/∂x
= (∂L/∂f) × (∂f/∂x)
```

而不需要单独得到 `∂f/∂x` 的全部矩阵元素。

这也是 Reverse-Mode Automatic Differentiation 特别适合神经网络训练的原因：

```
输入/参数很多
输出 Loss 通常只有一个标量
```

一次反向就能得到 Loss 对所有参数的梯度。

十五、Autograd 实际保存了什么

以 PyTorch 动态计算图为例：

```
前向执行算子
→ 产生 Tensor
→ 为需要梯度的结果记录 grad_fn
→ grad_fn 保存 backward 所需的上下文
```

不同算子需要保存不同内容：

算子	Backward 常需保存
Linear/MatMul	输入、权重或其必要视图

算子	Backward 常需保存
ReLU	输入符号或输出 Mask
Sigmoid	输出值，可由 `y(1−y)` 求导
Softmax	Softmax 输出
LayerNorm	输入、均值、逆标准差、Scale
Dropout	随机 Mask 或可复现 RNG 状态
Attention	Q/K/V、概率或重计算所需统计量，取决于实现

因此训练显存不仅有参数和梯度，还有 Saved Activations。

为什么 `torch.no_grad()` 会省显存

因为在 `no_grad` 下：

```
不建立反向图
不保存 backward 上下文
```

所以推理显存通常显著低于训练。

为什么执行完 Backward 后图通常被释放

默认 `loss.backward()` 后，Autograd 会释放许多中间 Saved Tensors。若要对同一图再次 Backward，需要：

```
retain_graph=True
```

但这会延长激活生命周期，显著增加显存，不应无目的使用。

十六、Gradient Accumulation 的两种含义

“梯度累加”容易混淆成两件事。

16.1 计算图分叉贡献累加

同一个 Tensor 通过多条路径影响 Loss：

```
dx = dx_path1 + dx_path2 + ...
```

这是链式法则本身要求的。

16.2 多个 Micro-batch 的 `.grad` 累加

PyTorch 默认不会在 Backward 前自动清空叶子参数的 `.grad`：

```
loss1.backward()
loss2.backward()
```

执行后：

```
param.grad
= grad_from_loss1 + grad_from_loss2
```

这使得显存不足时可以把一个大 Batch 拆成多个 Micro-batches。

若希望等价于大 Batch 的平均 Loss，通常每个 Micro-batch Loss 要除以累加步数：

```
loss = loss / accumulation_steps
loss.backward()
```

否则梯度总量会扩大 `accumulation_steps` 倍，相当于改变有效学习率。

为什么每个 Optimizer Step 后要 Zero Grad

```
optimizer.zero_grad(set_to_none=True)
```

如果忘记，下一训练 Step 会继续累加旧梯度。

`set_to_none=True` 与写零的区别：

- `None` 可以避免一遍清零内存写；
- 下一次 Backward 直接分配/写入梯度；
- 某些检查代码需要处理 `.grad is None`。

十七、用 PyTorch 验证全部手算

```
import torch

torch.set_printoptions(precision=6, sci_mode=False)
```

```
X = torch.tensor(
    [[ 1.0, 2.0],
     [-1.0, 1.0]],
    requires_grad=True,
)

W1 = torch.tensor(
    [[ 0.5, -1.0],
     [ 1.0,  0.5]],
    requires_grad=True,
)

b1 = torch.tensor(
    [0.0, -0.5],
    requires_grad=True,
)

W2 = torch.tensor(
    [[ 0.4],
     [-0.6]],
    requires_grad=True,
)

b2 = torch.tensor(
    [0.1],
    requires_grad=True,
)

Y = torch.tensor(
    [[1.0],
     [0.0]],
)

# Forward
Z1 = X @ W1 + b1
A1 = torch.relu(Z1)
Y_hat = A1 @ W2 + b2
B = X.shape[0]
loss = ((Y_hat - Y) ** 2).sum() / (2 * B)

# 保留非叶子 Tensor 的梯度，仅用于教学观察
Z1.retain_grad()
A1.retain_grad()
Y_hat.retain_grad()

# Backward
loss.backward()

print("Z1 =\n", Z1)
print("A1 =\n", A1)
print("Y_hat =\n", Y_hat)
print("loss =", loss.item())

print("dY_hat =\n", Y_hat.grad)
print("dW2 =\n", W2.grad)
print("db2 =\n", b2.grad)
print("dA1 =\n", A1.grad)
print("dZ1 =\n", Z1.grad)
print("dW1 =\n", W1.grad)
print("db1 =\n", b1.grad)
print("dX =\n", X.grad)
```

期望结果：

```
loss = 0.025

dY_hat = [[ 0.05],
          [-0.15]]

dW2 = [[ 0.05],
```

```
        [-0.15]]

db2 = [-0.10]

dA1 = [[ 0.02, -0.03],
       [-0.06,  0.09]]

dZ1 = [[ 0.02, 0.00],
       [-0.06, 0.09]]

dW1 = [[ 0.08, -0.09],
       [-0.02,  0.09]]

db1 = [-0.04, 0.09]

dX = [[ 0.01,  0.02],
      [-0.12, -0.015]]
```

为什么普通非叶子 Tensor 默认没有 `.grad`

在 PyTorch 中：

- 叶子参数的 `.grad` 默认保留，Optimizer 需要读取；
- 中间 Tensor 的梯度通常只在反向过程中临时流过，不默认保存在 `.grad`；
- 教学或调试时可调用 `retain_grad()`。

这能避免为所有中间 Tensor 长期保存梯度。

十八、用有限差分检查梯度

解析梯度可以通过数值梯度验证。

对参数 `w`：

```
∂L/∂w
≈ [L(w+ε) - L(w-ε)] ÷ (2ε)
```

这是中心差分。

例如检查 `W1[0,0]`：

```
with torch.no_grad():
    eps = 1e-4
    original = W1[0, 0].item()

    W1[0, 0] = original + eps
    loss_plus = ((((torch.relu(X @ W1 + b1) @ W2 + b2) - Y) ** 2).sum() / (2 * B))

    W1[0, 0] = original - eps
    loss_minus = ((((torch.relu(X @ W1 + b1) @ W2 + b2) - Y) ** 2).sum() / (2 * B))
```

```
    W1[0, 0] = original

numerical = (loss_plus - loss_minus) / (2 * eps)
print("autograd:", W1.grad[0, 0].item())  # 约 0.08
print("numerical:", numerical.item())     # 应接近 0.08
```

有限差分的边界：

- ε 太大：截断误差大；
- ε 太小：浮点舍入误差大；
- ReLU 在 0 附近不可导，扰动可能跨过分段边界；
- 大模型不可能逐参数检查，只适合小模块和随机抽样；
- FP16/BF16 不适合严格 Grad Check，通常使用 FP64 小规模验证。

十九、Mixed Precision 下反向传播多了什么

BF16/FP16 训练不改变链式法则，但会改变数值表示和状态精度。

FP16 Loss Scaling

FP16 的小梯度可能下溢到 0，因此先放大 Loss：

```
L_scaled = S × L
```

Backward 得到：

```
g_scaled = S × g
```

Optimizer Step 前再 Unscale：

```
g = g_scaled ÷ S
```

然后检查 Inf/NaN、执行 Gradient Clipping 和参数更新。

顺序必须正确：

```
Backward scaled loss
→ Unscale gradients
→ Inf/NaN check
→ Gradient clipping
→ Optimizer step
```

若在 Unscale 前 Clip，裁剪的是放大后的梯度，阈值语义会错误。

BF16

BF16 指数范围接近 FP32，通常不需要动态 Loss Scaling，但尾数精度较低。具体算子仍可能使用 FP32 Accumulation。

二十、分布式训练中的反向传播

DDP

每个 Rank 对自己的 Local Batch 做 Forward/Backward，得到本地参数梯度：

```
g_r = ∂L_r/∂θ
```

DDP 对参数梯度执行 AllReduce/Average：

```
g_global
= (1/DP) × Σ_r g_r
```

然后每个 Rank 使用相同的全局梯度更新相同参数。

为了通信计算重叠，Autograd 在某个 Gradient Bucket 准备完成后就可启动异步 AllReduce，同时继续计算更前面层的 Backward。

FSDP / ZeRO

参数、梯度或优化器状态被分片：

- Forward/Backward 前按需 AllGather 参数；
- Backward 计算局部梯度；
- 用 ReduceScatter 聚合并分片梯度；
- 每 Rank 只保留或更新自己负责的状态分片。

链式法则没有变化，改变的是 Tensor 布局、通信时机和生命周期。

Pipeline Parallel

每个 Stage 只保存自己层的激活和参数。Backward 时：

```
后一个 Stage 计算本 Stage 的输入梯度
→ P2P Send 到前一个 Stage
→ 前一个 Stage 把它作为上游梯度继续 Backward
```

跨 Stage 发送的 Activation Gradient，本质上就是链式法则中的上游梯度。

二十一、Activation Checkpointing 为什么能省显存

普通训练：

```
Forward 保存中间激活
Backward 直接读取激活计算梯度
```

Activation Checkpointing：

```
Forward 只保存少量边界激活
Backward 到该区间时重新执行一次 Forward
再利用重算出的中间值做 Backward
```

它做的交换：

```
更少激活显存
↔ 更多重计算 FLOP 和时间
```

它不改变理论梯度，但要注意：

- Dropout/RNG 状态必须正确恢复；
- 有状态算子和副作用需要特殊处理；
- 重计算区间选择影响显存收益和额外计算；
- Mixed Precision 与算子非确定性可能导致微小数值差异。

二十二、反向传播的性能瓶颈

Backward 往往比 Forward 更复杂，因为一个算子通常需要计算：

```
输入梯度 dX
权重梯度 dW
偏置梯度 db
```

以 Linear 为例：

```
Forward:  Y = XW
Backward: dX = dY W^T
          dW = X^T dY
```

Forward 一个主 GEMM，Backward 通常至少两个主 GEMM，所以训练计算成本显著高于只做 Forward 的推理。

主要瓶颈包括：

- GEMM/Attention Backward 计算；
- Saved Activation 读写 HBM；
- Gradient Tensor 写回；
- DDP AllReduce 或 FSDP ReduceScatter；
- Activation Checkpoint 重算；
- 小算子和 Kernel Launch；
- Gradient Accumulation 的额外迭代；
- Optimizer State 的 HBM 访问。

优化时需要区分：

```
Backward Compute Time
Gradient Communication Time
Exposed Communication Time
Activation Recompute Time
Optimizer Step Time
```

二十三、常见失败模式与误区

误区 1：反向传播是在“把 Loss 传回去”

更准确地说，传回去的是：

```
Loss 对各个中间变量的梯度
```

不是 Loss 数值本身。

误区 2：每层直接算 Loss 对参数的导数

实现上每层只需要：

```
上游梯度 × 局部导数
```

不需要知道整个下游网络细节。

误区 3：矩阵转置只是记忆公式

转置来自 Shape 和收缩轴：

```
Y = XW

dW = X^T dY
dX = dY W^T
```

先写 Shape，再推公式，比背公式可靠。

误区 4：一个 Tensor 只能收到一份梯度

如果有多条下游路径，梯度必须相加。Residual 和参数共享都依赖这一点。

误区 5：`loss.backward()` 会自动清空旧梯度

不会。叶子参数 `.grad` 默认累加，必须由训练循环显式清理。

误区 6：梯度正确就一定训练稳定

即使 Backward 数学正确，训练仍可能因以下因素失败：

- Learning Rate 过大；
- Loss Scaling 错误；
- Gradient Explosion；
- 数据或 Label 错误；
- 分布式 Collective 不一致；
- 低精度溢出；
- Optimizer 状态损坏；
- 错误的 Mask、Reduction 或 Loss 归一化。

误区 7：Backward 必须保存所有前向结果

不一定。可以只保存必要统计量，或使用 Activation Checkpointing 重算。

二十四、从头到尾再串一次

使用本例，完整数据流为：

```
X [2,2]
  ↓ Linear 1: XW1+b1
Z1 [[2.5,-0.5],[0.5,1.0]]
  ↓ ReLU
A1 [[2.5,0],[0.5,1.0]]
  ↓ Linear 2: A1W2+b2
Y_hat [[1.1],[-0.3]]
  ↓ MSE with Y
L = 0.025
```

从标量种子开始：

```
dL/dL = 1
```

反向：

```
MSE backward
→ dY_hat = [[0.05],[-0.15]]

Linear 2 backward
→ dW2 = [[0.05],[-0.15]]
→ db2 = [-0.10]
→ dA1 = [[0.02,-0.03],[-0.06,0.09]]

ReLU backward
→ Mask = [[1,0],[1,1]]
→ dZ1 = [[0.02,0],[-0.06,0.09]]

Linear 1 backward
→ dW1 = [[0.08,-0.09],[-0.02,0.09]]
→ db1 = [-0.04,0.09]
→ dX = [[0.01,0.02],[-0.12,-0.015]]
```

Optimizer：

```
读取 W1.grad、b1.grad、W2.grad、b2.grad
→ 更新参数
→ 清理 grad
→ 下一 Batch 使用新参数重新 Forward
```

这就是一次完整反向传播。

可选自测

1. Shape 题

若：

```
X:[B,D]
W:[D,H]
Y=XW:[B,H]
dY:[B,H]
```

请只根据 Shape 推出 `dW` 和 `dX` 的矩阵乘法表达式。

2. 梯度累加题

对于：

```
y = x² + 3x
L = y²
```

当 `x=2` 时，手算 `dL/dx`。注意 `x` 有两条路径到达 `y`。

3. 工程题

为什么 DDP 可以在最后几层的梯度计算完成后立即启动 AllReduce，同时 GPU 继续计算前面层的 Backward？它依赖什么数据依赖条件？

继续学习

1. 前置：矩阵微积分与 VJP/JVP —— 从 Shape 出发严格推导 Linear、Softmax 和 LayerNorm Backward。
2. 同层对比：手写 Autograd Engine —— 用拓扑排序和局部 backward 函数实现一个最小自动微分框架。
3. 下游应用：Transformer 完整 Backward —— 逐项讲解 Attention、Softmax、RMSNorm、SwiGLU 的梯度与显存。

回复 1、2 或 3 继续；也可以说“稍后”或“不感兴趣”。

事实来源与版本边界

本讲解使用稳定的微积分、矩阵微积分和 Reverse-Mode Automatic Differentiation 原理。PyTorch 示例遵循常见 eager-mode Autograd 语义；具体 `grad_fn`、Saved Tensor、Fused Backward、AOTAutograd、`torch.compile`、DDP Bucket 和 FSDP Hook 行为会随框架版本与编译路径变化。

生产环境中应以目标 PyTorch/CUDA/通信栈版本的源码、Profiler Timeline、数值对照和 Grad Check 为准。手算例子采用 FP32 小矩阵，旨在解释机制；它不代表大模型低精度训练中的舍入顺序、融合算子实现或分布式 Reduction 顺序。

大模型训练 LM Head 全解：前向、反向、FLOPs、显存与优化

> **原始问题：**详细讲解大模型训练中lm head这部分，包括前反向的每一步。举一个简单例子说明清楚，给出每一步的shape，计算公式，flops，作用和显存开销等大模型训练关心的数据。配合图片辅助我进行理解。我当前在做一个lm head显存优化的工作，讲完上述内容后再重点关注这部分变化，帮我画出一个动态显存开销图，在网页上清晰地展现出来每一步的计算步骤，计算量和显存开销变化。

提问：2026/08/27 10:35　更新：2026/08/27 10:51　重要度：5/5

大模型训练 LM Head 全解：前向、反向、FLOPs、显存与优化

一句话结论

LM Head（Language Modeling Head）是把 Transformer 最后一层每个 Token 的隐藏向量 `[H]` 投影成全词表分数 `[V]` 的输出层。 训练时它通常紧接稳定的 Softmax Cross Entropy：

```
Final hidden states H_final [B,S,H]
→ reshape 为 X [N,H]
→ Vocabulary projection: Z = XW_vocab^T
→ Logits Z [N,V]
→ Cross Entropy with target token ids [N]
→ scalar Loss
```

其中：

```
B  Micro Batch Size
S  Sequence Length
N  参与 Loss 的有效 Token 数, N ≤ B×S
H  Hidden Size
V  Vocabulary Size
W_vocab [V,H]
```

LM Head 训练最关键的系统事实是：

1. **计算量大**：前向矩阵乘约 `2NHV FLOPs`；连同 `dH` 和 `dW` 两个反向 GEMM，标准训练主 GEMM 总量约 `6NHV FLOPs`。
2. **Logits 显存可能巨大**：完整 Logits 有 `N×V` 个元素，和 Hidden Size 无关；长序列、大 Batch、大词表时，它可能比最后层 Hidden 大几十倍。
3. **Cross Entropy 并不需要长期保存完整 Logits**：每个 Token 的 Loss 只需要目标类别 Logit 和所有类别的 `logsumexp`；通过 Vocabulary Parallel、在线 Softmax、Chunk、Fusion 和重计算，可以避免完整 `[N,V]` Tensor 长时间驻留。
4. **显存优化不是免费午餐**：分块减小峰值，却可能增加重计算、Kernel Launch、通信、Buffer 管理和尾部低利用率。
5. **你当前做 LM Head 显存优化时，应优先定位峰值到底来自哪一份大 Tensor**：Logits、保存的 Softmax/Log-probs、`dLogits`、Gather 后的全词表 Logits、还是框架额外副本。

动态分析网页：

```
https://psgmessi.github.io/ai-infra-learning/lm-head-memory-explorer.html
```

网页支持动态调整 `B/S/H/V/dtype/TP/Token Chunk/Vocab Chunk`，逐阶段查看 Shape、FLOPs、显存峰值和实现方案之间的变化。

一、LM Head 在大模型中的位置

以 Decoder-only Transformer 为例：

```
Token IDs [B,S]
→ Token Embedding [B,S,H]
→ L 个 Transformer Blocks
→ Final RMSNorm / LayerNorm
→ H_final [B,S,H]
→ LM Head
→ Logits [B,S,V]
→ Shifted Cross Entropy
→ Loss scalar
```

LM Head 本身通常就是一个无 Bias 的 Linear：

```
logits[b,s,v]
= Σ_h H_final[b,s,h] × W_vocab[v,h]
```

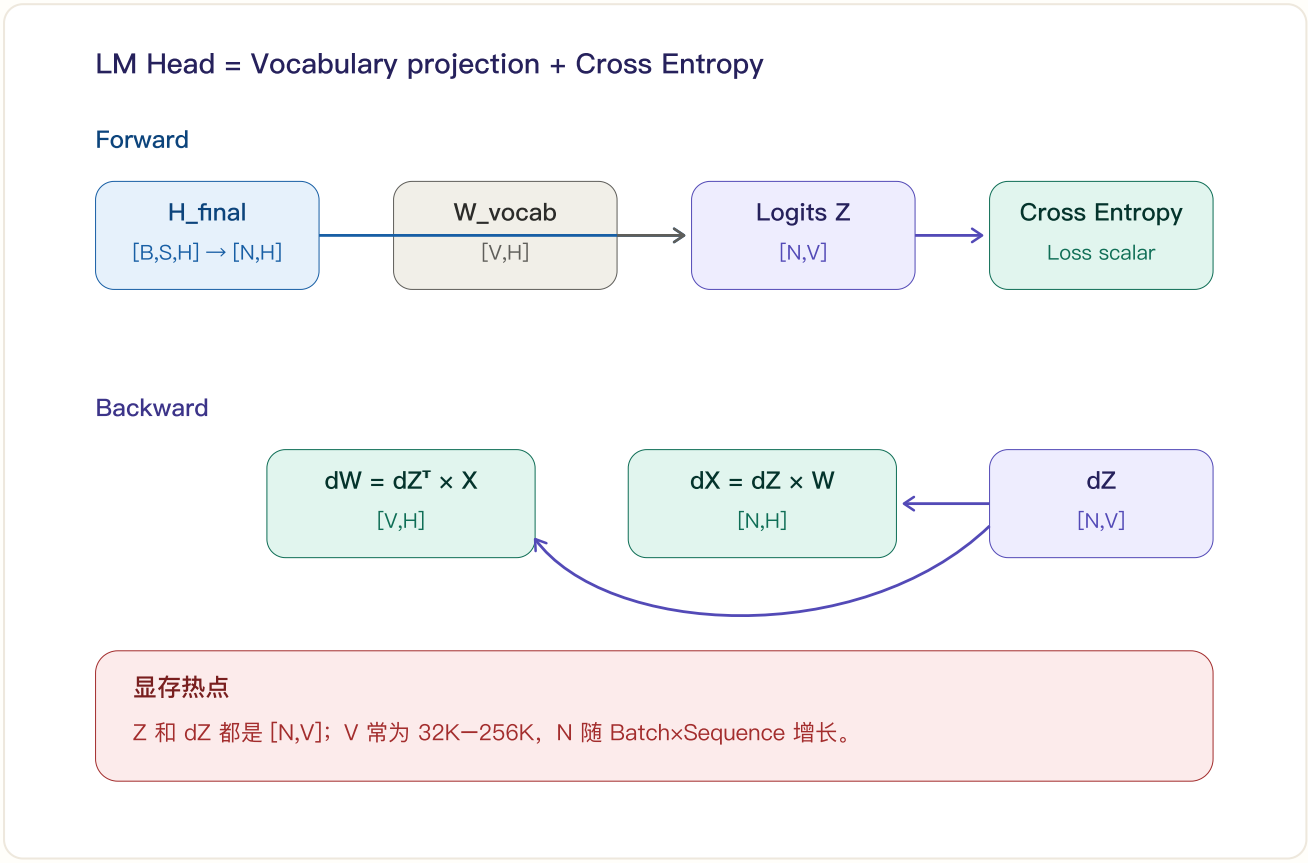
矩阵写法：

```
X = reshape(H_final, [N,H])
```

```
Z = X × W_vocab^T
```

Shape：

```
X          [N,H]
W_vocab    [V,H]
W_vocab^T  [H,V]
Z          [N,V]
```



LM Head 的作用

LM Head 把每个位置的上下文表示转换成：

```
对词表中每个 Token 的未归一化分数
```

注意它输出的是 Logits，不是概率：

```
Z[n,v] ∈ (−∞,+∞)
```

概率由 Softmax 得到：

```
P[n,v] = exp(Z[n,v]) ÷ Σ_j exp(Z[n,j])
```

训练目标是让目标 Token 的概率增加，非目标 Token 的总概率降低。

二、Causal LM 的 Label Shift 与有效 Token 数

Decoder-only Causal LM 使用当前位置的 Hidden State 预测下一个 Token。

假设 Token IDs：

```
[t0, t1, t2, t3]
```

训练配对：

```
位置 0 的 Hidden → 预测 t1
位置 1 的 Hidden → 预测 t2
位置 2 的 Hidden → 预测 t3
位置 3 没有下一个 Token，通常忽略
```

因此代码常见：

```
shift_logits = logits[:, :-1, :]
shift_labels = labels[:, 1:]
```

实际有效 Token 数不是机械的 `B×S`：

```
N_valid
= 非 Padding
- Ignore Index
- Sequence 边界不参与预测的位置
- 被 Loss Mask 排除的位置
```

性能估算必须明确使用：

```
N = 有效或实际进入 LM Head/CE 的 Token 数
```

有些实现先对全部 `B×S` 位置做 LM Head，再在 Loss 里 Mask；这种情况下投影计算量仍按 `B×S`，即使有效 Loss Token 更少。若能在投影前 Compact 有效 Token，才会真正减少 `N×V` 计算与显存，但会引入 Gather/Scatter、动态 Shape 和数据布局成本。

三、Weight Tying：LM Head 和 Embedding 是否共用参数

很多语言模型使用权重共享：

```
Input Embedding Weight E [V,H]
LM Head Weight W_vocab [V,H]

W_vocab 与 E 指向同一参数
```

优点：

- 少存一份 `[V,H]` 参数；
- 少存一份参数梯度和优化器状态；
- 输入和输出 Token 表示共享几何空间。

但反向时同一个参数有两条梯度来源：

```
dW_total
= dW_from_lm_head
+ dW_from_input_embedding
```

这不是覆盖，而是累加。

对于输入 Embedding，只有本 Batch 出现过的 Token 行有直接 Embedding Lookup 梯度；LM Head 的 `dW_vocab` 通常是 Dense 的，因为每个词表类别都参与 Softmax 分母。

如果模型使用 Untied LM Head，则参数状态额外增加：

```
参数：V×H×param_bytes
梯度：V×H×grad_bytes
Adam m/v：V×H×8 Byte（FP32）
FP32 Master Weight：可再加 V×H×4 Byte
```

四、统一小例子：2 个 Token、Hidden=3、Vocab=4

设置：

```
N = 2
H = 3
V = 4
```

最后层 Hidden：

```
X [N,H]
= [[1.0, 0.0, -1.0],
   [0.5, 1.0,  0.0]]
```

LM Head 权重：

```
W [V,H]
= [[1.0,  0.0, 0.0],    # Token 0
   [0.0,  1.0, 0.0],    # Token 1
   [0.0,  0.0, 1.0],    # Token 2
   [1.0, -1.0, 0.5]]    # Token 3
```

目标 Token：

```
y [N] = [0,2]
```

含义：

- 第一个位置应该预测 Token 0；
- 第二个位置应该预测 Token 2。

五、LM Head 前向第 1 步：Vocabulary Projection

公式：

```
Z = XW^T
```

Shape：

```
X    [2,3]
W^T  [3,4]
Z    [2,4]
```

第一个 Token Hidden：

```
x0 = [1,0,-1]
```

与四个词表向量做点积：

```
z0,0 = [1,0,-1]·[1,0,0]      = 1
z0,1 = [1,0,-1]·[0,1,0]      = 0
```

```
z0,2 = [1,0,-1]·[0,0,1]      = -1
z0,3 = [1,0,-1]·[1,-1,0.5]   = 0.5
```

第二个 Token：

```
x1 = [0.5,1,0]
```

```
z1,0 = 0.5
z1,1 = 1.0
z1,2 = 0.0
z1,3 = 0.5-1 = -0.5
```

所以：

```
Z [2,4]
= [[1.0, 0.0, -1.0,  0.5],
   [0.5, 1.0,  0.0, -0.5]]
```

Projection FLOPs

一个长度为 `H` 的 Dot Product：

```
H 次乘法 + H-1 次加法
≈ 2H FLOPs
```

共有 `N×V` 个 Dot Products：

```
FLOPs_forward_projection
≈ 2NHV
```

本例：

```
2×2×3×4 = 48 FLOPs
```

这里遵循常见口径：一次乘法和一次加法分别计 1 FLOP；Tensor Core FMA 常按 2 FLOPs 计。

六、LM Head 前向第 2 步：稳定 Softmax Cross Entropy

不能直接计算：

```
exp(z_v) ÷ Σ_j exp(z_j)
```

因为大 Logit 的 `exp` 可能溢出。稳定算法先减最大值。

对每个 Token `n`：

```
m_n = max_v Z[n,v]

s_n = Σ_v exp(Z[n,v]-m_n)

logsumexp_n = m_n + log(s_n)

loss_n = logsumexp_n - Z[n,y_n]
```

这与直接 Softmax 完全等价，但数值更稳定。

第一个 Token

```
z0 = [1,0,-1,0.5]
m0 = 1
```

减最大值：

```
[0,-1,-2,-0.5]
```

指数：

```
[1,
 exp(-1),
 exp(-2),
 exp(-0.5)]

≈ [1,0.367879,0.135335,0.606531]
```

求和：

```
s0 ≈ 2.109745
```

概率：

```
P0
≈ [0.473991,
   0.174371,
   0.064148,
   0.287490]
```

目标是 Token 0：

```
loss0 = -log(P0[0])
      ≈ 0.746567
```

第二个 Token

```
z1 = [0.5,1,0,-0.5]
m1 = 1
```

概率：

```
P1
≈ [0.276004,
   0.455054,
   0.167405,
   0.101536]
```

目标为 Token 2：

```
loss1 = -log(P1[2])
      ≈ 1.787339
```

Mean Loss：

```
L = (loss0+loss1) ÷ 2
  ≈ 1.266953
```

Cross Entropy 计算量

每个 Logit 大致经历：

```
比较 max
减法
exp
sum
归一化或 logsumexp
```

可用线性量级表示：

```
FLOPs_CE ≈ O(NV)
```

若把比较、减法、加法等粗略按 4–6 次标量操作计：

```
≈ 4NV 到 6NV 标量操作 + N 次 log
```

对于大 `H`，`2NHV` 的投影 GEMM 通常在 FLOPs 上远高于 `O(NV)`；但 Cross Entropy 的 Exp、Reduce 和大 Tensor HBM 读写仍可能成为显著性能与显存瓶颈。

七、Cross Entropy 反向：为什么 dLogits = P − OneHot

对单个 Token：

```
loss = log(Σ_j exp(z_j)) − z_y
```

对任意类别 `k`：

```
∂loss/∂z_k
= exp(z_k) ÷ Σ_j exp(z_j) − 1[k=y]

= P_k − 1[k=y]
```

若 Loss 对 `N` 个 Token 取 Mean：

```
dZ[n,v]
= (P[n,v] − 1[v=y_n]) ÷ N
```

本例 `N=2`。

第一个 Token，目标为 0

```
P0 − one_hot(0)
= [0.473991−1,
   0.174371,
   0.064148,
   0.287490]

= [−0.526009,
    0.174371,
    0.064148,
    0.287490]
```

除以 2：

```
dZ0
≈ [−0.263005,
    0.087186,
    0.032074,
    0.143745]
```

第二个 Token，目标为 2

```
dZ1
≈ [ 0.138002,
    0.227527,
   −0.416297,
    0.050768]
```

最终：

```
dZ [N,V]
≈ [[-0.263005, 0.087186, 0.032074, 0.143745],
   [ 0.138002, 0.227527,-0.416297, 0.050768]]
```

解释：

- 目标类别梯度为负，梯度下降会提高目标 Logit；
- 非目标类别梯度为正，梯度下降会降低这些 Logits；
- 每行梯度之和接近 0，因为 Softmax 对所有 Logits 同时加常数不敏感。

检查：

```
Σ_v dZ[n,v] ≈ 0
```

八、LM Head Linear Backward：dHidden 和 dWeight

前向：

```
Z = XW^T
```

收到：

```
dZ [N,V]
```

8.1 对隐藏状态的梯度

```
dX = dZ × W
```

Shape：

```
dZ [N,V]
W  [V,H]
dX [N,H]
```

本例：

```
dX
≈ [[-0.119260,-0.056559, 0.103946],
```

```
   [ 0.188770, 0.176759,-0.390913]]
```

这个 `dX` 会继续传给 Final Norm 和最后一个 Transformer Block。

FLOPs：

```
FLOPs_dX ≈ 2NVH
```

8.2 对 LM Head 权重的梯度

```
dW = dZ^T × X
```

Shape：

```
dZ^T [V,N]
X    [N,H]
dW   [V,H]
```

本例：

```
dW
≈ [[-0.194003, 0.138002, 0.263005],
   [ 0.200949, 0.227527,-0.087186],
   [-0.176075,-0.416297,-0.032074],
   [ 0.169129, 0.050768,-0.143745]]
```

FLOPs：

```
FLOPs_dW ≈ 2NVH
```

8.3 LM Head 训练总主 GEMM FLOPs

```
Forward Projection ≈ 2NHV
dHidden            ≈ 2NHV
dWeight            ≈ 2NHV
```

合计：

```
FLOPs_LMHead_GEMM_train
≈ 6NHV
```

再加：

```
Softmax / Cross Entropy ≈ O(NV)
```

如果使用不保存完整 Logits、Backward 重算 Projection 的 Fused Linear CE，则通常还要增加一次近似：

```
Recompute Logits ≈ 2NHV
```

保守主 GEMM 总量变成：

```
≈ 8NHV
```

相对标准 `6NHV`，理论上增加约：

```
(8−6) ÷ 6 = 33.3%
```

注意这是 **LM Head 子模块内部** 的增量，不代表整个模型 Step Time 增加 33%。如果 LM Head 只占全模型计算的一小部分，端到端增量会更小；如果长序列、大词表让 LM Head 占比很高，影响会更明显。

九、真实规模下的 FLOPs 与显存

使用一个可复算配置：

```
B = 2
S = 8192
N = B×S = 16384
H = 4096
V = 128256
Logits dtype = BF16 = 2 Byte
```

9.1 完整 Logits

元素数：

```
N×V
= 16384×128256
= 2,101,346,304 elements
```

显存：

```
2,101,346,304 × 2 Byte
= 4,202,692,608 Byte
≈ 3.914 GiB
```

一份 BF16 Logits 就接近 `3.914 GiB`。

若 Backward 峰值同时存在一份同 Shape 的 `dLogits`：

```
Logits + dLogits
≈ 7.828 GiB
```

这还没有包括：

- Softmax/LogSoftmax 保存结果；
- Cross Entropy Workspace；
- Hidden States；
- LM Head Weight 和 `dW`；
- cuBLAS/Triton Workspace；
- CUDA Allocator 碎片；
- TP Gather Buffer；
- 模型其他层。

具体实现可能原地复用 Logits Buffer，因此“同时两份”是一个显式 Tensor 上界模型，而不是所有 Kernel 的固定事实。

9.2 Hidden States

```
N×H×2 Byte
= 16384×4096×2
= 134,217,728 Byte
= 128 MiB
```

对比：

```
Logits / Hidden
= V / H
= 128256 / 4096
≈ 31.31
```

因此同 dtype 下，完整 Logits 是最后层 Hidden 的约 `31.3 倍`。

9.3 LM Head Weight

```
V×H×2 Byte
= 128256×4096×2
≈ 0.979 GiB
```

`dW` 若为 BF16 同样约 `0.979 GiB`；若梯度或参数副本使用 FP32，则翻倍。

9.4 FLOPs

Forward Projection：

```
2NHV
= 2×16384×4096×128256
≈ 17.21 TFLOPs
```

标准训练三个 GEMM：

```
6NHV
≈ 51.64 TFLOPs
```

这是每个 Micro-batch 的算法 FLOPs，不是 FLOP/s，也没有包含 Final Norm、CE 标量操作或通信。

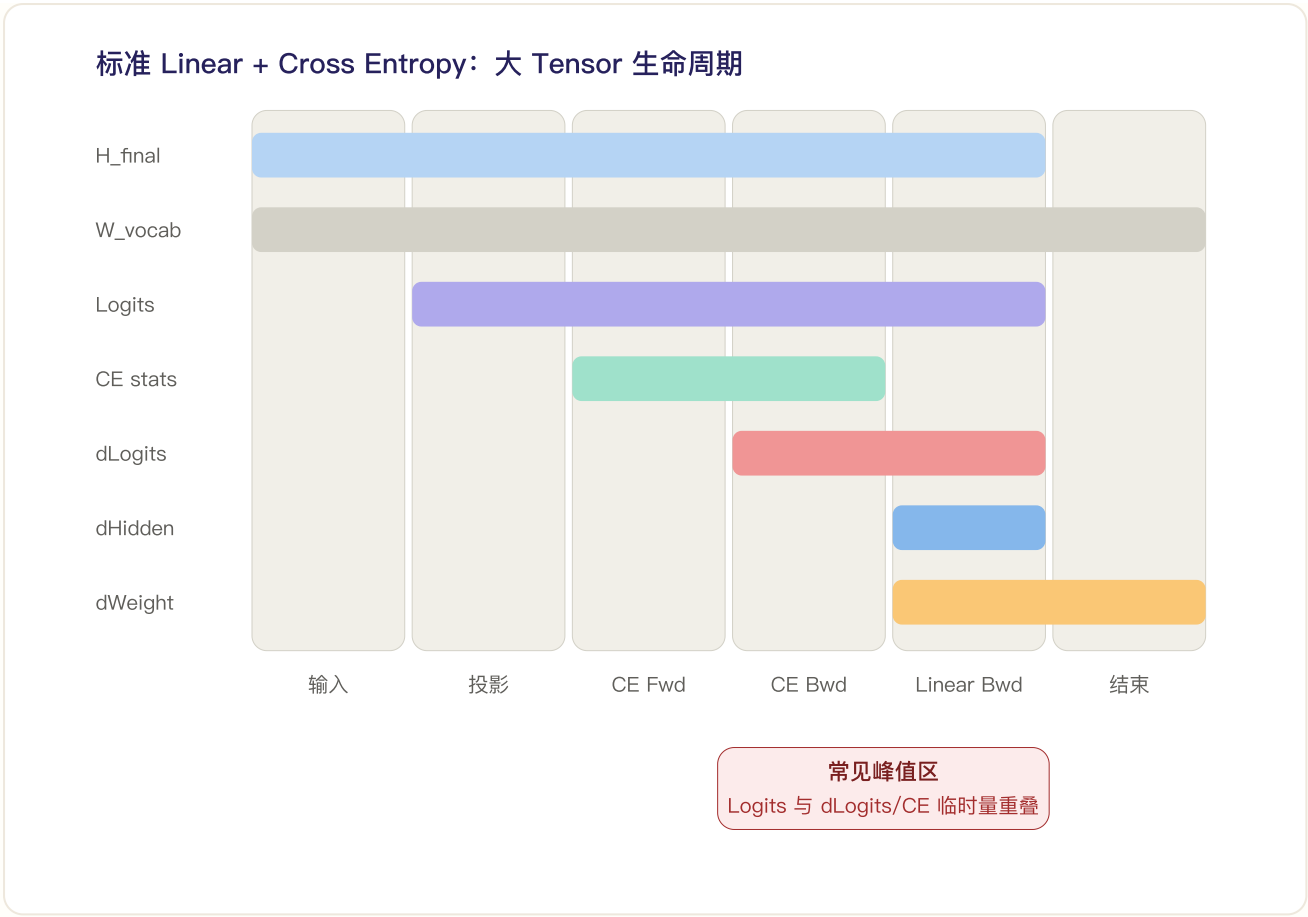
十、标准实现的动态显存时间线

标准路径：

```
logits = hidden @ weight.T
loss = cross_entropy(logits, labels)
loss.backward()
```

一个保守的生命周期模型：

```
阶段 1: H_final 和 W 已存在
阶段 2: 生成完整 Logits [N,V]
阶段 3: CE 计算/保存 Softmax 所需信息
阶段 4: 产生 dLogits [N,V]
阶段 5: 用 dLogits 计算 dH 和 dW
阶段 6: 释放 Logits/dLogits，保留参数梯度
```



为什么单看 `torch.cuda.memory_allocated()` 可能误判

需要区分：

```
allocated: Tensor 实际占用
reserved: CUDA Caching Allocator 向驱动保留
peak allocated: 历史最大 Tensor 占用
workspace: 算子临时空间
fragmentation: 空闲块无法满足当前分配
```

LM Head 优化后，`allocated` 峰值可能下降，但 `reserved` 不一定立即下降；反过来，Kernel 内部 Workspace 可能让实际峰值高于手算 Tensor 总和。

十一、显存优化路线 1：降低 Logits 精度

若 Logits 从 FP32 改为 BF16：

```
[N,V] × 4 Byte
→ [N,V] × 2 Byte
```

显存减半。

但稳定 Softmax 通常仍建议：

```
输入 Logits：BF16/FP16
max / sum-exp accumulation：FP32
Loss：FP32
```

风险：

- FP16 指数范围小；
- 大词表下极小概率更容易下溢；
- Label Smoothing、Logit Soft-Capping 等会改变数值路径；
- FP8 Logits 需要专门的缩放和误差验证，不能只按 1 Byte 容量推断可用性。

这是低风险的容量优化，但无法消除 `N×V` 规模本身。

十二、显存优化路线 2：Vocabulary Parallel LM Head

令 Tensor Parallel Degree 为 `P`，沿 Vocabulary 维切分：

```
W_vocab [V,H]
→ 每 Rank W_r [V/P,H]
```

每 Rank 只产生：

```
Z_r [N,V/P]
```

参数、`dW` 和 Local Logits 约缩小到 `1/P`。

12.1 Distributed Stable Softmax

每 Rank 先算 Local Max：

```
m_r[n] = max_{v in local shard} Z_r[n,v]
```

跨 TP 做 MAX AllReduce：

```
m[n] = max_r m_r[n]
```

各 Rank 计算：

```
s_r[n]
= Σ_{v in local shard} exp(Z_r[n,v]-m[n])
```

跨 TP 做 SUM AllReduce：

```
s[n] = Σ_r s_r[n]
```

只有拥有目标 Token 的 Rank 取出 Target Logit；其他 Rank 填 0：

```
t_r[n] = local target logit or 0
```

SUM AllReduce 得到全局 Target Logit：

```
t[n] = Σ_r t_r[n]
```

Loss：

```
loss[n]
= m[n] + log(s[n]) - t[n]
```

Megatron Core 的 `VocabParallelCrossEntropy` 就采用这类路径：全局 Max、Target Logit SUM、Sum-Exp SUM，并把 Local Exp Logits 原地归一化为 Local Softmax 保存给 Backward。

12.2 TP Backward

每 Rank 只生成 Local `dZ_r [N,V/P]`：

```
dZ_r = P_r - local_one_hot
```

Local Weight Gradient：

```
dW_r = dZ_r^T X
```

Local Hidden Gradient Contribution：

```
dX_r = dZ_r W_r
```

全局 `dX`：

```
dX = Σ_r dX_r
```

因此通常需要 TP SUM AllReduce，或与 Sequence Parallel 组合成 ReduceScatter。

12.3 通信量

Forward 的统计通信主要是长度 `N` 的 FP32 向量：

```
Global Max
Global Sum-Exp
Target Logit
```

量级约：

```
3 × N × 4 Byte
```

Backward 的 `dX` 通信量级：

```
N × H × grad_bytes
```

通常比 Forward 标量统计大得多。

12.4 适用条件

适合：

- 已使用 Tensor Parallel；
- `V` 很大；
- LM Head Weight 本身也需要分片；
- 网络拓扑足够快。

代价：

- 多次 TP Collective；
- 最慢 Rank 决定同步完成时间；
- 若错误地 AllGather 全词表 Logits，会重新引入 `[N,V]` 峰值，抵消分片收益。

十三、显存优化路线 3：Token Chunk

把 `N` 个 Token 分成大小 `C_t` 的块：

```
X [N,H]
→ X_0 [C_t,H], X_1 [C_t,H], ...
```

每块：

```
Z_i = X_i W^T        [C_t,V]
loss_i = CE(Z_i,y_i)
```

瞬时 Logits 从：

```
N×V
```

降到：

```
C_t×V
```

真实配置：

```
C_t = 256
V = 128256
BF16
```

Chunk Logits：

```
256×128256×2 Byte
≈ 62.6 MiB
```

相比完整 `3.914 GiB`，单块明显更小。

关键陷阱：普通 Python Loop 不一定真正省 Backward 峰值

如果只是：

```
loss = 0
for chunk in chunks:
    logits = chunk @ W.T
    loss = loss + cross_entropy(logits, labels_chunk)
loss.backward()
```

Autograd 可能为每个 Chunk 保存一张子图，直到最后统一 `backward()`。所有 Chunk 的 Saved Tensors 仍可能累计，不能仅凭“循环里覆盖了 Python 变量 logits”就断言显存变成一个 Chunk。

真正释放通常需要：

- 自定义 `autograd.Function`；
- Forward 只保存小统计量，Backward 重算 Chunk；
- 或逐 Chunk Backward 并正确缩放/累加梯度；
- 或框架提供的 Fused Linear Cross Entropy。

十四、显存优化路线 4：Vocabulary Chunk + Online Softmax

把 Vocabulary 分块：

```
W [V,H]
→ W_0 [C_v,H], W_1 [C_v,H], ...
```

每次只产生：

```
Z_chunk [N,C_v]
```

问题是 Softmax 分母需要全词表：

```
Σ_v exp(z_v)
```

可以使用 Online Softmax 合并每块统计量。

假设当前已经处理一部分 Vocabulary，维护：

```
m_old = 当前全局最大值
l_old = Σ exp(z-m_old)
```

新 Chunk 的统计：

```
m_chunk = max(z_chunk)
l_chunk = Σ exp(z_chunk-m_chunk)
```

合并：

```
m_new = max(m_old,m_chunk)

l_new
= l_old × exp(m_old-m_new)
+ l_chunk × exp(m_chunk-m_new)
```

处理完全部 Vocabulary 后：

```
logsumexp = m_final + log(l_final)
```

同时记录目标 Token 所在 Chunk 的 Target Logit，就能精确计算 Cross Entropy，而无需完整 `[N,V]` Logits。

显存从：

```
N×V×bytes
```

降到：

```
N×C_v×bytes
+ O(N) FP32 statistics
```

代价：

- 多个小 GEMM；
- Chunk 太小会降低 Tensor Core 利用率；
- 更多 Kernel Launch；
- Backward 通常需要重算 Chunk Logits。

十五、显存优化路线 5：Fused Linear Cross Entropy

标准边界：

```
Linear 输出完整 Logits
→ Framework 保存 Logits/Softmax
→ Cross Entropy
→ Backward 生成完整 dLogits
→ Linear Backward
```

Fusion 后：

```
linear_cross_entropy(hidden, weight, labels)
```

把 Projection 和 CE 视为一个整体算子。

Forward

分 Chunk/Tile 执行：

```
1. 计算一小块 Logits
2. 更新 per-token max 和 sum-exp
3. 如果目标 Token 在当前块，保存 Target Logit
```

```
4. 立即覆盖或释放当前 Logits Tile
5. 最终得到 Loss
```

只需长期保存：

```
Hidden [N,H]
Weight [V,H]
Target [N]
Max [N] FP32
SumExp [N] FP32
```

Backward

为了计算：

```
dZ = softmax(Z) − one_hot
```

如果 Forward 没保存完整 Z，就必须：

```
重新按 Chunk 计算 Logits
→ 使用保存的 max/sum-exp 恢复概率
→ 生成一个 dZ Chunk
→ 立即累计 dH 和 dW
→ 释放 dZ Chunk
```

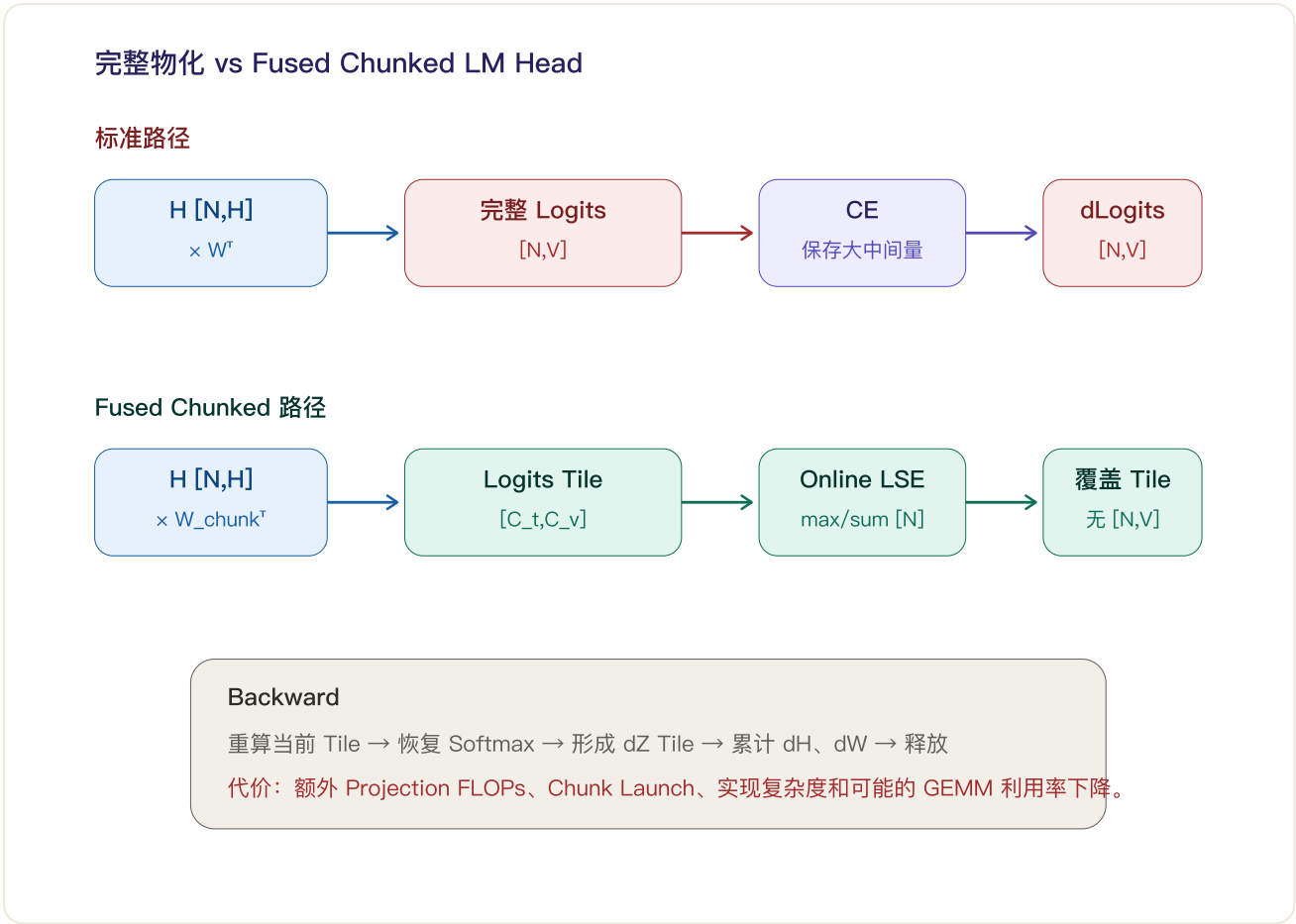
峰值由：

```
Token Chunk × Vocab Chunk
```

而不是：

```
N × V
```

决定。



一个重要的实现分歧

不同 Fused Linear CE 可能采用不同策略：

1. **Forward 只保存统计，Backward 重算**：低峰值、增加约一次 Projection；
2. **Forward 直接预计算部分梯度**：减少 Backward 重算，但需保存 `dH/dW` 或特殊状态；
3. **Token Chunk**：Tile `[C_t,V]`；
4. **Vocab Chunk**：Tile `[N,C_v]`；
5. **二维 Tile**：Tile `[C_t,C_v]`；
6. **TP + Chunk**：每 Rank 在 `V/P` 内继续分块。

不能只看到“Fused Linear CE”名字就假设显存、FLOPs 和通信完全一样，必须看具体实现。

十六、Cut Cross Entropy（CCE）

Cut Cross Entropy 的核心目标同样是：

```
不把所有 Token × Vocabulary Logits 写入全局显存
```

其核心思路：

- 只显式关注正确类别 Logit；
- 对所有类别的 `logsumexp` 在 Tile 中在线归约；
- 自定义 Kernel 把矩阵乘和 LSE Reduction 紧密结合；
- 中间 Logits 尽量停留在片上存储或短生命周期 Tile 中；
- Backward 可利用 Softmax 梯度的数值稀疏性，跳过低于数值精度、贡献可忽略的项，以改善吞吐。

Apple CCE 工作报告：在其 Gemma 2 2B 配置中，Loss 计算内存从 24 GB 降至约 1 MB，Classifier Head 总训练内存从 28 GB 降至约 1 GB。**这些是特定模型、实现与测量口径的报告结果，不能直接外推到你的模型。**

CCE 与普通 Chunk 的关键差异不只是“写一个 for 循环”，而是：

```
矩阵乘 Tile
+ Online LSE
+ Target Gather
+ Backward Gradient Tile
```

在定制 Kernel 中协同调度，避免全局显存往返和 Autograd 子图堆积。

十七、优化方案的计算—显存—通信对比

方案	完整 Logits	每 Rank 权重	主 GEMM FLOPs	通信	主要风险
标准 Linear+CE	`[N,V]`	`[V,H]`	约 `6NHV`	DP 梯度同步	峰值大
BF16 Logits	`[N,V]`	`[V,H]`	近似不变	不变	仍是 `N×V`
Vocab TP=P	`[N,V/P]` Local	`[V/P,H]`	每 Rank约 `6NHV/P`	Max/Sum/Target + dH AllReduce	网络与同步
Token Chunk	`[C_t,V]`	`[V,H]`	若重算约 `8NHV`	通常不新增	小 GEMM/Autograd 图陷阱
Vocab Chunk	`[N,C_v]`	`[V,H]`	若重算约 `8NHV`	通常不新增	多次扫 Hidden/Weight
TP + Chunk	`[C_t,C_v]`	`[V/P,H]`	每 Rank约 `8NHV/P`	TP Collectives	实现复杂

方案	完整 Logits	每 Rank 权重	主 GEMM FLOPs	通信	主要风险
CCE-like	Tile，不全局物化	`[V,H]` 或分片	上界约 `8NHV`，可有稀疏跳过	依并行实现	专用 Kernel 与数值验证

结论翻转条件

- `V/H` 很小、序列很短：完整 Logits 未必是峰值，Fusion 收益有限；
- `V` 很大、`N` 很大：LM Head 优化收益显著；
- GPU 算力强但 HBM 紧张：重算换显存通常更有吸引力；
- 网络慢：Vocab TP 可能省显存但通信暴露变大；
- Chunk 太小：显存继续下降，但 Tensor Core 利用率和 Launch 效率恶化；
- Weight Tied：参数显存已共享，但 Logits 动态显存仍然存在；
- Sequence Parallel/Context Parallel：每 Rank `N` 可能下降，但最后是否 Gather、如何计算 Loss 决定真实峰值。

十八、针对你当前 LM Head 显存优化工作的分析框架

18.1 第一件事：确认峰值 Tensor 身份

不要先猜方案，先记录峰值前后的大分配：

```
H_final             [N,H]
LM Head Weight      [V,H] 或 [V/P,H]
Logits              [N,V] 或 [N,V/P]
Saved Softmax       [N,V] 或 Local
Logits Gradient     [N,V] 或 Local
Weight Gradient     [V,H] 或 Local
Hidden Gradient     [N,H]
Gather/Reduce Buffer
Kernel Workspace
```

推荐同时看：

```
torch.cuda.reset_peak_memory_stats()
# run target region
torch.cuda.synchronize()
print(torch.cuda.max_memory_allocated() / 2**30)
print(torch.cuda.max_memory_reserved() / 2**30)
```

以及：

```
torch.cuda.memory._record_memory_history()
# run step
torch.cuda.memory._dump_snapshot("lm_head_snapshot.pickle")
```

再结合 Nsight Systems 确认 Kernel 和 Collective 时间线。

18.2 建立分阶段 NVTX 标记

```
with torch.cuda.nvtx.range("final_norm"):
    hidden = final_norm(hidden)

with torch.cuda.nvtx.range("lm_head_projection"):
    logits = lm_head(hidden)

with torch.cuda.nvtx.range("cross_entropy"):
    loss = cross_entropy(logits, labels)

with torch.cuda.nvtx.range("backward"):
    loss.backward()
```

Fused 后继续细分：

```
fwd_chunk_projection
fwd_online_lse
bwd_recompute_projection
bwd_dhidden
bwd_dweight
TP stats allreduce
dHidden allreduce/reduce-scatter
```

18.3 必须同时记录的指标

显存

```
Peak allocated / reserved
Logits Tensor bytes
Saved Tensor bytes
dLogits bytes
Workspace bytes
Allocator fragmentation
每阶段生命周期
```

计算

```
Projection GEMM duration
CE duration
Backward recompute duration
dH GEMM duration
dW GEMM duration
Tensor Core utilization
Achieved TFLOP/s
```

带宽

```
HBM read/write throughput
L2 hit rate
Weight 重读次数
Hidden 重读次数
```

通信

```
TP max/sum/target collectives
dH AllReduce/ReduceScatter
通信是否与其他计算重叠
Gather Buffer 是否完整物化全 Vocab
```

数值

```
Loss absolute/relative error
dHidden max/mean error
dWeight max/mean error
长训练 Loss Curve
NaN/Inf
不同 dtype 和极端 Logit 范围
Ignore Index / Label Smoothing
```

18.4 公平对比方法

比较 Baseline 与优化版本时固定：

```
同一 Hidden 和 Labels
同一 Weight
同一 Loss Reduction
同一 Ignore Index
同一 Label Smoothing
同一 dtype/accumulation dtype
同一 TP/SP/CP 配置
```

校验：

```
Forward Loss
Per-token Loss
LM Head dW
dHidden
Tied Embedding 总梯度
一个完整 Optimizer Step 后参数
```

性能要先 Warmup，再多次测量 P50/P95，避免把首轮编译、Autotune 或 CUDA Context 初始化计入。

十九、Chunk Size 应如何选择

Chunk Size 是典型的 U 型权衡。

Chunk 太大

- 峰值显存高；
- 可并发/流水机会少；
- 可能再次 OOM。

Chunk 太小

- GEMM M/N 维太小；
- Tensor Core 利用率下降；
- Kernel Launch 数量增加；
- Online Reduction 和索引开销占比提高；
- Weight/Hidden 可能被重复从 HBM 读取；
- TP Collective 过碎时网络效率下降。

建议实验矩阵：

```
Token Chunk: 64,128,256,512,1024
Vocab Chunk: 1024,2048,4096,8192,16384
```

每个点记录：

```
Peak Memory
Step Time
LM Head Time
Achieved TFLOP/s
HBM Bandwidth
数值误差
```

最后画：

```
x 轴：Peak Memory
y 轴：Step Time
```

选择 Pareto Frontier，而不是只选显存最低的配置。

二十、动态图网页如何使用

打开：

```
https://psgmessi.github.io/ai-infra-learning/lm-head-memory-explorer.html
```

可切换：

```
标准 Linear + CE
Vocabulary Parallel
Token Chunk Fused CE
Vocabulary Chunk Fused CE
TP + 双维 Chunk
CCE-like Tile Kernel
```

可动态修改：

```
B、S、有效 Token 比例、H、V、dtype、TP、Token Chunk、Vocab Chunk
```

网页会更新：

1. 有效 Token 数；
2. 每 Rank Forward FLOPs；
3. 每 Rank训练 FLOPs；
4. 逐阶段解析峰值；
5. Weight、dW、Hidden、Logits、dLogits、Stats、dHidden 和通信 Buffer 的生命周期；
6. 与标准实现相比的峰值变化；
7. 每一步的 Shape、FLOPs 和活跃 Tensor。

默认真实规模：

```
B=2, S=8192, H=4096, V=128256, BF16
```

也提供长上下文 Preset。网页使用解析模型，最终结论必须以真实代码 Profiling 为准。

二十一、最小 PyTorch 基线代码

```
import torch
import torch.nn.functional as F
```

```
B, S, H, V = 2, 8, 16, 32

torch.manual_seed(0)
hidden = torch.randn(B, S, H, device="cuda", dtype=torch.bfloat16, requires_grad=True)
weight = torch.randn(V, H, device="cuda", dtype=torch.bfloat16, requires_grad=True)
labels = torch.randint(0, V, (B, S), device="cuda")

# Causal LM shift
x = hidden[:, :-1, :].contiguous()       # [B,S-1,H]
y = labels[:, 1:].contiguous()          # [B,S-1]

logits = x @ weight.t()                  # [B,S-1,V]
loss = F.cross_entropy(
    logits.view(-1, V),
    y.reshape(-1),
    reduction="mean",
)

loss.backward()

print("logits:", logits.shape)
print("loss:", float(loss))
print("dHidden:", hidden.grad.shape)
print("dWeight:", weight.grad.shape)
```

一个容易忽略的问题

如果 Logits 在 Loss 之前被额外用于：

- Accuracy/Top-k Metric；
- Distillation；
- KL Loss；
- Logit Regularization；
- 多任务 Loss；
- 保存或日志；

那么 Fused Linear CE 不能简单丢弃完整 Logits，或者需要为这些消费者提供新的流式接口。优化前必须梳理所有消费者。

二十二、最小 Fused Chunk 伪代码

下面展示算法结构，不是生产 Kernel：

```
class LinearCrossEntropy(torch.autograd.Function):
    @staticmethod
    def forward(ctx, hidden, weight, target, chunk_size):
        # hidden [N,H], weight [V,H]
        # 只保存小统计，不保存完整 [N,V] logits
        max_value = full((N,), -inf, fp32)
        sum_exp = zeros((N,), fp32)
```

```
        target_logit = zeros((N,), fp32)

        for v_start in range(0, V, chunk_size):
            Wc = weight[v_start:v_end]          # [Cv,H]
            Zc = hidden @ Wc.T                  # [N,Cv]

            # Online logsumexp merge
            chunk_max = Zc.max(-1)
            new_max = maximum(max_value, chunk_max)
            sum_exp = (
                sum_exp * exp(max_value-new_max)
                + exp(Zc-new_max[:,None]).sum(-1)
            )
            max_value = new_max

            # 当前块包含目标 token 时提取 target logit
            gather_target_if_owned(...)

        loss = max_value + log(sum_exp) - target_logit
        ctx.save_for_backward(hidden, weight, target, max_value, sum_exp)
        return loss.mean()

    @staticmethod
    def backward(ctx, grad_output):
        # 重新分块计算 logits/softmax，不生成完整 [N,V]
        d_hidden = zeros_like(hidden)
        d_weight = zeros_like(weight)

        for each vocab_chunk:
            Zc = hidden @ Wc.T
            Pc = exp(Zc-max_value[:,None]) / sum_exp[:,None]
            dZc = Pc
            subtract_one_hot_if_target_owned(dZc)
            dZc *= grad_output / N

            d_hidden += dZc @ Wc
            d_weight[v_start:v_end] += dZc.T @ hidden

        return d_hidden, d_weight, None, None
```

生产实现需要进一步处理：

- BF16/FP16 输入与 FP32 Reduction；
- Label Smoothing；
- Ignore Index；
- Z-loss / Logit Soft-Capping；
- TP/SP/CP；
- Weight Tying；
- Autocast；
- Distributed Gradient Semantics；
- Stream 与 Buffer 生命周期；
- Kernel Fusion 和 Tile 调度；
- 非整除 Vocabulary Padding。

二十三、常见误区

误区 1：只有目标 Token 的 Logit 有用，所以其他 Logits 不用算

Cross Entropy 分母需要：

```
Σ_v exp(z_v)
```

因此所有类别都参与归一化。可以不把所有 Logits 写入全局显存，但通常仍需计算其对 `logsumexp` 的贡献。

误区 2：把 logits 变量覆盖，就释放了显存

Autograd 可能仍通过计算图持有底层 Tensor。Python 变量名被覆盖不等于 Tensor 生命周期结束。

误区 3：Token Chunk 循环一定把峰值降为一个 Chunk

若所有 Chunk 的计算图都保留到最后统一 Backward，Saved Tensors 仍可能累计。需要自定义 Backward、重计算或逐块 Backward。

误区 4：Vocabulary Parallel 不需要完整词表信息

它不需要 Gather 完整 Logits，但仍需要跨 Rank 合并 Global Max、Sum-Exp 和 Target Logit；Backward 还需合并 `dHidden`。

误区 5：Fused 一定更快

Fusion 通常省 HBM，但分块重算和小 GEMM 可能增加计算与 Launch。是否更快由 `N/H/V/Chunk/GPU/TP` 共同决定。

误区 6：Weight Tying 消除了 LM Head 显存问题

它减少参数、梯度和优化器状态副本，但不消除 `[N,V]` Logits 动态显存。

误区 7：只比较 Forward Loss 就够了

必须比较 `dHidden`、`dWeight`、Tied Embedding 总梯度和完整 Optimizer Step。很多错误只出现在 Backward、Mask、Reduction 或分布式路径。

二十四、建议你的优化工作按这个顺序推进

第 1 阶段：精确诊断

```
确认完整 Logits 是否实际物化
确认保存的是 Logits、Softmax 还是 LogProbs
确认 Backward 是否又分配完整 dLogits
确认 TP 是否误 AllGather 全词表
确认 Metrics/Distillation 是否持有 Logits
确认峰值是在 CE Forward、CE Backward、dW 还是通信阶段
```

第 2 阶段：建立正确性基线

```
Tiny FP64/FP32 Case
Forward Loss 对齐
Per-token Loss 对齐
dHidden 对齐
dWeight 对齐
Ignore Index 对齐
Label Smoothing 对齐
TP 1/2/4 对齐
```

第 3 阶段：逐层加入优化

推荐增量顺序：

```
1. BF16 Logits + FP32 Reduction
2. 避免全词表 AllGather，使用 Vocab Parallel CE
3. 自定义 Fused Linear CE，先做较大 Chunk
4. 扫 Token/Vocab Chunk Size
5. 融合 Online LSE、Target Gather 和 dZ
6. 优化 dH/dW 累加与 Buffer 复用
7. 再考虑 CCE-like 片上 Tile 与梯度过滤
```

每一步都做数值和峰值回归，避免同时改变太多变量。

第 4 阶段：端到端裁决

最终不是看 LM Head 单算子，而是：

```
Peak Memory 是否下降
是否能增加 Sequence 或 Micro Batch
Step Time 是否下降
MFU 是否变化
通信是否暴露
```

```
训练 Loss Curve 是否一致
Checkpoint/Resume 是否正确
```

如果省下的显存可以把 Micro Batch 从 1 提高到 2，即使 Fused LM Head 自身略慢，端到端吞吐也可能提高；反之，若模型本来不 OOM，过度切 Chunk 可能只带来额外开销。

二十五、关键总结

1. LM Head 前向是 `Z=XW^T`，输出 Shape `[N,V]`，Projection FLOPs 约 `2NHV`。
2. Stable Cross Entropy 对每个 Token 计算 Global Max、Sum-Exp 和 Target Logit；数学上需要所有类别的贡献，但不要求完整 Logits 长期驻留显存。
3. CE Backward：`dZ=(P-OneHot)/N`；Linear Backward：`dX=dZW`、`dW=dZ^TX`。
4. 标准训练主 GEMM FLOPs 约 `6NHV`；重算型 Fused CE 上界约 `8NHV`。
5. 完整 Logits 显存为 `N×V×dtype_bytes`，真实例子中单份 BF16 Logits 可达 3.914 GiB。
6. Weight Tying 节省参数状态，但不解决动态 Logits 峰值。
7. Vocab TP 把 Weight、Logits、dW 分片，但增加全局 Softmax 统计和 `dHidden` 通信。
8. 真正的 Chunk/Fusion 必须控制 Autograd Saved Tensor 生命周期；普通 Python Loop 不保证峰值下降。
9. 优化应同时看显存、重算 FLOPs、GEMM 利用率、HBM、通信、数值误差与端到端 Step Time。
10. 动态网页可以先做解析探索，最终必须用 Memory Snapshot 和 Nsight 验证真实实现。

可选自测

1. Shape 与 FLOPs

若：

```
N=32768, H=4096, V=128000
```

请计算：

```
Logits Shape
BF16 Logits GiB
Forward Projection FLOPs
标准训练三个 GEMM FLOPs
```

2. 反向推导

从：

```
Z=XW^T
```

只根据 Shape 推导：

```
dX = ?
dW = ?
```

并指出各自收缩哪个轴。

3. 工程决策

如果你的优化把 LM Head 峰值从 8 GiB 降到 1 GiB，但 LM Head 时间增加 20%，你会检查哪些端到端指标来判断它是否值得上线?

继续学习

1. 前置：Stable Softmax、LogSumExp 与 Online Softmax —— 理解不物化全 Logits 的数学基础。
2. 同层对比：Megatron Vocab Parallel CE 与 CCE/Fused Linear CE —— 比较通信、重计算和 Tile 策略。
3. 下游应用：LM Head Kernel Profiling 与 Nsight 调优 —— 将显存模型落到真实 CUDA/Triton 时间线。

回复 1、2 或 3 继续；也可以说“稍后”或“不感兴趣”。

事实来源与版本边界

截至 2026-08-27，本讲解基于稳定的矩阵微积分、Softmax Cross Entropy 原理，以及公开实现/资料：NVIDIA Megatron-LM `VocabParallelCrossEntropy` 源码、NVIDIA NeMo RL Chunked Linear Cross-Entropy 文档、Cut Cross-Entropy（Wijmans et al., ICLR 2025）与公开 Fused Linear Cross Entropy 实现说明。

不同框架对以下内容的实现可能不同：

```
是否保存 Logits 或 Softmax
是否原地覆盖
Backward 是否重算
Reduction dtype
Label Smoothing
Ignore Index
TP/SP/CP 支持
Chunk 维度
通信与计算重叠
```

文中的峰值是显式 Tensor 生命周期解析模型，不包含 Allocator 碎片和所有 Kernel Workspace；公开项目报告的显存/性能提升只适用于其测量配置，不能直接当作你的训练任务结果。